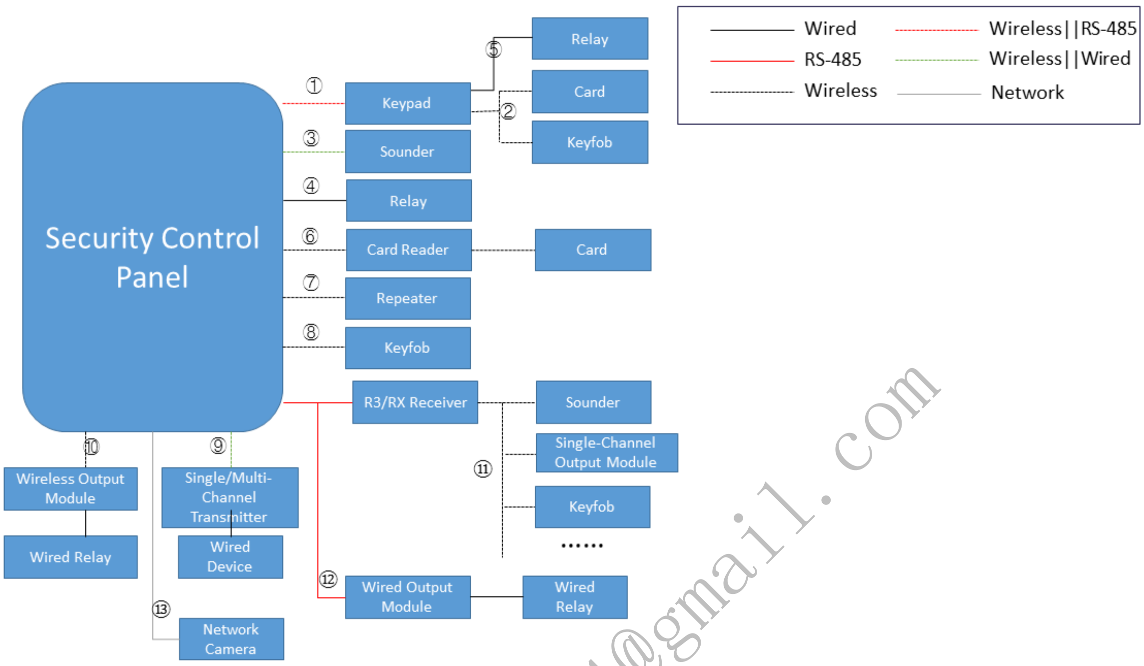
1 (access wired detectors via onboard transmitters), 3 (access wireless detectors via internal wireless receivers), 4 (access 1 to 2 wired detectors by enabling the function of extending zones for some magnetic contact detectors), 9 (access wired detectors by connecting wireless receivers with single-channel / multi-channel transmitters)

9.2.2 Topology of Peripherals



The hybrid security control panel can access peripherals in the following ways:

1 (access keypads via the RS-485 bus), 2 (access wireless cards and keyfobs via keypads), 3 (access wired sounders via onboard sounder modules), 4 (access wired relays via onboard relay modules), 5 (access wired relays via keypads), 11 (access wireless sounders, single-channel output modules (wireless relays), and keyfobs via the receivers connected to the RS-485 bus), 12 (access wired relays via the wired output module connected to the RS-485 bus), 13 (access network cameras by login via the network)

The wireless security control panel can access peripherals in the following ways:

1 (access keypas via wireless network), 3 (access wireless sounders via internal wireless receivers), 4 (access wired relays via onboard transmitters), 6 (access card readers via wireless network), 7 (access repeaters via wireless network), 8 (access keyfobs via wireless network), 9 (access wired relays by connecting wireless receivers with single-channel / multi-channel transmitters), 10 (access wired relays by connecting wireless receivers with wireless output modules), 13 (access network cameras by login via the network)

10 API Reference

10.1 Device (General)

10.1.1 Event Subscription

10.1.1.1 Get the alarm/event subscription capability

Request URL

GET /ISAPI/Event/notification/subscribeEventCap

Query Parameter

None

Request Message

None

Response Message

```
<?xml version="1.0" encoding="UTF-8"?>
<SubscribeEventCap xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, picture uploading modes of all events which contain pictures, attr:version{req, string, protocolVersion}-->
  <heartbeat min="1" max="180">
    <!--ro, opt, int, heartbeat interval time, range:[1,180], unit:s, attr:min{req, int},max{req, int}-->1
  </heartbeat>
  <channelMode opt="all,list">
    <!--ro, opt, enum, channel subscription mode, subType:string, attr:opt{req, string}, desc:"all" (subscribe to all channels), "list" (subscribe to
channels according to channel list)-->list
  </channelMode>
  <EventList>
    <!--ro, opt, array, event type list for subscription, subType:object, desc:this node is valid when eventMode is "list"-->
    <Event>
      <!--ro, opt, object, subscription of a specified alarm/event-->
      <type>
        <!--ro, req, enum, event type, subType:string, desc:refer to event type list (eventType): "ADAS"(advanced driving assistance system), "ADASAlarm"
(advanced driving assistance alarm), "AID"(traffic incident detection), "ANPR"(automatic number plate recognition), "AccessControllerEvent" (access
controller event), "CDsStatus" (CD burning status), "DBD"(driving behavior detection) "GPSUpload" (GPS information upload), "HFPD"(frequently appeared
person detection), "IO"(I/O alarm), "IOTD" (IoT device detection), "LES" (logistics scanning event), "LFPD"(rarely appeared person detection), "PALMismatch"
(video standard mismatch), "PIR", "PeopleCounting" (people counting), "PeopleNumChange" (people number change detection), "Standup"(standing up detection),
"TMA"(thermometry alarm), "TMPA"(temperature measurement pre-alarm), "VMD"(motion detection), "abnormalAcceleration", "abnormalDriving", "advReachHeight",
"alarmResult", "attendance", "attendedBaggage", "audioAbnormal", "audioexception", "behaviorResult"(abnormal event detection), "blindSpotDetection"(blind
spot detection alarm), "cardMatch", "changedStatus", "collision", "containerDetection", "crowdSituationAnalysis", "databaseException", "defocus"(defocus
detection), "diskUnformat"(disk unformatted), "diskerror", "diskfull", "driverConditionMonitor"(driver status monitoring alarm); "emergencyAlarm",
"faceCapture", "faceSnapModeling", "facedetection", "failDown"(People Falling Down), "faultAlarm", "fielddetection"(intrusion detection), "fireDetection",
"fireEscapeDetection", "flowOverrun", "framesPeopleCounting", "getUp"(getting up detection), "group" (people gathering), "hdBadBlock"(HDD bad sector
detection event), "hdImpact"(HDD impact detection event), "heatmap"(heat map alarm), "highHDTemperature"(HDD high temperature detection event),
"highTempAlarm"(HDD high temperature alarm), "hotSpare"(hot spare exception), "illaccess"(invalid access), "ipcTransferAbnormal", "ipconflict"(IP address
conflicts), "keyPersonGetUp"(key person getting up detection), "leavePosition"(absence detection), "linedetection"(line crossing detection),
"listSyncException"(list synchronization exception), "loitering"(loitering detection), "lowHDTemperature"(HDD low temperature detection event),
"mixedTargetDetection"(multi-target-type detection), "modelError", "nicbroken"(network disconnected), "nodeOffline"(node disconnected),
"nonPoliceIntrusion", "overSpeed"(overspeed alarm), "overtimeTarry"(staying overtime detection), "parking"(parking detection), "peopleNumChange",
"peopleNumCounting", "personAbnormalAlarm"(person ID exception alarm), "personDensityDetection", "personQueueCounting", "personQueueDetection",
"personQueueRealTime"(real-time data of people queuing-up detection), "personQueueTime"(waiting time detection), "playCellphone"(playing mobile phone
detection), "pocException"(video exception), "poe"(POE power exception), "policeAbsent", "radarAlarm", "radarFieldDetection", "radarLineDetection",
"radarPerimeterRule"(radar rule data), "radarTargetDetection", "radarVideoDetection"(radar-assisted target detection), "raidException", "rapidMove",
"reachHeight"(climbing detection), "recordCycleAbnormal"(insufficient recording period), "recordException", "regionEntrance", "regionExiting", "retention"
(people overstay detection), "rollover", "running"(people running), "safetyHelmetDetection"(hard hat detection), "scenechangedetection", "sensorAlarm"
(angular acceleration alarm), "severeHDFailure"(HDD major fault detection), "shelteralarm"(video tampering alarm), "shipsDetection", "sitQuietly"(sitting
detection), "smokeAndFireDetection", "smokeDetection", "softIO", "spacingChange"(distance exception), "sysStorFull"(storaging full alarm of cluster system),
"takingElevatorDetection"(elevator electric moped detection), "targetCapture", "temperature"(temperature difference alarm), "thermometry"(temperature
alarm), "thirdPartyException", "toiletTarry"(in-toilet overtime detection), "tollCodeInfo"(QR code information report), "tossing"(thrown object detection),
"unattendedBaggage", "vehicleMatchResult"(uploading list alarms), "vehicleRcogResult", "versionAbnormal"(cluster version exception), "videoException",
"videoloss", "violationAlarm", "violentMotion"(violent motion detection), "yardTarry"(playground overstay detection), "AccessControllerEvent",
"IDCardInfoEvent", "FaceTemperatureMeasurementEvent", "QRCodeEvent"(QR code event of access control), "CertificateCaptureEvent"(person ID capture comparison
event), "UncertificateCompareEvent", "ConsumptionAndTransactionRecordEvent", "ConsumptionEvent", "TFS" (traffic enforcement event),
"TransactionRecordEvent", "HealthInfoSyncQuery" (health information search event), "SetMealQuery"(searching consumption set meals), "ConsumptionStatusQuery"
(searching the consumption status), "certificateRevocation" (certificate expiry), "humanBodyComparison" (human body comparison),
"regionTargetNumberCounting" (regional target statistics)-->mixedTargetDetection
      </type>
      <minorAlarm opt="0x400,0x401,0x402,0x403">
        <!--ro, opt, string, minor alarm type, attr:opt{req, string}, desc:"IDCardInfoEvent" is required when the type of event is "AccessControllerEvent"--
>0x400,0x401
      </minorAlarm>
      <minorException opt="0x400,0x401,0x402,0x403">
        <!--ro, opt, string, minor exception type, attr:opt{req, string}, desc:"IDCardInfoEvent" is required when the type of event is
"AccessControllerEvent"-->0x400,0x401
      </minorException>
      <minorOperation opt="0x400,0x401,0x402,0x403">
        <!--ro, opt, string, minor operation type, attr:opt{req, string}, desc:"IDCardInfoEvent" is required when the type of event is
"AccessControllerEvent"-->0x400,0x401
      </minorOperation>
      <minorEvent opt="0x01,0x02,0x03,0x04">
        <!--ro, opt, string, minor event type, attr:opt{req, string}, desc:"IDCardInfoEvent" is required when the type of event is "AccessControllerEvent"--
>0x400,0x401
      </minorEvent>
    </Event>
  </EventList>
</SubscribeEventCap>
```

10.1.1.2 Event subscription heartbeat

EventType:heartBeat

```xml
<?xml version="1.0" encoding="UTF-8"?>

<EventNotificationAlert xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, opt, object, alarm message, attr:version{opt, string, protocolVersion}-->
  <ipAddress>
    <!--ro, req, string, IPv4 address of the device that triggers the alarm-->172.6.64.7
  </ipAddress>
  <ipv6Address>
    <!--ro, opt, string, IPv6 address of the device that triggers the alarm-->1080:0:0:0:8:800:200C:417A
  </ipv6Address>
  <portNo>
    <!--ro, opt, int, communication port No. of the device that triggers the alarm-->80
  </portNo>
  <protocol>
    <!--ro, opt, enum, transmission communication protocol type, subType:string, desc:when ISAPI protocol is transmitted via HCNetSDK, the channel No. is the video channel No. of private protocol. When ISAPI protocol is transmitted via EZ protocol, the channel No. is the video channel No. of EZ protocol. When ISAPI protocol is transmitted via ISUP, the channel No. is the video channel No. of ISUP-->HTTP
  </protocol>
  <macAddress>
    <!--ro, opt, string, MAC address-->01:17:24:45:D9:F4
  </macAddress>
  <channelID>
    <!--ro, opt, int, channel No. of the device that triggers the alarm, desc:video channel No. that triggers the alarm-->1
  </channelID>
  <dateTime>
    <!--ro, req, datetime, alarm trigger time-->2004-05-03T17:30:08+08:00
  </dateTime>
  <activePostCount>
    <!--ro, opt, int, times that the same alarm has been uploaded, desc:event triggering frequency-->1
  </activePostCount>
  <eventType>
    <!--ro, req, string, event type-->heartBeat
  </eventType>
  <eventState>
    <!--ro, req, enum, event status, subType:string, desc:for durative event: "active" (valid), "inactive" (invalid)-->active
  </eventState>
  <eventDescription>
    <!--ro, req, string, event description-->heartBeat
  </eventDescription>
  <channelName>
    <!--ro, opt, string, channel name, range:[1,64]-->test
  </channelName>
  <deviceID>
    <!--ro, opt, string, device ID, desc:it should be returned for ISUP alarms, e.g., test0123 (Ehome2.0, Ehome4.0, and ISUP5.0)-->12345
  </deviceID>
</EventNotificationAlert>
```


10.1.2 Port Settings

10.1.2.1 Get the capability of a specific serial port

Request URL

GET /ISAPI/System/Serial/ports/<portID>/capabilities

Query Parameter

Parameter Name	Parameter Type	Description
portID	string	--

Request Message

None

Response Message

```xml
<?xml version="1.0" encoding="UTF-8"?>
<SerialPort xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, opt, object, port No., attr:version{req, string, protocolVersion}-->
  <id>
    <!--ro, req, int, ID-->0
  </id>
  <serialPortType opt="RS485,RS422,RS232">
    <!--ro, opt, string, serial port type, attr:opt{req, string}-->RS485
  </serialPortType>
  <serialAddress min="0" max="10">
    <!--ro, opt, int, serial port address, attr:min{req, int},max{req, int}, desc:serial port address-->1
  </serialAddress>
  <duplexMode opt="half,full">
    <!--ro, opt, string, duplex mode of the serial port, attr:opt{req, string}-->half
  </duplexMode>
  <baudRate opt="1200,2400,4800,9600,19200,38400,57600,115200">
    <!--ro, opt, int, attr:opt{req, string}-->1200
  </baudRate>
  <dataBits opt="6,7,8">
    <!--ro, opt, int, attr:opt{req, string}-->6
  </dataBits>
  <parityType opt="none,even,odd,mark,space">
    <!--ro, opt, string, attr:opt{req, string}-->none
  </parityType>
  <stopBits opt="1,1.5,2">
    <!--ro, opt, string, stop bit, attr:opt{req, string}-->1
  </stopBits>
  <workMode opt="console,transparent,audiomixe,screenCtrl,ptzCtrl,keyboard,matrix,audioMixers">
    <!--ro, opt, string, working mode, attr:opt{req, string}-->console
  </workMode>
  <flowCtrl opt="none,software,hardware">
    <!--ro, opt, string, flowCtrl, attr:opt{req, string}-->none
  </flowCtrl>
  <mode opt="readerMode,clientMode,externMode,stairsControl,accessControlHost,disabled,custom,cardReceiver,QRCodeReader">
    <!--ro, opt, string, working mode, attr:opt{req, string}-->readerMode
  </mode>
  <outputDataType opt="cardNo,employeeNo,auto">
    <!--ro, opt, string, output data type, attr:opt{req, string}, desc:data type output from the door station to the elevator controller: floorNumber (floor No.,default),cardNo (card No.)-->cardNo
  </outputDataType>
</SerialPort>
```


10.1.2.2 Set the parameters of a specific serial port supported by the device

Request URL

PUT /ISAPI/System/Serial/ports/<portID>?permissionController=<indexID>&childDevID=<childDevID>&deviceIndex=<deviceIndex>

Query Parameter

Parameter Name	Parameter Type	Description
portID	string	--
indexID	string	--
childDevID	string	--
deviceIndex	string	--

Request Message

```xml
<?xml version="1.0" encoding="UTF-8"?>
<SerialPort xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--opt, object, port, attr:version{req, string, protocolVersion}-->
  <id>
    <!--req, int, serial port ID-->0
  </id>
  <serialPortType>
    <!--opt, enum, serial port type: "RS485","RS422","RS232", subType:string, desc:serial port type: "RS485","RS422","RS232"-->RS485
  </serialPortType>
  <serialAddress>
    <!--opt, int-->1
  </serialAddress>
  <duplexMode>
    <!--opt, enum, duplex mode of the serial port, subType:string, desc:"half", "full”-->half
  </duplexMode>
  <baudRate>
    <!--opt, enum, subType:int-->2400
  </baudRate>
  <dataBits>
    <!--opt, int-->6
  </dataBits>
  <parityType>
    <!--opt, enum, parity type, subType:string, desc:"none, even, odd, mark, space”-->none
  </parityType>
  <stopBits>
    <!--opt, string, stop bit: "1,1.5,2"-->1
  </stopBits>
  <workMode>
    <!--opt, enum, work mode, subType:string, desc:working mode: "console","transparent","audiomixer","stairsControl"-elevator control,"cardReader"-card
reader,"disabled","custom". This node is required only when <serialPortType> is set to "RS232"-->console
  </workMode>
  <flowCtrl>
    <!--opt, enum, "none,software,hardware", subType:string, desc:"none, software, hardware”-->none
  </flowCtrl>
  <mode>
    <!--opt, enum, work mode, subType:string, desc:deq,working mode: "readerMode,clientMode,externMode,accessControlHost,disabled",this node is valid only
when <serialPortType> is "RS485"-->readerMode
  </mode>
  <outputDataType>
    <!--opt, enum, output data type, subType:string, dep:and,{$.SerialPort.mode,eq,accessControlHost}, desc:"cardNo,employeeNo", this node is valid when
<mode>is "accessControlHost”-->cardNo
  </outputDataType>
</SerialPort>
```

Response Message

```xml
<?xml version="1.0" encoding="UTF-8"?>
<ResponseStatus xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, response message, attr:version{ro, req, string, protocolVersion}-->
  <requestURL>
    <!--ro, req, string, request URL-->null
  </requestURL>
  <statusCode>
    <!--ro, req, enum, status code, subType:int, desc:0 (OK), 1 (OK), 2 (Device Busy), 3 (Device Error), 4 (Invalid Operation), 5 (Invalid XML Format), 6
(Invalid XML Content), 7 (Reboot Required)-->0
  </statusCode>
  <statusString>
    <!--ro, req, enum, status description, subType:string, desc:“OK” (succeeded), “Device Busy”, “Device Error”, “Invalid Operation”, “Invalid XML Format”,
“Invalid XML Content”, “Reboot” (reboot device)-->OK
  </statusString>
  <subStatusCode>
    <!--ro, req, string, error reason description in detail, desc:error reason description in detail-->OK
  </subStatusCode>
</ResponseStatus>
```

10.1.2.3 Get the parameters of a specific port supported by the device

Request URL

GET /ISAPI/System/Serial/ports/<portID>?permissionController=<indexID>&childDevID=<childDevID>&deviceIndex=<deviceIndex>

Query Parameter

Parameter Name	Parameter Type	Description
portID	string	--
indexID	string	--
childDevID	string	--
deviceIndex	string	--

Request Message

None

Response Message


```
<?xml version="1.0" encoding="UTF-8"?>
<SerialPort xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, opt, object, port, attr:version{req, string, protocolVersion}-->
  <id>
    <!--ro, req, int, serial port ID-->0
  </id>
  <serialPortType>
    <!--ro, opt, enum, serial port type, subType:string, desc:"RS485", "RS422", "RS232"-->RS485
  </serialPortType>
  <serialAddress>
    <!--ro, opt, int-->1
  </serialAddress>
  <duplexMode>
    <!--ro, opt, enum, duplex mode of the serial port, subType:string, desc:"half", "full"-->half
  </duplexMode>
  <baudRate>
    <!--ro, opt, enum, subType:int-->2400
  </baudRate>
  <dataBits>
    <!--ro, opt, int-->6
  </dataBits>
  <parityType>
    <!--ro, opt, enum, parity type, subType:string, desc:parity type: "none,even,odd,mark,space"-->none
  </parityType>
  <stopBits>
    <!--ro, opt, enum, stop bit, subType:string, desc:stop bit-->1
  </stopBits>
  <workMode>
    <!--ro, opt, enum, working mode, subType:string, desc:"console", "transparent", "audiomixer", "screenCtrl", "ptzCtrl", "keyboard", "matrix",
"audioMixers"-->console
  </workMode>
  <flowCtrl>
    <!--ro, opt, enum, "none,software,hardware", subType:string, desc:"none", "software", "hardware"-->none
  </flowCtrl>
  <mode>
    <!--ro, opt, enum, working mode, subType:string, desc:deq,working mode: "readerMode,clientMode,externMode,accessControlHost,disabled",this node is valid
only when <serialPortType> is "RS485"-->readerMode
  </mode>
  <outputDataType>
    <!--ro, opt, enum, output data type, subType:string, dep:and,{$.SerialPort.mode,eq,accessControlHost}, desc:output data type: "cardNo,employeeNo",this
node is valid when <mode>is "accessControlHost"-->cardNo
  </outputDataType>
</SerialPort>
```


10.1.3 System Maintenance

10.1.3.1 Get the system security capability

Request URL

GET /ISAPI/Security/capabilities?username=<userName>&deviceIndex=<deviceIndex>

Query Parameter

Parameter Name	Parameter Type	Description
userName	string	user name
deviceIndex	string	--

Request Message

None

Response Message

```xml
<?xml version="1.0" encoding="UTF-8"?>
<SecurityCap xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, system security capability, attr:version{req, string, protocolVersion}-->
  <supportUserNums>
    <!--ro, opt, int, supported maximum number of users-->0
  </supportUserNums>
  <userBondIpNums>
    <!--ro, opt, int, supported maximum number of IP addresses that can be bound-->0
  </userBondIpNums>
  <userBondMacNums>
    <!--ro, opt, int, supported maximum number of MAC addresses that can be bound-->0
  </userBondMacNums>
  <issupIllegalLoginLock>
    <!--ro, opt, bool, whether the device supports locking login-->true
  </issupIllegalLoginLock>
  <isSupportOnlineUser>
    <!--ro, opt, bool, whether the device supports the online user configuration-->true
  </isSupportOnlineUser>
  <isSupportAnonymous>
    <!--ro, opt, bool, whether the device supports anonymous login-->true
  </isSupportAnonymous>
  <securityVersion opt="1,2">
    <!--ro, opt, int, encryption capability set, attr:opt{req, string}, desc:the encryption capability of each version consists of two parts: encryption algorithm and the range of encrypted nodes currently 1 refers to AES128 encryption and 2 refers to AES256 encryption, the range of encrypted nodes is described in each protocol-->1
  </securityVersion>
  <keyIterateNum>
    <!--ro, opt, int, secret key iteration times, dep:or,{$.SecurityCap.securityVersion,eq,1},{$.SecurityCap.securityVersion,eq,2}, desc:this node depends on the node securityVersion, the range is between 100 and 1000-->100
  </keyIterateNum>
  <isSupportUserCheck>
    <!--ro, opt, bool, whether the device supports verifying the login password when editing (editing/adding/deleting) user parameters, dep:or,{$.SecurityCap.securityVersion,eq,0},{$.SecurityCap.securityVersion,eq,1}, desc:it is an added capability, which indicates that whether supporting the login password verification for editing/adding/deleting user parameters, this node depends on the node securityVersion, which means that it is only valid for the versions that support encrypting the sensitive information-->true
  </isSupportUserCheck>
  <isIrreversible>
    <!--ro, opt, bool, whether the device supports irreversible password storage, desc:If this function is not supported, the plaintext password of the user information will be stored in the device; otherwise, the password will be hashed for storage in the device.-->true
  </isIrreversible>
  <salt>
    <!--ro, opt, string, the specific salt used by the user to log in-->test
  </salt>
</SecurityCap>
```

10.1.3.2 Get the device's system capability

Request URL

GET /ISAPI/System/capabilities

Query Parameter

None

Request Message

None

Response Message

```xml
<?xml version="1.0" encoding="UTF-8"?>
<DeviceCap xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, system capability of the device, attr:version{opt, string, protocolVersion}-->
  <SysCap>
    <!--ro, opt, object, system capability-->
    <isSupportDst>
      <!--ro, opt, bool, whether the device supports DST (Daylight Saving Time)-->true
    </isSupportDst>
    <SerialCap>
      <!--ro, opt, object, range of RS-485 serial port numbers supported by the device-->
    </SerialCap>
    <isSupportExternalDevice>
      <!--ro, opt, bool, whether the device supports connecting to peripherals: true (support), desc:related URI: /ISAPI/System/externalDevice/capabilities-
->true
    </isSupportExternalDevice>
    <isSupportSubscribeEvent>
      <!--ro, opt, bool, whether the device supports subscribing to events, desc:related URI: /ISAPI/Event/notification/subscribeEventCap-->true
    </isSupportSubscribeEvent>
  </SysCap>
  <isSupportSnapshot>
    <!--ro, opt, bool, whether the device supports capturing pictures-->true
  </isSupportSnapshot>
  <isSupportGIS>
    <!--ro, opt, bool, whether the device supports GIS, desc:related URI: /ISAPI/GIS/channels-->true
  </isSupportGIS>
  <isSupportCompass>
    <!--ro, opt, bool, whether the device supports compass configuration, desc:related URI: /ISAPI/Compass/channels/<ID>/capabilities-->true
  </isSupportCompass>
  <isSupportRoadInfoOverlays>
    <!--ro, opt, bool, whether the device supports overlaying the lane information-->true
  </isSupportRoadInfoOverlays>
  <isSupportFaceCaptureStatistics>
    <!--ro, opt, bool, whether the device supports face capture statistics, desc:related URI: /ISAPI/Intelligent/channels/<ID>/faceCaptureStatistics/search-
->true
  </isSupportFaceCaptureStatistics>
  <isSupportElectronicsEnlarge>
    <!--ro, opt, bool, whether the device supports electronics enlarge-->true
  </isSupportElectronicsEnlarge>
  <isSupportCloud>
    <!--ro, opt, bool, whether the device supports cloud storage-->true
  </isSupportCloud>
  <isSupportRecordHost>
    <!--ro, opt, bool, whether the device supports configuring the education sharing server, desc:related URI: /ISAPI/RecordHost/capabilities-->true
  </isSupportRecordHost>
  <isSupportAcsUpdate>
    <!--ro, opt, bool, whether the device supports peripheral module upgrade (true, support), desc:related URI: /ISAPI/System/AcsUpdate/capabilities-->true
  </isSupportAcsUpdate>
  <isSupportAccessControlCap>
    <!--ro, opt, bool, access control capability, desc:related URI: /ISAPI/AccessControl/capabilities-->true
  </isSupportAccessControlCap>
</DeviceCap>
```


10.1.3.3 Get the languages supported by the device

Request URL

GET /ISAPI/System/DeviceLanguage

Query Parameter

None

Request Message

None

Response Message


```xml
<?xml version="1.0" encoding="UTF-8"?>

<DeviceLanguage xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, languages supported by the device, attr:version{req, string, protocolVersion}-->
  <language>
    <!--ro, req, enum, language, subType:string, desc:"SimChinese" (simplified Chinese), "TraChinese" (traditional Chinese), "English", "Russian",
"Bulgarian", "Hungarian", "Greek", "German", "Italian", "Czech", "Slovakia", "French", "Polish", "Dutch", "Portuguese", "Spanish", "Romanian", "Turkish",
"Japanese", "Danish", "Swedish", "Norwegian", "Finnish", "Korean", "Thai", "Estonia", "Vietnamese", "Hebrew", "Latvian", "Arabic", "Sovenian"-Slovenian,
"Croatian", "Lithuanian", "Serbian", "BrazilianPortuguese"-Brazilian Portuguese, "Indonesian", "Ukrainian", "EURSpanish", "Sovenian", "Uzbek", "Kazak",
"Kirghiz", "Farsi", "Azerbaidzhan", "Burmese", "Mongolian"-->SimChinese
  </language>
</DeviceLanguage>
```


10.1.3.4 Set device language parameters

Request URL

PUT /ISAPI/System/DeviceLanguage

Query Parameter

None

Request Message

```xml
<?xml version="1.0" encoding="UTF-8"?>

<DeviceLanguage xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--req, object, languages supported by the device, attr:version{req, string, protocolVersion}-->
  <language>
    <!--req, enum, language, subType:string, desc:"SimChinese" (simplified Chinese), "TraChinese" (traditional Chinese), "English", "Russian", "Bulgarian", "Hungarian", "Greek", "German", "Italian", "Czech", "Slovakia", "French", "Polish", "Dutch", "Portuguese", "Spanish", "Romanian", "Turkish", "Japanese", "Danish", "Swedish", "Norwegian", "Finnish", "Korean", "Thai", "Estonia", "Vietnamese", "Hebrew", "Latvian", "Arabic", "Sovenian" (Slovenian), "Croatian", "Lithuanian", "Serbian", "BrazilianPortuguese" (Brazilian Portuguese), "Indonesian", "Ukrainian", "EURSpanish"-->SimChinese
  </language>
</DeviceLanguage>
```

Response Message

```xml
<?xml version="1.0" encoding="UTF-8"?>

<ResponseStatus xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, response message, attr:version{ro, req, string, protocolVersion}-->
  <requestURL>
    <!--ro, req, string, request URL-->null
  </requestURL>
  <statusCode>
    <!--ro, req, enum, status code, subType:int, desc:0 (OK), 1 (OK), 2 (Device Busy), 3 (Device Error), 4 (Invalid Operation), 5 (Invalid XML Format), 6 (Invalid XML Content), 7 (Reboot Required)-->0
  </statusCode>
  <statusString>
    <!--ro, req, enum, status information, subType:string, desc:"OK" (succeeded), "Device Busy", "Device Error", "Invalid Operation", "Invalid XML Format", "Invalid XML Content", "Reboot" (reboot device)-->OK
  </statusString>
  <subStatusCode>
    <!--ro, req, string, sub status code, which describes the error in details, desc:sub status code, which describes the error in details-->OK
  </subStatusCode>
</ResponseStatus>
```

10.1.3.5 Get the capability of configuring the device language

Request URL

GET /ISAPI/System/DeviceLanguage/capabilities

Query Parameter

None

Request Message

None

Response Message

```xml
<?xml version="1.0" encoding="UTF-8"?>

<DeviceLanguage xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, opt, object, device language configuration, attr:version{req, string, protocolVersion}-->
  <language opt="SimChinese,TraChinese,English,Russian,Bulgarian,Hungarian,Greek,German,Italian,Czech,Slovakia,French,Polish,Dutch,Portuguese,Spanish,Romanian,Turkish,Japanese,Danish,Swedish,Norwegian,Finnish,Korean,Thai,Estonia,Vietnamese,Hebrew,Latvian,Arabic,Sovenian,Croatian,Lithuanian,Serbian,BrazilianPortuguese,Indonesian,Ukrainian,EURSpanish,Uzbek,Kazak,Kirghiz,Farsi,Azerbaidzhan,Burmese,Mongolian,Anglicism,Estonian">
    <!--ro, req, enum, language, subType:string, attr:opt{req, string}, desc:"SimChinese" (simplified Chinese), "TraChinese" (traditional Chinese), "English", "Russian", "Bulgarian", "Hungarian", "Greek", "German", "Italian", "Czech", "Slovakia", "French", "Polish", "Dutch", "Portuguese", "Spanish", "Romanian", "Turkish", "Japanese", "Danish", "Swedish", "Norwegian", "Finnish", "Korean", "Thai", "Estonia", "Vietnamese", "Hebrew", "Latvian", "Arabic", "Sovenian"-Slovenian, "Croatian", "Lithuanian", "Serbian", "BrazilianPortuguese"-Brazilian Portuguese, "Indonesian", "Ukrainian", "EURSpanish", "Sovenian", "Uzbek", "Kazak", "Kirghiz", "Farsi", "Azerbaidzhan", "Burmese", "Mongolian"-->SimChinese
  </language>
  <upgradeFirmWareEnabled>
    <!--ro, opt, bool, whether to enable upgrading the firmware-->true
  </upgradeFirmWareEnabled>
</DeviceLanguage>
```

10.1.3.6 Get the network service capability

Request URL

GET /ISAPI/System/Network/capabilities

Query Parameter

None

Request Message

None

Response Message

```xml
<?xml version="1.0" encoding="UTF-8"?>
<NetworkCap xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, get the network service capability, attr:version{opt, string, protocolVersion}-->
  <isSupportWireless>
    <!--ro, req, bool, whether the device supports wireless network-->true
  </isSupportWireless>
  <isSupportNtp>
    <!--ro, opt, bool, whether the device supports NTP (Network Time Protocol)-->true
  </isSupportNtp>
  <isSupportSSH>
    <!--ro, opt, bool, whether the device supports SSH-->true
  </isSupportSSH>
  <isSupportEhome>
    <!--ro, opt, bool, whether the device supports EHome (ISUP) protocol-->true
  </isSupportEhome>
  <isSupportWirelessServer>
    <!--ro, opt, bool, whether the device supports wireless server-->true
  </isSupportWirelessServer>
</NetworkCap>
```

10.1.3.7 Set SSH parameters

Request URL

PUT /ISAPI/System/Network/ssh?readerID=<readerID>&type=<type>&SOCChipID=<SOCChipID>

Query Parameter

Parameter Name	Parameter Type	Description
readerID	string	--
type	enum	--
SOCChipID	string	--

Request Message

```xml
<?xml version="1.0" encoding="UTF-8"?>

<SSH xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--wo, opt, object, attr:version{opt, string, protocolVersion}-->
  <enabled>
    <!--wo, req, bool, whether to enable the function-->true
  </enabled>
  <port>
    <!--wo, opt, int-->22
  </port>
</SSH>
```

Response Message

```xml
<?xml version="1.0" encoding="UTF-8"?>

<ResponseStatus xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, response message, attr:version{ro, req, string, protocolVersion}-->
  <requestURL>
    <!--ro, req, string, request URL-->null
  </requestURL>
  <statusCode>
    <!--ro, req, enum, status code, subType:int, desc:0 (OK), 1 (OK), 2 (Device Busy), 3 (Device Error), 4 (Invalid Operation), 5 (Invalid XML Format), 6 (Invalid XML Content), 7 (Reboot Required)-->0
  </statusCode>
  <statusString>
    <!--ro, req, enum, read-only,status description: OK,Device Busy,Device Error,Invalid Operation,Invalid XML Format,Invalid XML Content,Reboot,Additional Error, subType:string, desc:"OK" (succeeded), "Device Busy", "Device Error", "Invalid Operation", "Invalid XML Format", "Invalid XML Content", "Reboot" (reboot device)-->OK
  </statusString>
  <subStatusCode>
    <!--ro, req, string, sub status code, which describes the error in details, desc:sub status code, which describes the error in details-->OK
  </subStatusCode>
</ResponseStatus>
```

10.1.3.8 Reboot device

Request URL

PUT /ISAPI/System/reboot?childDevID=<devIndex>&module=<module>&loginPassword=<loginPassword>

Query Parameter

Parameter Name	Parameter Type	Description
devIndex	string	--
module	enum	--
loginPassword	string	--

Request Message

None

Response Message

```xml
<?xml version="1.0" encoding="UTF-8"?>

<ResponseStatus xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, response message, attr:version{ro, req, string, protocolVersion}-->
  <requestURL>
    <!--ro, req, string, request URL-->null
  </requestURL>
  <statusCode>
    <!--ro, req, enum, status code, subType:int, desc:0 (OK), 1 (OK), 2 (Device Busy), 3 (Device Error), 4 (Invalid Operation), 5 (Invalid XML Format), 6 (Invalid XML Content), 7 (Reboot Required)-->0
  </statusCode>
  <statusString>
    <!--ro, req, enum, status description, subType:string, desc:“OK” (succeeded), “Device Busy”, “Device Error”, “Invalid Operation”, “Invalid XML Format”, “Invalid XML Content”, “Reboot” (reboot device)-->OK
  </statusString>
  <subStatusCode>
    <!--ro, req, string, sub status code, desc:sub status code-->OK
  </subStatusCode>
</ResponseStatus>
```


10.1.4 System Upgrade

10.1.4.1 Get the capabilities of peripheral module upgrade

Request URL

GET /ISAPI/System/AcsUpdate/capabilities

Query Parameter

None

Request Message

None

Response Message

```
<?xml version="1.0" encoding="UTF-8"?>

<AcsUpdate xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, capabilities of peripheral module upgrade, attr:version{req, string, protocolVersion}-->
  <type opt="cardReader,FPModule,securityModule,extendModule,channelController,IRModule,lampModule,elevatorController,FPAlgorithmProgram,uboot,keypad,wirelessRecv,wiredZone,sirenIndoor,sirenOutdoor,sirenAudio,repeater,bluetoothModule,single,wallSwitch,smartPlug,detector,transmitter,remoteCtrl,subModule,dispModule,netReader,lockControlBoard,networkZoneModule,userInterfaceBoard,subPermissionController,electricLock,heatingModule,RS485Module,QRCodeModule,R3WirelessRecv,RXWirelessRecv,wiredOutput,electricGenie,zigbeeModule,PMRModule,networkInputAndOutputModule,infoModule,keypadAndCardReader,socialSecurityCardModule,localController">
    <!--ro, opt, enum, upgrade type: upgrade peripheral, subType:string, attr:opt{req, string}, desc:"cardReader" (485 card reader), "FPModule" (fingerprint module), "securityModule" (security module), "extendModule" (I/O extended module), "channelController" (lane controller), "IRModule" (infrared module), "lampModule" (indicator module), "elevatorController" (sub elevator controller), "FPAlgorithmProgram" (fingerprint algorithm programma of card reader), "uboot" (uboot upgrade), "keypad" (keypad), "wirelessRecv" (wireless receiver module), "wiredZone" (wired zone module), "sirenIndoor" (indoor sounder), "sirenOutdoor" (outdoor sounder), "sirenAudio" (sounder two-way audio), "repeater" (repeater), "bluetoothModule" (bluetooth module), "single" (single output module), "wallSwitch" (wall mounted switch), "smartPlug" (smart plug), "detector" (detector), "transmitter" (transmitter peripheral), "remoteCtrl" (keyfob), "subModule" (sub module), "dispModule" (display module), "netReader" (network card reader), "faceModule" (face module), "touchScreenModule" (touch screen module), "temperatureModule" (temperature measurement module), "lockControlBoard" (lock control board), "networkZoneModule" (network zone module), "userInterfaceBoard" (interface board), "subPermissionController" (sub permission controller), "electricLock" (electric lock), "heatingModule" (heating module), "RS485Module" (RS-485 module), "QRCodeModule" (QR code module), "electricGenie" (electric Genie), "zigbeeModule" (zigbee module), "PMRModule" (PMR module), "networkInputAndOutputModule" (network input and output module via network TAP)-->cardReader
  </type>
  <batchType opt="electricGenie,wiredOutput,networkZoneModule,networkInputAndOutputModule">
    <!--ro, opt, enum, batch upgrade type (peripheral type), subType:string, attr:opt{req, string},
desc:peripheral types which support batch upgrading;
if this node is supported, the following operations are supported: 1. Upgrade devices: /ISAPI/System/updateFirmware?type=<type>; 2. Start upgrading the specified analysis unit (device of the cluster): /ISAPI/System/upgradeStatus/startUpgrade?format=json; 3. Get the device upgrade progress: /ISAPI/System/upgradeStatus?format=json-->electricGenie
  </batchType>
  <cardReaderNo min="1" max="10" opt="1,4">
    <!--ro, opt, int, card reader No. range, it is valid when the returned value of type consists cardReader, attr:min{opt, int, step:1},max{opt, int, step:1},opt{opt, string}-->1
  </cardReaderNo>
  <FPModuleNo min="1" max="10">
    <!--ro, opt, int, fingerprint module No. range, it is valid when the returned value of type consists FPModule, attr:min{req, int},max{req, int}-->1
  </FPModuleNo>
  <securityModuleNo min="1" max="10">
    <!--ro, opt, int, security module No. range, it is valid when the returned value of type consists securityModule, attr:min{req, int},max{req, int}-->1
  </securityModuleNo>
  <extendModuleNo min="1" max="10">
    <!--ro, opt, int, No. range of I/O extended module, it is valid when the returned value of type consists extendModule, attr:min{req, int},max{req, int}-->1
  </extendModuleNo>
  <channelControllerNo min="1" max="10">
    <!--ro, opt, int, lane controller No. range, it is valid when the returned value of type consists channelController, attr:min{req, int},max{req, int}-->1
  </channelControllerNo>
  <IRModuleNo min="1" max="10">
    <!--ro, opt, int, infrared module No. range, it is valid when the returned value of type consists IRModule, attr:min{req, int},max{req, int}-->1
  </IRModuleNo>
  <lampModuleNo min="1" max="10">
    <!--ro, opt, int, indicator module No. range, it is valid when the returned value of type consists lampModule, attr:min{req, int},max{req, int}-->1
  </lampModuleNo>
  <elevatorControllerNo min="1" max="10">
    <!--ro, opt, int, sub elevator controller No. range, it is valid when the returned value of type consists elevatorController, attr:min{req, int},max{req, int}-->1
  </elevatorControllerNo>
  <FPAlgorithmProgramNo min="1" max="10">
    <!--ro, opt, int, No. range of fingerprint algorithm programma of card reader, it is valid when the returned value of type consists FPAlgorithmProgram, attr:min{req, int},max{req, int}-->1
  </FPAlgorithmProgramNo>
  <faceModuleNo min="1" max="10">
    <!--ro, opt, int, face module No. range, it is valid when the returned value of type consists faceModule, attr:min{req, int},max{req, int}-->1
  </faceModuleNo>
  <touchScreenModuleNo min="1" max="10">
    <!--ro, opt, int, touch screen module No. range, it is valid when the returned value of type consists touchScreenModule, attr:min{req, int},max{req, int}-->1
  </touchScreenModuleNo>
  <temperatureModuleNo min="1" max="10">
    <!--ro, opt, int, No. range of temperature measurement module, it is valid when the returned value of type consists temperatureModule, attr:min{req, int},max{req, int}-->1
  </temperatureModuleNo>
  <keypadAddress opt="1,3,5">
    <!--ro, opt, int, keypad address range, it is valid when the returned value of type consists keypad, attr:opt{req, string}-->1
  </keypadAddress>
  <wirelessRecvAddress opt="1,3,5">
    <!--ro, opt, int, No. range of wireless receiver module, it is valid when the returned value of type consists wirelessRecv, attr:opt{req, string}-->1
  </wirelessRecvAddress>
  <wiredZoneAddress opt="1,3,5">
    <!--ro, opt, int, No. range of wired zone module, it is valid when the returned value of type consists wiredZone, attr:opt{req, string}-->1
  </wiredZoneAddress>
  <sirenIndoorNo opt="1,3,5">
    <!--ro, opt, int, indoor sounder No. range, it is valid when the returned value of type consists sirenIndoor, attr:opt{req, string}-->1
  </sirenIndoorNo>
  <sirenOutdoorNo opt="1,3,5">
    <!--ro, opt, int, outdoor sounder No. range, it is valid when the returned value of type consists sirenOutdoor, attr:opt{req, string}-->1
  </sirenOutdoorNo>
  <repeaterNo opt="1,3,5">
    <!--ro, opt, int, repeater No. range, it is valid when the returned value of type consists repeater, attr:opt{req, string}-->1
  </repeaterNo>
```

```
  <subModuleNo opt="1,3,5">
    <!--ro, opt, int, sub module No. range, it is valid when the returned value of type consists subModule, attr:opt{req, string}-->1
  </subModuleNo>
  <dispModuleNo opt="1,3,5">
    <!--ro, opt, int, display module No. range, it is valid when the returned value of type consists dispModule, attr:opt{req, string}-->1
  </dispModuleNo>
  <single opt="1,3,5">
    <!--ro, opt, int, No. range of single output module, it is valid when the returned value of type consists single, attr:opt{req, string}-->1
  </single>
  <wallSwitch opt="1,3,5">
    <!--ro, opt, int, No. range of wall mounted switch, it is valid when the returned value of type consists wallSwitch, attr:opt{req, string}-->1
  </wallSwitch>
  <smartPlug opt="1,3,5">
    <!--ro, opt, int, smart plug No. range, it is valid when the returned value of type consists smartPlug, attr:opt{req, string}-->1
  </smartPlug>
  <zoneNo opt="1,3,5">
    <!--ro, opt, int, zone  No. range, it is valid when the returned value of type consists detector, attr:opt{req, string}, desc:get the detector via the
zone No.-->1
  </zoneNo>
  <transmitterNo opt="1,3,5">
    <!--ro, opt, int, transmitter No. range, it is valid when the returned value of type consists transmitter, attr:opt{req, string}-->1
  </transmitterNo>
  <remoteCtrlNo opt="1,3,5">
    <!--ro, opt, int, keyfob No. range, it is valid when the returned value of type consists remoteCtrl, attr:opt{req, string}-->1
  </remoteCtrlNo>
  <netReaderNo min="1" max="10">
    <!--ro, opt, int, No. range of network card reader, it is valid when the returned value of type consists netReader, attr:min{req, int},max{req, int}-->1
  </netReaderNo>
  <lockControlBoardLNo opt="1,2,3,4,5,6,7,8">
    <!--ro, opt, int, No. range of the left part of the bracket, it is valid when the value of type returns, attr:opt{req, string}-->1
  </lockControlBoardLNo>
  <lockControlBoardRNo opt="1,2,3,4,5,6,7,8">
    <!--ro, opt, int, No. range of the right part of the bracket, it is valid when the value of type returns, attr:opt{req, string}-->1
  </lockControlBoardRNo>
  <networkZoneModuleNo opt="1,3,5">
    <!--ro, opt, int, No. range of network zone module, it is valid when the returned value of type consists networkZoneModule, attr:opt{req, string}-->1
  </networkZoneModuleNo>
  <userInterfaceBoardNo opt="0,1">
    <!--ro, opt, enum, interface board No. range, it is valid when the returned value of type consists userInterfaceBoard, subType:string, attr:opt{req,
string}, desc:0-main user extended interface board, 1-sub user extended interface board-->0
  </userInterfaceBoardNo>
  <electricLockNo opt="1,3,5">
    <!--ro, opt, int, electric lock No. range, it is valid when the returned value of type consists electricLock, attr:opt{req, string}-->1
  </electricLockNo>
  <heatingModuleNo opt="1,3,5">
    <!--ro, opt, int, heating module No. range, it is valid when the returned value of type consists heatingModule, attr:opt{req, string}, desc:the heating
module starts to work to increase the device temperature when the devices runs under low temperature-->1
  </heatingModuleNo>
  <sirenAudioNo opt="1,3,5">
    <!--ro, opt, int, sounder (two-way audio) No. range, it is valid when the returned value of type consists sirenAudio, attr:opt{req, string}-->1
  </sirenAudioNo>
  <QRCodeModuleNo min="1" max="10">
    <!--ro, opt, int, No. range of QR code module, it is valid when the returned value of type consists QRCodeModule, attr:min{req, int},max{req, int}-->1
  </QRCodeModuleNo>
  <R3WirelessRecvNo opt="1,2,3,4">
    <!--ro, opt, int, No. range of R3 wireless receiver module, it is valid when the returned value of type consists R3WirelessRecv, attr:opt{req, string}--
>1
  </R3WirelessRecvNo>
  <RXWirelessRecvNo opt="1,2,3,4">
    <!--ro, opt, int, No. range of RX wireless receiver module, it is valid when the returned value of type consists RXWirelessRecv, attr:opt{req, string}--
>1
  </RXWirelessRecvNo>
  <wiredOutputNo opt="1,2,3,4">
    <!--ro, opt, int, No. range of wired output module , it is valid when the returned value of type consists wiredOutput, attr:opt{req, string}-->1
  </wiredOutputNo>
  <electricGenieNo opt="1,2,3,4">
    <!--ro, opt, int, No. range of electric Genie, it is valid when the returned value of type or batchType consists electricGenie, attr:opt{req, string}--
>1
  </electricGenieNo>
  <networkInputAndOutputModuleNo opt="1,2,3,4">
    <!--ro, opt, int, No. range of network input and output module, it is valid when the returned value of type consists networkInputAndOutputModule,
attr:opt{req, string}-->1
  </networkInputAndOutputModuleNo>
  <infoModuleNo opt="1,2,3,4">
    <!--ro, opt, int, dep:or,{$.AcsUpdate.type,eq,infoModule}, attr:opt{req, string}-->1
  </infoModuleNo>
  <keypadAndCardReaderModuleNo opt="1,2,3,4">
    <!--ro, opt, int, dep:or,{$.AcsUpdate.type,eq,keypadAndCardReaderModule}, attr:opt{req, string}-->1
  </keypadAndCardReaderModuleNo>
  <threeDimensionalStructuredLightModuleNo opt="1,2,3,4">
    <!--ro, opt, int, dep:or,{$.AcsUpdate.type,eq,threeDimensionalStructuredLightModule}, attr:opt{req, string}-->1
  </threeDimensionalStructuredLightModuleNo>
  <socialSecurityCardModuleNo opt="1">
    <!--ro, opt, int, dep:or,{$.AcsUpdate.type,eq,socialSecurityCardModule}, attr:opt{req, string}-->1
  </socialSecurityCardModuleNo>
  <localControllerNo min="1" max="62">
    <!--ro, opt, int, attr:min{req, int},max{req, int}-->1
  </localControllerNo>
</AcsUpdate>
```

10.1.4.2 Upgrade devices

Request URL

POST /ISAPI/System/updateFirmware?type=<type>&moduleAddress=<moduleAddress>&id=<indexID>&childDevID=<devIndex>&isUpdateAuxDevice=<isUpdateAuxDevice>

Query Parameter

Parameter Name	Parameter Type	Description
type	enum	device type 1.alarm device: "keypad" (keypad), "wirelessRecv" (wireless receiver module), "wiredZone" (wired zone module), "sirenIndoor" (indoor sounder), "sirenOutdoor" (outdoor sounder), "sirenAudio" (sounder two-way audio), "repeater" (repeater), "bluetoothModule" (bluetooth module), "single" (single output module), "wallSwitch" (wall mounted switch), "smartPlug" (smart plug), "detector" (detector), "transmitter" (transmitter peripheral), "remoteCtrl" (keyfob), "subModule" (sub module), "dispModule" (display module), "netReader" (network card reader), "faceModule" (face module), "touchScreenModule" (touch screen module), "temperatureModule" (temperature measurement module), "lockControlBoard" (lock control board), "networkZoneModule" (network zone module), "userInterfaceBoard" (interface board), "subPermissionController" (sub permission controller), "electricLock" (electric lock), "heatingModule" (heating module), "RS485Module" (RS-485 module), "QRCodeModule" (QR code module), "electricGenie" (electric Genie), "zigbeeModule" (zigbee module), "PMRModule" (PMR module), "networkInputAndOutputModule" (network input and output module via network TAP) 2. access control device: "cardReader" (485 card reader), "FPModule" (fingerprint module), "securityModule" (security module), "extendModule" (I/O extended module), "channelController" (lane controller), "IRModule" (infrared module), "lampModule" (indicator module), "elevatorController" (sub elevator controller), "FPAlgorithmProgram" (fingerprint algorithm program of card reader), "bluetoothModule" (bluetooth module), "MCU"-MCU upgrade, "temperatureModule" (temperature measurement module), "faceModule" (face module), "touchScreenModule" (touch screen module), "netReader" (network card reader), "userInterfaceBoard" (interface board), "subPermissionController" (sub permission controller), "QRCodeModule" (QR code module) 3. intelligent cabinet: "lockControlBoard" (lock control board) 4. "ledReceiverCard"-receiving card; 5. "subsystem"-sub system; 6. radar-assisted device: "warningScreenFirmware"-firmware upgrade of pre-alarm screen, "warningScreenFont"-font library upgrade of pre-alarm screen, "warningScreenVoice"-audio upgrade of pre-alarm screen; 7. "electricGenie"-electric Genie
moduleAddress	string	module address is an exclusive node for Ax Hybrid example: keypad: GET /ISAPI/SecurityCP/Configuration/outputs?format=json; relay: GET /ISAPI/SecurityCP/Configuration/outputs?format=json; sounder: GET /ISAPI/SecurityCP/Configuration/wirelessSiren?format=json; output module: GET /ISAPI/SecurityCP/Configuration/outputModules?format=json; extension module: GET /ISAPI/SecurityCP/Configuration/extensionModule?format=json; zone: GET /ISAPI/SecurityCP/Configuration/zones?format=json
indexID	string	device No. 1. AX Hybrid Pro and wireless security control upgrade by ID; 2. access control devices upgrade by ID; 3. intelligent cabinet upgrade by IS
devIndex	string	--
isUpdateAuxDevice	string	--

Request Message

None

Response Message

```
<?xml version="1.0" encoding="UTF-8"?>

<ResponseStatus xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, response message, attr:version{ro, req, string, protocolVersion}-->
  <requestURL>
    <!--ro, req, string, request URL, range:[0,1024]-->null
  </requestURL>
  <statusCode>
    <!--ro, req, enum, status code, subType:int, desc:0 (OK), 1 (OK), 2 (Device Busy), 3 (Device Error), 4 (Invalid Operation), 5 (Invalid XML Format), 6
(Invalid XML Content), 7 (Reboot Required)-->0
  </statusCode>
  <statusString>
    <!--ro, req, enum, status information, subType:string, desc:"OK" (succeeded), "Device Busy", "Device Error", "Invalid Operation", "Invalid XML Format",
"Invalid XML Content", "Reboot" (reboot device)-->OK
  </statusString>
  <subStatusCode>
    <!--ro, req, string, sub status code, desc:it describes the error in details-->OK
  </subStatusCode>
  <description>
    <!--ro, opt, string, custom error information, range:[0,1024], desc:this node is used for debugging-->badXmlFormat
  </description>
</ResponseStatus>
```

10.1.4.3 Get the device upgrade progress

Request URL

GET /ISAPI/System/upgradeStatus?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "requestURL":  "/ISAPI/Streaming/channels/1",
    /*ro, opt, string, request URL*/
    "statusCode":  "test",
    /*ro, req, string, status code*/
    "statusString":  "test",
    /*ro, req, string, status description*/
    "subStatusCode":  "test",
    /*ro, req, string, sub status code*/
    "errorCode":  1,
    /*ro, opt, int, This field is required when the value of statusCode is not 1, and it corresponds to subStatusCode.*/
    "errorMsg":  "ok",
    /*ro, opt, string, This field is required when the value of statusCode is not 1. Detailed error description of a certain parameter can be provided*/
    "upgrading":  "TRUE",
    /*ro, opt, string, whether the device is upgrading: "TRUE" (upgrading), "FALSE" (not in upgrading)*/
    "percent":  22,
    /*ro, opt, int, upgrade progress (% complete)*/
    "idList": [
    /*ro, opt, array, ID list, subType:object*/
        {
            "id":  "test",
            /*ro, req, string, analysis unit ID*/
            "percent":  22,
            /*ro, opt, int, upgrade progress (% complete)*/
            "status":  "test"
            /*ro, opt, string, "backingUp" (backing up upgrade)*/
        }
    ]
}
```


10.1.4.4 Get the device upgrading status and progress

Request URL

GET /ISAPI/System/upgradeStatus?type=<Type>&childDevID=<devIndex>

Query Parameter

Parameter Name	Parameter Type	Description
Type	string	--
devIndex	string	--

Request Message

None

Response Message

```
<?xml version="1.0" encoding="UTF-8"?>

<upgradeStatus xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, upgrade status and result, attr:version{req, string, protocolVersion}-->
  <upgrading>
    <!--ro, req, bool, upgrade status-->true
  </upgrading>
  <percent>
    <!--ro, req, int, upgrade progress (% complete), range:[0,100]-->1
  </percent>
</upgradeStatus>
```

10.1.5 Time Settings

10.1.5.1 Get device time synchronization management parameters

Request URL

GET /ISAPI/System/time

Query Parameter

None

Request Message

None

Response Message

```
<?xml version="1.0" encoding="UTF-8"?>

<Time xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, opt, object, time management, attr:version{opt, string, protocolVersion}-->
  <timeMode>
    <!--ro, req, enum, time synchronization mode, subType:string, desc:"NTP" (NTP time synchronization), "manual" (manual time synchronization), "satellite"
(satellite time synchronization), "platform" (platform time synchronization), "NONE" (time synchronization is not allowed or no time synchronization
source), "GB28181" (GB28181 time synchronization)-->NTP
  </timeMode>
  <localTime>
    <!--ro, opt, string, local time, range:[0,256], dep:and,{$.Time.timeMode,eq,manual}-->2019-02-28T10:50:44+08:00
  </localTime>
  <timeZone>
    <!--ro, opt, string, time zone, range:[0,256], dep:and,{$.Time.timeMode,eq,manual},{$.Time.timeMode,eq,NTP}-->CST-
8:00:00DST00:30:00,M4.1.0/02:00:00,M10.5.0/02:00:00
  </timeZone>
  <satelliteInterval>
    <!--ro, opt, int, satellite time synchronization interval, step:1, unit:min, desc:unit: minute-->60
  </satelliteInterval>
  <isSummerTime>
    <!--ro, opt, bool, whether the device time returned currently is in DST (Daylight Saving Time) system-->true
  </isSummerTime>
  <platformType>
    <!--ro, opt, enum, platform type, subType:string, dep:and,{$.Time.timeMode,eq,platform}, desc:exists only when the timeMode is selected as platform--
>EZVIZ
  </platformType>
  <platformNo>
    <!--ro, opt, int, platform No., range:[1,2], dep:and,{$.Time.timeMode,eq,GB28181}, desc:it is the only ID, which is configured via platformNo in
GB28181List, related URI: /ISAPI/System/Network/SIP/<SIPServerID>-->1
  </platformNo>
</Time>
```

10.1.5.2 Set device time synchronization management parameters

Request URL

PUT /ISAPI/System/time

Query Parameter

None

Request Message

```xml
<?xml version="1.0" encoding="UTF-8"?>

<Time xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--req, object, time management, attr:version{opt, string, protocolVersion}-->
  <timeMode>
    <!--req, enum, time synchronization mode, subType:string, desc:“NTP” (NTP time synchronization), “manual” (manual time synchronization), “satellite” (satellite time synchronization), “platform” (platform time synchronization), “NONE” (time synchronization is not allowed or no time synchronization source), “GB28181” (GB28181 time synchronization)-->NTP
  </timeMode>
  <localTime>
    <!--opt, string, local time, range:[0,256], dep:and,{$.Time.timeMode,eq,manual}-->2019-02-28T10:50:44+08:00
  </localTime>
  <timeZone>
    <!--opt, string, time zone, range:[0,256], dep:and,{$.Time.timeMode,eq,manual},{$.Time.timeMode,eq,NTP}-->CST-8:00:00DST00:30:00,M4.1.0/02:00:00,M10.5.0/02:00:00
  </timeZone>
  <satelliteInterval>
    <!--opt, int, satellite time synchronization interval, step:1, unit:min, desc:unit: minute-->60
  </satelliteInterval>
  <isSummerTime>
    <!--opt, bool, whether the time returned by the current device is that in the DST (daylight saving time)-->true
  </isSummerTime>
  <platformType>
    <!--opt, enum, platform type, subType:string, dep:and,{$.Time.timeMode,eq,platform}, desc:exists only when the timeMode is selected as platform, related URI: /ISAPI/System/Network/EZVIZ-->EZVIZ
  </platformType>
  <platformNo>
    <!--opt, int, platform No., range:[1,2], dep:and,{$.Time.timeMode,eq,GB28181}, desc:it is the only ID, which is configured via platformNo in GB28181List, related URI: /ISAPI/System/Network/SIP/<SIPServerID>-->1
  </platformNo>
</Time>
```

Response Message

```xml
<?xml version="1.0" encoding="UTF-8"?>

<ResponseStatus xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, response message, attr:version{ro, req, string, protocolVersion}-->
  <requestURL>
    <!--ro, req, string, request URL-->null
  </requestURL>
  <statusCode>
    <!--ro, req, enum, status code, subType:int, desc:0 (OK), 1 (OK), 2 (Device Busy), 3 (Device Error), 4 (Invalid Operation), 5 (Invalid XML Format), 6 (Invalid XML Content), 7 (Reboot Required)-->0
  </statusCode>
  <statusString>
    <!--ro, req, enum, status description, subType:string, desc:“OK” (succeeded), “Device Busy”, “Device Error”, “Invalid Operation”, “Invalid XML Format”, “Invalid XML Content”, “Reboot” (reboot device)-->OK
  </statusString>
  <subStatusCode>
    <!--ro, req, string, sub status code, desc:sub status code-->OK
  </subStatusCode>
  <FailedNodeInfoList>
    <!--ro, opt, object, information list of failed nodes, desc:for the manual time synchronization of central analysis cluster, this field is returned if time synchronization failed-->
    <FailedNodeInfo>
      <!--ro, opt, object, information of failed nodes-->
      <nodeID>
        <!--ro, req, string, node ID, range:[0,64]-->test
      </nodeID>
      <nodeIP>
        <!--ro, req, string, node IP, range:[0,20]-->test
      </nodeIP>
      <reason>
        <!--ro, opt, string, reason why the node failed to synchronize time, range:[0,128]-->test
      </reason>
    </FailedNodeInfo>
  </FailedNodeInfoList>
</ResponseStatus>
```

10.1.5.3 Get the capability of device time synchronization management

Request URL

GET /ISAPI/System/time/capabilities

Query Parameter

None

Request Message

None

Response Message


```xml
<?xml version="1.0" encoding="UTF-8"?>

<Time xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, opt, object, time management capability set, attr:version{opt, string, protocolVersion}-->
  <timeMode opt="NTP,manual,satellite,SDK,28181,ONVIF,ALL(任何支持的校时方式都允许校时),NONE(不允校时或无校时源),platform">
    <!--ro, req, enum, time synchronization mode, subType:string, attr:opt{opt, string}, desc:“NTP” (NTP time synchronization), “manual” (manual time synchronization), “satellite” (satellite time synchronization), “platform” (platform time synchronization), “NONE” (time synchronization is not allowed or no time synchronization source), “GB28181” (GB28181 time synchronization)-->NTP
  </timeMode>
  <localTime min="0" max="256">
    <!--ro, opt, string, local time, range:[0,256], attr:min{opt, string},max{opt, string}-->test
  </localTime>
  <timeZone min="0" max="256">
    <!--ro, opt, string, time zone, range:[0,256], attr:min{opt, string},max{opt, string}-->test
  </timeZone>
  <satelliteInterval min="0" max="3600">
    <!--ro, opt, int, satellite time synchronization interval, step:1, unit:min, attr:min{opt, string},max{opt, string}, desc:unit: minute-->60
  </satelliteInterval>
  <timeType opt="local,UTC">
    <!--ro, opt, enum, time type, subType:string, attr:opt{opt, string}, desc:“local” (local time), “UTC” (UTC time)-->local
  </timeType>
  <platformType opt="EZVIZ">
    <!--ro, opt, enum, platform type, subType:string, dep:and,{$.Time.timeMode,eq,platform}, attr:opt{opt, string}, desc:platform type-->EZVIZ
  </platformType>
  <platformNo min="1" max="2">
    <!--ro, opt, int, platform No., range:[1,2], dep:and,{$.Time.timeMode,eq,GB28181}, attr:min{req, int},max{req, int}, desc:it is the only ID, which is configured via platformNo in GB28181List, related URI: /ISAPI/System/Network/SIP/<SIPServerID>-->1
  </platformNo>
  <isSupportHistoryTime>
    <!--ro, opt, bool, supported capability of the historical time synchronization list, desc:related URI: /ISAPI/System/time/historyInfo?format=json-->true
  </isSupportHistoryTime>
  <isSupportTimeFilter>
    <!--ro, opt, bool, supported capability of filtering time synchronization, desc:related URI: /ISAPI/System/time/filter/capabilities?format=json-->true
  </isSupportTimeFilter>
  <displayFormat opt="MM/dd/yyyy hh:mm,mm,dd-MM-yyyy hh,MM-dd-yyyy hh:mm,yyyy-MM-dd hh:mm">
    <!--ro, opt, enum, time display format, subType:string, attr:opt{req, string}, desc:if this node is returned, it indicates that the device supports configuring time display format, related URI: /ISAPI/System/time/timeType?format=json-->MM/dd/yyyy hh:mm
  </displayFormat>
  <isSupportSyncDeviceNTPInfoToCamera>
    <!--ro, opt, bool, the capability of synchronizing device's NTP service information with the camera, desc:related URI: /ISAPI/System/time/SyncDeviceNTPInfoToCamera/capabilities?format=json-->true
  </isSupportSyncDeviceNTPInfoToCamera>
  <isSupportNTPService>
    <!--ro, opt, bool-->true
  </isSupportNTPService>
</Time>
```


10.1.5.4 Set device local time

Request URL

PUT /ISAPI/System/time/localTime

Query Parameter

None

Request Message

None

Response Message

```xml
<?xml version="1.0" encoding="UTF-8"?>

<ResponseStatus xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, response message, attr:version{ro, req, string, protocolVersion}-->
  <requestURL>
    <!--ro, req, string, request URL-->null
  </requestURL>
  <statusCode>
    <!--ro, req, enum, status code, subType:int, desc:0 (OK), 1 (OK), 2 (Device Busy), 3 (Device Error), 4 (Invalid Operation), 5 (Invalid XML Format), 6 
(Invalid XML Content), 7 (Reboot Required)-->0
  </statusCode>
  <statusString>
    <!--ro, req, enum, status information, subType:string, desc:"OK" (succeeded), "Device Busy", "Device Error", "Invalid Operation", "Invalid XML Format", 
"Invalid XML Content", "Reboot" (reboot device)-->OK
  </statusString>
  <subStatusCode>
    <!--ro, req, string, sub status code, desc:sub status code description-->OK
  </subStatusCode>
</ResponseStatus>
```

10.1.5.5 Set parameters of all NTP servers

Request URL

PUT /ISAPI/System/time/ntpServers

Query Parameter

None

Request Message

```xml
<?xml version="1.0" encoding="UTF-8"?>

<NTPServerList xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--req, array, NTP server information list, subType:object, attr:version{opt, string, protocolVersion}-->
  <NTPServer>
    <!--opt, object, NTP server information-->
    <id>
      <!--req, string, ID-->1
    </id>
    <addressingFormatType>
      <!--req, enum, NTP server address type, subType:string, desc:"ipaddress" (IP address), "hostname" (domain name)-->hostname
    </addressingFormatType>
    <hostName>
      <!--opt, string, NTP server domain name, range:[1,64]-->12345
    </hostName>
    <ipAddress>
      <!--opt, string, IPv4 address, range:[1,32]-->192.168.1.112
    </ipAddress>
    <ipv6Address>
      <!--opt, string, IPv6 address, range:[1,128]-->1030:C9B4:FF12:48AA:1A2B
    </ipv6Address>
    <portNo>
      <!--opt, int, port No., range:[1,65535], desc:the default port No. is 123-->123
    </portNo>
    <synchronizeInterval>
      <!--opt, int, time synchronization interval, range:[1,10800], unit:min-->1440
    </synchronizeInterval>
    <enabled>
      <!--opt, bool, whether to enable, desc:disabled (by default)-->false
    </enabled>
  </NTPServer>
</NTPServerList>
```


Response Message

```xml
<?xml version="1.0" encoding="UTF-8"?>

<ResponseStatus xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, response message, attr:version{ro, req, string, protocolVersion}-->
  <requestURL>
    <!--ro, req, string, request URL-->/ISAPI/xxxx
  </requestURL>
  <statusCode>
    <!--ro, req, enum, status code, subType:int, desc:0 (OK), 1 (OK), 2 (Device Busy), 3 (Device Error), 4 (Invalid Operation), 5 (Invalid XML Format), 6
(Invalid XML Content), 7 (Reboot Required)-->0
  </statusCode>
  <statusString>
    <!--ro, req, enum, status description, subType:string, desc:"OK" (succeeded), "Device Busy", "Device Error", "Invalid Operation", "Invalid XML Format",
"Invalid XML Content", "Reboot" (reboot device)-->OK
  </statusString>
  <subStatusCode>
    <!--ro, req, string, sub status code, desc:sub status code-->OK
  </subStatusCode>
</ResponseStatus>
```

10.1.5.6 Get the parameters of a specific NTP (Network Time Protocol) server

Request URL

GET /ISAPI/System/time/ntpServers

Query Parameter

None

Request Message

None

Response Message

```xml
<?xml version="1.0" encoding="UTF-8"?>

<NTPServerList xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, array, NTP server information list, subType:object, attr:version{req, string, protocolVersion}-->
  <NTPServer>
    <!--ro, opt, object, NTP server information-->
    <id>
      <!--ro, req, string, ID-->1
    </id>
    <addressingFormatType>
      <!--ro, req, enum, NTP server address type, subType:string, desc:"ipaddress" (IP address), "hostname" (domain name)-->hostname
    </addressingFormatType>
    <hostName>
      <!--ro, opt, string, NTP server domain name, range:[1,64]-->12345
    </hostName>
    <ipAddress>
      <!--ro, opt, string, IPv4 address, range:[1,32]-->192.168.1.112
    </ipAddress>
    <ipv6Address>
      <!--ro, opt, string, IPv6 address, range:[1,128]-->1030:C9B4:FF12:48AA:1A2B
    </ipv6Address>
    <portNo>
      <!--ro, opt, int, port No., range:[1,65535], desc:the default port No. is 123-->123
    </portNo>
    <synchronizeInterval>
      <!--ro, opt, int, time synchronization interval, range:[1,10800], unit:min-->1440
    </synchronizeInterval>
    <enabled>
      <!--ro, opt, bool, whether to enable, desc:disabled (by default)-->false
    </enabled>
  </NTPServer>
</NTPServerList>
```

10.1.5.7 Set the parameters of a NTP server

Request URL

PUT /ISAPI/System/time/ntpServers/<NTPServerID>

Query Parameter

Parameter Name	Parameter Type	Description
NTPServerID	string	--

Request Message

```xml
<?xml version="1.0" encoding="UTF-8"?>

<NTPServer xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--req, object, NTP server information, attr:version{req, string, protocolVersion}-->
  <id>
    <!--req, string, ID-->1
  </id>
  <addressingFormatType>
    <!--req, enum, IP address type of NTP server, subType:string, desc:"ipaddress" (IP address), "hostname" (domain name)-->hostname
  </addressingFormatType>
  <hostName>
    <!--opt, string, NTP server domain name地址去掉, range:[1,64]-->12345
  </hostName>
  <ipAddress>
    <!--opt, string, IPv4 address, range:[1,32], desc:IPv4 address-->192.168.1.112
  </ipAddress>
  <ipv6Address>
    <!--opt, string, IPv6 address, range:[1,128], desc:IPv6 address-->1030:C9B4:FF12:48AA:1A2B
  </ipv6Address>
  <portNo>
    <!--opt, int, port No., range:[1,65535], step:1, desc:port No.-->1
  </portNo>
  <synchronizeInterval>
    <!--opt, int, time synchronization interval, range:[1,10800], step:1, unit:min, desc:NTP time synchronization interval, unit: minute-->1440
  </synchronizeInterval>
  <enabled>
    <!--opt, bool-->false
  </enabled>
</NTPServer>
```

Response Message

```xml
<?xml version="1.0" encoding="UTF-8"?>

<ResponseStatus xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, response message, attr:version{ro, req, string, protocolVersion}-->
  <requestURL>
    <!--ro, req, string, request URL-->null
  </requestURL>
  <statusCode>
    <!--ro, req, enum, status code, subType:int, desc:0 (OK), 1 (OK), 2 (Device Busy), 3 (Device Error), 4 (Invalid Operation), 5 (Invalid XML Format), 6 
(Invalid XML Content), 7 (Reboot Required)-->0
  </statusCode>
  <statusString>
    <!--ro, req, enum, status description, subType:string, desc:"OK" (succeeded), "Device Busy", "Device Error", "Invalid Operation", "Invalid XML Format", 
"Invalid XML Content", "Reboot" (reboot device)-->OK
  </statusString>
  <subStatusCode>
    <!--ro, req, string, sub status code, desc:sub status code-->OK
  </subStatusCode>
</ResponseStatus>
```


10.1.5.8 Get the parameters of a NTP server

Request URL

GET /ISAPI/System/time/ntpServers/<NTPServerID>

Query Parameter

Parameter Name	Parameter Type	Description
NTPServerID	string	--

Request Message

None

Response Message

```xml
<?xml version="1.0" encoding="UTF-8"?>

<NTPServer xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, NTP server information, attr:version{req, string, protocolVersion}-->
  <id>
    <!--ro, req, string, ID-->1
  </id>
  <addressingFormatType>
    <!--ro, req, enum, IP address type of NTP server, subType:string, desc:"ipaddress" (IP address), "hostname" (domain name)-->hostname
  </addressingFormatType>
  <hostName>
    <!--ro, opt, string, NTP server domain name, range:[1,64]-->12345
  </hostName>
  <ipAddress>
    <!--ro, opt, string, IPv4 address, range:[1,32], desc:IPv4 address-->192.168.1.112
  </ipAddress>
  <ipv6Address>
    <!--ro, opt, string, IPv6 address, range:[1,128], desc:IPv6 address-->1030:C9B4:FF12:48AA:1A2B
  </ipv6Address>
  <portNo>
    <!--ro, opt, int, port No., range:[1,65535], step:1, desc:port No.-->1
  </portNo>
  <synchronizeInterval>
    <!--ro, opt, int, time synchronization interval, range:[1,10800], step:1, unit:min, desc:NTP time synchronization interval, unit: minute-->1440
  </synchronizeInterval>
  <enabled>
    <!--ro, opt, bool-->false
  </enabled>
</NTPServer>
```

10.1.5.9 Get time zone

Request URL

GET /ISAPI/System/time/timeZone

Query Parameter

None

Request Message

None

Response Message

None

10.1.5.10 Set time zone

Request URL

PUT /ISAPI/System/time/timeZone

Query Parameter

None

Request Message

None

Response Message

```xml
<?xml version="1.0" encoding="UTF-8"?>

<ResponseStatus xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, response message, attr:version{ro, req, string, protocolVersion}-->
  <requestURL>
    <!--ro, req, string, request URL-->null
  </requestURL>
  <statusCode>
    <!--ro, req, enum, status code, subType:int, desc:0 (OK), 1 (OK), 2 (Device Busy), 3 (Device Error), 4 (Invalid Operation), 5 (Invalid XML Format), 6 
(Invalid XML Content), 7 (Reboot Required)-->0
  </statusCode>
  <statusString>
    <!--ro, req, enum, status description, subType:string, desc:"OK" (succeeded), "Device Busy", "Device Error", "Invalid Operation", "Invalid XML Format", 
"Invalid XML Content", "Reboot" (reboot device)-->OK
  </statusString>
  <subStatusCode>
    <!--ro, req, string, sub status code, desc:sub status code-->OK
  </subStatusCode>
</ResponseStatus>
```

10.2 Network Settings

10.2.1 HTTP Listening

10.2.1.1 Delete receiving server(s)

Request URL

DELETE /ISAPI/Event/notification/httpHosts

Query Parameter

None

Request Message

None

Response Message


```xml
<?xml version="1.0" encoding="UTF-8"?>

<ResponseStatus xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, response message, attr:version{ro, req, string, protocolVersion}-->
  <requestURL>
    <!--ro, req, string, request URL-->null
  </requestURL>
  <statusCode>
    <!--ro, req, enum, status code, subType:int, desc:0 (OK), 1 (OK), 2 (Device Busy), 3 (Device Error), 4 (Invalid Operation), 5 (Invalid XML Format), 6 (Invalid XML Content), 7 (Reboot Required)-->0
  </statusCode>
  <statusString>
    <!--ro, req, enum, status information, subType:string, desc:"OK" (succeeded), "Device Busy", "Device Error", "Invalid Operation", "Invalid XML Format", "Invalid XML Content", "Reboot" (reboot device)-->OK
  </statusString>
  <subStatusCode>
    <!--ro, req, string, sub status code, which describes the error in details, desc:sub status code, which describes the error in details-->OK
  </subStatusCode>
</ResponseStatus>
```


10.2.1.2 Delete a HTTP listening server

Request URL

DELETE /ISAPI/Event/notification/httpHosts/<hostID>

Query Parameter

Parameter Name	Parameter Type	Description
hostID	string	--

Request Message

None

Response Message


```xml
<?xml version="1.0" encoding="UTF-8"?>

<ResponseStatus xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, response message, attr:version{ro, req, string, protocolVersion}-->
  <requestURL>
    <!--ro, req, string, request URL-->null
  </requestURL>
  <statusCode>
    <!--ro, req, enum, status code, subType:int, desc:0-OK, 1-OK, 2-Device Busy, 3-Device Error, 4-Invalid Operation, 5-Invalid XML Format, 6-Invalid XML Content, 7-Reboot Required-->0
  </statusCode>
  <statusString>
    <!--ro, req, enum, status description, subType:string, desc:“OK” (succeeded), “Device Busy”, “Device Error”, “Invalid Operation”, “Invalid XML Format”, “Invalid XML Content”, “Reboot” (reboot device)-->OK
  </statusString>
  <subStatusCode>
    <!--ro, req, string, sub status code, desc:sub status code-->OK
  </subStatusCode>
</ResponseStatus>
```

10.2.1.3 Set the parameters of a listening host

Request URL

PUT /ISAPI/Event/notification/httpHosts/<hostID>

Query Parameter

Parameter Name	Parameter Type	Description
hostID	string	--

Request Message

```xml
<?xml version="1.0" encoding="UTF-8"?>
<HttpHostNotification xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--req, object, listening host, attr:version{req, string, protocolVersion}-->
  <id>
    <!--req, string, listening host ID, range:[1,10]-->test
  </id>
  <url>
    <!--req, string, URL-->test
  </url>
  <protocolType>
    <!--req, enum, protocol type, subType:string, desc:"HTTP", "HTTPS", "EHome" (ISUP)-->HTTP
  </protocolType>
  <parameterFormatType>
    <!--req, enum, parameter format type, subType:string, desc:"JSON", "XML"-->JSON
  </parameterFormatType>
  <addressingFormatType>
    <!--req, enum, address type, subType:string, desc:"hostname", "ipaddress"-->hostname
  </addressingFormatType>
  <ipAddress>
    <!--opt, string, IP address, dep:or,{$.HttpHostNotification.addressingFormatType,eq,ipAddress}, desc:IP address-->test
  </ipAddress>
  <portNo>
    <!--opt, int, port number-->1
  </portNo>
  <httpAuthenticationMethod>
    <!--req, enum, authentication method, subType:string, desc:"MD5digest"(MD5), "none", "base64"-->MD5digest
  </httpAuthenticationMethod>
</HttpHostNotification>
```

Response Message

```xml
<?xml version="1.0" encoding="UTF-8"?>

<ResponseStatus xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, response message, attr:version{ro, req, string, protocolVersion}-->
  <requestURL>
    <!--ro, req, string, request URL, range:[0,1024]-->null
  </requestURL>
  <statusCode>
    <!--ro, req, enum, status code, subType:int, desc:0 (OK), 1 (OK), 2 (Device Busy), 3 (Device Error), 4 (Invalid Operation), 5 (Invalid XML Format), 6 
(Invalid XML Content), 7 (Reboot Required)-->0
  </statusCode>
  <statusString>
    <!--ro, req, enum, status description, subType:string, desc:"OK"(succeeded), "Device Busy", "Device Error", "Invalid Operation", "Invalid XML Format", 
"Invalid XML Content", "Reboot"(reboot device)-->OK
  </statusString>
  <subStatusCode>
    <!--ro, req, string, sub status code, desc:sub status code-->OK
  </subStatusCode>
  <description>
    <!--ro, opt, string, range:[0,1024]-->badXmlFormat
  </description>
</ResponseStatus>
```


10.2.1.4 Get the parameters of a listening host

Request URL

GET /ISAPI/Event/notification/httpHosts/<hostID>

Query Parameter

Parameter Name	Parameter Type	Description
hostID	string	--

Request Message

None

Response Message

```
<?xml version="1.0" encoding="UTF-8"?>

<HttpHostNotification xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, listening host, attr:version{req, string, protocolVersion}-->
  <id>
    <!--ro, req, string, listening host ID, range:[1,10]-->test
  </id>
  <url>
    <!--ro, req, string, URL-->test
  </url>
  <protocolType>
    <!--ro, req, enum, protocol type, subType:string, desc:"HTTP", "HTTPS", "EHome"-->HTTP
  </protocolType>
  <parameterFormatType>
    <!--ro, req, enum, parameter format type, subType:string, desc:"JSON", "XML"-->JSON
  </parameterFormatType>
  <addressingFormatType>
    <!--ro, req, enum, address type, subType:string, desc:"hostname", "ipaddress"-->hostname
  </addressingFormatType>
  <hostName>
    <!--ro, opt, string, host name, dep:and,{$.HttpHostNotification.addressingFormatType,eq,hostName}-->test
  </hostName>
  <ipAddress>
    <!--ro, opt, string, IP address, dep:or,{$.HttpHostNotification.addressingFormatType,eq,ipAddress}, desc:IP address-->test
  </ipAddress>
  <ipv6Address>
    <!--ro, opt, string, IPv6 address, dep:or,{$.HttpHostNotification.addressingFormatType,eq,ipAddress}, desc:IPv6 address-->test
  </ipv6Address>
  <portNo>
    <!--ro, opt, int, port number-->1
  </portNo>
  <userName>
    <!--ro, opt, string, user name, dep:and,{$.HttpHostNotification.httpAuthenticationMethod,eq,MD5digest}-->test
  </userName>
  <httpAuthenticationMethod>
    <!--ro, req, enum, authentication method, subType:string, desc:"MD5digest" (MD5), "none", "base64"-->MD5digest
  </httpAuthenticationMethod>
  <ANPR>
    <!--ro, opt, object, ANPR-->
    <detectionUpLoadPicturesType>
      <!--ro, opt, enum, uploaded pictures type, subType:string, desc:"all", "licensePlatePicture" (license plate picture), "detectionPicture" (detected
picture)-->all
    </detectionUpLoadPicturesType>
  </ANPR>
  <Extensions>
    <!--ro, opt, object, range-->
    <intervalBetweenEvents>
      <!--ro, opt, int, event interval-->1
    </intervalBetweenEvents>
  </Extensions>
  <uploadImagesDataType>
    <!--ro, opt, enum, picture data type, subType:string, desc:"URL", "binary" (default value). For cloud storage, only "URL" is supported-->URL
  </uploadImagesDataType>
  <httpBroken>
    <!--ro, opt, bool, whether to enable the automatic network replenishment, desc:if the ANR function is enabled, it will be applied to all events-->true
  </httpBroken>
  <SubscribeEvent>
    <!--ro, opt, object, picture uploading modes of all events which contain pictures-->
    <heartbeat>
      <!--ro, opt, int, heartbeat interval time-->30
    </heartbeat>
    <eventMode>
      <!--ro, req, enum, event mode, subType:string, desc:"all" (all alarms need to be reported), "list" (only listed alarms need to be reported)-->all
    </eventMode>
    <EventList>
      <!--ro, opt, array, event list, subType:object-->
      <Event>
        <!--ro, opt, object, channel information linked to event-->
        <type>
          <!--ro, req, enum, event type, subType:string, desc:refer to event type list (eventType): "ADAS"(advanced driving assistance system), "ADASAlarm"
(advanced driving assistance alarm), "AID"(traffic incident detection), "ANPR"(automatic number plate recognition), "AccessControllerEvent" (access
controller event), "CDsStatus" (CD burning status), "DBD"(driving behavior detection) "GPSUpload" (GPS information upload), "HFPD"(frequently appeared
person detection), "IO"(I/O alarm), "IOTD" (IoT device detection), "LES" (logistics scanning event), "LFPD"(rarely appeared person detection), "PALMismatch"
(video standard mismatch), "PIR", "PeopleCounting" (people counting), "PeopleNumChange" (people number change detection), "Standup"(standing up detection),
"TMA"(thermometry alarm), "TMPA"(temperature measurement pre-alarm), "VMD"(motion detection), "abnormalAcceleration", "abnormalDriving", "advReachHeight",
"alarmResult", "attendance", "attendedBaggage", "audioAbnormal", "audioexception", "behaviorResult"(abnormal event detection), "blindSpotDetection"(blind
spot detection alarm), "cardMatch", "changedStatus", "collision", "containerDetection", "crowdSituationAnalysis", "databaseException", "defocus"(defocus
detection), "diskUnformat"(disk unformatted), "diskerror", "diskfull", "driverConditionMonitor"(driver status monitoring alarm); "emergencyAlarm",
"faceCapture", "faceSnapModeling", "facedetection", "failDown"(People Falling Down), "faultAlarm", "fielddetection"(intrusion detection), "fireDetection",
"fireEscapeDetection", "flowOverrun", "framesPeopleCounting", "getUp"(getting up detection), "group" (people gathering), "hdBadBlock"(HDD bad sector
detection event), "hdImpact"(HDD impact detection event), "heatmap"(heat map alarm), "highHDTemperature"(HDD high temperature detection event),
"highTempAlarm"(HDD high temperature alarm), "hotSpare"(hot spare exception), "illaccess"(invalid access), "ipcTransferAbnormal", "ipconflict"(IP address
conflicts), "keyPersonGetUp"(key person getting up detection), "leavePosition"(absence detection), "linedetection"(line crossing detection),
"listSyncException"(list synchronization exception), "loitering"(loitering detection), "lowHDTemperature"(HDD low temperature detection event),
"mixedTargetDetection"(multi-target-type detection), "modelError", "nicbroken"(network disconnected), "nodeOffline"(node disconnected),
"nonPoliceIntrusion", "overSpeed"(overspeed alarm), "overtimeTarry"(staying overtime detection), "parking"(parking detection), "peopleNumChange"
```

```
"nonPoliceIntrusion", "overSpeed"(overspeed alarm), "overtimeTarry"(staying overtime detection), "parking"(parking detection), "peopleNumChange", "peopleNumCounting", "personAbnormalAlarm"(person ID exception alarm), "personDensityDetection", "personQueueCounting", "personQueueDetection", "personQueueRealTime"(real-time data of people queuing-up detection), "personQueueTime"(waiting time detection), "playCellphone"(playing mobile phone detection), "pocException"(video exception), "poe"(POE power exception), "policeAbsent", "radarAlarm", "radarFieldDetection", "radarLineDetection", "radarPerimeterRule"(radar rule data), "radarTargetDetection", "radarVideoDetection"(radar-assisted target detection), "raidException", "rapidMove", "reachHeight"(climbing detection), "recordCycleAbnormal"(insufficient recording period), "recordException", "regionEntrance", "regionExiting", "retention"(people overstay detection), "rollover", "running"(people running), "safetyHelmetDetection"(hard hat detection), "scenechangedetection", "sensorAlarm"(angular acceleration alarm), "severeHDFailure"(HDD major fault detection), "shelteralarm"(video tampering alarm), "shipsDetection", "sitQuietly"(sitting detection), "smokeAndFireDetection", "smokeDetection", "softIO", "spacingChange"(distance exception), "sysStorFull"(storaging full alarm of cluster system), "takingElevatorDetection"(elevator electric moped detection), "targetCapture", "temperature"(temperature difference alarm), "thermometry"(temperature alarm), "thirdPartyException", "toiletTarry"(in-toilet overtime detection), "tollCodeInfo"(QR code information report), "tossing"(thrown object detection), "unattendedBaggage", "vehicleMatchResult"(uploading list alarms), "vehicleRcogResult", "versionAbnormal"(cluster version exception), "videoException", "videoloss", "violationAlarm", "violentMotion"(violent motion detection), "yardTarry"(playground overstay detection), "AccessControllerEvent", "IDCardInfoEvent", "FaceTemperatureMeasurementEvent", "QRCodeEvent"(QR code event of access control), "CertificateCaptureEvent"(person ID capture comparison event), "UncertificateCompareEvent", "ConsumptionAndTransactionRecordEvent", "ConsumptionEvent", "TransactionRecordEvent", "SetMealQuery"(searching consumption set meals), "ConsumptionStatusQuery"(searching the consumption status), "humanBodyComparison" (human body comparison), "regionTargetNumberCounting" (regional target statistics)-->mixedTargetDetection
        </type>
        <minorAlarm>
          <!--ro, opt, string, alarm sub type, desc:refer to the macro definition of uploaded events. "IDCardInfoEvent" is required when the type of event is "AccessControllerEvent"-->0x400,0x401,0x402,0x403
        </minorAlarm>
        <minorException>
          <!--ro, opt, string, minor exception type, desc:refer to the macro definition of uploaded events. "IDCardInfoEvent" is required when the type of event is "AccessControllerEvent"-->0x400,0x401,0x402,0x403
        </minorException>
        <minorOperation>
          <!--ro, opt, string, minor operation type, desc:refer to the macro definition of uploaded events. "IDCardInfoEvent" is required when the type of event is "AccessControllerEvent"-->0x400,0x401,0x402,0x403
        </minorOperation>
        <minorEvent>
          <!--ro, opt, string, minor event type, desc:refer to the macro definition of uploaded events. "IDCardInfoEvent" is required when the type of event is "AccessControllerEvent"-->0x01,0x02,0x03,0x04
        </minorEvent>
        <pictureURLType>
          <!--ro, opt, enum, alarm picture format of the specified event, subType:string, desc:"binary", "localURL" (local URL), "cloudStorageURL" (cloud storage URL), "EZVIZURL" (EZ URL)-->binary
        </pictureURLType>
        <channels>
          <!--ro, opt, string, listen to the events on the specified channel No. list, desc:if all channels are being listened to, the node shall not be applied. If some channels are being listened to, the channel No. shall be listed and separated by commas-->1,2,3,4
        </channels>
      </Event>
    </EventList>
    <channels>
      <!--ro, opt, string, listen to the specified channel No. list, desc:if all channels are being listened to, the node shall not be applied. If some channels are being listened to, the channel No. shall be listed and separated by commas-->1,2,3,4
    </channels>
    <pictureURLType>
      <!--ro, opt, enum, alarm picture format, subType:string, desc:"binary", "localURL" (local URL), "cloudStorageURL" (cloud storage URL), "EZVIZURL" (EZ URL). The node indicates the upload mode of all event pictures. If the node is applied, <pictureURLType> of <Event> will be invalid. If the node is not applied, the pictures are uploaded in the default mode. The default data type of uploaded pictures for front-end devices is binary, and for back-end devices is local URL of the device-->binary
    </pictureURLType>
    <ChangedUploadSub>
      <!--ro, opt, object, subscribe to messages-->
      <interval>
        <!--ro, opt, int, the lifecycle of arming GUID, desc:within the interval, if the client software does not reconnect to the device, a new GUID will be generated by the device-->5
      </interval>
      <StatusSub>
        <!--ro, opt, object, sub status-->
        <all>
          <!--ro, opt, bool, whether to subscribe to all-->true
        </all>
        <channel>
          <!--ro, opt, bool, channel subscription status (whether the channel is subscribed), desc:it is not required if the value of <all> is true-->true
        </channel>
        <hd>
          <!--ro, opt, bool, HDD subscription status (whether the HDD is subscribed), desc:it is not required if the value of <all> is true-->true
        </hd>
        <capability>
          <!--ro, opt, bool, subscription status of capability set change (whether the capability set change is subscribed), desc:it is not required if the value of <all> is true-->true
        </capability>
      </StatusSub>
    </ChangedUploadSub>
  </SubscribeEvent>
  <PackingSpaceRecognition>
    <!--ro, opt, object, current control parameters of event listened by parking space detector on the security control panel, desc:related event: PackingSpaceRecognition-->
    <upLoadPicturesType>
      <!--ro, req, enum, uploaded picture type, subType:string, desc:"all", "picturesTypes"(upload specified types of pictures), "notUpload"(not upload pictures)-->all
    </upLoadPicturesType>
    <PicturesTypes>
      <!--ro, opt, array, specified list of uploaded picture types, subType:object, dep:and, {$.HttpHostNotification.PackingSpaceRecognition.upLoadPicturesType,eq,picturesTypes}-->
      <picturesType>
        <!--ro, opt, enum, uploaded picture type, subType:string, desc:"backgroundImage"(captured background picture), "plateImage"(license plate thumbnail)-->backgroundImage
      </picturesType>
    </PicturesTypes>
  </PackingSpaceRecognition>
```

```
    &lt;enabled&gt;
      &lt;!--ro, opt, bool--&gt;true
    &lt;/enabled&gt;
    &lt;netWork&gt;
      &lt;!--ro, opt, enum, subType:int--&gt;1
    &lt;/netWork&gt;
  &lt;/HttpHostNotification&gt;
```

10.2.1.5 Get the capabilities of listening hosts parameters

Request URL

GET /ISAPI/Event/notification/httpHosts/capabilities?type=<type>

Query Parameter

Parameter Name	Parameter Type	Description
type	enum	--

Request Message

None

Response Message

```xml
<?xml version="1.0" encoding="UTF-8"?>
<HttpHostNotificationCap xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, capabilities of subscribing to event types and event channels, attr:version{req, string, protocolVersion}-->
  <hostNumber>
    <!--ro, req, int, the number of listening hosts-->0
  </hostNumber>
  <urlLen max="10">
    <!--ro, req, string, URL length, attr:max{req, int}-->test
  </urlLen>
  <protocolType opt="HTTP,HTTPS,EHome">
    <!--ro, req, enum, protocol type, subType:string, attr:opt{req, string}, desc:"HTTP", "HTTPS", "EHome"-->HTTP
  </protocolType>
  <parameterFormatType opt="XML,querystring,JSON">
    <!--ro, req, enum, alarm parameter format type, subType:string, attr:opt{req, string}, desc:"xml" (XML format), "querystring", "json" (JSON format)-->XML
  </parameterFormatType>
  <addressingFormatType opt="ipaddress,hostname">
    <!--ro, req, enum, address format type, subType:string, attr:opt{req, string}, desc:"ipaddress", "hostname"-->ipaddress
  </addressingFormatType>
  <ipAddress opt="ipv4,ipv6">
    <!--ro, opt, string, IP address, attr:opt{req, string}, desc:it is valid when addressingFormatType is "ipaddress"-->test
  </ipAddress>
  <portNo min="0" max="10">
    <!--ro, opt, string, port number, range:[0,65535], attr:min{req, int},max{req, int}-->1
  </portNo>
  <SubscribeEventCap>
    <!--ro, opt, string, subscribe to changing status of capability set-->test
    <heartbeat min="0" max="10">
      <!--ro, opt, string, heartbeat interval time, attr:min{req, int},max{req, int}-->1
    </heartbeat>
    <channelMode opt="all,list">
      <!--ro, opt, enum, channel mode, subType:string, attr:opt{req, string}, desc:"all" (the device does not support subscribing to event channels separately, "list" (the device support subscribing to event channels separately). If both channelMode and eventMode return all, the device does not support subscribing to event types and event channels.-->all
    </channelMode>
    <eventMode opt="all,list">
      <!--ro, opt, enum, event mode, subType:string, attr:opt{req, string}, desc:"all" (subscribe to all channels and events on the device), "list" (subscribe to all channels and events on the device)-->all
    </eventMode>
    <EventList>
      <!--ro, opt, object, event list-->
      <Event>
        <!--ro, opt, string, event subscription information-->test
        <type>
          <!--ro, req, enum, see details in event type list, subType:string, desc:"AccessControllerEvent", "IDCardInfoEvent", "FaceTemperatureMeasurementEvent", "QRCodeEvent", "CertificateCaptureEvent", "UncertificateCompareEvent", "ConsumptionAndTransactionRecordEvent", "ConsumptionEvent", "TransactionRecordEvent", "HealthInfoSyncQuery", "SetMealQuery", "ConsumptionStatusQuery", "humanBodyComparison", "regionTargetNumberCounting"-->AccessControllerEvent
        </type>
        <minorAlarm opt="0x400,0x401,0x402,0x403">
          <!--ro, opt, string, minor alarm type, attr:opt{req, string}, desc:"IDCardInfoEvent" is required when the type of event is "AccessControllerEvent"-->0x400,0x401
        </minorAlarm>
        <minorException opt="0x400,0x401,0x402,0x403">
          <!--ro, opt, string, minor exception type, attr:opt{req, string}, desc:"IDCardInfoEvent" is required when the type of event is "AccessControllerEvent"-->0x400,0x401
        </minorException>
        <minorOperation opt="0x400,0x401,0x402,0x403">
          <!--ro, opt, string, minor operation type, attr:opt{req, string}, desc:"IDCardInfoEvent" is required when the type of event is "AccessControllerEvent"-->0x400,0x401
        </minorOperation>
        <minorEvent opt="0x01,0x02,0x03,0x04">
          <!--ro, opt, string, minor event type, attr:opt{req, string}, desc:"IDCardInfoEvent" is required when the type of event is "AccessControllerEvent"-->0x400,0x401
        </minorEvent>
      </Event>
    </EventList>
  </SubscribeEventCap>
</HttpHostNotificationCap>
```


10.2.1.6 Set IP address of receiving server(s)

Request URL

PUT /ISAPI/Event/notification/httpHosts

Query Parameter

None

Request Message

```xml
<?xml version="1.0" encoding="UTF-8"?>
<HttpHostNotificationList xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--req, array, listing host list, subType:object, attr:version{req, string, protocolVersion}-->
  <HttpHostNotification>
    <!--opt, object, listing host-->
    <id>
      <!--req, string, ID-->test
    </id>
    <url>
      <!--req, string, URL-->test
    </url>
    <protocolType>
      <!--req, enum, protocol type, subType:string, desc:"HTTP", "HTTPS", "EHome"-->HTTP
    </protocolType>
    <parameterFormatType>
      <!--req, enum, parameter format type, subType:string, desc:"JSON", "XML"-->JSON
    </parameterFormatType>
    <addressingFormatType>
      <!--req, enum, address types, subType:string, desc:"hostname" (host name), "ipaddress" (ip address)-->hostname
    </addressingFormatType>
    <ipAddress>
      <!--opt, string, IP address-->test
    </ipAddress>
    <portNo>
      <!--opt, int, port No.-->1
    </portNo>
    <userName>
      <!--opt, string, user name, dep:and,{$.HttpHostNotification.httpAuthenticationMethod,eq,MD5digest}-->test
    </userName>
    <httpAuthenticationMethod>
      <!--req, enum, authentication method, subType:string, desc:"MD5digest" (MD5), "none"-->MD5digest
    </httpAuthenticationMethod>
    <SubscribeEvent>
      <!--opt, object, picture uploading modes of all events which contain pictures-->
      <eventMode>
        <!--req, enum, event mode, subType:string, desc:event mode-->all
      </eventMode>
    </SubscribeEvent>
  </HttpHostNotification>
</HttpHostNotificationList>
```


Response Message

```xml
<?xml version="1.0" encoding="UTF-8"?>

<ResponseStatus xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, response message, attr:version{ro, req, string, protocolVersion}-->
  <requestURL>
    <!--ro, req, string, request URL, range:[0,1024]-->null
  </requestURL>
  <statusCode>
    <!--ro, req, enum, status code, subType:int, desc:0 (OK), 1 (OK), 2 (Device Busy), 3 (Device Error), 4 (Invalid Operation), 5 (Invalid XML Format), 6
(Invalid XML Content), 7 (Reboot Required)-->0
  </statusCode>
  <statusString>
    <!--ro, req, enum, status description, subType:string, desc:"OK" (succeeded), "Device Busy", "Device Error", "Invalid Operation", "Invalid XML Format",
"Invalid XML Content", "Reboot" (reboot device)-->OK
  </statusString>
  <subStatusCode>
    <!--ro, req, string, sub status code, which describes the error in details, desc:sub status code, which describes the error in details-->OK
  </subStatusCode>
  <description>
    <!--ro, opt, string, range:[0,1024]-->badXmlFormat
  </description>
</ResponseStatus>
```

10.2.1.7 Get parameters of all listening hosts

Request URL

GET /ISAPI/Event/notification/httpHosts

Query Parameter

None

Request Message

None

Response Message

```xml
<?xml version="1.0" encoding="UTF-8"?>

<HttpHostNotificationList xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
```

```xml
<HttpHostNotificationList xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, array, listening host list, subType:object, attr:version{req, string, protocolVersion}-->
  <HttpHostNotification>
    <!--ro, opt, object, listening host-->
    <id>
      <!--ro, req, string, subscribe to ID, range:[1,10]-->test
    </id>
    <url>
      <!--ro, req, string, URL-->test
    </url>
    <protocolType>
      <!--ro, req, enum, protocol type, subType:string, desc:"HTTP", "HTTPS", "EHome" (ISUP)-->HTTP
    </protocolType>
    <parameterFormatType>
      <!--ro, req, enum, parameter format type, subType:string, desc:"JSON", "XML"-->JSON
    </parameterFormatType>
    <addressingFormatType>
      <!--ro, req, enum, address type, subType:string, desc:"hostname", "ipaddress"-->hostname
    </addressingFormatType>
    <hostName>
      <!--ro, opt, string, host name, dep:and,{$.HttpHostNotification.addressingFormatType,eq,hostName}-->test
    </hostName>
    <ipAddress>
      <!--ro, opt, string, IP address, dep:or,{$.HttpHostNotification.addressingFormatType,eq,ipAddress}, desc:IP address-->test
    </ipAddress>
    <ipv6Address>
      <!--ro, opt, string, IPv6 Address, dep:or,{$.HttpHostNotification.addressingFormatType,eq,ipAddress}, desc:IPv6 Address-->test
    </ipv6Address>
    <portNo>
      <!--ro, opt, int, port number-->1
    </portNo>
    <userName>
      <!--ro, opt, string, user name, dep:and,{$.HttpHostNotification.httpAuthenticationMethod,eq,MD5digest}-->test
    </userName>
    <httpAuthenticationMethod>
      <!--ro, req, enum, authentication method, subType:string, desc:"MD5digest"(MD5), "none", "base64"-->MD5digest
    </httpAuthenticationMethod>
    <ANPR>
      <!--ro, opt, object, ANPR-->
      <detectionUpLoadPicturesType>
        <!--ro, opt, enum, uploaded pictures type, subType:string, desc:"all", "licensePlatePicture", "detectionPicture"(detected picture)-->all
      </detectionUpLoadPicturesType>
    </ANPR>
    <Extensions>
      <!--ro, opt, object, range-->
      <intervalBetweenEvents>
        <!--ro, opt, int, event interval-->1
      </intervalBetweenEvents>
    </Extensions>
    <uploadImagesDataType>
      <!--ro, opt, enum, picture data type, subType:string, desc:"URL", "binary" (default value). For cloud storage, only "URL" is supported-->URL
    </uploadImagesDataType>
    <httpBroken>
      <!--ro, opt, bool, whether to enable the automatic network replenishment, desc:if the ANR function is enabled, it will be applied to all events-->true
    </httpBroken>
    <SubscribeEvent>
      <!--ro, opt, object, picture uploading modes of all events which contain pictures-->
      <heartbeat>
        <!--ro, opt, int, heartbeat interval time-->30
      </heartbeat>
      <eventMode>
        <!--ro, req, enum, event mode, subType:string, desc:"all" (all alarms need to be reported), "list" (only listed alarms need to be reported)-->all
      </eventMode>
      <EventList>
        <!--ro, opt, array, event list, subType:object-->
        <Event>
          <!--ro, opt, object, channel information linked to event-->
          <type>
            <!--ro, req, enum, event type, subType:string, desc:refer to event type list (eventType): "ADAS"(advanced driving assistance system),
"ADASAlarm"(advanced driving assistance alarm), "AID"(traffic incident detection), "ANPR"(automatic number plate recognition), "AccessControllerEvent"
(access controller event), "CDsStatus" (CD burning status), "DBD"(driving behavior detection) "GPSUpload" (GPS information upload), "HFPD"(frequently
appeared person detection), "IO"(I/O alarm), "IOTD" (IoT device detection), "LES" (logistics scanning event), "LFPD"(rarely appeared person detection),
"PALMismatch" (video standard mismatch), "PIR", "PeopleCounting" (people counting), "PeopleNumChange" (people number change detection), "Standup"(standing
up detection), "TMA"(thermometry alarm), "TMPA"(temperature measurement pre-alarm), "VMD"(motion detection), "abnormalAcceleration", "abnormalDriving",
"advReachHeight", "alarmResult", "attendance", "attendedBaggage", "audioAbnormal", "audioexception", "behaviorResult"(abnormal event detection),
"blindSpotDetection"(blind spot detection alarm), "cardMatch", "changedStatus", "collision", "containerDetection", "crowdSituationAnalysis",
"databaseException", "defocus"(defocus detection), "diskUnformat"(disk unformatted), "diskerror", "diskfull", "driverConditionMonitor"(driver status
monitoring alarm); "emergencyAlarm", "faceCapture", "faceSnapModeling", "facedetection", "failDown"(People Falling Down), "faultAlarm", "fielddetection"
(intrusion detection), "fireDetection", "fireEscapeDetection", "flowOverrun", "framesPeopleCounting", "getUp"(getting up detection), "group" (people
gathering), "hdBadBlock"(HDD bad sector detection event), "hdImpact"(HDD impact detection event), "heatmap"(heat map alarm), "highHDTemperature"(HDD high
temperature detection event), "highTempAlarm"(HDD high temperature alarm), "hotSpare"(hot spare exception), "illaccess"(invalid access),
"ipcTransferAbnormal", "ipconflict"(IP address conflicts), "keyPersonGetUp"(key person getting up detection), "leavePosition"(absence detection),
"linedetection"(line crossing detection), "listSyncException"(list synchronization exception), "loitering"(loitering detection), "lowHDTemperature"(HDD low
temperature detection event), "mixedTargetDetection"(multi-target-type detection), "modelError", "nicbroken"(network disconnected), "nodeOffline"(node
disconnected), "nonPoliceIntrusion", "overSpeed"(overspeed alarm), "overtimeTarry"(staying overtime detection), "parking"(parking detection),
"peopleNumChange", "peopleNumCounting", "personAbnormalAlarm"(person ID exception alarm), "personDensityDetection", "personQueueCounting",
"personQueueDetection", "personQueueRealTime"(real-time data of people queuing-up detection), "personQueueTime"(waiting time detection), "playCellphone"
(playing mobile phone detection), "pocException"(video exception), "poe"(POE power exception), "policeAbsent", "radarAlarm", "radarFieldDetection",
"radarLineDetection", "radarPerimeterRule"(radar rule data), "radarTargetDetection", "radarVideoDetection"(radar-assisted target detection),
"raidException", "rapidMove", "reachHeight"(climbing detection), "recordCycleAbnormal"(insufficient recording period), "recordException", "regionEntrance",
"regionExiting", "retention"(people overstay detection), "rollover", "running"(people running), "safetyHelmetDetection"(hard hat detection),
"scenechangedetection", "sensorAlarm"(angular acceleration alarm), "severeHDFailure"(HDD major fault detection), "shelteralarm"(video tampering alarm),
"shipsDetection", "sitQuietly"(sitting detection), "smokeAndFireDetection", "smokeDetection", "softIO", "spacingChange"(distance exception), "sysStorFull"
(storaging full alarm of cluster system), "takingElevatorDetection"(elevator electric moped detection), "targetCapture", "temperature"(temperature
```

```
difference alarm), "thermometry"(temperature alarm), "thirdPartyException", "toiletTarry"(in-toilet overtime detection), "tollCodeInfo"(QR code information report), "tossing"(thrown object detection), "unattendedBaggage", "vehicleMatchResult"(uploading list alarms), "vehicleRcogResult", "versionAbnormal" (cluster version exception), "videoException", "videoloss", "violationAlarm", "violentMotion"(violent motion detection), "yardTarry"(playground overstay detection), "AccessControllerEvent", "IDCardInfoEvent", "FaceTemperatureMeasurementEvent", "QRCodeEvent"(QR code event of access control), "CertificateCaptureEvent"(person ID capture comparison event), "UncertificateCompareEvent", "ConsumptionAndTransactionRecordEvent", "ConsumptionEvent", "TransactionRecordEvent", "SetMealQuery"(searching consumption set meals), "ConsumptionStatusQuery"(searching the consumption status), "humanBodyComparison" (human body comparison), "regionTargetNumberCounting" (regional target statistics)-->mixedTargetDetection
          </type>
          <minorAlarm>
            <!--ro, opt, string, alarm sub type, desc:refer to the macro definition of uploaded events. "IDCardInfoEvent" is required when the type of event is "AccessControllerEvent"-->0x400,0x401,0x402,0x403
          </minorAlarm>
          <minorException>
            <!--ro, opt, string, exception sub type, desc:refer to the macro definition of uploaded events. "IDCardInfoEvent" is required when the type of event is "AccessControllerEvent"-->0x400,0x401,0x402,0x403
          </minorException>
          <minorOperation>
            <!--ro, opt, string, operation sub type, desc:refer to the macro definition of uploaded events. "IDCardInfoEvent" is required when the type of event is "AccessControllerEvent"-->0x400,0x401,0x402,0x403
          </minorOperation>
          <minorEvent>
            <!--ro, opt, string, event sub type, desc:refer to the macro definition of uploaded events. "IDCardInfoEvent" is required when the type of event is "AccessControllerEvent"-->0x01,0x02,0x03,0x04
          </minorEvent>
          <pictureURLType>
            <!--ro, opt, enum, alarm picture format of the specified event, subType:string, desc:alarm picture format of the specified event-->binary
          </pictureURLType>
          <channels>
            <!--ro, opt, string, listen to the events on the specified channel No. list, desc:if all channels are being listened to, the node shall not be applied. if some channels are being listened to, the channel No. shall be listed and separated by commas-->1,2,3,4
          </channels>
        </Event>
      </EventList>
      <channels>
        <!--ro, opt, string, listen to the specified channel No. list, desc:if all channels are being listened to, the node shall not be applied. if some channels are being listened to, the channel No. shall be listed and separated by commas-->1,2,3,4
      </channels>
      <pictureURLType>
        <!--ro, opt, enum, alarm picture format, subType:string, desc:the node indicates the picture types of all events which contain pictures to be uploaded. if the node is applied, the pictureURLType of the Event will not take effect. if the node is not applied, the pictures reported from the device by default mode: the default data type of uploaded pictures captured from front-end devices is binary. the default data type of uploaded pictures reported from storage devices is local URL of the device.-->binary
      </pictureURLType>
      <ChangedUploadSub>
        <!--ro, opt, object, subscribe to messages-->
        <interval>
          <!--ro, opt, int, the lifecycle of arming GUID, desc:if GUID is not reconnected in the internal, a new GUID will be generated since the device start a new arm period-->5
        </interval>
        <StatusSub>
          <!--ro, opt, object, sub status-->
          <all>
            <!--ro, opt, bool, subscribe to all?-->true
          </all>
          <channel>
            <!--ro, opt, bool, subscribe to channel status, desc:reporting is not required if all is true-->true
          </channel>
          <hd>
            <!--ro, opt, bool, subscribe to HDD status, desc:reporting is not required if all is true-->true
          </hd>
          <capability>
            <!--ro, opt, bool, subscribe to changing status of capability set, desc:reporting is not required if all is true-->true
          </capability>
        </StatusSub>
      </ChangedUploadSub>
    </SubscribeEvent>
    <PackingSpaceRecognition>
      <!--ro, opt, object, current control parameters of event listened by parking space detector on the security control panel, desc:related event: PackingSpaceRecognition-->
      <upLoadPicturesType>
        <!--ro, opt, enum, uploaded picture type, subType:string, desc:"all", "picturesTypes" (upload specified types of pictures), "notUpload" (not upload pictures)-->all
      </upLoadPicturesType>
      <PicturesTypes>
        <!--ro, opt, array, specified list of uploaded picture type, subType:object, dep:and, {$.HttpHostNotificationList[*].HttpHostNotification.PackingSpaceRecognition.upLoadPicturesType,eq,picturesTypes}-->
        <picturesType>
          <!--ro, opt, enum, uploaded picture type, subType:string, desc:"backgroundImage" (captured background picture), "plateImage" (license plate thumbnail)-->backgroundImage
        </picturesType>
      </PicturesTypes>
    </PackingSpaceRecognition>
  </HttpHostNotification>
  <enabled>
    <!--ro, opt, bool-->true
  </enabled>
  <netWork>
    <!--ro, opt, enum, subType:int-->1
  </netWork>
</HttpHostNotificationList>
```

10.3 Access Control (General)

10.3.1 Access Control Event Management

10.3.1.1 Get the capability of searching for access control events

Request URL

GET /ISAPI/AccessControl/AcsEvent/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "AcsEvent": {
    /*ro, req, object, access control events*/
        "AcsEventCond": {
        /*ro, opt, object, search conditions*/
            "searchID": {
            /*ro, req, object, search ID, it is used to check whether the current search requester is the same as the previous one. If they are the same,
the search record will be stored in the device to speed up the next search*/
                "@min":  1,
                /*ro, req, int, the minimum value*/
                "@max":  1
                /*ro, req, int, the maximum value*/
            },
            "searchResultPosition": {
            /*ro, req, object, the start position of the search result in the result list*/
                "@min":  1,
                /*ro, req, int, the minimum value*/
                "@max":  1
                /*ro, req, int, the maximum value*/
            },
            "maxResults": {
            /*ro, req, object, the maximum number of search results that can be obtained by calling this URL*/
                "@min":  1,
                /*ro, req, int, the minimum value*/
                "@max":  1
                /*ro, req, int, the maximum value*/
            },
            "major": {
            /*ro, opt, object, major alarm type (the type value should be transformed to the decimal number)*/
                "@opt":  "0,1,2,3,5"
                /*ro, req, string, major type*/
            },
            "minorAlarm": {
            /*ro, opt, object, minor alarm type (the type value should be transformed to the decimal number)*/
                "@opt":  "1024,1025,1026,1027…"
                /*ro, req, string, minor alarm type*/
            },
            "minorException": {
            /*ro, opt, object, minor exception type (the type value should be transformed to the decimal number)*/
                "@opt":  "39,58,59,1024…"
                /*ro, req, string, minor exception type*/
            },
            "minorOperation": {
            /*ro, opt, object, minor operation type (the type value should be transformed to the decimal number)*/
                "@opt":  "80,90,112,113…"
                /*ro, req, string, minor operation type*/
            },
            "minorEvent": {
            /*ro, opt, object, minor event type (the type value should be transformed to the decimal number)*/
                "@opt":  "1,2,3,4…"
                /*ro, req, string, minor event type*/
            },
            "startTime": {
            /*ro, opt, object, start time*/
                "@min":  0,
                /*ro, req, int, the minimum value, range:[0,32]*/
                "@max":  32
                /*ro, req, int, the maximum value, range:[0,32]*/
            },
            "endTime": {
            /*ro, opt, object, end time*/
                "@min":  0,
                /*ro, req, int, the minimum value, range:[0,32]*/
                "@max":  32
                /*ro, req, int, the maximum value, range:[0,32]*/
            },
            "cardNo": {
            /*ro, opt, object, card No.*/
```

```
                "@min":  1,
                /*ro, req, int, the minimum value*/
                "@max":  1
                /*ro, req, int, the maximum value*/
            },
            "beginSerialNo": {
            /*ro, opt, object, start serial No.*/
                "@min":  1,
                /*ro, req, int, the minimum value*/
                "@max":  1
                /*ro, req, int, the maximum value*/
            },
            "endSerialNo": {
            /*ro, opt, object, end serial No.*/
                "@min":  1,
                /*ro, req, int, the minimum value*/
                "@max":  1
                /*ro, req, int, the maximum value*/
            },
            "employeeNoString": {
            /*ro, opt, object, employee No.*/
                "@min":  1,
                /*ro, req, int, the minimum value*/
                "@max":  1
                /*ro, req, int, the maximum value*/
            },
        },
        "InfoList": {
        /*ro, opt, object, information list*/
            "maxSize":  10,
            /*ro, opt, int, the maximum value*/
            "time": {
            /*ro, opt, object, time (UTC time)*/
                "@min":  0,
                /*ro, req, int, the minimum value, range:[0,32]*/
                "@max":  32
                /*ro, req, int, the maximum value, range:[0,32]*/
            },
            "netUser": {
            /*ro, opt, object, user name*/
                "@min":  1,
                /*ro, req, int, the minimum value*/
                "@max":  1
                /*ro, req, int, the maximum value*/
            },
            "remoteHostAddr": {
            /*ro, opt, object, remote host address*/
                "@min":  1,
                /*ro, req, int, the minimum value*/
                "@max":  1
                /*ro, req, int, the maximum value*/
            },
            "cardNo": {
            /*ro, opt, object, card No.*/
                "@min":  1,
                /*ro, req, int, the minimum value*/
                "@max":  1
                /*ro, req, int, the maximum value*/
            },
            "cardType": {
            /*ro, opt, object, card type*/
                "@opt":  "1,2,3,4,5,6,7,8"
                /*ro, req, string, 1 (normal card), 2 (disability card), 3 (blocklist card), 4 (patrol card), 5 (duress card), 6 (super card), 7 (visitor
card), 8 (dismiss card)*/
            },
            "reportChannel": {
            /*ro, opt, object, channel type for uploading alarms/events*/
                "@opt":  "1,2,3"
                /*ro, req, string, "1" (for uploading arming information), "2" (for uploading by central group 1), "3" (for uploading by central group 2)*/
            },
            "cardReaderKind": {
            /*ro, opt, object, card reader type*/
                "@opt":  "1,2,3,4"
                /*ro, req, string, "1" (IC card reader), "2" (ID card reader), "3" (QR code scanner), "4" (fingerprint module)*/
            },
            "cardReaderNo": {
            /*ro, opt, object, card reader No.*/
                "@min":  1,
                /*ro, req, int, the minimum value*/
                "@max":  1
                /*ro, req, int, the maximum value*/
            },
            "doorNo": {
            /*ro, opt, object, door (floor) No.*/
                "@min":  1,
                /*ro, req, int, the minimum value*/
                "@max":  1
                /*ro, req, int, the maximum value*/
            },
            "verifyNo": {
            /*ro, opt, object, multiple authentication No.*/
                "@min":  1,
                /*ro, req, int, the minimum value*/
                "@max":  1
```

```
                "@max": 1
                /*ro, req, int, the maximum value*/
            },
            "alarmOutNo": {
            /*ro, opt, object, alarm output No.*/
                "@min": 1,
                /*ro, req, int, the minimum value*/
                "@max": 1
                /*ro, req, int, the maximum value*/
            },
            "caseSensorNo": {
            /*ro, opt, object, event trigger No.*/
                "@min": 1,
                /*ro, req, int, the minimum value*/
                "@max": 1
                /*ro, req, int, the maximum value*/
            },
            "deviceNo": {
            /*ro, opt, object, device No.*/
                "@min": 1,
                /*ro, req, int, the minimum value*/
                "@max": 1
                /*ro, req, int, the maximum value*/
            },
            "MACAddr": {
            /*ro, opt, object, MAC Address*/
                "@min": 1,
                /*ro, req, int, the minimum value*/
                "@max": 1
                /*ro, req, int, the maximum value*/
            },
            "serialNo": {
            /*ro, opt, object, event serial No.*/
                "@min": 1,
                /*ro, req, int, the minimum value*/
                "@max": 1
                /*ro, req, int, the maximum value*/
            },
            "userType": {
            /*ro, opt, object, person types*/
                "@opt": "normal,visitor,blackList,administrators"
                /*ro, req, string, "normal" (normal person (household)), "visitor" (visitor), "blacklist" (person in blocklist), "administrators" (administrator)*/
            },
            "currentVerifyMode": {
            /*ro, opt, object, current authentication mode of the card reader*/
                "@opt": "cardAndPw,card,cardOrPw,fp,fpAndPw,fpOrCard,fpAndCard,fpAndCardAndPw,faceOrFpOrCardOrPw,faceAndFp,faceAndPw,faceAndCard,face,employeeNoAndPw,fpOrPw,employeeNoAndFp,employeeNoAndFpAndPw,faceAndFpAndCard,faceAndPwAndFp,employeeNoAndFace,faceOrfaceAndCard,fpOrface,cardOrfaceOrPw,iris,faceOrFpOrCardOrPwOrIris,faceOrCardOrPwOrIris"
                /*ro, req, string, "cardAndPw"-card+password, "card", "cardOrPw"-card or password, "fp"-fingerprint, "fpAndPw"-fingerprint+password, "fpOrCard"-fingerprint or card, "fpAndCard"-fingerprint+card, "fpAndCardAndPw"-fingerprint+card+password, "faceOrFpOrCardOrPw"-face or fingerprint or card or password, "faceAndFp"-face+fingerprint, "faceAndPw"-face+password, "faceAndCard"-face+card, "face", "employeeNoAndPw"-employee No.+password, "fpOrPw"-fingerprint or password, "employeeNoAndFp"-employee No.+fingerprint, "employeeNoAndFpAndPw"-employee No.+fingerprint+password, "faceAndFpAndCard"-face+fingerprint+card, "faceAndPwAndFp"-face+password+fingerprint, "employeeNoAndFace"-employee No.+face, "faceOrfaceAndCard"-face or face+card, "fpOrface"-fingerprint or face, "cardOrfaceOrPw"-card or face or password, "cardOrFpOrPw"-card or fingerprint or password*/
            },
            "attendanceStatus": {
            /*ro, opt, object, attendance status, desc:"undefined", "checkIn" (check-in), "checkOut" (check-out), "breakOut" (start of break), "breakIn" (end of break), "overtimeIn" (start of overtime), "overTimeOut" (end of overtime)*/
                "@opt": "undefined,checkIn,checkOut,breakOut,breakIn,overtimeIn,overtimeOut"
                /*ro, req, string, options*/
            },
            "statusValue": {
            /*ro, opt, object, status value*/
                "@min": 1,
                /*ro, req, int, the minimum value*/
                "@max": 1
                /*ro, req, int, the maximum value*/
            },
        }
    }
}
```


10.3.1.2 Search for access control events

Request URL

POST /ISAPI/AccessControl/AcsEvent?format=json

Query Parameter

None

Request Message

```
{
    "AcsEventCond": {
    /*req, object, access control events*/
        "searchID":  "test",
        /*req, string, search ID, desc:it is used to check whether the current search requester is the same as the previous one. If they are the same, the search record will be stored in the device to speed up the next search*/
        "searchResultPosition":  0,
        /*req, int, the start position of the search result in the result list:, desc:in a single search, if you cannot get all the records in the result list, you can mark the end position and get the following records after the marked position in the next search. If the maximum number of totalMatches supported by the device is M and the number of totalMatches stored in the device now is N (N<=M), the valid range of this node is 0 to N-1*/
        "maxResults":  30,
        /*req, int, the maximum number of search results, which is defined by the device capability, will be returned if the value of maxResults reaches the limit, desc:if maxResults exceeds the range returned by the device capability, the device will return the maximum number of search results according to the device capability and will not return error message*/
        "major":  1,
        /*req, int, major type, desc:the type value should be transformed to the decimal number; see Access Control Event Types for details*/
        "minor":  1024,
        /*req, int, minor type, desc:the type value should be transformed to the decimal number; see Access Control Event Types for details*/
        "startTime":  "1970-01-01T00:00:00+08:00",
        /*opt, datetime, start time (UTC time)*/
        "endTime":  "1970-01-01T00:00:00+08:00",
        /*opt, datetime, end time (UTC time)*/
        "cardNo":  "test",
        /*opt, string, card No.*/
        "name":  "test",
        /*opt, string, name of the card holder*/
        "videoChannel":  1,
        /*opt, int, video channel No., range:[1,86400], desc:this node is newly added to DeepinMind devices for attendance*/
        "picEnable":  true,
        /*opt, bool, whether to upload the picture along with the event information, desc:false (no), true (yes, default value); (1. all matched events will be uploaded without pictures; 2. all matched events will be uploaded with pictures if there are any; 3. if this node is not configured, the default value is true)*/
        "beginSerialNo":  1,
        /*opt, int, start serial No.*/
        "endSerialNo":  1,
        /*opt, int, end serial No.*/
        "employeeNoString":  "test",
        /*opt, string, employee No. (person ID)*/
        "timeReverseOrder":  true,
        /*opt, bool, whether to return events in descending order of time (later events will be returned first), desc:true (yes), false or this node is not returned (no)*/
        "isAbnomalTemperature":  true,
        /*opt, bool, whether the skin-surface temperature is abnormal*/
        "temperatureSearchCond":  "all",
        /*opt, enum, temperature search condition, subType:string, desc:when this node and isAbnormalTemperature both exist, isAbnormalTemperature is invalid; "all" (event with temperature), "normal" (event with normal temperature), "abnormal" (event with abnormal temperature)*/
        "isAttendanceInfo":  true,
        /*opt, bool, whether it contains attendance records, desc:this node is newly added to HEOP protocol; if this node is true, main type, minor type, employee No., name, and time will be returned*/
        "hasRecordInfo":  true
        /*opt, bool*/
    }
}
```


Response Message

```
{
    "AcsEvent": {
    /*ro, req, object, access control events*/
        "searchID":  "test",
        /*ro, req, string, search ID, it is used to check whether the current search requester is the same as the previous one. If they are the same, the search record will be stored in the device to speed up the next search*/
        "responseStatusStrg":  "OK",
        /*ro, req, string, searching status description*/
        "numOfMatches":  1,
        /*ro, req, int, number of results returned this time*/
        "totalMatches":  1,
        /*ro, req, int, total number of matched results*/
        "InfoList": [
        /*ro, opt, array, information list, subType:object*/
            {
                "major":  1,
                /*ro, req, int, major alarm type*/
                "minor":  1,
                /*ro, req, int, minor alarm type*/
                "time":  "2016-12-12T17:30:08+08:00",
                /*ro, req, string, time (UTC time)*/
                "netUser":  "test",
                /*ro, opt, string, user name*/
                "remoteHostAddr":  "test",
                /*ro, opt, string, remote host address*/
                "videoChannel":  1,
                /*ro, opt, int, video channel No., range:[1,86400], desc:this node is newly added to DeepinMind devices for attendance*/
                "cardNo":  "test",
                /*ro, opt, string, card No.*/
                "cardType":  1,
                /*ro, opt, enum, card type, subType:int, desc:1 (normal card), 2 (disability card), 3 (blocklist card), 4 (patrol card), 5 (duress card), 6 (super card), 7 (visitor card), 8 (dismiss card)*/
                "whiteListNo":  1,
```

```
        /*ro, opt, int, allowlist No.*/
        "reportChannel":  1,
        /*ro, opt, int, channel type for uploading alarm/event*/
        "cardReaderKind":  1,
        /*ro, opt, int, card reader type: 1 (IC card reader)*/
        "cardReaderNo":  1,
        /*ro, opt, int, card reader No.*/
        "doorNo":  1,
        /*ro, opt, int, door or floor No.*/
        "verifyNo":  1,
        /*ro, opt, int, multi-factor authentication No.*/
        "alarmInNo":  1,
        /*ro, opt, int, alarm input No.*/
        "alarmOutNo":  1,
        /*ro, opt, int, alarm output No.*/
        "caseSensorNo":  1,
        /*ro, opt, int, event trigger No.*/
        "RS485No":  1,
        /*ro, opt, int, RS-485 channel No.*/
        "multiCardGroupNo":  1,
        /*ro, opt, int, group No.*/
        "accessChannel":  1,
        /*ro, opt, int, RS-485 channel No.*/
        "deviceNo":  1,
        /*ro, opt, int, device No.*/
        "distractControlNo":  1,
        /*ro, opt, int, distributed controller No.*/
        "employeeNoString":  "test",
        /*ro, opt, string, employee No. (person ID)*/
        "localControllerID":  1,
        /*ro, opt, int, distributed controller No.*/
        "InternetAccess":  1,
        /*ro, opt, int, network interface No.*/
        "type":  1,
        /*ro, opt, int, zone type, desc:0 (instant alarm zone), 1 (24-hour zone), 2 (delayed zone), 3 (internal zone), 4 (key zone), 5 (fire alarm
zone), 6 (perimeter zone), 7 (24-hour silent zone), 8 (24-hour auxiliary zone), 9 (24-hour shock zone), 10 (emergency door open zone), 11 (emergency door
closed zone), 255 (none)*/
        "MACAddr":  "test",
        /*ro, opt, string, MAC address*/
        "swipeCardType":  1,
        /*ro, opt, enum, card swiping type, subType:int, desc:0 (invalid), 1 (QR code)*/
        "serialNo":  1,
        /*ro, opt, int, event serial No.*/
        "channelControllerID":  1,
        /*ro, opt, int, lane controller ID*/
        "channelControllerLampID":  1,
        /*ro, opt, int, light board ID of lane controller, range:[1,255]*/
        "channelControllerIRAdaptorID":  1,
        /*ro, opt, int, IR adapter ID of lane controller, range:[1,255]*/
        "channelControllerIREmitterID":  1,
        /*ro, opt, int, active infrared intrusion detector No. of lane controller, range:[1,255]*/
        "userType":  "normal",
        /*ro, opt, string, person type*/
        "currentVerifyMode":  "cardAndPw",
        /*ro, opt, enum, current authentication mode of the card reader, subType:string, desc:"cardAndPw" (card + password); "card", "cardOrPw"
(card or password),  "fp" (fingerprint), "fpAndPw" (fingerprint + password), "fpOrCard" "fingerprint or card", "fpAndCard" (fingerprint + card),
"fpAndCardAndPw" (fingerprint + card + password), "faceOrFpOrCardOrPw" (face or fingerprint or card or password), "faceAndFp" (face + fingerprint),
"faceAndPw" (face + password), "faceAndCard" (face + card), "face", "employeeNoAndPw" (emplyee No. +password), "fpOrPw" (fingerprint or password),
"employeeNoAndFp" (employee No. + fingerprint), "employeeNoAndFpAndPw" (employee No. + fingerprint + password), "faceAndFpAndCard" (face + fingerprint +
card), "faceAndPwAndFp" (face + password + fingerprint), "employeeNoAndFace" (employee No. + face), "faceOrfaceAndCard" (face or face + card), "fpOrface"
(fingerprint or face), "cardOrfaceOrPw" (card or face or password), "faceOrFpOrCardOrPwOrIris" (face or fingerprint or card or password or iris),
"faceOrCardOrPwOrIris" (face or card or password or iris), "sleep", "invalid"*/
        "QRCodeInfo":  "test",
        /*ro, opt, string, QR code information*/
        "thermometryUnit":  "celsius",
        /*ro, opt, enum, temperature unit, subType:string, desc:"celsius" (Celsius, default value), "fahrenheit" (Fahrenheit), "kelvin" (Kelvin)*/
        "currTemperature":  36.5,
        /*ro, opt, float, skin-surface temperature, which is accurate to one decimal place*/
        "isAbnomalTemperature":  true,
        /*ro, opt, bool, whether the skin-surface temperature is abnormal (true-yes)*/
        "RegionCoordinates": {
        /*ro, opt, object, coordinates of the skin-surface temperature*/
            "positionX":  254,
            /*ro, opt, int, normalized X-coordinate which is between 0 and 1000*/
            "positionY":  133
            /*ro, opt, int, normalized Y-coordinate which is between 0 and 1000*/
        },
        "mask":  "unknown",
        /*ro, opt, enum, whether the person wears a mask, subType:string, desc:"unknown"*/
        "pictureURL":  "test",
        /*ro, opt, string, picture URL*/
        "filename":  "picture1",
        /*ro, opt, string, file name, desc:if multiple pictures are returned at a time, filename of each picture should be unique*/
        "attendanceStatus":  "undefined",
        /*ro, opt, enum, attendance status, subType:string, desc:"undefined", "checkIn" (check-in), "checkOut" (check-out), "breakOut" (start of
break), "breakIn" (end of break), "overtimeIn" (start of overtime), "overTimeOut" (end of overtime)*/
        "label":  "test",
        /*ro, opt, string, custom attendance name*/
        "statusValue":  1,
        /*ro, opt, int, status value*/
        "helmet":  "unknown",
        /*ro, opt, enum, whether the person wears a hard hat, subType:string, desc:"unknown", "yes", "no"*/
        "visibleLightPicUrl":  "test",
        /*ro, opt, string, visible light picture URL*/
```

```
                /*ro, opt, string, visible light picture URL*/
                "thermalPicUrl":  "test",
                /*ro, opt, string, URL of the thermal imaging picture*/
                "appType":  "attendance",
                /*ro, opt, enum, application type, subType:string, desc:"attendance" (Time & Attendance module), "signIn" (Check-In module, which is only
used for FocSign products)*/
                "HealthInfo": {
                /*ro, opt, object, health information*/
                    "healthCode":  1,
                    /*ro, opt, enum, health code status, subType:int, desc:0 (no request), 1 (no health code), 2 (green QR code), 3 (yellow QR code), 4 (red
QR code), 5 (no such person), 6 (other error, e.g., searching failed due to API exception), 7 (searching for the health code timed out)*/
                    "NADCode":  1,
                    /*ro, opt, enum, nucleic acid test result, subType:int, desc:0 (no result), 1 (negative, which means normal), 2 (positive, which means
diagnosed), 3 (the result has expired)*/
                    "travelCode":  1,
                    /*ro, opt, enum, trip code, subType:int, desc:0 (no trip in the past 14 days), 1 (has left the current area in the past 14 days), 2 (has
been to the high-risk area in the past 14 days), 3 (other)*/
                    "travelInfo":  "test",
                    /*ro, opt, string*/
                    "vaccineStatus":  1,
                    /*ro, opt, enum, whether the person is vaccinated, subType:int, desc:0 (not vaccinated), 1 (vaccinated)*/
                    "vaccineNum":  1
                    /*ro, opt, int, step:1*/
                },
                "meetingID":  "test",
                /*ro, opt, string, meeting ID*/
                "PersonInfoExtends": [
                /*ro, opt, array, additional person information, subType:object, desc:this node displays additional person information on the device*/
                    {
                        "id":  1,
                        /*ro, opt, int, extended ID of the additional person information, range:[1,32], desc:related URL:
/ISAPI/AccessControl/personInfoExtendName?format=json; this node is used for displaying the name of value; if ID does not exists, it starts from 1*/
                        "value":  "test"
                        /*ro, opt, string, extended content of the additional person information*/
                    }
                ],
                "name":  "test",
                /*ro, opt, string, name, desc:person name*/
                "FaceRect": {
                /*ro, opt, object, rectangle frame for human face, desc:the origin is the upper-left corner of the screen*/
                    "height":  1.000,
                    /*ro, req, float, height, range:[0.000,1.000]*/
                    "width":  1.000,
                    /*ro, req, float, width, range:[0.000,1.000]*/
                    "x":  0.000,
                    /*ro, req, float, X-coordinate of the upper-left corner of the frame, range:[0.000,1.000]*/
                    "y":  0.000
                    /*ro, req, float, Y-coordinate of the upper-left corner of the frame, range:[0.000,1.000]*/
                },
                "RecordInfo": {
                /*ro, opt, object*/
                    "startTime":  "1970-01-01T00:00:00+08:00",
                    /*ro, opt, datetime, recording start time*/
                    "endTime":  "1970-01-01T00:00:00+08:00",
                    /*ro, opt, datetime, recording end time*/
                    "playbackURL":  "rtsp://10.65.130.168:554/ISAPI/Streaming/tracks/201/?starttime=20190213T091134Z&amp;endtime=20190213T092116Z"
                    /*ro, opt, string, range:[0,256]*/
                },
                "currentAuthenticationTimes":  1,
                /*ro, opt, int, range:[0,255], step:1*/
                "allowAuthenticationTimes":  1
                /*ro, opt, int, range:[0,255], step:1*/
            }
        ]
    }
}
```

10.3.1.3 Get the capability of getting total number of access control events by specific conditions

Request URL

GET /ISAPI/AccessControl/AcsEventTotalNum/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "AcsEvent": {
    /*ro, opt, object*/
        "AcsEventTotalNumCond": {
        /*ro, opt, object, search conditions*/
            "major": {
```

```
            /*ro, req, object, major alarm type*/
                "@opt":  "0,1,2,3,5"
                /*ro, opt, string, major alarm type*/
            },
            "minorAlarm": {
            /*ro, req, object, minor alarm type*/
                "@opt":  "1024,1025,1026,1027"
                /*ro, opt, string, minor alarm type*/
            },
            "minorException": {
            /*ro, req, object, minor exception type*/
                "@opt":  "39,58,59,1024"
                /*ro, opt, string, minor exception type*/
            },
            "minorOperation": {
            /*ro, req, object, minor operation type*/
                "@opt":  "80,90,112,113"
                /*ro, opt, string, minor operation type*/
            },
            "minorEvent": {
            /*ro, opt, object, minor event type*/
                "@opt":  "1,2,3,4"
                /*ro, opt, string, minor event type*/
            },
            "startTime": {
            /*ro, opt, object, start time*/
                "@min":  1,
                /*ro, opt, int, start time (UTC time)*/
                "@max":  1
                /*ro, opt, int, end time (UTC time)*/
            },
            "endTime": {
            /*ro, opt, object, end time*/
                "@min":  1,
                /*ro, opt, int, start time (UTC time)*/
                "@max":  1
                /*ro, opt, int, end time (UTC time)*/
            },
            "cardNo": {
            /*ro, opt, object, card No.*/
                "@min":  1,
                /*ro, opt, int, card No.*/
                "@max":  32
                /*ro, opt, int, card No.*/
            },
            "name": {
            /*ro, opt, object, name of the card holder*/
                "@min":  1,
                /*ro, opt, int, name of the card holder*/
                "@max":  32
                /*ro, opt, int, name of the card holder*/
            },
            "picEnable":  "true,false",
            /*ro, opt, string*/
            "beginSerialNo": {
            /*ro, opt, object, start serial No.*/
                "@min":  1,
                /*ro, opt, int, start serial No.*/
                "@max":  1
                /*ro, opt, int, start serial No.*/
            },
            "endSerialNo": {
            /*ro, opt, object, end serial No.*/
                "@min":  1,
                /*ro, opt, int, end serial No.*/
                "@max":  1
                /*ro, opt, int, end serial No.*/
            },
            "employeeNoString": {
            /*ro, opt, object, employee No. (person ID)*/
                "@min":  1,
                /*ro, opt, int, employee No. (person ID)*/
                "@max":  32
                /*ro, opt, int, employee No. (person ID)*/
            }
        },
        "totalNum": {
        /*ro, req, object*/
            "@min":  1,
            /*ro, opt, int*/
            "@max":  1
            /*ro, opt, int*/
        }
    }
}
```


10.3.1.4 Get the total number of access control events by specific conditions

Request URL

POST /ISAPI/AccessControl/AcsEventTotalNum?format=json

Query Parameter

None

Request Message

```
{
    "AcsEventTotalNumCond": {
    /*req, object*/
        "major":  1,
        /*req, int, major alarm type, desc:(the type value should be transformed to the decimal number), refer to Access Control Event Types for details*/
        "minor":  1024,
        /*req, int, sub type, step:1, desc:(the type value should be transformed to the decimal number),refer to Access Control Event Types for details*/
        "startTime":  "1970-01-01+08:00",
        /*opt, date, start time (UTC time)*/
        "endTime":  "1970-01-01+08:00",
        /*opt, date, end time (UTC time)*/
        "cardNo":  "test",
        /*opt, string, card No.*/
        "name":  "test",
        /*opt, string, name of the card holder*/
        "picEnable":  true,
        /*opt, bool, whether to upload the picture along with the event information, desc:whether to contain pictures: "true"-yes,"false"-no*/
        "beginSerialNo":  1,
        /*opt, int, start serial No.*/
        "endSerialNo":  100,
        /*opt, int, end serial No.*/
        "employeeNoString":  "test"
        /*opt, string, employee No. (person ID), range:[1,32]*/
    }
}
```

Response Message

```
{
    "AcsEventTotalNum": {
    /*ro, req, object*/
        "totalNum":  1,
        /*ro, req, int, total number of events that match the search conditions*/
        "existedEventNum":  1
        /*ro, opt, int*/
    }
}
```

10.3.1.5 Get the capability of clearing event and card linkage parameters

Request URL

GET /ISAPI/AccessControl/ClearEventCardLinkageCfg/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "ClearEventCardLinkageCfg": {
    /*ro, opt, object, clear event and card linkage parameters*/
        "ClearFlags": {
        /*ro, opt, object*/
            "eventCardLinkage":  "true,false"
            /*ro, req, string, event and card linkage parameters*/
        }
    }
}
```

10.3.1.6 Clear event card linkage configurations

Request URL

PUT /ISAPI/AccessControl/ClearEventCardLinkageCfg?format=json

Query Parameter

None

Request Message

```
{
    "ClearEventCardLinkageCfg": {
    /*req, object*/
        "ClearFlags": {
        /*opt, object*/
            "eventCardLinkage":  true
            /*req, bool, whether to clear event and card linkage parameters*/
        }
    }
}
```

Response Message

```
{
    "statusCode":  1,
    /*ro, opt, int, status code, desc:1 (succeeded); it is required when an error occurred*/
    "statusString":  "ok",
    /*ro, opt, string, status description, range:[1,64], desc:"ok" (succeeded); it is required when an error occurred*/
    "subStatusCode":  "ok",
    /*ro, opt, string, sub status code, range:[1,64], desc:"ok" (succeeded); it is required when an error occurred*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:when the value of statusCode is not 1, it corresponds to subStatusCode*/
    "errorMsg":  "ok"
    /*ro, opt, string, error information, desc:this node must be returned when the value of statusCode is not 1*/
}
```

10.3.1.7 Getting arming information

Request URL

GET /ISAPI/AccessControl/DeployInfo

Query Parameter

None

Request Message

None

Response Message

```
<?xml version="1.0" encoding="UTF-8"?>

<DeployInfo xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, opt, object, arming information, attr:version{req, string, protocolVersion}-->
  <DeployList size="5">
    <!--ro, opt, array, arming list, subType:object, attr:size{req, int}-->
    <Content>
      <!--ro, opt, object, subscribe to messages-->
      <deployNo>
        <!--ro, req, int, arming No.-->1
      </deployNo>
      <deployType>
        <!--ro, req, enum, arming type, subType:int-->1
      </deployType>
      <protocolType>
        <!--ro, opt, enum, protocol type, subType:string, dep:or,{$.DeployInfo.DeployList[*].Content.deployType,eq,2},
{$.DeployInfo.DeployList[*].Content.deployType,eq,3}, desc:"HTTP", "HTTPS"-->HTTP
      </protocolType>
      <ipAddr>
        <!--ro, req, string, IP address-->test
      </ipAddr>
      <port>
        <!--ro, opt, int, port No., range:[1,65535]-->1
      </port>
      <eventType>
        <!--ro, opt, enum, subType:string-->AccessController
      </eventType>
    </Content>
  </DeployList>
</DeployInfo>
```

10.3.1.8 Getting arming information capability

Request URL

GET /ISAPI/AccessControl/DeployInfo/capabilities

Query Parameter

None

Request Message

None

Response Message


```
<?xml version="1.0" encoding="UTF-8"?>

<DeployInfo xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, opt, object, arming Information, attr:version{req, string, protocolVersion}-->
  <DeployList size="5">
    <!--ro, opt, array, arming list, subType:object, attr:size{req, int}-->
    <Content>
      <!--ro, opt, object-->
      <deployNo min="1" max="10">
        <!--ro, req, int, arming No., attr:min{req, int},max{req, int}-->1
      </deployNo>
      <deployType opt="0,1,2,3">
        <!--ro, req, int, arming type, attr:opt{req, string}-->1
      </deployType>
      <protocolType opt="HTTP,HTTPS">
        <!--ro, opt, enum, protocol type, subType:string, attr:opt{req, string}, desc:"HTTP", "HTTPS"-->HTTP
      </protocolType>
      <ipAddr min="1" max="10">
        <!--ro, req, string, IP address, attr:min{req, int},max{req, int}-->test
      </ipAddr>
      <port min="0" max="10">
        <!--ro, opt, int, port No., range:[1,65535], attr:min{req, int},max{req, int}-->1
      </port>
      <eventType opt="AccessController,Consumer,AccessControllerAndConsumer">
        <!--ro, opt, enum, subType:string, attr:opt{req, string}-->AccessController
      </eventType>
    </Content>
  </DeployList>
</DeployInfo>
```


10.3.1.9 Set the event card linkage parameters

Request URL

PUT /ISAPI/AccessControl/EventCardLinkageCfg/<ACEID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
ACEID	string	--

Request Message


```
{
    "EventCardLinkageCfg": {
    /*req, object, event card linkage parameters*/
        "proMode":  "event",
        /*req, enum, linkage type, subType:string, desc:"event" (event linkage), "card" (card linkage), "mac" (MAC address linkage), "employee" (employee No., i.e., person ID)*/
        "EventLinkageInfo": {
        /*opt, object, event linkage parameters, desc:it is valid when proMode is "event"*/
            "mainEventType":  0,
            /*opt, enum, major event type, subType:int, desc:0-device event, 1-alarm input event, 2-access control point event, 3-authentication unit (card reader, fingerprint module) event*/
            "subEventType":  54
            /*opt, int, sub event type, desc:minor event type,refer to Event Linkage Types for details*/
        },
        "eventSourceID":  1,
        /*opt, int, event source ID, desc:it is valid when proMode is "event". For device event (mainEventType is 0), this field is invalid; for access control point event (mainEventType is 2), this field refers to the access control point No.; for authentication unit event (mainEventType is 3), this field refers to the authentication unit No.; for alarm input event (mainEventType is 1), this field refers to the zone alarm input ID or the event alarm input ID 65535-all*/
        "mainDevBuzzer":  true,
        /*opt, bool, whether to enable buzzer linkage of the access controller (start buzzing):, desc:false-no, true-yes*/
        "mainDevStopBuzzer":  true,
        /*opt, bool, whether to enable buzzer linkage of access controller (stop buzzing), desc:false-no, true-yes*/
    }
}
```

Response Message

```
{
    "requestURL":  "test",
    /*ro, opt, string, URI*/
    "statusCode":  "test",
    /*ro, opt, string, status code*/
    "statusString":  "test",
    /*ro, opt, string, status description*/
    "subStatusCode":  "test",
    /*ro, opt, string, sub status code*/
    "errorCode":  1,
    /*ro, req, int, error code*/
    "errorMsg":  "ok"
    /*ro, req, string, error description*/
}
```

10.3.1.10 Get the event and card linkage configuration parameters

Request URL

GET /ISAPI/AccessControl/EventCardLinkageCfg/<ACEID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
ACEID	string	--

Request Message

None

Response Message

```
{
    "EventCardLinkageCfg": {
    /*ro, req, object*/
        "proMode":  "event",
        /*ro, req, enum, linkage type, subType:string, desc:"event"-event linkage, "card"-card linkage, "mac"-MAC address linkage, "employee"-employee No.
(person ID)*/
        "EventLinkageInfo": {
        /*ro, opt, object, event linage parameters, desc:it is valid when proMode is "event"*/
            "mainEventType":  0,
            /*ro, opt, enum, major event type, subType:int, desc:0-device event,1-alarm input event,2-access control point event,3-authentication unit (card
reader, fingerprint module) event*/
            "subEventType":  54
            /*ro, opt, int, event sub type, desc:minor event type,refer to Event Linkage Types for details*/
        },
        "eventSourceID":  1,
        /*ro, opt, int, event source ID, desc:it is valid when proMode is "event",65535-all. For device event (mainEventType is 0),this field is invalid;
for access control point event (mainEventType is 2),this field refers to the access control point No.; for authentication unit event (mainEventType is
3,this field refers to the authentication unit No.; for alarm input event (mainEventType is 1),this field refers to the zone alarm input ID or the event
alarm input ID*/
        "mainDevBuzzer":  true,
        /*ro, opt, bool, whether to enable buzzer linkage of the access controller (start buzzing), desc:"false"-no, "true"-yes*/
        "mainDevStopBuzzer":  true,
        /*ro, opt, bool, whether to enable buzzer linkage of access controller (stop buzzing), desc:"false"-no,"true"-yes*/
    }
}
```

10.3.1.11 Get the configuration capability of the event and card linkage

Request URL

GET /ISAPI/AccessControl/EventCardLinkageCfg/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "EventCardLinkageCfg": {
    /*ro, req, object, parameters of the event and card linkage*/
        "eventID": {
```

```
"eventID": {
/*ro, opt, object, event ID*/
    "@min":  1,
    /*ro, opt, int*/
    "@max":  1
    /*ro, opt, int*/
},
"proMode": {
/*ro, req, object, linkage type*/
    "@opt":  "event,card,mac,employee"
    /*ro, opt, string, linkage method*/
},
"EventLinkageInfo": {
/*ro, opt, object, event linkage information*/
    "mainEventType": {
    /*ro, opt, object, event main type*/
        "@opt":  "0,1,2,3"
        /*ro, opt, string, event main type*/
    },
    "devSubEventType": {
    /*ro, opt, object, minor event type*/
        "@opt":  "0,1,2,3,54…"
        /*ro, opt, string, minor event type*/
    },
    "alarmSubEventType": {
    /*ro, opt, object, minor type of alarm input event*/
        "@opt":  "0,1,2,3,52…"
        /*ro, opt, string, minor type of alarm input event*/
    },
    "doorSubEventType": {
    /*ro, opt, object, minor type of access control point event*/
        "@opt":  "0,1,2,3…"
        /*ro, opt, string, minor type of access control point event*/
    },
    "cardReaderSubEventType": {
    /*ro, opt, object, minor type of authentication unit event*/
        "@opt":  "0,1,2,3…"
        /*ro, opt, string, minor type of authentication unit event*/
    }
},
"CardNoLinkageInfo": {
/*ro, opt, object, card linkage parameters*/
    "cardNo": {
    /*ro, opt, object, card No.*/
        "@min":  1,
        /*ro, opt, int*/
        "@max":  32
        /*ro, opt, int*/
    }
},
"EmployeeInfo": {
/*ro, opt, object, person ID*/
    "employeeNo": {
    /*ro, opt, object, person ID*/
        "@min":  1,
        /*ro, opt, int, employee No. (person ID)*/
        "@max":  32
        /*ro, opt, int*/
    }
},
"eventSourceID": {
/*ro, opt, object, event source ID*/
    "@min":  1,
    /*ro, opt, int*/
    "@max":  1
    /*ro, opt, int*/
},
"alarmout": {
/*ro, opt, object, linked alarm output No.*/
    "@min":  1,
    /*ro, opt, int*/
    "@max":  1
    /*ro, opt, int*/
},
"openDoor": {
/*ro, opt, object, linked door No. to open*/
    "@min":  1,
    /*ro, opt, int*/
    "@max":  1
    /*ro, opt, int*/
},
"closeDoor": {
/*ro, opt, object, linked door No. to close*/
    "@min":  1,
    /*ro, opt, int*/
    "@max":  1
    /*ro, opt, int*/
},
"alwaysOpen": {
/*ro, opt, object, array,linked door No. to remain unlocked*/
    "@min":  1,
    /*ro, opt, int*/
    "@max":  1
    /*ro, opt, int*/
```

```
        },
        "alwaysClose": {
        /*ro, opt, object, linked door No. to remain locked*/
            "@min":  1,
            /*ro, opt, int*/
            "@max":  1
            /*ro, opt, int*/
        },
        "mainDevBuzzer":  "true,false",
        /*ro, opt, string, buzzer linkage of the access controller*/
        "mainDevStopBuzzer":  "true,false",
        /*ro, opt, string, whether to enable buzzer linkage of the access controller (stop buzzing): "false"-no,"true"-yes*/
        "readerBuzzer": {
        /*ro, opt, object, linked buzzer*/
            "@min":  1,
            /*ro, opt, int*/
            "@max":  1,
            /*ro, opt, int*/
        },
        "alarmOutClose": {
        /*ro, opt, object, array,linked alarm output No.*/
            "@min":  1,
            /*ro, opt, int*/
            "@max":  1
            /*ro, opt, int*/
        },
        "readerStopBuzzer": {
        /*ro, opt, object, linked buzzer No. to stop buzzing*/
            "@min":  1,
            /*ro, opt, int*/
            "@max":  1,
            /*ro, opt, int*/
        },
    }
}
```

10.3.1.12 Get the capability of the list of event and card linkage ID

Request URL

GET /ISAPI/AccessControl/EventCardNoList/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "EventCardNoList": {
    /*ro, opt, object*/
        "id": {
        /*ro, opt, object, range of event ID that can be configured*/
            "@min":  1,
            /*ro, opt, int*/
            "@max":  1
            /*ro, opt, int*/
        }
    }
}
```

10.3.1.13 Get the list of event and card linkage ID

Request URL

GET /ISAPI/AccessControl/EventCardNoList?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "EventCardNoList": {
    /*ro, opt, object*/
        "id": [1, 2, 3]
        /*ro, req, array, list of configured event and card linkage ID, subType:int, desc:[1, 2, 3] indicates that the device is configured with event
linkage 1, 2, and 3*/
    }
}
```

10.3.1.14 Get the configuration capability of event optimization

Request URL

GET /ISAPI/AccessControl/EventOptimizationCfg/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "EventOptimizationCfg": {
    /*ro, opt, object*/
        "enable":  "true,false",
        /*ro, opt, string, whether to enable event optimization*/
        "isCombinedLinkageEvents":  "true,false"
        /*ro, opt, string, whether to enable linked event combination*/
    }
}
```

10.3.1.15 Set the event optimization parameters

Request URL

PUT /ISAPI/AccessControl/EventOptimizationCfg?format=json

Query Parameter

None

Request Message

```
{
    "EventOptimizationCfg": {
    /*opt, object*/
        "enable":  true,
        /*opt, bool, whether to enable event optimization*/
        "isCombinedLinkageEvents":  true
        /*opt, bool, whether to enable linked event combination*/
    }
}
```

Response Message

```
{
    "requestURL":  "test",
    /*ro, opt, string, URI*/
    "statusCode":  "test",
    /*ro, opt, string, status code*/
    "statusString":  "test",
    /*ro, opt, string, status description*/
    "subStatusCode":  "test",
    /*ro, opt, string, sub status code*/
    "errorCode":  1,
    /*ro, req, int, error code*/
    "errorMsg":  "ok"
    /*ro, req, string, error details*/
}
```

10.3.1.16 Get the event optimization configuration parameters

Request URL

GET /ISAPI/AccessControl/EventOptimizationCfg?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "EventOptimizationCfg": {
    /*ro, opt, object*/
        "enable":  true,
        /*ro, opt, bool, whether to enable event optimization*/
        "isCombinedLinkageEvents":  true
        /*ro, opt, bool, whether to enable linked event combination*/
    }
}
```

10.3.1.17 Access control event

EventType:AccessControllerEvent

```
{
    "ipAddress":  "172.6.64.7",
    /*ro, req, string, IPv4 address of the device that triggers the alarm*/
    "ipv6Address":  "1080:0:0:0:8:800:200C:417A",
    /*ro, opt, string, IPv6 address of the device that triggers the alarm*/
    "portNo":  80,
    /*ro, opt, int, communication port No. of the device that triggers the alarm*/
    "protocol":  "HTTP",
    /*ro, opt, enum, transmission communication protocol type, subType:string, desc:when ISAPI protocol is transmitted via HCNetSDK, the channel No. is the video channel No. of private protocol. When ISAPI protocol is transmitted via EZ protocol, the channel No. is the video channel No. of EZ protocol. When ISAPI protocol is transmitted via ISUP, the channel No. is the video channel No. of ISUP*/
    "macAddress":  "01:17:24:45:D9:F4",
    /*ro, opt, string, MAC address*/
    "channelID":  1,
    /*ro, opt, int, channel No. of the device that triggers the alarm, desc:when ISAPI protocol is transmitted via HCNetSDK, the channel No. is the video channel No. of private protocol. When ISAPI protocol is transmitted via EZ protocol, the channel No. is the video channel No. of EZ protocol. When ISAPI protocol is transmitted via ISUP, the channel No. is the video channel No. of ISUP*/
    "dateTime":  "2004-05-03T17:30:08+08:00",
    /*ro, req, datetime, alarm trigger time*/
    "activePostCount":  1,
    /*ro, req, int, times that the same alarm has been uploaded, desc:times that the same alarm has been uploaded*/
    "eventType":  "AccessControllerEvent",
    /*ro, req, string, event type, desc:"AccessControllerEvent" (access control event)*/
    "eventState":  "active",
    /*ro, req, enum, event status, subType:string, desc:for durative event: "active" (valid event or event starts), "inactive" (invalid event or the event ends). For the heartbeat, the node value indicates the heartbeat data, and it is uploaded every 10 seconds*/
    "eventDescription":  "AccessControllerEvent",
    /*ro, req, string, event description, desc:"AccessControllerEvent" (access control event)*/
    "deviceID":  "test0123",
    /*ro, opt, string, device ID (PUID), desc:this node must be returned when ISAPI event information is transmitted via ISUP*/
    "AccessControllerEvent": {
    /*ro, req, object, access control event information*/
        "deviceName":  "test",
        /*ro, opt, string, device name*/
        "majorEventType":  1,
        /*ro, req, int, major alarm type, desc:the type value should be transformed to the decimal number; see Access Control Alarm Types for details*/
        "subEventType":  1,
        /*ro, req, int, minor alarm type, desc:the type value should be transformed to the decimal number; see Access Control Alarm Types for details*/
        "inductiveEventType":  "authenticated",
        /*ro, opt, enum, inductive event type, subType:string, desc:this node is used by storage devices; for access control devices, this node is invalid; "authenticated", "authenticationFailed", "openingDoor", "closingDoor", "doorException", "remoteOperation", "timeSynchronization", "deviceException", "deviceRecovered", "alarmTriggered", "alarmRecovered" (arming restoring event), "callCenter"*/
        "netUser":  "test",
        /*ro, opt, string, user name for network operations*/
        "remoteHostAddr":  "test",
        /*ro, opt, string, remote host address*/
        "cardNo":  "test",
        /*ro, opt, string, card No.*/
        "cardType":  1,
        /*ro, opt, enum, card type, subType:int, desc:1 (normal card), 2 (disability card), 3 (blocklist card), 4 (patrol card), 5 (duress card), 6 (super card), 7 (visitor card), 8 (dismiss card)*/
        "name":  "test",
        /*ro, opt, string, person name*/
        "sex":  "male",
        /*ro, opt, enum, subType:string, desc:"male", "female"*/
        "whiteListNo":  1,
        /*ro, opt, int, allowlist No.*/
        "reportChannel":  1,
        /*ro, opt, enum, channel type for uploading alarms/events, subType:int, desc:1 (uploading in arming mode), 2 (uploading by central group 1), 3 (uploading by central group 2)*/
        "cardReaderKind":  1,
```

```
        /*ro, opt, enum, card reader type, subType:int, desc:1 (IC card reader), 2 (ID card reader), 3 (QR code scanner), 4 (fingerprint module)*/
        "cardReaderNo":  1,
        /*ro, opt, int, card reader No., step:1, desc:card reader No.*/
        "doorNo":  1,
        /*ro, opt, int, door (floor) No.*/
        "verifyNo":  1,
        /*ro, opt, int, multiple authentication No.*/
        "alarmInNo":  1,
        /*ro, opt, int, alarm input No.*/
        "alarmOutNo":  1,
        /*ro, opt, int, alarm output No.*/
        "caseSensorNo":  1,
        /*ro, opt, int, event trigger No.*/
        "RS485No":  1,
        /*ro, opt, int, RS-485 channel No.*/
        "multiCardGroupNo":  1,
        /*ro, opt, int, group No.*/
        "accessChannel":  1,
        /*ro, opt, int, turnstile No.*/
        "deviceNo":  1,
        /*ro, opt, int, device No.*/
        "distractControlNo":  1,
        /*ro, opt, int, distributed access controller No.*/
        "employeeNo":  1,
        /*ro, opt, int, employee No. (person ID)*/
        "employeeNoString":  "test",
        /*ro, opt, string, employee No. (person ID), desc:if the node employeeNo exists or the value of employeeNoString can be converted to that of
employeeNo, this node is required. For the upper-layer platform or client software, the node employeeNoString will be parsed in priority; if
employeeNoString is not configured, the node employeeNo will be parsed*/
        "employeeName":  "test",
        /*ro, opt, string, person name, desc:this node is only used for FocSign products*/
        "localControllerID":  1,
        /*ro, opt, int, distributed access controller No., desc:0 (access controller), 1 to 64 (distributed access controller No. 1 to distributed access
controller No. 64)*/
        "InternetAccess":  1,
        /*ro, opt, enum, network interface No., subType:int, desc:1 (upstream network interface No. 1), 2 (upstream network interface No. 2), 3 (downstream
network interface No. 1)*/
        "type":  1,
        /*ro, opt, enum, zone type, subType:int, desc:0 (instant zone), 1 (24-hour zone), 2 (delayed zone), 3 (internal zone), 4 (key zone), 5 (fire alarm
zone), 6 (perimeter zone), 7 (24-hour silent zone), 8 (24-hour auxiliary zone), 9 (24-hour shock zone), 10 (emergency door open zone), 11 (emergency door
closed zone), 255 (none)*/
        "MACAddr":  "test",
        /*ro, opt, string, MAC address*/
        "swipeCardType":  1,
        /*ro, opt, enum, card swiping types, subType:int, desc:0 (invalid), 1 (QR code)*/
        "serialNo":  1,
        /*ro, opt, int, event serial No., range:[1,100000], desc:it starts at 1 and each record increases by 1. It will be overwritten repeatedly when
reaching the maximum value supported by the device*/
        "channelControllerID":  1,
        /*ro, opt, enum, lane controller ID, subType:int, desc:1 (main lane controller), 2 (sub-lane controller)*/
        "channelControllerLampID":  1,
        /*ro, opt, int, light board ID of lane controller, range:[1,255]*/
        "channelControllerIRAdaptorID":  1,
        /*ro, opt, int, IR adaptor ID of the lane controller, range:[1,255]*/
        "channelControllerIREmitterID":  1,
        /*ro, opt, int, active infrared intrusion detector No. of the lane controller, range:[1,255]*/
        "userType":  "normal",
        /*ro, opt, enum, person type, subType:string, desc:"normal" (normal person (resident)), "visitor" (visitor), "blacklist" (person in the blocklist),
"administrators" (administrator)*/
        "currentVerifyMode":  "cardAndPw",
        /*ro, opt, enum, current authentication mode of the card reader, subType:string, desc:"cardAndPw" (card+password), "card" (card), "cardOrPw" (card
or password), "fp" (fingerprint), "fpAndPw" (fingerprint+password), "fpOrCard" (fingerprint or card), "fpAndCard" (fingerprint+card), "fpAndCardAndPw"
(fingerprint+card+password), "faceOrFpOrCardOrPw" (face or fingerprint or card or password), "faceAndFp" (face+fingerprint), "faceAndPw" (face+password),
"faceAndCard" (face+card), "face" (face), "employeeNoAndPw" (employee No.+password), "fpOrPw" (fingerprint or password), "employeeNoAndFp" (employee
No.+fingerprint), "employeeNoAndFpAndPw" (employee No.+fingerprint+password), "faceAndFpAndCard" (face+fingerprint+card), "faceAndPwAndFp"
(face+password+fingerprint), "employeeNoAndFace" (employee No.+face), "faceOrfaceAndCard" (face or face+card), "fpOrface" (fingerprint or face),
"cardOrfaceOrPw" (card or face or password), "iris" (iris), "faceOrFpOrCardOrPwOrIris" (face, fingerprint, card, password, or iris), "faceOrCardOrPwOrIris"
(face, card, password, or iris)*/
        "currentEvent":  true,
        /*ro, opt, bool, whether it is a real-time event*/
        "QRCodeInfo":  "test",
        /*ro, opt, string, QR code information*/
        "thermometryResult":  "success",
        /*ro, opt, enum, temperature screening result, subType:string, desc:"success", "fail"*/
        "thermometryUnit":  "celsius",
        /*ro, opt, enum, temperature unit, subType:string, desc:"celsius" (Celsius, default value), "fahrenheit" (Fahrenheit), "kelvin" (Kelvin)*/
        "currTemperature":  36.1,
        /*ro, opt, float, skin-surface temperature, which is accurate to one decimal place*/
        "isAbnomalTemperature":  true,
        /*ro, opt, bool, whether the skin-surface temperature is abnormal*/
        "RegionCoordinates": {
        /*ro, opt, object, coordinates of the skin-surface temperature*/
            "positionX":  0,
            /*ro, opt, int, normalized X-coordinate which is between 0 and 1000, range:[0,1000]*/
            "positionY":  0
            /*ro, opt, int, normalized Y-coordinate which is between 0 and 1000, range:[0,1000]*/
        },
        "remoteCheck":  true,
        /*ro, opt, bool, whether remote verification is required: true-yes, false-no (default)*/
        "mask":  "unknown",
        /*ro, opt, enum, whether the person wears a mask, subType:string, desc:"unknown", "yes", "no"*/
        "frontSerialNo":  1,
        /*ro, opt, int, serial No. of the previous event, desc:if this node does not exist, the platform will check whether the event loss occurred
according to the node serialNo. If both the serialNo and frontSerialNo are returned, the platform will check whether the event loss occurred according to
```

```
according to the node serialNo. If both the serialNo and frontSerialNo are returned, the platform will check whether the event loss occurred according to
both nodes. It is mainly used to solve the problem that the serialNo is inconsistent after subscribing events or alarms*/
        "attendanceStatus":  "checkIn",
        /*ro, opt, enum, attendance status, subType:string, desc:"checkIn" (check-in), "checkOut" (check-out), "breakOut" (start of break), "breakIn" (end
of break), "overtimeIn" (start of overtime), "overTimeOut" (end of overtime)*/
        "label":  "test",
        /*ro, opt, string, self-defined attendance name*/
        "statusValue":  1,
        /*ro, opt, int, status value*/
        "pictureURL":  "test",
        /*ro, opt, string, URL of the captured picture, range:[0,256]*/
        "visibleLightURL":  "test",
        /*ro, opt, string, visible light picture URL of the thermal imaging camera, range:[0,256]*/
        "thermalURL":  "test",
        /*ro, opt, string, URL of the thermal picture, range:[0,256]*/
        "faceBasemapURL":  "test",
        /*ro, opt, string, range:[0,256]*/
        "picturesNumber":  1,
        /*ro, opt, int, number of captured pictures*/
        "unlockType":  "password",
        /*ro, opt, enum, unlocking type, subType:string, desc:this node is returned when the minor type is MINOR_UNCLOCK_RECORD; "password" (unlock by
password), "hijcking" (unlock by duress), "card" (unlock by card), "householder" (unlock by householder), "centerplatform" (unlock by management center),
"bluetooth" (unlock by bluetooth), "qrcode" (unlocked via QR code), "face" (unlock by recognizing face), "fingerprint" (unlock by fingerprint)*/
        "classroomId":  "test",
        /*ro, opt, string, class ID*/
        "classroomName":  "test",
        /*ro, opt, string, class name*/
        "analysisModule":  "signageApp",
        /*ro, opt, enum, analysis module, subType:string, desc:this node is not returned, and the value is report via signage App; "signageApp" (signage
App), "faceSDK" (face picture SDK)*/
        "customInfo":  "test",
        /*ro, opt, string, custom information*/
        "helmet":  "unknown",
        /*ro, opt, enum, whether the person is wearing hard hat, subType:string, desc:"unknown", "yes", "no"*/
        "purePwdVerifyEnable":  true,
        /*ro, opt, bool, whether the device supports opening the door only by password,
        desc:opening the door only by password:
        the password in authentication method is person password; checking the repetition of person password is not supported by the device, it should be
performed by the upper-layer platform; adding, deleting, editing, and searching for person password locally is not supported by the device*/
        "appType":  "attendance",
        /*ro, opt, enum, application type (for FocSign products), subType:string, desc:"attendance" (Time & Attendance module), "signIn" (Check-In module)*/
        "HealthInfo": {
        /*ro, opt, object, health information*/
            "healthCode":  1,
            /*ro, opt, enum, health code status, subType:int, desc:0 (no request), 1 (no health code), 2 (green QR code), 3 (yellow QR code), 4 (red QR
code), 5 (no such person), 6 (other error, e.g., searching failed due to API exception), 7 (searching for the health code timed out)*/
            "NADCode":  1,
            /*ro, opt, enum, nucleic acid test result, subType:int, desc:0 (no result), 1 (negative, which means normal), 2 (positive, which means
diagnosed), 3 (the result has expired)*/
            "NADMsg":  "test",
            /*ro, opt, string, range:[0,64]*/
            "NADTime":  1,
            /*ro, opt, enum, subType:int*/
            "travelCode":  1,
            /*ro, opt, enum, trip code, subType:int, desc:0 (no trip in the past 14 days), 1 (has left the current area left in the past 14 days), 2 (has
been to the high-risk area in the past 14 days), 3 (other)*/
            "travelInfo":  "test",
            /*ro, opt, string, trip information, desc:the empty string indicates that searching trip failed*/
            "vaccineStatus":  1,
            /*ro, opt, enum, whether the person is vaccinated, subType:int, desc:0 (not vaccinated), 1 (vaccinated)*/
            "vaccineNum":  1,
            /*ro, opt, int, step:1*/
            "vaccineMsg":  "test",
            /*ro, opt, string, range:[0,64]*/
            "ANTCode":  1,
            /*ro, opt, enum, subType:int*/
            "ANTMsg":  "test"
            /*ro, opt, string, range:[0,64]*/
        },
        "PhysicalInfo": {
        /*ro, opt, object, BMI information, desc:this node is obtained after authentication by BMI scales which is connected to MinMoe terminals*/
            "weight":  7000,
            /*ro, opt, int, weight, unit:kg*/
            "height":  18000
            /*ro, opt, int, height, unit:cm*/
        },
        "meetingID":  "test",
        /*ro, opt, string, meeting ID, range:[1,32]*/
        "PersonInfoExtends": [
        /*ro, opt, array, additional person information, subType:object, desc:this node displays additional person information on the device*/
            {
                "id":  1,
                /*ro, opt, int, extended ID of the additional person information, range:[1,32], desc:related URL: /ISAPI/AccessControl/personInfoExtendName?
format=json; this node is used for displaying the name of value; if ID does not exists, it starts from 1*/
                "value":  "test"
                /*ro, opt, string, extended content of the additional person information*/
            }
        ],
        "customPrompt":  "test",
        /*ro, opt, string, custom prompt message, range:[1,128], desc:this node is displayed when the authentication result is authenticated, authentication
failed, or stranger*/
        "FaceRect": {
        /*ro, opt, object, rectangle frame for human face, desc:the origin is the upper-left corner of the screen*/
            "height":  1.000,
```

```
            /*ro, req, float, height, range:[0.000,1.000]*/
            "width":  1.000,
            /*ro, req, float, width, range:[0.000,1.000]*/
            "x":  0.000,
            /*ro, req, float, X-coordinate of the upper-left corner of the frame, range:[0.000,1.000]*/
            "y":  0.000
            /*ro, req, float, Y-coordinate of the upper-left corner of the frame, range:[0.000,1.000]*/
        },
        "faceSimilarity":  90,
        /*ro, opt, int, Similarity, range:[0,100]*/
        "faceRecognitionDistance":  0.1,
        /*ro, opt, float, unit:m*/
        "eyesDistance":  20,
        /*ro, opt, int, range:[0,100], step:1*/
        "faceRecognitionFailedReason":  "attackBlacklist",
        /*ro, opt, enum, subType:string*/
        "currentAuthenticationTimes":  1,
        /*ro, opt, int, range:[0,255], step:1*/
        "allowAuthenticationTimes":  1,
        /*ro, opt, int, range:[0,255], step:1*/
        "LocalAttendanceData": {
        /*ro, opt, object*/
            "attendanceResult": [
            /*ro, opt, array, subType:object*/
                {
                    "date":  "1970-01-01",
                    /*ro, opt, date*/
                    "week":  1,
                    /*ro, opt, enum, subType:int, desc:1 (Monday), 2 (Tuesday), 3 (Wednesday), 4 (Thursday), 5 (Friday), 6 (Saturday), 7 (Sunday)*/
                    "personalAttendanceStatus":  "normal"
                    /*ro, opt, enum, subType:string*/
                }
            ]
        },
        "hasRecord":  true
        /*ro, opt, bool*/
    },
    "URLCertificationType":  "digest"
    /*ro, opt, enum, picture URL authentication method, subType:string, desc:"no" (no authentication, it is used for the cloud protocol), "digest" (digest authentication, it is used for local picture URL returned by NVR or DVR)*/
}
```

Parameter Name	Parameter Value	Parameter Type(Content-Type)	Content-ID	File Name	Description
AccessControllerEvent	[Message content]	application/json	--	--	--
Picture	[Binary picture data]	image/jpeg	pictureImage	Picture.jpg	--
VisibleLight	[Binary picture data]	image/jpeg	visibleLight_image	VisibleLight.jpg	--
Thermal	[Binary picture data]	image/jpeg	thermal_image	Thermal.jpg	--

Note: The protocol is transmitted in form format. See Chapter 4.5.1.4 for form framework description, as shown in the instance below.

```
--<frontier>
Content-Disposition: form-data; name=Parameter Name;filename=File Name
Content-Type: Parameter Type
Content-Length: ****
Content-ID: Content ID
Parameter Value
```

- Parameter Name: the name property of Content-Disposition in the header of form unit; it refers to the form unit name.
- Parameter Type (Content-Type): the Content-Type property in the header of form unit.
- File Name (filename): the filename property of Content-Disposition of form unit Headers. It exists only when the transmitted data of form unit is file, and it refers to the file name of form unit body.
- Parameter Value: the body content of form unit.

10.3.2 Access Control Schedule Management

10.3.2.1 Get the capability of clearing access control schedule configuration.

Request URL

GET /ISAPI/AccessControl/ClearPlansCfg/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "ClearPlansCfg": {
    /*ro, req, object*/
        "ClearFlags": {
        /*ro, opt, object*/
            "doorStatusWeekPlan":  "true,false",
            /*ro, opt, string, whether to clear the week schedule of the door control*/
            "cardReaderWeekPlan":  "true,false",
            /*ro, opt, string, whether to clear the week schedule of the card reader authentication mode control*/
            "userRightWeekPlan":  "true,false",
            /*ro, opt, string, whether to clear the week schedule of the access permission control*/
            "doorStatusHolidayPlan":  "true,false",
            /*ro, opt, string, whether to clear the holiday schedule of the door control*/
            "cardReaderHolidayPlan":  "true,false",
            /*ro, opt, string, whether to clear the holiday schedule of the card reader authentication mode control*/
            "userRightHolidayPlan":  "true,false",
            /*ro, opt, string, whether to clear the holiday schedule of the access permission control*/
            "doorStatusHolidayGroup":  "true,false",
            /*ro, opt, string, whether to clear the holiday group of the door control*/
            "cardReaderHolidayGroup":  "true,false",
            /*ro, opt, string, whether to clear the holiday group of the card reader authentication mode control*/
            "userRightHolidayGroup":  "true,false",
            /*ro, opt, string, whether to clear the holiday group of the access permission control*/
            "doorStatusTemplate":  "true,false",
            /*ro, opt, string, whether to clear the schedule template of the door control*/
            "cardReaderTemplate":  "true,false",
            /*ro, opt, string, whether to clear the control schedule template of the card reader authentication mode*/
            "userRightTemplate":  "true,false"
            /*ro, opt, string, whether to clear the schedule template of the access permission control*/
        }
    }
}
```


10.3.2.2 Clear access control schedule configuration parameters

Request URL

PUT /ISAPI/AccessControl/ClearPlansCfg?format=json

Query Parameter

None

Request Message

```
{
    "ClearPlansCfg": {
    /*opt, object*/
        "ClearFlags": {
        /*opt, object*/
            "doorStatusWeekPlan":  true,
            /*opt, bool, whether to clear the week schedule of the door control*/
            "cardReaderWeekPlan":  true,
            /*opt, bool, whether to clear the week schedule of the card reader authentication mode control*/
            "userRightWeekPlan":  true,
            /*opt, bool, whether to clear the week schedule of the access permission control*/
            "doorStatusHolidayPlan":  true,
            /*opt, bool, whether to clear the holiday schedule of the door control*/
            "cardReaderHolidayPlan":  true,
            /*opt, bool, whether to clear the holiday schedule of the card reader authentication mode control*/
            "userRightHolidayPlan":  true,
            /*opt, bool, whether to clear the holiday schedule of the access permission control*/
            "doorStatusHolidayGroup":  true,
            /*opt, bool, whether to clear the holiday group of the door control*/
            "cardReaderHolidayGroup":  true,
            /*opt, bool, whether to clear the holiday group of the card reader authentication mode control*/
            "userRightHolidayGroup":  true,
            /*opt, bool, whether to clear the holiday group of the access permission control*/
            "doorStatusTemplate":  true,
            /*opt, bool, whether to clear the schedule template of the door control*/
            "cardReaderTemplate":  true,
            /*opt, bool, whether to clear the control schedule template of card reader authentication mode*/
            "userRightTemplate":  true
            /*opt, bool, whether to clear the schedule template of access permission control*/
        }
    }
}
```

Response Message

```
{
    "statusCode":  1,
    /*ro, opt, int, status code, desc:1 (succeeded); it is required when an error occurred*/
    "statusString":  "ok",
    /*ro, opt, string, status description, range:[1,64], desc:"ok" (succeeded); it is required when an error occurred*/
    "subStatusCode":  "ok",
    /*ro, opt, string, sub status code, range:[1,64], desc:"ok" (succeeded); it is required when an error occurred*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:when the value of statusCode is not 1, it corresponds to subStatusCode*/
    "errorMsg":  "ok"
    /*ro, opt, string, error information, desc:this node must be returned when the value of statusCode is not 1*/
}
```

10.3.2.3 Set holiday group parameters of door control schedule

Request URL

PUT /ISAPI/AccessControl/DoorStatusHolidayGroupCfg/<holidayGroupID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
holidayGroupID	string	--

Request Message

```
{
    "DoorStatusHolidayGroupCfg": {
    /*opt, object*/
        "enable":  true,
        /*req, bool, whether to enable, desc:whether to enable*/
        "groupName":  "test",
        /*req, string, holiday group name*/
        "holidayPlanNo":  "1,3,5"
        /*opt, string*/
    }
}
```

Response Message

```
{
    "statusCode":  1,
    /*ro, opt, int, status code, desc:1 (succeeded); it is required when an error occurred*/
    "statusString":  "ok",
    /*ro, opt, string, status description, range:[1,64], desc:"ok" (succeeded); it is required when an error occurred*/
    "subStatusCode":  "ok",
    /*ro, opt, string, sub status code, range:[1,64], desc:"ok" (succeeded); it is required when an error occurred*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:when the value of statusCode is not 1, it corresponds to subStatusCode*/
    "errorMsg":  "ok"
    /*ro, opt, string, error information, desc:this node must be returned when the value of statusCode is not 1*/
}
```

10.3.2.4 Get the holiday group configuration parameters of the door control schedule

Request URL

GET /ISAPI/AccessControl/DoorStatusHolidayGroupCfg/<holidayGroupID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
holidayGroupID	string	--

Request Message

None

Response Message

```
{
    "DoorStatusHolidayGroupCfg": {
    /*ro, opt, object*/
        "enable":  true,
        /*ro, req, bool, whether to enable: "true"-enable, "false"-disable*/
        "groupName":  "test",
        /*ro, req, string, holiday group name*/
        "holidayPlanNo":  "1,3,5"
        /*ro, req, string*/
    }
}
```

10.3.2.5 Get the configuration capability of door status parameters of holiday group

Request URL

GET /ISAPI/AccessControl/DoorStatusHolidayGroupCfg/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "DoorStatusHolidayGroupCfg": {
    /*ro, opt, object*/
        "groupNo": {
        /*ro, opt, object, holiday group No.*/
            "@min":  1,
            /*ro, opt, int, the minimum value*/
            "@max":  16
            /*ro, opt, int, the maximum value*/
        },
        "enable":  "true,false",
        /*ro, opt, string, whether to enable, desc:true (enable)*/
        "groupName": {
        /*ro, opt, object, length of holiday group name*/
            "@min":  1,
            /*ro, opt, int, the minimum length*/
            "@max":  32
            /*ro, opt, int, the maximum length*/
        },
        "holidayPlanNo": {
        /*ro, opt, object, holiday group plan No.*/
            "@min":  1,
            /*ro, opt, int, the minimum value*/
            "@max":  16
            /*ro, opt, int, the maximum value*/
        }
    }
}
```

10.3.2.6 Get the configuration parameters of the door control holiday schedule

Request URL

GET /ISAPI/AccessControl/DoorStatusHolidayPlanCfg/<holidayPlanID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
holidayPlanID	string	--

Request Message

None

Response Message

```
{
    "DoorStatusHolidayPlanCfg": {
    /*ro, req, object*/
        "enable":  true,
        /*ro, req, bool, whether to enable, desc:true (enable), false (disable)*/
        "beginDate":  "2017-10-01",
        /*ro, req, date, start date of the holiday*/
        "endDate":  "2017-10-08",
        /*ro, req, date, end date of the holiday*/
        "HolidayPlanCfg": [
        /*ro, req, array, holiday schedule parameters, subType:object*/
            {
                "id":  1,
                /*ro, req, int, time period No., range:[1,8]*/
                "enable":  true,
                /*ro, req, bool, whether to enable, desc:true (enable), false (disable)*/
                "doorStatus":  "remainClosed",
                /*ro, req, enum, door status, subType:string, desc:“remainOpen”-remain open (access without authentication), “remainClosed”-remain closed (access is not allowed), “normal”-access by authentication, "sleep", "invalid”, “induction”, “barrierFree”*/
                "TimeSegment": {
                /*ro, opt, object, time*/
                    "beginTime":  "00:00:00",
                    /*ro, req, time, start time of the time period, desc:device local time*/
                    "endTime":  "10:00:00"
                    /*ro, req, time, end time of the time period, desc:device local time*/
                }
            }
        ]
    }
}
```

10.3.2.7 Set parameters of door control holiday schedule

Request URL

PUT /ISAPI/AccessControl/DoorStatusHolidayPlanCfg/<holidayPlanID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
holidayPlanID	string	--

Request Message

```
{
    "DoorStatusHolidayPlanCfg": {
    /*ro, req, object*/
        "enable":  true,
        /*ro, req, bool, whether to enable, desc:"true"-enable, "false"-disable*/
        "beginDate":  "2017-10-01",
        /*ro, req, date, start date of the holiday*/
        "endDate":  "2017-10-08",
        /*ro, req, date, end data of the holiday*/
        "HolidayPlanCfg": [
        /*ro, req, array, holiday schedule parameters, subType:object*/
            {
                "id":  1,
                /*ro, req, int, time period No., range:[1,8]*/
                "enable":  true,
                /*ro, req, bool, whether to enable, desc:"true"-enable, "false"-disable*/
                "doorStatus":  "remainClosed",
                /*ro, req, enum, door status, subType:string, desc:"remainOpen"-remain open (access without authentication), "remainClosed"-remain closed
(access is not allowed), "normal"-access by authentication, "sleep", "invalid”*/
                "TimeSegment": {
                /*ro, opt, object, time*/
                    "beginTime":  "00:00:00",
                    /*ro, req, time, start time, desc:device local time*/
                    "endTime":  "10:00:00"
                    /*ro, req, time, end time, desc:device local time*/
                }
            }
        ]
    }
}
```

Response Message

```
{
    "statusCode":  1,
    /*ro, opt, int, status code, desc:1 (succeeded); it is required when an error occurred*/
    "statusString":  "ok",
    /*ro, opt, string, status description, range:[1,64], desc:"ok" (succeeded); it is required when an error occurred*/
    "subStatusCode":  "ok",
    /*ro, opt, string, sub status code, range:[1,64], desc:"ok" (succeeded); it is required when an error occurred*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:when the value of statusCode is not 1, it corresponds to subStatusCode*/
    "errorMsg":  "ok"
    /*ro, opt, string, error information, desc:this node must be returned when the value of statusCode is not 1*/
}
```

10.3.2.8 Get the configuration capability of the door control holiday schedule

Request URL

GET /ISAPI/AccessControl/DoorStatusHolidayPlanCfg/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "DoorStatusHolidayPlanCfg": {
    /*ro, opt, object*/
        "planNo": {
        /*ro, opt, object, holiday schedule No.*/
            "@min":  1,
            /*ro, opt, int*/
            "@max":  16
            /*ro, opt, int*/
        },
        "enable":  "true,false",
        /*ro, opt, string, whether to enable: "true"-enable,"false"-disable, desc:"true" (enable), "false" (disable)*/
        "beginDate":  "1970-01-01",
        /*ro, opt, date, start date of the holiday*/
        "endDate":  "1971-01-01",
        /*ro, opt, date, end date of the holiday*/
        "HolidayPlanCfg": {
        /*ro, opt, object*/
            "maxSize":  8,
            /*ro, opt, int*/
            "id": {
            /*ro, opt, object, time period No.*/
                "@min":  1,
                /*ro, opt, int*/
                "@max":  8
                /*ro, opt, int*/
            },
            "enable":  "true,false",
            /*ro, opt, string, whether to enable: "true"-enable,"false"-disable, desc:"true" (enable), "false" (disable)*/
            "doorStatus": {
            /*ro, opt, object, door status*/
                "@opt":  "remainOpen,remainClosed,normal,sleep,invlid,induction,barrierFree"
                /*ro, opt, string, desc:"remainOpen" (remain open (access without authentication)), "remainClosed" (remain closed (access is not allowed)),
"normal" (access by authentication), "sleep", "invalid"*/
            },
            "TimeSegment": {
            /*ro, opt, object, time*/
                "beginTime":  "00:00:00",
                /*ro, opt, time, start time of the time period (device local time), desc:device local time*/
                "endTime":  "00:00:00",
                /*ro, opt, time, end time of the time period (device local time), desc:device local time*/
                "validUnit":  "minute"
                /*ro, opt, enum, time accuracy, subType:string, desc:"hour", "minute", "second"; if this node is not returned, the default time accuracy is
"minute"*/
            }
        }
    }
}
```

10.3.2.9 Get the configuration parameters of the door control schedule

Request URL

GET /ISAPI/AccessControl/DoorStatusPlan/<doorID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
doorID	string	--

Request Message

None

Response Message

```
{
    "DoorStatusPlan": {
    /*ro, req, object*/
        "templateNo":  1
        /*ro, req, int, schedule template No., desc:0-cancel linking the template with the schedule and restore to the default status (normal status)*/
    }
}
```

10.3.2.10 Set parameters of door control schedule

Request URL

PUT /ISAPI/AccessControl/DoorStatusPlan/<doorID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
doorID	string	--

Request Message

```
{
    "DoorStatusPlan": {
    /*req, object*/
        "templateNo":  1
        /*req, int, schedule template No., desc:0-cancel linking the template with the schedule and restore to the default status (normal status)*/
    }
}
```

Response Message

```
{
    "statusCode":  1,
    /*ro, opt, int, status code, desc:1 (succeeded); it is required when an error occurred*/
    "statusString":  "ok",
    /*ro, opt, string, status description, range:[1,64], desc:"ok" (succeeded); it is required when an error occurred*/
    "subStatusCode":  "ok",
    /*ro, opt, string, sub status code, range:[1,64], desc:"ok" (succeeded); it is required when an error occurred*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:when the value of statusCode is not 1, it corresponds to subStatusCode*/
    "errorMsg":  "ok"
    /*ro, opt, string, error information, desc:this node must be returned when the value of statusCode is not 1*/
}
```

10.3.2.11 Get the configuration capability of the door control schedule

Request URL

GET /ISAPI/AccessControl/DoorStatusPlan/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "DoorStatusPlan": {
    /*ro, opt, object*/
        "doorNo": {
        /*ro, opt, object, door No.*/
            "@min":  1,
            /*ro, opt, int*/
            "@max":  4
            /*ro, opt, int*/
        },
        "templateNo": {
        /*ro, opt, object, schedule template No.*/
            "@min":  1,
            /*ro, opt, int*/
            "@max":  16
            /*ro, opt, int*/
        }
    }
}
```

10.3.2.12 Set parameters of door control schedule template

Request URL

PUT /ISAPI/AccessControl/DoorStatusPlanTemplate/<planTemplateID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
planTemplateID	string	--

Request Message


```
{
    "DoorStatusPlanTemplate": {
    /*ro, opt, object, door control schedule template*/
        "enable":  true,
        /*ro, req, bool, whether to enable, desc:"true"-enable, "false"-disable*/
        "templateName":  "test",
        /*ro, req, string, template name*/
        "weekPlanNo":  1,
        /*ro, req, int, weekly schedule No.*/
        "holidayGroupNo":  "1,3,5"
        /*ro, req, string, holiday group No., desc:holiday group No.*/
    }
}
```


Response Message


```
{
    "statusCode":  1,
    /*ro, opt, int, status code, desc:1 (succeeded); it is required when an error occurred*/
    "statusString":  "ok",
    /*ro, opt, string, status description, range:[1,64], desc:"ok" (succeeded); it is required when an error occurred*/
    "subStatusCode":  "ok",
    /*ro, opt, string, sub status code, range:[1,64], desc:"ok" (succeeded); it is required when an error occurred*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:when the value of statusCode is not 1, it corresponds to subStatusCode*/
    "errorMsg":  "ok"
    /*ro, opt, string, error information, desc:this node must be returned when the value of statusCode is not 1*/
}
```


10.3.2.13 Get parameters of door control schedule template

Request URL

GET /ISAPI/AccessControl/DoorStatusPlanTemplate/<planTemplateID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
planTemplateID	string	--

Request Message

None

Response Message


```
{
    "DoorStatusPlanTemplate": {
    /*ro, opt, object, door control schedule template*/
        "enable":  true,
        /*ro, req, bool, whether to enable, desc:"true"-enable, "false"-disable*/
        "templateName":  "test",
        /*ro, req, string, template name*/
        "weekPlanNo":  1,
        /*ro, req, int, weekly schedule No.*/
        "holidayGroupNo":  "1,3,5"
        /*ro, req, string, holiday group No.*/
    }
}
```


10.3.2.14 Get the configuration capability of the door control schedule template

Request URL

GET /ISAPI/AccessControl/DoorStatusPlanTemplate/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "DoorStatusPlanTemplate": {
    /*ro, opt, object, schedule template*/
        "templateNo": {
        /*ro, opt, object, schedule template No.*/
            "@min":  1,
            /*ro, opt, int, the minimum value of schedule template No.*/
            "@max":  16
            /*ro, opt, int, the maximum value of schedule template No.*/
        },
        "enable":  "true,false",
        /*ro, opt, string, whether to enable, desc:true (enable), false (disable)*/
        "templateName": {
        /*ro, opt, object, template name length*/
            "@min":  1,
            /*ro, opt, int, the minimum value of template name length*/
            "@max":  32
            /*ro, opt, int, the maximum value of template name length*/
        },
        "weekPlanNo": {
        /*ro, opt, object, weekly schedule No.*/
            "@min":  1,
            /*ro, opt, int, the minimum value of weekly schedule No.*/
            "@max":  16
            /*ro, opt, int, the maximum value of weekly schedule No.*/
        },
        "holidayGroupNo": {
        /*ro, opt, object, holiday group No.*/
            "@min":  1,
            /*ro, opt, int, the minimum value of holiday group No.*/
            "@max":  16
            /*ro, opt, int, the maximum value of holiday group No.*/
        }
    }
}
```

10.3.2.15 Set parameters of door control weekly schedule

Request URL

PUT /ISAPI/AccessControl/DoorStatusWeekPlanCfg/<weekPlanID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
weekPlanID	string	--

Request Message

```
{
    "DoorStatusWeekPlanCfg": {
    /*opt, object, weekly schedule of door control*/
        "enable":  true,
        /*req, bool, whether to enable, desc:"true"-enable, "false"-disable*/
        "WeekPlanCfg": [
        /*req, array, weekly schedule parameters, subType:object*/
            {
                "week":  "Monday",
                /*req, enum, days of the week, subType:string, desc:"Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"*/
                "id":  1,
                /*req, int, time period No., range:[1,8]*/
                "enable":  true,
                /*req, bool, whether to enable*/
                "doorStatus":  "remainClosed",
                /*req, enum, door control schedule, subType:string, desc:"remainOpen"-remain open (access without authentication), "remainClosed"-remain
closed (access is not allowed), "normal"-access by authentication, "sleep", "invalid”*/
                "TimeSegment": {
                /*req, object, time*/
                    "beginTime":  "00:00:00",
                    /*req, time, start time, desc:device local time*/
                    "endTime":  "10:00:00"
                    /*req, time, end time, desc:device local time*/
                }
            }
        ]
    }
}
```

Response Message

```
{
    "statusCode":  1,
    /*ro, opt, int, status code, desc:1 (succeeded); it is required when an error occurred*/
    "statusString":  "ok",
    /*ro, opt, string, status description, range:[1,64], desc:"ok" (succeeded); it is required when an error occurred*/
    "subStatusCode":  "ok",
    /*ro, opt, string, sub status code, range:[1,64], desc:"ok" (succeeded); it is required when an error occurred*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:when the value of statusCode is not 1, it corresponds to subStatusCode*/
    "errorMsg":  "ok"
    /*ro, opt, string, error information, desc:this node must be returned when the value of statusCode is not 1*/
}
```

10.3.2.16 Get the configuration parameters of the door control week schedule

Request URL

GET /ISAPI/AccessControl/DoorStatusWeekPlanCfg/<weekPlanID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
weekPlanID	string	--

Request Message

None

Response Message

```
{
    "DoorStatusWeekPlanCfg": {
    /*ro, opt, object, door control week schedule*/
        "enable":  true,
        /*ro, req, bool, whether to enable, desc:true (enable), false (disable)*/
        "WeekPlanCfg": [
        /*ro, req, array, week schedule parameters, subType:object*/
            {
                "week":  "Monday",
                /*ro, req, enum, days of the week, subType:string, desc:"Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"*/
                "id":  1,
                /*ro, req, int, time period No., range:[1,8]*/
                "enable":  true,
                /*ro, req, bool, whether to enable: "true"-enable, "false"-disable*/
                "doorStatus":  "remainClosed",
                /*ro, req, enum, door status, subType:string, desc:"remainOpen"-remain open (access without authentication), "remainClosed"-remain closed
(access is not allowed), "normal"-access by authentication, "sleep","invalid", "induction", "barrierFree"*/
                "TimeSegment": {
                /*ro, req, object, time*/
                    "beginTime":  "00:00:00",
                    /*ro, req, time, start time of the time period, desc:device local time*/
                    "endTime":  "10:00:00"
                    /*ro, req, time, end time of the time period, desc:device local time*/
                }
            }
        ]
    }
}
```

10.3.2.17 Get the configuration capability of the door control week schedule

Request URL

GET /ISAPI/AccessControl/DoorStatusWeekPlanCfg/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "DoorStatusWeekPlanCfg": {
    /*ro, opt, object*/
        "planNo": {
        /*ro, opt, object, week schedule No.*/
            "@min":  1,
            /*ro, opt, int*/
            "@max":  16
            /*ro, opt, int*/
        },
        "enable":  "true,false",
        /*ro, opt, string, whether to enable: "true"-enable,"false"-disable, desc:"true" (enable), "false" (disable)*/
        "WeekPlanCfg": {
        /*ro, opt, object, week schedule parameters*/
            "maxSize":  56,
            /*ro, opt, int*/
            "week": {
            /*ro, opt, object*/
                "@opt":  "Monday,Tuesday,Wednesday,Thursday,Friday,Saturday,Sunday"
                /*ro, opt, string, desc:"Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"*/
            },
            "id": {
            /*ro, opt, object, weekly schedule No.*/
                "@min":  1,
                /*ro, opt, int*/
                "@max":  8
                /*ro, opt, int*/
            },
            "enable":  "true,false",
            /*ro, opt, string, whether to enable: "true"-enable,"false"-disable, desc:"true" (enable), "false" (disable)*/
            "doorStatus": {
            /*ro, opt, object, door status*/
                "@opt":  "remainOpen,remainClosed,normal,sleep,invalid,induction,barrierFree"
                /*ro, opt, string, desc:"remainOpen" (remain open (access without authentication)), "remainClosed" (remain closed (access is not allowed)),
"normal" (access by authentication), "sleep", "invalid"*/
            },
            "TimeSegment": {
            /*ro, opt, object, time*/
                "beginTime":  "00:00:00",
                /*ro, opt, time, start time of the time period (device local time), desc:device local time*/
                "endTime":  "10:00:00",
                /*ro, opt, time, end time of the time period (device local time), desc:device local time*/
                "validUnit":  "minute"
                /*ro, opt, enum, time accuracy, subType:string, desc:"hour", "minute", "second"; if this node is not returned, the default time accuracy is
"minute"*/
            }
        }
    }
}
```

10.3.3 Access Controller Management

10.3.3.1 Get the configuration capability of the access controller

Request URL

GET /ISAPI/AccessControl/AcsCfg/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "AcsCfg": {
    /*ro, req, object*/
        "voicePrompt":  "true,false",
        /*ro, opt, string, whether to enable voice prompt, desc:"true" (yes), "false" (no)*/
    }
}
```

10.3.3.2 Set the parameters of the access controller

Request URL

PUT /ISAPI/AccessControl/AcsCfg?format=json

Query Parameter

None

Request Message

```
{
    "AcsCfg": {
    /*req, object, parameters of the access controller*/
        "voicePrompt":  true,
        /*opt, bool, whether to enable voice prompt*/
    }
}
```

Response Message

```
{
    "statusCode":  1,
    /*ro, opt, int, status code, desc:status code*/
    "statusString":  "ok",
    /*ro, opt, string, status description, range:[1,64], desc:status description*/
    "subStatusCode":  "ok",
    /*ro, opt, string, sub status code, range:[1,64], desc:sub status code*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:error code*/
    "errorMsg":  "ok"
    /*ro, opt, string, error description, desc:error description*/
}
```

10.3.3.3 Get the configuration parameters of the access controller

Request URL

GET /ISAPI/AccessControl/AcsCfg?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "AcsCfg": {
    /*ro, req, object*/
        "RS485Backup":  true,
        /*ro, opt, bool, whether to enable downstream RS-485 communication redundancy*/
        "showCapPic":  true,
        /*ro, opt, bool, whether to display the captured picture*/
        "showUserInfo":  true,
        /*ro, opt, bool, whether to display user information*/
        "overlayUserInfo":  true,
        /*ro, opt, bool, whether to overlay user information*/
        "voicePrompt":  true,
        /*ro, opt, bool, whether to enable voice prompt*/
        "uploadCapPic":  true,
        /*ro, opt, bool, whether to upload the picture from linked capture, desc:whether to upload the picture from linked capture*/
        "saveCapPic":  true,
        /*ro, opt, bool, whether to save the captured picture, desc:whether to save the captured picture*/
        "inputCardNo":  true,
        /*ro, opt, bool, whether to allow inputting card No. on keypad*/
        "enableWifiDetect":  true,
        /*ro, opt, bool, whether to enable Wi-Fi probe*/
        "enable3G4G":  true,
        /*ro, opt, bool, whether to enable 3G/4G*/
        "protocol":  "Private",
        /*ro, opt, enum, communication protocol type of the card reader, subType:string, desc:"Private” (private protocol), "OSDP” (OSDP protocol)*/
        "enableCaptureCertificate":  true,
        /*ro, opt, bool, whether to enable capturing the ID picture, desc:true (yes), false (no). If this node does not exist, it indicates that this
function is not supported*/
        "showPicture":  true,
        /*ro, opt, bool, whether to display the authenticated picture, desc:whether to display the authenticated picture*/
        "showPresetPicture":  true,
        /*ro, opt, bool*/
        "showEmployeeNo":  true,
        /*ro, opt, bool, whether to display the authenticated employee ID*/
        "showName":  true,
        /*ro, opt, bool, whether to display the authenticated name*/
        "showPhoneNo":  true,
        /*ro, opt, bool*/
        "desensitiseEmployeeNo":  true,
        /*ro, opt, bool, whether to enable employee No. de-identification for local UI display, dep:or,{$.AcsCfg.showEmployeeNo,eq,true}*/
        "desensitiseName":  true
```

```
        "desensitiseName": true,
        /*ro, opt, bool, whether to enable name de-identification for local UI display, dep:or,{$.AcsCfg.showName,eq,true}*/
        "desensitisePhoneNo":  true,
        /*ro, opt, bool, dep:or,{$.AcsCfg.showPhoneNo,eq,true}*/
        "thermalEnabled":  true,
        /*ro, opt, bool, whether to enable temperature measurement*/
        "thermalMode":  true,
        /*ro, opt, bool, whether to enable temperature measurement only mode*/
        "thermalPictureEnabled":  true,
        /*ro, opt, bool, whether to enable uploading visible light pictures in temperature measurement only mode: true-enable,false-disable (default). This field is used to control uploading captured pictures and visible light pictures*/
        "thermalIp":  "192.168.1.1",
        /*ro, opt, string, IP address of the thermography device, desc:for access control devices, each device only requires one IP address; for metal detector doors, this field does not need to be configured*/
        "highestThermalThreshold":  37.3,
        /*ro, opt, float, upper limit of the temperature threshold*/
        "lowestThermalThreshold":  38.5,
        /*ro, opt, float, lower limit of the temperature threshold*/
        "thermalDoorEnabled":  false,
        /*ro, opt, bool, whether to open the door according to the temperature threshold, desc:whether to open the door when the temperature is above the upper limit (highestThermalThreshold) or below the lower limit (lowestThermalThreshold) of the threshold: true (open the door), false (not open the door (default))*/
        "QRCodeEnabled":  false,
        /*ro, opt, bool, whether to enable QR code function*/
        "remoteCheckDoorEnabled":  false,
        /*ro, opt, bool, whether to enable controlling the door by remote verification, desc:whether to enable controlling the door by remote verification*/
        "checkChannelType":  "Ezviz",
        /*ro, opt, enum, verification channel type, subType:string, dep:or,{$.AcsCfg.remoteCheckDoorEnabled,eq,true}, desc:verification channel type*/
        "channelIp":  "test",
        /*ro, opt, string, IP address of the verification channel, dep:and,{$.AcsCfg.checkChannelType,eq,PrivateSDK}, desc:this field is valid when checkChannelType is "PrivateSDK"*/
        "needDeviceCheck":  true,
        /*ro, opt, bool, dep:or,{$.AcsCfg.remoteCheckDoorEnabled,eq,true}*/
        "remoteCheckTimeout":  5,
        /*ro, opt, int, range:[1,10], unit:s, dep:or,{$.AcsCfg.remoteCheckDoorEnabled,eq,true}*/
        "remoteCheckVerifyMode":  1,
        /*ro, opt, enum, subType:int, dep:or,{$.AcsCfg.remoteCheckDoorEnabled,eq,true}*/
        "offlineDevCheckOpenDoorEnabled":  false,
        /*ro, opt, bool, dep:or,{$.AcsCfg.remoteCheckDoorEnabled,eq,true}*/
        "remoteCheckWithISAPIListen":  "async",
        /*ro, opt, enum, subType:string, dep:or,{$.AcsCfg.checkChannelType,eq,ISAPIListen}*/
        "uploadVerificationPic":  true,
        /*ro, opt, bool, whether to upload the authenticated picture, desc:whether to upload the authenticated picture*/
        "uploadVerificationPicType":  0,
        /*ro, opt, enum, subType:int*/
        "saveVerificationPic":  true,
        /*ro, opt, bool, whether to save the authenticated picture, desc:whether to save the authenticated picture*/
        "saveFacePic":  true,
        /*ro, opt, bool, whether to save the registered face picture, desc:whether to save the registered face picture*/
        "thermalUnit":  "celsius",
        /*ro, opt, enum, temperature unit, subType:string, desc:"celsius", "fahrenheit"*/
        "highestThermalThresholdF":  1.0,
        /*ro, opt, float, the maximum value of the temperature threshold, desc:the value is accurate to one decimal place, and the unit is Fahrenheit this node is used to check whether to open the door when the temperature is higher than the threshold*/
        "lowestThermalThresholdF":  1.0,
        /*ro, opt, float, the minimum value of the temperature threshold, desc:the value is accurate to one decimal place, and the unit is Fahrenheit this node is used to check whether to open the door when the temperature is higher than the threshold*/
        "enable5G":  true,
        /*ro, opt, bool, whether to enable 5G*/
        "thermalCompensation":  1.0,
        /*ro, opt, float, temperature compensation, desc:float,temperature compensation,the value is accurate to one decimal place. The unit depends on the node thermalUnit. If the node thermalUnit does not exist,the default unit is Celsius*/
        "externalCardReaderEnabled":  true,
        /*ro, opt, bool*/
        "combinationAuthenticationTimeout":  1,
        /*ro, opt, int, range:[1,20], step:1, unit:s*/
        "combinationAuthenticationLimitOrder":  true,
        /*ro, opt, bool*/
        "passwordEnabled":  true,
        /*ro, opt, bool*/
        "showGender":  true,
        /*ro, opt, bool, whether to display gender*/
        "showSignInTime":  true,
        /*ro, opt, bool*/
        "showsCustomInfo":  true,
        /*ro, opt, bool*/
        "showMobileWebQRCode":  true,
        /*ro, opt, bool*/
        "fireAlarmInputType":  "alwaysOpen",
        /*ro, opt, enum, subType:string*/
        "buzzerEnabled":  true,
        /*ro, opt, bool*/
        "saveVPAudioFile":  false,
        /*ro, opt, bool*/
        "saveVPAudioFileByAuth":  false,
        /*ro, opt, bool*/
        "faceDuplicateCheckEnabled":  false,
        /*ro, opt, bool*/
        "localControllerBackupMode":  1,
        /*ro, opt, enum, subType:int*/
        "maxlocalControllerNum":  64,
        /*ro, opt, int*/
        "saveFpPicByCollectionMode":  false,
        /*ro, opt, bool*/
```

```
        "faceBatchModelingMode":  "recognitionPriority",
        /*ro, opt, enum, subType:string*/
        "externalAuthResultDisplayEnabled":  false
        /*ro, opt, bool*/
    }
}
```

10.3.3.4 Get the capability of getting the working status of

the access controller **Request URL**

GET /ISAPI/AccessControl/AcsWorkStatus/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "AcsWorkStatus": {
    /*ro, req, object*/
        "doorLockStatus": {
        /*ro, opt, object, door lock status (relay status): 0-normally close, 1-normally open, 2-short-circuit alarm, 3-broken-circuit alarm, 4-exception
alarm*/
            "@opt":  "0,1,2,3,4"
            /*ro, opt, string*/
        },
        "doorStatus": {
        /*ro, opt, object, door (floor) status: 1-sleep, 2-remain unlocked (free), 3-remain locked (disabled), 4-normal status (controlled)*/
            "@opt":  "1,2,3,4"
            /*ro, opt, string*/
        },
        "magneticStatus": {
        /*ro, opt, object, magnetic contact status: 0-normally close,1-normally open, 2-short-circuit alarm, 3-broken-circuit alarm, 4-exception alarm*/
            "@opt":  "0,1,2,3,4"
            /*ro, opt, string, magnetic contact status*/
        },
        "powerSupplyStatus": {
        /*ro, opt, object, device power supply status: "ACPowerSupply"-alternative current, "BatteryPowerSupply"-storage battery power supply*/
            "@opt":  "ACPowerSupply,BatteryPowerSupply"
            /*ro, opt, string*/
        },
        "hostAntiDismantleStatus": {
        /*ro, opt, object, tampering status of the access control device: "close"-disabled, "open"-enabled*/
            "@opt":  "close,open"
            /*ro, opt, string*/
        },
        "cardReaderOnlineStatus": {
        /*ro, opt, object, online status of the authentication unit*/
            "@min":  1,
            /*ro, opt, int, the minimum value, range:[1,8]*/
            "@max":  1
            /*ro, opt, int, the maximum value, range:[1,8]*/
        },
        "cardReaderAntiDismantleStatus": {
        /*ro, opt, object, tampering status of the authentication unit*/
            "@min":  1,
            /*ro, opt, int*/
            "@max":  1
            /*ro, opt, int*/
        },
        "cardReaderVerifyMode": {
        /*ro, opt, object, current authentication mode of the authentication unit, desc:1-sleep,2-card+password,3-card,4-card or password,5-fingerprint,6-
fingerprint+password,7-fingerprint or card,8-fingerprint+card,9-fingerprint+card+password,10-face or fingerprint or card or password,11-face+fingerprint,12-
face+password,13-face+card,14-face,15-employee No.+password,16-fingerprint or password,17-employee No.+fingerprint,18-employee No.+fingerprint+password,19-
face+fingerprint+card,20-face+password+fingerprint,21-employee No.+face,22-face or face+card,23-fingerprint or face,24-card or face or password,25-card or
face,26-card or face or fingerprint,27-card or fingerprint or password*/
            "@opt":  "1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32,33,34"
            /*ro, opt, string*/
        },
        "alarmInStatus": {
        /*ro, opt, object, No. of input port with alarms*/
            "@min":  1,
            /*ro, opt, int*/
            "@max":  1
            /*ro, opt, int*/
        },
        "alarmOutStatus": {
        /*ro, opt, object, No. of output port with alarms*/
            "@min":  1,
            /*ro, opt, int*/
            "@max":  1
            /*ro, opt, int*/
        },
        "cardNum": {
        /*ro, opt, object, number of added cards*/
            "@min":  1,
            /*ro, opt, int*/
            "@max":  1
            /*ro, opt, int*/
        },
    }
}
```


10.3.3.5 Get the working status of the access controller

Request URL

GET /ISAPI/AccessControl/AcsWorkStatus?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "AcsWorkStatus": {
    /*ro, req, object*/
        "doorLockStatus":  [1, 2, 1, 2],
        /*ro, opt, enumarray, door lock status (relay status), subType:int, desc:door lock status (relay status): 0 (normally close), 1 (normally open), 2 (short-circuit alarm), 3 (broken-circuit alarm), 4 (exception alarm). For example, [1,2,1,2] indicates that door lock 1 is normally open, door lock 2 triggers short-circuit alarm, door lock 3 is normally open, and door lock 4 triggers short-circuit alarm*/
        "doorStatus":  [1, 2, 1, 2],
        /*ro, opt, enumarray, door (floor) status, subType:int, desc:door (floor) status: 1 (sleep), 2 (remain unlocked (free)), 3 (remain locked (disabled)), 4 (normal status (controlled)). For example, [1,2,1,2] indicates that door 1 is sleeping, door 2 remains unlocked, door 3 is sleeping, and door 4 remains unlocked*/
        "magneticStatus":  [1, 2, 1, 2],
        /*ro, opt, enumarray, magnetic contact status, subType:int, desc:magnetic contact status: 0 (normally close), 1 (normally open), 2 (short-circuit alarm), 3 (broken-circuit alarm), 4 (exception alarm). For example, [1,2,1,2] indicates that magnetic contact No.1 is normally open, magnetic contact No.2 triggers short-circuit alarm, magnetic contact No.3 is normally open, and magnetic contact No.4 triggers short-circuit alarm*/
        "caseStatus": [1, 3, 5],
        /*ro, opt, array, event trigger status, subType:int, desc:event trigger status, e.g., [1,3,5] indicates that event trigger No.1, No.3, and No.5 have input*/
        "batteryVoltage":  50,
        /*ro, opt, int, storage battery power voltage, desc:storage battery power voltage, the actual value will be 10 times of this value, unit: Volt*/
        "batteryLowVoltage":  false,
        /*ro, opt, bool, whether the storage battery is in low voltage status, desc:"true” (yes), "false” (no)*/
        "powerSupplyStatus":  "ACPowerSupply",
        /*ro, opt, enum, device power supply status, subType:string, desc:"ACPowerSupply” (alternative current), "BatteryPowerSupply” (storage battery power supply)*/
        "multiDoorInterlockStatus":  "close",
        /*ro, opt, enum, multi-door interlocking status, subType:string, desc:"close” (disabled), "open” (enabled)*/
        "antiSneakStatus":  "open",
        /*ro, opt, enum, anti-passback status, subType:string, desc:"close” (disabled), "open” (enabled)*/
        "hostAntiDismantleStatus":  "open",
        /*ro, opt, enum, tampering status of the access control device, subType:string, desc:"close” (disabled), "open” (enabled)*/
        "indicatorLightStatus":  "onLine",
        /*ro, opt, enum, indicator status, subType:string, desc:"offLine” (offline), "onLine” (online)*/
        "cardReaderOnlineStatus": [1, 3, 5],
        /*ro, opt, array, online status of the authentication unit, subType:int, desc:online status of the authentication unit, e.g., [1,3,5] indicates that authentication unit No.1, No.3, and No.5 are online*/
        "netReaderOnlineStatus": [1, 3, 5],
        /*ro, opt, array, subType:int*/
        "POEPortList": [
        /*ro, opt, array, subType:object, range:[1,8]*/
            {
                "port":  1,
                /*ro, req, int, range:[1,8]*/
                "readerID": [1, 2, 3]
                /*ro, opt, array, subType:int*/
            }
        ],
        "cardReaderAntiDismantleStatus": [1, 3, 5],
        /*ro, opt, array, tampering status of the authentication unit, subType:int, desc:tampering status of the authentication unit, e.g., [1,3,5] indicates that the tampering function of authentication unit No.1, No.3, and No.5 is enabled*/
        "cardReaderVerifyMode":  [3, 5, 3, 5],
        /*ro, opt, enumarray, current authentication mode of the authentication unit, subType:int, desc:1 (sleep), 2 (card + password), 3 (card), 4 (card or password), 5 (fingerprint), 6 (fingerprint + password), 7 (fingerprint or card), 8 (fingerprint + card), 9 (fingerprint + card + password),10 (face or fingerprint or card or password), 11 (face + fingerprint), 12 (face + password), 13 (face + card), 14 (face), 15 (employee No. + password), 16 (fingerprint or password), 17 (employee No. + fingerprint), 18 (employee No. + fingerprint + password), 19 (face + fingerprint + card), 20 (face + password + fingerprint), 21 (employee No. + face), 22 (face or face + card), 23 (fingerprint or face), 24 (card or face or password), 25 (card or face), 26 (card or face or fingerprint), 27 (card or fingerprint or password). For example, [3,5,3,5] indicates that the authentication mode of authentication unit 1 is "card", the authentication mode of authentication unit 2 is "fingerprint", the authentication mode of authentication unit 3 is "card", and the authentication mode of authentication unit 4 is "fingerprint"*/
        "setupAlarmStatus": [1, 3, 5],
        /*ro, opt, array, No. of armed input port, subType:int, desc:No. of armed input port, e.g., [1,3,5] indicates that input port No.1, No.3, and No.5 are armed*/
        "alarmInStatus": [1, 3, 5],
        /*ro, opt, array, No. of input port with alarms, subType:int, desc:No. of input port with alarms, e.g., [1,3,5] indicates that input port No.1, No.3, and No.5 trigger alarms*/
        "alarmOutStatus": [1, 3, 5],
        /*ro, opt, array, No. of output port with alarms, subType:int, desc:No. of output port with alarms, e.g., [1,3,5] indicates that output port No.1, No.3, and No.5 trigger alarms*/
        "cardNum":  3,
        /*ro, opt, int, number of added cards, desc:number of added cards*/
        "fireAlarmStatus":  "normal",
        /*ro, opt, enum, fire alarm status, subType:string, desc:fire alarm status: "normal", "shortCircuit” (short-circuit alarm), "brokenCircuit” (broken-circuit alarm)*/
        "batteryChargeStatus":  "charging",
        /*ro, opt, enum, battery charging status, subType:string, desc:battery charging status: "charging", "uncharged"*/
        "masterChannelControllerStatus":  "onLine",
        /*ro, opt, enum, online status of the main-lane controller, subType:string, desc:online status of the main lane controller: "offLine” (offline), "onLine” (online)*/
        "slaveChannelControllerStatus":  "onLine",
        /*ro, opt, enum, online status of the sub lane controller, subType:string, desc:online status of the sub lane controller: "offLine” (offline), "onLine” (online)*/
        "antiSneakServerStatus":  "normal",
        /*ro, opt, enum, anti-passback server status, subType:string, desc:anti-passback server status: "disable” (disabled), "normal", "disconnect” (disconnected)*/
        "netStatus":  "connect",
        /*ro, opt, enum, subType:string*/
        "InterfaceStatusList": [
        /*ro, opt, array, subType:object*/
```

```
        /*ro, opt, array, subType:object*/
            {
                "id":  1,
                /*ro, opt, int*/
                "netStatus":  "connect"
                /*ro, opt, enum, subType:string*/
            }
        ],
        "signalStatus":  "noSignal",
        /*ro, opt, enum, subType:string*/
        "sipStatus":  "connect",
        /*ro, opt, enum, subType:string*/
        "ezvizStatus":  "unregistered",
        /*ro, opt, enum, subType:string*/
        "voipStatus":  "unregistered",
        /*ro, opt, enum, subType:string*/
        "wifiStatus":  "connect",
        /*ro, opt, enum, subType:string*/
        "TFCardStatus":  "mounted",
        /*ro, opt, enum, subType:string*/
        "acrossHostInterlockStatus":  "disable",
        /*ro, opt, enum, subType:string*/
        "faceReceivingStatus":  "receiving",
        /*ro, opt, enum, subType:string*/
        "QRCodeReaderStatusList": [
        /*ro, opt, array, subType:object*/
            {
                "id":  1,
                /*ro, opt, int*/
                "QRCodeReaderStatus":  "onLine"
                /*ro, opt, enum, subType:string*/
            }
        ]
    }
}
```


10.3.4 Access Point Unit Management

10.3.4.1 Get the door configuration capability

Request URL

GET /ISAPI/AccessControl/Door/param/<doorID>/capabilities

Query Parameter

Parameter Name	Parameter Type	Description
doorID	string	--

Request Message

None

Response Message


```
<?xml version="1.0" encoding="UTF-8"?>

<DoorParam xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, attr:version{req, string, protocolVersion}-->
  <doorNo min="1" max="10">
    <!--ro, opt, int, door No., attr:min{req, int},max{req, int}-->1
  </doorNo>
  <doorName min="1" max="32">
    <!--ro, opt, string, door name, attr:min{req, int},max{req, int}-->test
  </doorName>
  <magneticType opt="alwaysClose,alwaysOpen">
    <!--ro, opt, enum, magnetic contact type, subType:string, attr:opt{req, string}, desc:"alwaysClose" (remain locked), "alwaysOpen" (remain unlocked)--
>alwaysClose
  </magneticType>
  <openButtonType opt="alwaysClose,alwaysOpen">
    <!--ro, opt, enum, door button type, subType:string, attr:opt{req, string}, desc:"alwaysClose" (remain locked), "alwaysOpen" (remain unlocked)--
>alwaysClose
  </openButtonType>
  <openDuration min="1" max="255">
    <!--ro, opt, int, door open duration (floor relay action time), attr:min{req, int},max{req, int}-->1
  </openDuration>
  <disabledOpenDuration min="1" max="255">
    <!--ro, opt, int, door open duration by disability card (delay duration of closing the door), attr:min{req, int},max{req, int}-->1
  </disabledOpenDuration>
  <magneticAlarmTimeout min="0" max="255">
    <!--ro, opt, int, alarm time of magnetic contact detection timeout, range:[0,255], unit:s, attr:min{req, int},max{req, int}, desc:alarm time of magnetic
contact detection timeout,which is between 0 and 255,0 refers to not triggering alarm,unit: second-->1
  </magneticAlarmTimeout>
  <enableDoorLock opt="true,false">
    <!--ro, opt, bool, whether to enable locking door when the door is closed, attr:opt{req, string}-->true
  </enableDoorLock>
```

```xml
    </enableDoorLock>
    <enableLeaderCard opt="true,false">
      <!--ro, opt, bool, whether to enable remaining open with first card, attr:opt{req, string}-->true
    </enableLeaderCard>
    <leaderCardMode opt="disable,alwaysOpen,authorize">
      <!--ro, opt, enum, first card mode, subType:string, attr:opt{req, string}, desc:"disable", "alwaysOpen" (remain open with first card), "authorize" (first card authentication)-->disable
    </leaderCardMode>
    <leaderCardOpenDuration min="1" max="1440">
      <!--ro, opt, int, duration of remaining open with first card, attr:min{req, int},max{req, int}-->1
    </leaderCardOpenDuration>
    <stressPassword min="1" max="8">
      <!--ro, opt, string, duress password, attr:min{req, int},max{req, int}, desc:the maximum length is 8 bytes, and the duress password should be encoded by Base64 for transmission-->test
    </stressPassword>
    <superPassword min="1" max="8">
      <!--ro, opt, string, super password, attr:min{req, int},max{req, int}, desc:the maximum length is 8 bytes, and the duress password should be encoded by Base64 for transmission-->test
    </superPassword>
    <unlockPassword min="1" max="8">
      <!--ro, opt, string, dismiss password,t, attr:min{req, int},max{req, int}, desc:the maximum length is 8 bytes, and the duress password should be encoded by Base64 for transmission-->test
    </unlockPassword>
    <useLocalController opt="true,false">
      <!--ro, opt, bool, whether it is connected to the distributed controller, attr:opt{req, string}-->true
    </useLocalController>
    <localControllerID min="0" max="64">
      <!--ro, opt, int, distributed controller No., range:[0,64], attr:min{req, int},max{req, int}, desc:ro,distributed controller No.,which is between 1 and 64,0-unregistered-->1
    </localControllerID>
    <localControllerDoorNumber min="0" max="4">
      <!--ro, opt, int, distributed controller door No., range:[0,4], attr:min{req, int},max{req, int}, desc:ro,distributed controller door No.,which is between 1 and 4,0-unregistered-->1
    </localControllerDoorNumber>
    <localControllerStatus opt="0,1,2,3,4,5,6,7,8,9">
      <!--ro, opt, enum, online status of the distributed controller, subType:int, attr:opt{req, string}, desc:0-offline, 1-network online, 2-RS-485 serial port 1 on loop circuit 1, 3-RS-485 serial port 2 on loop circuit 1, 4-RS-485 serial port 1 on loop circuit 2, 5-RS-485 serial port 2 on loop circuit 2, 6-RS-485 serial port 1 on loop circuit 3, 7-RS-485 serial port 2 on loop circuit 3,8-RS-485 serial port 1 on loop circuit 4,  9-RS-485 serial port 2 on loop circuit 4-->1
    </localControllerStatus>
    <lockInputCheck opt="true,false">
      <!--ro, opt, bool, whether to enable door lock input detection, attr:opt{req, string}-->true
    </lockInputCheck>
    <lockInputType opt="alwaysClose,alwaysOpen">
      <!--ro, opt, enum, door lock input type, subType:string, attr:opt{req, string}, desc:"alwaysClose" (remain locked), "alwaysOpen" (remain unlocked)-->alwaysClose
    </lockInputType>
    <doorTerminalMode opt="preventCutAndShort,preventCutAndShort,common">
      <!--ro, opt, enum, working mode of door terminal, subType:string, attr:opt{req, string}, desc:working mode of door terminal: "preventCutAndShort"-prevent from broken-circuit and short-circuit (default),"common"-->preventCutAndShort
    </doorTerminalMode>
    <openButton opt="true,false">
      <!--ro, opt, bool, whether to enable door button, attr:opt{req, string}, desc:whether to enable door button: "true"-yes (default),"false"-no-->true
    </openButton>
    <ladderControlDelayTime min="1" max="255">
      <!--ro, opt, int, elevator control delay time (for visitor), attr:min{req, int},max{req, int}-->1
    </ladderControlDelayTime>
    <Leader>
      <!--ro, opt, object-->
      <continuousVerificationTimes min="1" max="10">
        <!--ro, opt, int, range:[1,10], attr:min{req, int},max{req, int}-->1
      </continuousVerificationTimes>
      <continuousVerificationDuration min="1" max="10">
        <!--ro, opt, int, range:[5,60], unit:s, attr:min{req, int},max{req, int}-->20
      </continuousVerificationDuration>
      <effectiveTimeEnabled opt="true,false">
        <!--ro, req, bool, attr:opt{req, string}-->true
      </effectiveTimeEnabled>
      <dayBeginTime>
        <!--ro, opt, time, dep:and,{$.DoorParam.Leader.effectiveTimeEnabled,eq,true}-->00:00:00
      </dayBeginTime>
      <beginEffectiveTime>
        <!--ro, opt, time, dep:and,{$.DoorParam.Leader.effectiveTimeEnabled,eq,true}-->00:00:00
      </beginEffectiveTime>
      <endEffectiveTime>
        <!--ro, opt, time, dep:and,{$.DoorParam.Leader.effectiveTimeEnabled,eq,true}-->00:00:00
      </endEffectiveTime>
    </Leader>
    <verificationPassOpenDoor opt="true,false">
      <!--ro, opt, bool, attr:opt{req, string}-->true
    </verificationPassOpenDoor>
    <relayReverseEnabled opt="true,false" def="false">
      <!--ro, opt, bool, attr:opt{req, string},def{req, bool}-->true
    </relayReverseEnabled>
  </DoorParam>
```


10.3.4.2 Set the door (floor) parameters

Request URL

PUT /ISAPI/AccessControl/Door/param/<doorID>

Query Parameter

Parameter Name	Parameter Type	Description
doorID	string	--

Request Message


```xml
<?xml version="1.0" encoding="UTF-8"?>

<DoorParam xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--req, object, door parameter, attr:version{req, string, protocolVersion}-->
  <doorName>
    <!--opt, string, door name-->test
  </doorName>
  <magneticType>
    <!--opt, enum, door magnetic sensor type, subType:string, desc:"alwaysClose” (remain locked), "alwaysOpen” (remain unlocked)-->alwaysClose
  </magneticType>
  <openButtonType>
    <!--opt, enum, open door button type, subType:string, desc:"alwaysClose” (remain locked), "alwaysOpen” (remain unlocked)-->alwaysClose
  </openButtonType>
  <openDuration>
    <!--opt, int, door open duration, range:[1,255], unit:s-->1
  </openDuration>
  <disabledOpenDuration>
    <!--opt, int, door open duration by disability card (delay duration of closing the door), range:[1,255], unit:s-->1
  </disabledOpenDuration>
  <magneticAlarmTimeout>
    <!--opt, int, alarm time of magnetic contact detection timeout, range:[0,255], unit:s, desc:0 refers to not triggering alarm-->1
  </magneticAlarmTimeout>
  <enableDoorLock>
    <!--opt, bool, lock door when door closed-->true
  </enableDoorLock>
  <enableLeaderCard>
    <!--opt, bool, whether to enable remaining open with first card-->true
  </enableLeaderCard>
  <leaderCardMode>
    <!--opt, enum, first card mode, subType:string, desc:first card mode: "disable","alwaysOpen"-remain open with first card,"authorize"-first card
authentication. If this node is configured,the node <enableLeaderCard> is invalid-->disable
  </leaderCardMode>
  <leaderCardOpenDuration>
    <!--opt, int, duration of remaining open with first card, range:[0,1440], unit:min, dep:and,{$.DoorParam.leaderCardMode,eq,alwaysOpen}-->1
  </leaderCardOpenDuration>
  <stressPassword>
    <!--opt, string, duress password, desc:the maximum length is 8 bytes, and the duress password should be encoded by Base64 for transmission-->test
  </stressPassword>
  <superPassword>
    <!--opt, string, super password, desc:the maximum length is 8 bytes, and the duress password should be encoded by Base64 for transmission-->test
  </superPassword>
  <unlockPassword>
    <!--opt, string, unlock password, desc:the maximum length is 8 bytes, and the duress password should be encoded by Base64 for transmission-->test
  </unlockPassword>
  <useLocalController>
    <!--opt, bool, whether it is connected to the distributed controller-->true
  </useLocalController>
  <localControllerID>
    <!--opt, int, distributed controller No., range:[0,64], desc:ro,distributed controller No.,which is between 1 and 64,0-unregistered-->1
  </localControllerID>
  <localControllerDoorNumber>
    <!--opt, int, distributed controller door No., range:[0,4], desc:ro,distributed controller door No.,which is between 1 and 4,0-unregistered-->1
  </localControllerDoorNumber>
  <localControllerStatus>
    <!--opt, enum, online status of the distributed controller, subType:int, desc:ro,online status of the distributed controller: 0-offline,1-network
online,2-RS-485 serial port 1 on loop circuit 1,3-RS-485 serial port 2 on loop circuit 1,4-RS-485 serial port 1 on loop circuit 2,5-RS-485 serial port 2 on
loop circuit 2,6-RS-485 serial port 1 on loop circuit 3,7-RS-485 serial port 2 on loop circuit 3,8-RS-485 serial port 1 on loop circuit 4,9-RS-485 serial
port 2 on loop circuit 4-->1
  </localControllerStatus>
  <lockInputCheck>
    <!--opt, bool, whether to enable door lock input detection-->true
  </lockInputCheck>
  <lockInputType>
    <!--opt, enum, door lock input type, subType:string, desc:"alwaysClose” (remain locked), "alwaysOpen” (remain unlocked), Lt remains locked by default--
>alwaysClose
  </lockInputType>
  <doorTerminalMode>
    <!--opt, enum, working mode of door terminal, subType:string, desc:"preventCutAndShort” (prevent from broken-circuit and short-circuit
(default)),"common"-->preventCutAndShort
  </doorTerminalMode>
  <openButton>
    <!--opt, bool, whether to enable door button-->true
  </openButton>
  <ladderControlDelayTime>
    <!--opt, int, elevator control delay time (for visitor), range:[1,255], unit:min-->1
  </ladderControlDelayTime>
  <Leader>
    <!--opt, object-->
    <continuousVerificationTimes>
      <!--opt, int, range:[1,10]-->1
```

```
        <!--opt, int, range:[1,10]-->1
      </continuousVerificationTimes>
      <continuousVerificationDuration>
        <!--opt, int, range:[5,60], unit:s-->20
      </continuousVerificationDuration>
      <effectiveTimeEnabled>
        <!--req, bool-->true
      </effectiveTimeEnabled>
      <dayBeginTime>
        <!--opt, time, dep:and,{$.DoorParam.Leader.effectiveTimeEnabled,eq,true}-->00:00:00
      </dayBeginTime>
      <beginEffectiveTime>
        <!--opt, time, dep:and,{$.DoorParam.Leader.effectiveTimeEnabled,eq,true}-->00:00:00
      </beginEffectiveTime>
      <endEffectiveTime>
        <!--opt, time, dep:and,{$.DoorParam.Leader.effectiveTimeEnabled,eq,true}-->00:00:00
      </endEffectiveTime>
    </Leader>
    <verificationPassOpenDoor>
      <!--opt, bool-->true
    </verificationPassOpenDoor>
    <relayReverseEnabled>
      <!--opt, bool-->true
    </relayReverseEnabled>
  </DoorParam>
```

Response Message

```
<?xml version="1.0" encoding="UTF-8"?>

<ResponseStatus xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, response message, attr:version{ro, req, string, protocolVersion}-->
  <requestURL>
    <!--ro, req, string, request URL-->null
  </requestURL>
  <statusCode>
    <!--ro, req, enum, status code, subType:int, desc:0 (OK), 1 (OK), 2 (Device Busy), 3 (Device Error), 4 (Invalid Operation), 5 (Invalid XML Format), 6 
(Invalid XML Content), 7 (Reboot Required)-->0
  </statusCode>
  <statusString>
    <!--ro, req, enum, status description, subType:string, desc:“OK” (succeeded), “Device Busy”, “Device Error”, “Invalid Operation”, “Invalid XML Format”, 
“Invalid XML Content”, “Reboot” (reboot device)-->OK
  </statusString>
  <subStatusCode>
    <!--ro, req, string, error code description, desc:error code description-->OK
  </subStatusCode>
</ResponseStatus>
```


10.3.4.3 Get the door (floor) configuration parameters

Request URL

GET /ISAPI/AccessControl/Door/param/<doorID>

Query Parameter

Parameter Name	Parameter Type	Description
doorID	string	--

Request Message

None

Response Message

```
<?xml version="1.0" encoding="UTF-8"?>

<DoorParam xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, attr:version{req, string, protocolVersion}-->
  <doorName>
    <!--ro, opt, string, door name-->test
  </doorName>
  <magneticType>
    <!--ro, opt, enum, magnetic contact type, subType:string, desc:"alwaysClose"-remain locked, "alwaysOpen"-remain unlocked-->alwaysClose
  </magneticType>
  <openButtonType>
    <!--ro, opt, enum, door button type, subType:string, desc:"alwaysClose"-remain locked, "alwaysOpen"-remain unlocked-->alwaysClose
  </openButtonType>
  <openDuration>
    <!--ro, opt, int, door open duration, range:[1,255], unit:s-->1
  </openDuration>
  <disabledOpenDuration>
    <!--ro, opt, int, door open duration by disability card (delay duration of closing the door), range:[1,255], unit:s-->1
  </disabledOpenDuration>
```

```
  </disabledOpenDuration>
  <magneticAlarmTimeout>
    <!--ro, opt, int, alarm time of magnetic contact detection timeout, range:[0,255], unit:s, desc:0 refers to not triggering alarm-->1
  </magneticAlarmTimeout>
  <enableDoorLock>
    <!--ro, opt, bool, whether to enable locking door when the door is closed-->true
  </enableDoorLock>
  <enableLeaderCard>
    <!--ro, opt, bool, whether to enable remaining open with first card. This node is invalid when leaderCardMode is configured-->true
  </enableLeaderCard>
  <leaderCardMode>
    <!--ro, opt, enum, first card mode, subType:string, desc:"disable","alwaysOpen"-remain open with first card,"authorize"-first card authentication. If
this node is configured,the node <enableLeaderCard> is invalid-->disable
  </leaderCardMode>
  <leaderCardOpenDuration>
    <!--ro, opt, int, duration of remaining open with first card, range:[0,1440], unit:min, dep:and,{$.DoorParam.leaderCardMode,eq,alwaysOpen}-->1
  </leaderCardOpenDuration>
  <stressPassword>
    <!--ro, opt, string, duress password, desc:the maximum length is 8 bytes, and the duress password should be encoded by Base64 for transmission-->test
  </stressPassword>
  <superPassword>
    <!--ro, opt, string, super password, desc:wo,super password,the maximum length is 8 bytes,and the super password should be encoded by Base64 for
transmission-->test
  </superPassword>
  <unlockPassword>
    <!--ro, opt, string, dismiss password, desc:the maximum length is 8 bytes, and the dismiss password should be encoded by Base64 for transmission-->test
  </unlockPassword>
  <useLocalController>
    <!--ro, opt, bool, whether it is connected to the distributed controller-->true
  </useLocalController>
  <localControllerID>
    <!--ro, opt, int, distributed controller No., which is between 1 and 64, range:[0,64], desc:0-unregistered-->1
  </localControllerID>
  <localControllerDoorNumber>
    <!--ro, opt, int, distributed controller door No., range:[0,4], desc:0-unregistered-->1
  </localControllerDoorNumber>
  <localControllerStatus>
    <!--ro, opt, enum, online status of the distributed controller, subType:int, desc:0-offline,1-network online,2-RS-485 serial port 1 on loop circuit 1,3-
RS-485 serial port 2 on loop circuit 1,4-RS-485 serial port 1 on loop circuit 2,5-RS-485 serial port 2 on loop circuit 2,6-RS-485 serial port 1 on loop
circuit 3,7-RS-485 serial port 2 on loop circuit 3,8-RS-485 serial port 1 on loop circuit 4,9-RS-485 serial port 2 on loop circuit 4-->1
  </localControllerStatus>
  <lockInputCheck>
    <!--ro, opt, bool, whether to enable door lock input detection-->true
  </lockInputCheck>
  <lockInputType>
    <!--ro, opt, enum, door lock input type, subType:string, desc:"alwaysClose"-remain locked (default), "alwaysOpen"-remain unlocked-->alwaysClose
  </lockInputType>
  <doorTerminalMode>
    <!--ro, opt, enum, working mode of door terminal, subType:string, desc:"preventCutAndShort"-prevent from broken-circuit and short-circuit
(default),"common”-->preventCutAndShort
  </doorTerminalMode>
  <openButton>
    <!--ro, opt, bool, whether to enable door button: "true"-yes (default), "false"-no-->true
  </openButton>
  <ladderControlDelayTime>
    <!--ro, opt, int, elevator control delay time (for visitor), range:[1,255], unit:min-->1
  </ladderControlDelayTime>
  <Leader>
    <!--ro, opt, object-->
    <continuousVerificationTimes>
      <!--ro, opt, int, range:[1,10]-->1
    </continuousVerificationTimes>
    <continuousVerificationDuration>
      <!--ro, opt, int, range:[5,60], unit:s-->20
    </continuousVerificationDuration>
    <effectiveTimeEnabled>
      <!--ro, req, bool-->true
    </effectiveTimeEnabled>
    <dayBeginTime>
      <!--ro, opt, time, dep:and,{$.DoorParam.Leader.effectiveTimeEnabled,eq,true}-->00:00:00
    </dayBeginTime>
    <beginEffectiveTime>
      <!--ro, opt, time, dep:and,{$.DoorParam.Leader.effectiveTimeEnabled,eq,true}-->00:00:00
    </beginEffectiveTime>
    <endEffectiveTime>
      <!--ro, opt, time, dep:and,{$.DoorParam.Leader.effectiveTimeEnabled,eq,true}-->00:00:00
    </endEffectiveTime>
  </Leader>
  <verificationPassOpenDoor>
    <!--ro, opt, bool-->true
  </verificationPassOpenDoor>
  <relayReverseEnabled>
    <!--ro, opt, bool-->true
  </relayReverseEnabled>
</DoorParam>
```


10.3.4.4 Get the status of the secure door control unit

Request URL

GET /ISAPI/AccessControl/DoorSecurityModule/moduleStatus

Query Parameter

None

Request Message

None

Response Message


```
<?xml version="1.0" encoding="UTF-8"?>

<ModuleStatus xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, opt, object, module status, attr:version{req, string, protocolVersion}-->
  <securityModuleNo min="1" max="256">
    <!--ro, req, string, secure door control unit No., attr:min{req, int},max{req, int}-->test
  </securityModuleNo>
  <onlineStatus opt="0,1">
    <!--ro, req, enum, online status, subType:int, attr:opt{req, string}, desc:0 (offline), 1(online)-->1
  </onlineStatus>
  <desmantelStatus opt="0,1">
    <!--ro, req, enum, tamper-proof status, subType:int, attr:opt{req, string}, desc:0 (the unit is not tampered), 1(the unit is tampered)-->1
  </desmantelStatus>
</ModuleStatus>
```


10.3.4.5 Get the capability of getting the status of the secure door control unit

Request URL

GET /ISAPI/AccessControl/DoorSecurityModule/moduleStatus/capabilities

Query Parameter

None

Request Message

None

Response Message


```
<?xml version="1.0" encoding="UTF-8"?>

<ModuleStatus xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, opt, object, module status, attr:version{req, string, protocolVersion}-->
  <securityModuleNo min="1" max="256">
    <!--ro, req, string, secure door control unit No., attr:min{req, int},max{req, int}-->test
  </securityModuleNo>
  <onlineStatus opt="0,1">
    <!--ro, req, enum, online status, subType:int, attr:opt{req, string}, desc:0 (offline), 1 (online)-->1
  </onlineStatus>
  <desmantelStatus opt="0,1">
    <!--ro, req, enum, tampering status, subType:int, attr:opt{req, string}, desc:0 (the unit is not tampered), 1 (the unit is tampered)-->1
  </desmantelStatus>
</ModuleStatus>
```


10.3.5 Anti-Passback

10.3.5.1 Get the anti-passing back configuration capability

Request URL

GET /ISAPI/AccessControl/AntiSneakCfg/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "AntiSneakCfg": {
    /*ro, req, object*/
        "enable":  "true,false",
        /*ro, req, string, whether to enable anti-passing back*/
        "startCardReaderNo": {
        /*ro, opt, object, first card reader No., desc:first card reader No.*/
            "@min":  1,
            /*ro, opt, int, the minimum value*/
            "@max":  4,
            /*ro, opt, int, the maximum value*/
            "@opt": [1, 4]
            /*ro, opt, array, subType:int*/
        }
    }
}
```

10.3.5.2 Get the parameters of anti-passback configuration

Request URL

GET /ISAPI/AccessControl/AntiSneakCfg?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "AntiSneakCfg": {
    /*ro, req, object*/
        "enable":  true,
        /*ro, req, bool, whether to enable anti-passback*/
        "startCardReaderNo":  1
        /*ro, opt, int, first card reader No., desc:first card reader No.,0-no first card reader*/
    }
}
```

10.3.5.3 Set the anti-passing back parameters

Request URL

PUT /ISAPI/AccessControl/AntiSneakCfg?format=json

Query Parameter

None

Request Message

```
{
    "AntiSneakCfg": {
    /*req, object*/
        "enable":  true,
        /*req, bool, whether to enable anti-passing back*/
        "startCardReaderNo":  1
        /*opt, int, first card reader No., desc:0-no first card reader*/
    }
}
```

Response Message

```
{
    "requestURL":  "test",
    /*ro, opt, string, URI*/
    "statusCode":  1,
    /*ro, opt, int, status code, desc:1 (succeeded). It is required when an error occurred*/
    "statusString":  "ok",
    /*ro, opt, string, status description, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "subStatusCode":  "ok",
    /*ro, opt, string, sub status code, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:when the value of statusCode is not 1, it corresponds to subStatusCode*/
    "errorMsg":  "ok"
    /*ro, opt, string, error information, desc:this node is required when the value of statusCode is not 1*/
}
```

10.3.5.4 Get the anti-passing back configuration parameters of a specified card reader

Request URL

GET /ISAPI/AccessControl/CardReaderAntiSneakCfg/<cardReaderID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
cardReaderID	string	--

Request Message

None

Response Message

```
{
    "CardReaderAntiSneakCfg": {
    /*ro, req, object*/
        "enable":  true,
        /*ro, req, bool, whether to enable the anti-passing back function of the card reader, desc:"true" (enable), "false" (disable)*/
        "followUpCardReader": [2, 3, 4]
        /*ro, opt, array, following card reader No. after the first card reader, subType:int, desc:[2,3,4] indicates that card reader No. 2, No. 3, or No. 4
can be swiped after the first card reader*/
    }
}
```

10.3.5.5 Set anti-passing back parameters of a card reader

Request URL

PUT /ISAPI/AccessControl/CardReaderAntiSneakCfg/<cardReaderID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
cardReaderID	string	--

Request Message

```
{
    "CardReaderAntiSneakCfg": {
    /*req, object, anti-passing back parameters of a card reader*/
        "enable":  true,
        /*req, bool, whether to enable the anti-passing back function of the card reader, desc:"true"-enable, "false"-disable*/
        "followUpCardReader": [2, 3, 4]
        /*opt, array, following card reader No. after the first card reader, subType:int, desc:e.g., [2,3,4] indicates that card reader No. 2, No. 3, and
No. 4 can be swiped after the first card reader*/
    }
}
```

Response Message

```
{
    "requestURL":  "test",
    /*ro, opt, string, URI*/
    "statusCode":  1,
    /*ro, opt, int, status code, desc:1 (succeeded); it is required when an error occurred*/
    "statusString":  "ok",
    /*ro, opt, string, status description, range:[1,64], desc:"ok" (succeeded); it is required when an error occurred*/
    "subStatusCode":  "ok",
    /*ro, opt, string, sub status code, range:[1,64], desc:"ok" (succeeded); it is required when an error occurred*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:when the value of statusCode is not 1, it corresponds to subStatusCode*/
    "errorMsg":  "ok"
    /*ro, opt, string, error information, desc:this node is required when the value of statusCode is not 1*/
}
```

10.3.5.6 Get the configuration capability of anti-passing back parameters of card readers

Request URL

GET /ISAPI/AccessControl/CardReaderAntiSneakCfg/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "CardReaderAntiSneakCfg": {
    /*ro, req, object*/
        "cardReaderNo": {
        /*ro, opt, object, card reader No.*/
            "@min":  1,
            /*ro, opt, int, the minimum value*/
            "@max":  512,
            /*ro, opt, int, the maximum value*/
            "@opt": [1, 4]
            /*ro, opt, array, subType:int*/
        },
        "enable":  "true,false",
        /*ro, req, string, whether to enable the anti-passing back function of the card reader*/
        "followUpCardReader": {
        /*ro, opt, object, array,following card reader No. after the first card reader*/
            "@min":  1,
            /*ro, opt, int*/
            "@max":  512,
            /*ro, opt, int*/
            "@opt": [1, 4]
            /*ro, opt, array, subType:int*/
        }
    }
}
```

10.3.5.7 Get the capability of clearing anti-passback records

Request URL

GET /ISAPI/AccessControl/ClearAntiSneak/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "ClearAntiSneak": {
    /*ro, req, object*/
        "EmployeeNoList": {
        /*ro, opt, object*/
            "maxSize":  32,
            /*ro, opt, int*/
            "employeeNo": {
            /*ro, opt, object, employee No. (person ID)*/
                "@min":  1,
                /*ro, opt, int*/
                "@max":  32
                /*ro, opt, int*/
            }
        },
        "clearMode": {
        /*ro, opt, object*/
            "@opt": ["employeeNo", "all"]
            /*ro, opt, array, subType:string*/
        }
    }
}
```

10.3.5.8 Clear anti-passback records in the device

Request URL

PUT /ISAPI/AccessControl/ClearAntiSneak?format=json

Query Parameter

None

Request Message

```
{
    "ClearAntiSneak": {
    /*req, object, clear anti-passback records*/
        "EmployeeNoList": [
        /*opt, array, person ID list, subType:object, dep:and,{$.ClearAntiSneak.clearMode,eq,employeeNo}*/
            {
                "employeeNo":  "test"
                /*opt, string, employee No. (person ID)*/
            }
        ],
        "clearMode":  "employeeNo"
        /*opt, enum, clearing mode, subType:string*/
    }
}
```


Response Message

```
{
    "requestURL":  "test",
    /*ro, opt, string, URI*/
    "statusCode":  1,
    /*ro, opt, int, status code, desc:1 (succeeded). It is required when an error occurred*/
    "statusString":  "ok",
    /*ro, opt, string, status description, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "subStatusCode":  "ok",
    /*ro, opt, string, sub status code, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:it is required when the value of statusCode is not 1, and it corresponds to subStatusCode*/
    "errorMsg":  "ok"
    /*ro, opt, string, error information, desc:this node is required when the value of statusCode is not 1*/
}
```

10.3.5.9 Get the capability of clearing anti-passback parameters

Request URL

GET /ISAPI/AccessControl/ClearAntiSneakCfg/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "ClearAntiSneakCfg": {
    /*ro, req, object, the capability of clearing anti-passback*/
        "ClearFlags": {
        /*ro, req, object*/
            "antiSneak":  "true,false"
            /*ro, req, string, whether to clear the anti-passback parameter*/
        }
    }
}
```

10.3.5.10 Clear anti-passing back parameters

Request URL

PUT /ISAPI/AccessControl/ClearAntiSneakCfg?format=json

Query Parameter

None

Request Message

```
{
    "ClearAntiSneakCfg": {
    /*req, object*/
        "ClearFlags": {
        /*req, object*/
            "antiSneak":  true
            /*req, bool, whether to clear the anti-passing back parameters*/
        }
    }
}
```

Response Message

```
{
    "requestURL":  "test",
    /*ro, opt, string, URI*/
    "statusCode":  1,
    /*ro, opt, int, status code, desc:1 (succeeded); it is required when an error occurred*/
    "statusString":  "ok",
    /*ro, opt, string, status description, range:[1,64], desc:"ok" (succeeded); it is required when an error occurred*/
    "subStatusCode":  "ok",
    /*ro, opt, string, sub status code, range:[1,64], desc:"ok" (succeeded); it is required when an error occurred*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:when the value of statusCode is not 1, it corresponds to subStatusCode*/
    "errorMsg":  "ok"
    /*ro, opt, string, error information, desc:this node is required when the value of statusCode is not 1*/
}
```


10.3.6 Authentication Schedule Management

10.3.6.1 Set control schedule parameters of card reader authentication mode

Request URL

PUT /ISAPI/AccessControl/CardReaderPlan/<cardReaderID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
cardReaderID	string	--

Request Message

```
{
    "CardReaderPlan": {
    /*req, object*/
        "templateNo":  1
        /*req, int, schedule template No., desc:0-cancel linking the template to the schedule and restore to the default status (normal status)*/
    }
}
```

Response Message

```
{
    "statusCode":  1,
    /*ro, opt, int, status code, desc:1 (succeeded); it is required when an error occurred*/
    "statusString":  "ok",
    /*ro, opt, string, status description, range:[1,64], desc:"ok" (succeeded); it is required when an error occurred*/
    "subStatusCode":  "ok",
    /*ro, opt, string, sub status code, range:[1,64], desc:"ok" (succeeded); it is required when an error occurred*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:when the value of statusCode is not 1, it corresponds to subStatusCode*/
    "errorMsg":  "ok"
    /*ro, opt, string, error information, desc:this node must be returned when the value of statusCode is not 1*/
}
```

10.3.6.2 Get the control schedule configuration parameters of the card reader authentication mode

Request URL

GET /ISAPI/AccessControl/CardReaderPlan/<cardReaderID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
cardReaderID	string	--

Request Message

None

Response Message

```
{
    "CardReaderPlan": {
    /*ro, req, object, schedule template structure*/
        "templateNo":  1
        /*ro, req, int, schedule template number, desc:0-cancel linking the template to the schedule and restore to the default status (normal status)*/
    }
}
```


10.3.6.3 Get the control schedule configuration capability of the card reader authentication mode

Request URL

GET /ISAPI/AccessControl/CardReaderPlan/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "CardReaderPlan": {
    /*ro, opt, object, card reader No. node*/
        "cardReaderNo": {
        /*ro, opt, object, card reader No.*/
            "@min":  1,
            /*ro, opt, int*/
            "@max":  4
            /*ro, opt, int*/
        },
        "templateNo": {
        /*ro, opt, object, schedule template No. node*/
            "@min":  1,
            /*ro, opt, int*/
            "@max":  16,
            /*ro, opt, int*/
            "@opt": [65535, 65534, 65533]
            /*ro, opt, array, subType:int*/
        },
        "verifyMode": {
        /*ro, opt, object, authentication mode, dep:and,{$.CardReaderPlan.templateNo.@opt[*],eq,65535}*/
            "@opt":
"cardAndPw,card,cardOrPw,fp,fpAndPw,fpOrCard,fpAndCard,fpAndCardAndPw,faceOrFpOrCardOrPw,faceAndFp,faceAndPw,faceAndCard,face,employeeNoAndPw,fpOrPw,employe
eNoAndFp,employeeNoAndFpAndPw,faceAndFpAndCard,faceAndPwAndFp,employeeNoAndFace,faceOrfaceAndCard,fpOrface,cardOrfaceOrPw,cardOrFace,cardOrFaceOrFp,cardOrFp
OrPw,faceOrPw,employeeNoAndFaceAndPw,cardOrFaceOrFaceAndCard,iris,faceOrFpOrCardOrPwOrIris,faceOrCardOrPwOrIris,sleep,invalid"
            /*ro, opt, string, desc:"cardAndPw” (card + password), "card” (card), "cardOrPw” (card or password), "fp” (fingerprint), "fpAndPw” (fingerprint
+ password), "fpOrCard” (fingerprint or card), "fpAndCard” (fingerprint + card), "fpAndCardAndPw” (fingerprint + card + password), "faceOrFpOrCardOrPw”
(face or fingerprint or card or password), "faceAndFp” (face + fingerprint), "faceAndPw” (face + password), "faceAndCard” (face + card), "face” (face),
"employeeNoAndPw” (employee No. + password), "fpOrPw” (fingerprint or password), "employeeNoAndFp” (employee No. + fingerprint), "employeeNoAndFpAndPw”
(employee No. + fingerprint + password), "faceAndFpAndCard” (face + fingerprint + card), "faceAndPwAndFp” (face + password + fingerprint),
"employeeNoAndFace” (employee No. + face), "faceOrfaceAndCard” (face or face + card), "fpOrface” (fingerprint or face), "cardOrfaceOrPw” (card or face or
password), "cardOrFace” (card or face), "cardOrFaceOrFp” (card or face or fingerprint), "cardOrFpOrPw” (card or fingerprint or password), "faceOrPw" (face
or password), "employeeNoAndFaceAndPw" (employee No. + face + card), "cardOrFaceOrFaceAndCard" (card or face or face + card), "iris",
"faceOrFpOrCardOrPwOrIris"(face or fingerprint or card or password or iris), "faceOrCardOrPwOrIris" (face or  card or password or iris), "sleep",
"invalid"*/
        }
    }
}
```

10.3.6.4 Get the holiday group configuration parameters of the control schedule of the card reader authentication mode

Request URL

GET /ISAPI/AccessControl/VerifyHolidayGroupCfg/<holidayGroupID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
holidayGroupID	string	--

Request Message

None

Response Message

```
{
    "VerifyHolidayGroupCfg": {
    /*ro, opt, object*/
        "enable":  true,
        /*ro, req, bool, whether to enable, desc:true (yes), false-no (default)*/
        "groupName":  "test",
        /*ro, req, string, holiday group name*/
        "holidayPlanNo":  "1,3,5"
        /*ro, opt, string, holiday group schedule No., desc:holiday group schedule No.*/
    }
}
```

10.3.6.5 Set holiday group parameters of control schedule of card reader authentication mode

Request URL

PUT /ISAPI/AccessControl/VerifyHolidayGroupCfg/<holidayGroupID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
holidayGroupID	string	--

Request Message

```
{
    "VerifyHolidayGroupCfg": {
    /*opt, object, holiday group parameters of control schedule of card reader authentication mode*/
        "enable":  true,
        /*req, bool, whether to enable, desc:"true"-enable, "false"-disable*/
        "groupName":  "test",
        /*req, string, holiday group name*/
        "holidayPlanNo":  "1,3,5"
        /*opt, string, holiday group schedule No., desc:holiday group schedule No.*/
    }
}
```

Response Message

```
{
    "statusCode":  1,
    /*ro, opt, int, status code, desc:1 (succeeded); it is required when an error occurred*/
    "statusString":  "ok",
    /*ro, opt, string, status description, range:[1,64], desc:"ok" (succeeded); it is required when an error occurred*/
    "subStatusCode":  "ok",
    /*ro, opt, string, sub status code, range:[1,64], desc:"ok" (succeeded); it is required when an error occurred*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:when the value of statusCode is not 1, it corresponds to subStatusCode*/
    "errorMsg":  "ok"
    /*ro, opt, string, error information, desc:this node must be returned when the value of statusCode is not 1*/
}
```


10.3.6.6 Get the holiday group configuration capability of the control schedule of the card reader authentication mode

Request URL

GET /ISAPI/AccessControl/VerifyHolidayGroupCfg/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "VerifyHolidayGroupCfg": {
    /*ro, opt, object*/
        "groupNo": {
        /*ro, opt, object, holiday group No.*/
            "@min":  1,
            /*ro, opt, int, the minimum value*/
            "@max":  16
            /*ro, opt, int, the maximum value*/
        },
        "enable":  "true,false",
        /*ro, opt, string, whether to enable, desc:whether to enable: "true"-enable,"false"-disable*/
        "groupName": {
        /*ro, opt, object, length of holiday group name*/
            "@min":  1,
            /*ro, opt, int, the minimum length*/
            "@max":  32
            /*ro, opt, int, the maximum length*/
        },
        "holidayPlanNo": {
        /*ro, opt, object, holiday group schedule No.*/
            "@min":  1,
            /*ro, opt, int, the minimum value*/
            "@max":  16
            /*ro, opt, int, the maximum value*/
        }
    }
}
```

10.3.6.7 Get holiday schedule parameters of the card reader authentication mode

Request URL

GET /ISAPI/AccessControl/VerifyHolidayPlanCfg/<holidayPlanID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
holidayPlanID	string	--

Request Message

None

Response Message

```
{
    "VerifyHolidayPlanCfg": {
    /*ro, opt, object, holiday schedule parameters of the card reader authentication mode*/
        "enable":  true,
        /*ro, req, bool, whether to enable, desc:true (enable), false (disable)*/
        "beginDate":  "1970-01-01",
        /*ro, req, date, start date of the holiday, desc:device local time*/
        "endDate":  "1970-01-02",
        /*ro, req, date, end date of the holiday, desc:device local time*/
        "HolidayPlanCfg": [
        /*ro, req, array, holiday schedule parameters, subType:object*/
            {
                "id":  1,
                /*ro, req, int, time period No., range:[1,8]*/
                "enable":  true,
                /*ro, req, bool, whether to enable the holiday schedule, desc:true (enable), false (disable)*/
                "verifyMode":  "cardAndPw",
                /*ro, req, enum, authentication mode, subType:string, desc:"cardAndPw" (card+password), "card" (card), "cardOrPw" (card or password), "fp" (fingerprint), "fpAndPw" (fingerprint+password), "fpOrCard" (fingerprint or card), "fpAndCard" (fingerprint+card), "fpAndCardAndPw" (fingerprint+card+password), "faceOrFpOrCardOrPw" (face or fingerprint or card or password), "faceAndFp" (face+fingerprint), "faceAndPw" (face+password), "faceAndCard" (face+card), "face" (face), "employeeNoAndPw" (employee No.+password), "fpOrPw" (fingerprint or password), "employeeNoAndFp" (employee No.+fingerprint), "employeeNoAndFpAndPw" (employee No.+fingerprint+password), "faceAndFpAndCard" (face+fingerprint+card), "faceAndPwAndFp" (face+password+fingerprint), "employeeNoAndFace" (employee No.+face), "faceOrfaceAndCard" (face or face+card), "fpOrface" (fingerprint or face), "cardOrfaceOrPw" (card or face or password), "cardOrFace" (card or face), "cardOrFaceOrFp" (card or face or fingerprint), "cardOrFpOrPw" (card or fingerprint or password), "iris" (iris), "faceOrFpOrCardOrPwOrIris" (face or fingerprint or card or password or iris), "faceOrCardOrPwOrIris" (face or card or password or iris), "cardOrFace" (card or face), "cardOrFaceOrFp" (card or face or password), "faceOrPw" (face or password), "employeeNoAndFaceAndPw" (employee No.+face+password), "cardOrFaceOrFaceAndCard" (card or face or face+card), "faceOrFpOrPw" (face or fingerprint or password), "cardOrFpOrFaceOrIris" (card or fingerpriont or face or iris), "fpOrFaceOrIrisOrPw" (fingerprint or face or iris or password), "cardOrFpOrIrisOrPw" (card or fingerprint or iris or password), "cardOrIrisOrPw" (card or iris or password), "cardAndIris" (card+ris), "fpAndIri" (fingerprint+iris), "faceAndIris" (face+iris), "irisAndPw" (iris+password), "cardAndIrisAndPw" (card+iris+password), "faceAndIrisAndPw" (face+iris+password), "cardAndFaceAndIris" (card+face+iris)*/
                "TimeSegment": {
                /*ro, opt, object, time*/
                    "beginTime":  "00:00:00",
                    /*ro, req, time, start time of the time period, desc:device local time*/
                    "endTime":  "10:00:00"
                    /*ro, req, time, end time of the time period, desc:device local time*/
                }
            }
        ]
    }
}
```

10.3.6.8 Set holiday schedule parameters of card reader authentication mode

Request URL

PUT /ISAPI/AccessControl/VerifyHolidayPlanCfg/<holidayPlanID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
holidayPlanID	string	--

Request Message

```
{
    "VerifyHolidayPlanCfg": {
    /*opt, object, holiday schedule parameters of the card reader authentication mode*/
        "enable":  true,
        /*req, bool, whether to enable, desc:"true"-enable, "false"-disable*/
        "beginDate":  "1970-01-01",
        /*req, date, start date of the holiday, desc:device local time*/
        "endDate":  "1970-01-02",
        /*req, date, end date of the holiday, desc:device local time*/
        "HolidayPlanCfg": [
        /*req, array, holiday schedule parameters, subType:object*/
            {
                "id":  1,
                /*req, int, time period No., range:[1,8]*/
                "enable":  true,
                /*req, bool, whether to enable, desc:"true"-enable, "false"-disable*/
                "verifyMode":  "cardAndPw",
                /*req, enum, authentication mode, subType:string, desc:"cardAndPw" (card+password), "card" (card), "cardOrPw" (card or password), "fp"
(fingerprint), "fpAndPw" (fingerprint+password), "fpOrCard" (fingerprint or card), "fpAndCard" (fingerprint+card), "fpAndCardAndPw"
(fingerprint+card+password), "faceOrFpOrCardOrPw" (face or fingerprint or card or password), "faceAndFp" (face+fingerprint), "faceAndPw" (face+password),
"faceAndCard" (face+card), "face" (face), "employeeNoAndPw" (employee No.+password), "fpOrPw" (fingerprint or password), "employeeNoAndFp" (employee
No.+fingerprint), "employeeNoAndFpAndPw" (employee No.+fingerprint+password), "faceAndFpAndCard" (face+fingerprint+card), "faceAndPwAndFp"
(face+password+fingerprint), "employeeNoAndFace" (employee No.+face), "faceOrfaceAndCard" (face or face+card), "fpOrface" (fingerprint or face),
"cardOrfaceOrPw" (card or face or password), "cardOrFace" (card or face), "cardOrFaceOrFp" (card or face or fingerprint), "cardOrFpOrPw" (card or
fingerprint or password), "iris" (iris), "faceOrFpOrCardOrPwOrIris" (face or fingerprint or card or password or iris), "faceOrCardOrPwOrIris" (face or card
or password or iris), "cardOrFace" (card or face), "cardOrFaceOrFp" (card or face or password), "faceOrPw" (face or password), "employeeNoAndFaceAndPw"
(employee No.+face+password), "cardOrFaceOrFaceAndCard" (card or face or face+card), "faceOrFpOrPw" (face or fingerprint or password),
"cardOrFpOrFaceOrIris" (card or fingerpriont or face or iris), "fpOrFaceOrIrisOrPw" (fingerprint or face or iris or password), "cardOrFpOrIrisOrPw" (card or
fingerprint or iris or password), "cardOrIrisOrPw" (card or iris or password), "cardAndIris" (card+ris), "fpAndIri" (fingerprint+iris), "faceAndIris"
(face+iris), "irisAndPw" (iris+password), "cardAndIrisAndPw" (card+iris+password), "faceAndIrisAndPw" (face+iris+password), "cardAndFaceAndIris"
(card+face+iris)*/
                "TimeSegment": {
                /*opt, object, time*/
                    "beginTime":  "00:00:00",
                    /*req, time, start time of the time period, desc:device local time*/
                    "endTime":  "10:00:00"
                    /*req, time, end time of the time period, desc:device local time*/
                }
            }
        ]
    }
}
```

Response Message

```
{
    "statusCode":  1,
    /*ro, opt, int, status code, desc:1 (succeeded); it is required when an error occurred*/
    "statusString":  "OK",
    /*ro, opt, string, status description, range:[1,64], desc:"ok" (succeeded); it is required when an error occurred*/
    "subStatusCode":  "ok",
    /*ro, opt, string, sub status code, range:[1,64], desc:"ok" (succeeded); it is required when an error occurred*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:when the value of statusCode is not 1, it corresponds to subStatusCode*/
    "errorMsg":  "ok"
    /*ro, opt, string, error information, desc:this node must be returned when the value of statusCode is not 1*/
}
```


10.3.6.9 Get the holiday schedule configuration capability of the card reader authentication mode

Request URL

GET /ISAPI/AccessControl/VerifyHolidayPlanCfg/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "VerifyHolidayPlanCfg": {
    /*ro, opt, object, holiday schedule of card reader authentication mode*/
        "planNo": {
        /*ro, opt, object, holiday schedule template No.*/
            "@min":  1,
            /*ro, opt, int, maximum value*/
            "@max":  16
            /*ro, opt, int, minimum value*/
        },
        "enable":  "true,false",
        /*ro, opt, string, whether to enable, desc:true (enable), false (disable)*/
        "beginDate":  "1970-01-01",
        /*ro, opt, date, start date of the holiday (device local time), desc:(device local time)*/
        "endDate":  "1970-01-02",
        /*ro, opt, date, end date of the holiday (device local time), desc:(device local time)*/
        "HolidayPlanCfg": {
        /*ro, opt, object, holiday schedule parameters*/
            "maxSize":  8,
            /*ro, opt, int, maximum value*/
            "id": {
            /*ro, opt, object, time period No.*/
                "@min":  1,
                /*ro, opt, int, minimum value*/
                "@max":  8
                /*ro, opt, int, maximum value*/
            },
            "enable":  "true,false",
            /*ro, opt, string, whether to enable, desc:true (enable), false (disable)*/
            "verifyMode": {
            /*ro, opt, object, authentication mode*/
                "@opt":
"cardAndPw,card,cardOrPw,fp,fpAndPw,fpOrCard,fpAndCard,fpAndCardAndPw,faceOrFpOrCardOrPw,faceAndFp,faceAndPw,faceAndCard,face,employeeNoAndPw,fpOrPw,employe
eNoAndFp,employeeNoAndFpAndPw,faceAndFpAndCard,faceAndPwAndFp,employeeNoAndFace,faceOrfaceAndCard,fpOrface,cardOrfaceOrPw,cardOrFace,cardOrFaceOrFp,faceOrPw
,employeeNoAndFaceAndPw,cardOrFaceOrFaceAndCard,iris,faceOrFpOrCardOrPwOrIris,faceOrCardOrPwOrIris,sleep,invalid,cardOrFpOrFaceOrIris,fpOrFaceOrIrisOrPw,car
dOrFpOrIrisOrPw,cardOrIrisOrPw,cardAndIris,fpAndIris,faceAndIris,irisAndPw,cardAndIrisAndPw,faceAndIrisAndPw,cardAndFaceAndIris,Pw,cardOrFpOrPw,faceOrFpOrPw
,cardAndVp,faceAndVp,fpAndVp,pwAndVp"
                /*ro, opt, string, authentication mode, desc:"cardAndPw" (card+password), "card" (card), "cardOrPw" (card or password), "fp" (fingerprint),
"fpAndPw" (fingerprint+password), "fpOrCard" (fingerprint or card), "fpAndCard" (fingerprint+card), "fpAndCardAndPw" (fingerprint+card+password),
"faceOrFpOrCardOrPw" (face or fingerprint or card or password), "faceAndFp" (face+fingerprint), "faceAndPw" (face+password), "faceAndCard" (face+card),
"face" (face), "employeeNoAndPw" (employee No.+password), "fpOrPw" (fingerprint or password), "employeeNoAndFp" (employee No.+fingerprint),
"employeeNoAndFpAndPw" (employee No.+fingerprint+password), "faceAndFpAndCard" (face+fingerprint+card), "faceAndPwAndFp" (face+password+fingerprint),
"employeeNoAndFace" (employee No.+face), "faceOrfaceAndCard" (face or face+card), "fpOrface" (fingerprint or face), "cardOrfaceOrPw" (card or face or
password), "cardOrFace" (card or face), "cardOrFaceOrFp" (card or face or fingerprint), "cardOrFpOrPw" (card or fingerprint or password), "iris" (iris),
"faceOrFpOrCardOrPwOrIris" (face or fingerprint or card or password or iris), "faceOrCardOrPwOrIris" (face or card or password or iris), "cardOrFace" (card
or face), "cardOrFaceOrFp" (card or face or password), "faceOrPw" (face or password), "employeeNoAndFaceAndPw" (employee No.+face+password),
"cardOrFaceOrFaceAndCard" (card or face or face+card), "faceOrFpOrPw" (face or fingerprint or password), "cardOrFpOrFaceOrIris" (card or fingerpriont or
face or iris), "fpOrFaceOrIrisOrPw" (fingerprint or face or iris or password), "cardOrFpOrIrisOrPw" (card or fingerprint or iris or password),
"cardOrIrisOrPw" (card or iris or password), "cardAndIris" (card+ris), "fpAndIri" (fingerprint+iris), "faceAndIris" (face+iris), "irisAndPw"
(iris+password), "cardAndIrisAndPw" (card+iris+password), "faceAndIrisAndPw" (face+iris+password), "cardAndFaceAndIris" (card+face+iris)*/
            },
            "TimeSegment": {
            /*ro, opt, object, time*/
                "beginTime":  "00:00:00",
                /*ro, opt, time, start time, desc:(device local time)*/
                "endTime":  "10:00:00",
                /*ro, opt, time, end time, desc:(device local time)*/
                "validUnit":  "minute"
                /*ro, opt, enum, time accuracy, subType:string, desc:"hour", "minute", "second". If this node is not returned, the default time accuracy is
"minute"*/
            }
        },
        "purePwdVerifyEnable":  true
        /*ro, opt, bool*/
    }
}
```

10.3.6.10 Get the schedule template parameters of card reader authentication mode

Request URL

GET /ISAPI/AccessControl/VerifyPlanTemplate/<planTemplateID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
planTemplateID	string	--

Request Message

None

Response Message

```
{
    "VerifyPlanTemplate": {
    /*ro, opt, object, the schedule template parameters of card reader authentication mode*/
        "enable":  true,
        /*ro, req, bool, whether to enable, desc:"true"-enable, "false"-disable*/
        "templateName":  "test",
        /*ro, req, string, template name*/
        "weekPlanNo":  1,
        /*ro, req, int, week schedule No.*/
        "holidayGroupNo":  "1,3,5"
        /*ro, req, string, holiday group No., desc:holiday group No.*/
    }
}
```

10.3.6.11 Set the schedule template parameters of the card reader authentication mode

Request URL

PUT /ISAPI/AccessControl/VerifyPlanTemplate/<planTemplateID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
planTemplateID	string	--

Request Message

```
{
    "VerifyPlanTemplate": {
    /*opt, object*/
        "enable":  true,
        /*req, bool, whether to enable, desc:true (yes), false (no)*/
        "templateName":  "test",
        /*req, string, template name*/
        "weekPlanNo":  1,
        /*req, int, week schedule No.*/
        "holidayGroupNo":  "1,3,5"
        /*req, string, holiday group No., desc:holiday group No.*/
    }
}
```

Response Message

```
{
    "statusCode":  1,
    /*ro, opt, int, status code, desc:1 (succeeded). It is required when an error occurred*/
    "statusString":  "ok",
    /*ro, opt, string, status description, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "subStatusCode":  "ok",
    /*ro, opt, string, sub status code, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:it is required when the value of statusCode is not 1, and it corresponds to subStatusCode*/
    "errorMsg":  "ok"
    /*ro, opt, string, error information, desc:this field is required when the value of statusCode is not 1*/
}
```

10.3.6.12 Get the schedule template configuration capability of the card reader authentication mode

Request URL

GET /ISAPI/AccessControl/VerifyPlanTemplate/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "VerifyPlanTemplate": {
    /*ro, opt, object*/
        "templateNo": {
        /*ro, opt, object, schedule template No.*/
            "@min":  1,
            /*ro, opt, int*/
            "@max":  16
            /*ro, opt, int*/
        },
        "enable":  "true,false",
        /*ro, opt, string, whether to enable, desc:true (yes), false (no)*/
        "templateName": {
        /*ro, opt, object, template name length*/
            "@min":  1,
            /*ro, opt, int*/
            "@max":  32
            /*ro, opt, int*/
        },
        "weekPlanNo": {
        /*ro, opt, object, weekly schedule No.*/
            "@min":  1,
            /*ro, opt, int*/
            "@max":  16
            /*ro, opt, int*/
        },
        "holidayGroupNo": {
        /*ro, opt, object, holiday group No.*/
            "@min":  1,
            /*ro, opt, int*/
            "@max":  16
            /*ro, opt, int*/
        }
    }
}
```

10.3.6.13 Get the week schedule configuration parameters of the card reader authentication mode

Request URL

GET /ISAPI/AccessControl/VerifyWeekPlanCfg/<weekPlanID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
weekPlanID	string	--

Request Message

None

Response Message

```
{
    "VerifyWeekPlanCfg": {
    /*ro, opt, object, the week schedule configuration parameters of the card reader authentication mode*/
        "enable":  true,
        /*ro, req, bool, whether to enable the week schedule configuration parameters of the card reader authentication mode, desc:true (enable), false
(disable)*/
        "WeekPlanCfg": [
        /*ro, req, array, week schedule parameters, subType:object*/
            {
                "week":  "Monday",
                /*ro, req, enum, day of a week, subType:string, desc:"Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"*/
                "id":  1,
                /*ro, req, int, time period No., range:[1,8]*/
                "enable":  true,
                /*ro, req, bool, whether to enable week schedule*/
                "verifyMode":  "cardAndPw",
                /*ro, req, enum, authentication mode, subType:string, desc:"cardAndPw" (card+password), "card" (card), "cardOrPw" (card or password), "fp"
(fingerprint), "fpAndPw" (fingerprint+password), "fpOrCard" (fingerprint or card), "fpAndCard" (fingerprint+card), "fpAndCardAndPw"
(fingerprint+card+password), "faceOrFpOrCardOrPw" (face or fingerprint or card or password), "faceAndFp" (face+fingerprint), "faceAndPw" (face+password),
"faceAndCard" (face+card), "face" (face), "employeeNoAndPw" (employee No.+password), "fpOrPw" (fingerprint or password), "employeeNoAndFp" (employee
No.+fingerprint), "employeeNoAndFpAndPw" (employee No.+fingerprint+password), "faceAndFpAndCard" (face+fingerprint+card), "faceAndPwAndFp"
(face+password+fingerprint), "employeeNoAndFace" (employee No.+face), "faceOrfaceAndCard" (face or face+card), "fpOrface" (fingerprint or face),
"cardOrfaceOrPw" (card or face or password), "cardOrFace" (card or face), "cardOrFaceOrFp" (card or face or fingerprint), "cardOrFpOrPw" (card or
fingerprint or password), "iris" (iris), "faceOrFpOrCardOrPwOrIris" (face or fingerprint or card or password or iris), "faceOrCardOrPwOrIris" (face or card
or password or iris), "cardOrFace" (card or face), "cardOrFaceOrFp" (card or face or password), "faceOrPw" (face or password), "employeeNoAndFaceAndPw"
(employee No.+face+password), "cardOrFaceOrFaceAndCard" (card or face or face+card), "faceOrFpOrPw" (face or fingerprint or password),
"cardOrFpOrFaceOrIris" (card or fingerpriont or face or iris), "fpOrFaceOrIrisOrPw" (fingerprint or face or iris or password), "cardOrFpOrIrisOrPw" (card or
fingerprint or iris or password), "cardOrIrisOrPw" (card or iris or password), "cardAndIris" (card+ris), "fpAndIris" (fingerprint+iris), "faceAndIris"
(face+iris), "irisAndPw" (iris+password), "cardAndIrisAndPw" (card+iris+password), "faceAndIrisAndPw" (face+iris+password), "cardAndFaceAndIris"
(card+face+iris)*/
                "TimeSegment": {
                /*ro, req, object, time*/
                    "beginTime":  "00:00:00",
                    /*ro, req, time, start time, desc:device local time*/
                    "endTime":  "10:00:00"
                    /*ro, req, time, end time, desc:device local time*/
                }
            }
        ]
    }
}
```

10.3.6.14 Set the week schedule parameters of the card reader authentication mode

Request URL

PUT /ISAPI/AccessControl/VerifyWeekPlanCfg/<weekPlanID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
weekPlanID	string	--

Request Message

```
{
    "VerifyWeekPlanCfg": {
    /*opt, object, the week schedule configuration parameters of the card reader authentication mode*/
        "enable":  true,
        /*req, bool, whether to enable, desc:"true" (enable), "false" (disable)*/
        "WeekPlanCfg": [
        /*req, array, week schedule parameters, subType:object*/
            {
                "week":  "Monday",
                /*req, enum, days of the week, subType:string, desc:"Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"*/
                "id":  1,
                /*req, int, time period No., range:[1,8]*/
                "enable":  true,
                /*req, bool, whether to enable week schedule*/
                "verifyMode":  "cardAndPw",
                /*req, enum, authentication mode, subType:string, desc:"cardAndPw" (card+password), "card" (card), "cardOrPw" (card or password), "fp" (fingerprint), "fpAndPw" (fingerprint+password), "fpOrCard" (fingerprint or card), "fpAndCard" (fingerprint+card), "fpAndCardAndPw" (fingerprint+card+password), "faceOrFpOrCardOrPw" (face or fingerprint or card or password), "faceAndFp" (face+fingerprint), "faceAndPw" (face+password), "faceAndCard" (face+card), "face" (face), "employeeNoAndPw" (employee No.+password), "fpOrPw" (fingerprint or password), "employeeNoAndFp" (employee No.+fingerprint), "employeeNoAndFpAndPw" (employee No.+fingerprint+password), "faceAndFpAndCard" (face+fingerprint+card), "faceAndPwAndFp" (face+password+fingerprint), "employeeNoAndFace" (employee No.+face), "faceOrfaceAndCard" (face or face+card), "fpOrface" (fingerprint or face), "cardOrfaceOrPw" (card or face or password), "cardOrFace" (card or face), "cardOrFaceOrFp" (card or face or fingerprint), "cardOrFpOrPw" (card or fingerprint or password), "iris" (iris), "faceOrFpOrCardOrPwOrIris" (face or fingerprint or card or password or iris), "faceOrCardOrPwOrIris" (face or card or password or iris), "cardOrFace" (card or face), "cardOrFaceOrFp" (card or face or password), "faceOrPw" (face or password), "employeeNoAndFaceAndPw" (employee No.+face+password), "cardOrFaceOrFaceAndCard" (card or face or face+card), "faceOrFpOrPw" (face or fingerprint or password), "cardOrFpOrFaceOrIris" (card or fingerpriont or face or iris), "fpOrFaceOrIrisOrPw" (fingerprint or face or iris or password), "cardOrFpOrIrisOrPw" (card or fingerprint or iris or password), "cardOrIrisOrPw" (card or iris or password), "cardAndIris" (card+ris), "fpAndIris" (fingerprint+iris), "faceAndIris" (face+iris), "irisAndPw" (iris+password), "cardAndIrisAndPw" (card+iris+password), "faceAndIrisAndPw" (face+iris+password), "cardAndFaceAndIris" (card+face+iris)*/
                "TimeSegment": {
                /*req, object, time*/
                    "beginTime":  "00:00:00",
                    /*req, time, start time of the time period, desc:device local time*/
                    "endTime":  "10:00:00"
                    /*req, time, end time of the time period, desc:device local time*/
                }
            }
        ]
    }
}
```

Response Message

```
{
    "statusCode":  1,
    /*ro, opt, int, status code, desc:1 (succeeded). It is required when an error occurred*/
    "statusString":  "OK",
    /*ro, opt, string, status description, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "subStatusCode":  "ok",
    /*ro, opt, string, sub status code, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:it is required when the value of statusCode is not 1, and it corresponds to subStatusCode*/
    "errorMsg":  "ok"
    /*ro, opt, string, error information, desc:this field is required when the value of statusCode is not 1*/
}
```

10.3.6.15 Get the weekly schedule configuration capability of the card reader authentication mode

Request URL

GET /ISAPI/AccessControl/VerifyWeekPlanCfg/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "VerifyWeekPlanCfg": {
    /*ro, opt, object, weekly schedule parameters of the card reader authentication mode*/
        "planNo": {
        /*ro, opt, object, weekly schedule No.*/
            "@min":  1,
            /*ro, opt, int, minimum value*/
            "@max":  16
            /*ro, opt, int, maximum value*/
        },
        "enable":  "true,false",
        /*ro, opt, string, whether to enable, desc:"true"-enable, "false"-disable*/
        "WeekPlanCfg": {
        /*ro, opt, object, week schedule parameters*/
            "maxSize":  56,
            /*ro, opt, int, maximum value*/
            "week": {
            /*ro, opt, object, day of a week*/
                "@opt":  "Monday,Tuesday,Wednesday,Thursday,Friday,Saturday,Sunday"
                /*ro, opt, string, day of a week, desc:"Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"*/
            },
            "id": {
            /*ro, opt, object, weekly schedule No.*/
                "@min":  1,
                /*ro, opt, int, minimum value*/
                "@max":  8
                /*ro, opt, int, maximum value*/
            },
            "enable":  "true,false",
            /*ro, opt, string, whether to enable, desc:true (enable), false (disable)*/
            "verifyMode": {
            /*ro, opt, object, authentication mode*/
                "@opt":
"cardAndPw,card,cardOrPw,fp,fpAndPw,fpOrCard,fpAndCard,fpAndCardAndPw,faceOrFpOrCardOrPw,faceAndFp,faceAndPw,faceAndCard,face,employeeNoAndPw,fpOrPw,employe
eNoAndFp,employeeNoAndFpAndPw,faceAndFpAndCard,faceAndPwAndFp,employeeNoAndFace,faceOrfaceAndCard,fpOrface,cardOrfaceOrPw,cardOrFace,cardOrFaceOrFp,faceOrPw
,employeeNoAndFaceAndPw,cardOrFaceOrFaceAndCard,iris,faceOrFpOrCardOrPwOrIris,faceOrCardOrPwOrIris,sleep,invalid,cardOrFpOrFaceOrIris,fpOrFaceOrIrisOrPw,car
dOrFpOrIrisOrPw,cardOrIrisOrPw,cardAndIris,fpAndIris,faceAndIris,irisAndPw,cardAndIrisAndPw,faceAndIrisAndPw,cardAndFaceAndIris,Pw,cardOrFpOrPw,faceOrFpOrPw
,cardAndVp,faceAndVp,fpAndVp,pwAndVp"
                /*ro, opt, string, authentication mode, desc:"cardAndPw" (card+password), "card" (card), "cardOrPw" (card or password), "fp" (fingerprint),
"fpAndPw" (fingerprint+password), "fpOrCard" (fingerprint or card), "fpAndCard" (fingerprint+card), "fpAndCardAndPw" (fingerprint+card+password),
"faceOrFpOrCardOrPw" (face or fingerprint or card or password), "faceAndFp" (face+fingerprint), "faceAndPw" (face+password), "faceAndCard" (face+card),
"face" (face), "employeeNoAndPw" (employee No.+password), "fpOrPw" (fingerprint or password), "employeeNoAndFp" (employee No.+fingerprint),
"employeeNoAndFpAndPw" (employee No.+fingerprint+password), "faceAndFpAndCard" (face+fingerprint+card), "faceAndPwAndFp" (face+password+fingerprint),
"employeeNoAndFace" (employee No.+face), "faceOrfaceAndCard" (face or face+card), "fpOrface" (fingerprint or face), "cardOrfaceOrPw" (card or face or
password), "cardOrFace" (card or face), "cardOrFaceOrFp" (card or face or fingerprint), "cardOrFpOrPw" (card or fingerprint or password), "iris" (iris),
"faceOrFpOrCardOrPwOrIris" (face or fingerprint or card or password or iris), "faceOrCardOrPwOrIris" (face or card or password or iris), "cardOrFace" (card
or face), "cardOrFaceOrFp" (card or face or password), "faceOrPw" (face or password), "employeeNoAndFaceAndPw" (employee No.+face+password),
"cardOrFaceOrFaceAndCard" (card or face or face+card), "faceOrFpOrPw" (face or fingerprint or password), "cardOrFpOrFaceOrIris" (card or fingerpriont or
face or iris), "fpOrFaceOrIrisOrPw" (fingerprint or face or iris or password), "cardOrFpOrIrisOrPw" (card or fingerprint or iris or password),
"cardOrIrisOrPw" (card or iris or password), "cardAndIris" (card+ris), "fpAndIri" (fingerprint+iris), "faceAndIris" (face+iris), "irisAndPw"
(iris+password), "cardAndIrisAndPw" (card+iris+password), "faceAndIrisAndPw" (face+iris+password), "cardAndFaceAndIris" (card+face+iris), "sleep",
"invalid"*/
            },
            "TimeSegment": {
            /*ro, opt, object, time*/
                "beginTime":  "00:00:00",
                /*ro, opt, time, start time, desc:start time of the time period (device local time)*/
                "endTime":  "10:00:00",
                /*ro, opt, time, end time, desc:end time of the time period (device local time)*/
                "validUnit":  "minute"
                /*ro, opt, enum, time accuracy, subType:string, desc:If this node is not returned, it indicates that the time accuracy is "minute". "hour",
"minute", "second"*/
            }
        },
        "purePwdVerifyEnable":  true
        /*ro, opt, bool*/
    }
}
```

10.3.7 Credential Recognition Unit Management

10.3.7.1 Set the card reader parameters

Request URL

PUT /ISAPI/AccessControl/CardReaderCfg/<cardReaderID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
cardReaderID	string	--

Request Message

```
{
    "CardReaderCfg": {
    /*req, object, card reader information*/
        "enable":  true,
        /*req, bool, whether to enable, desc:true-yes, false-no*/
        "okLedPolarity":  "cathode",
        /*opt, enum, OK LED polarity, subType:string, desc:"cathode", "anode”*/
        "errorLedPolarity":  "cathode",
        /*opt, enum, Error LED polarity, subType:string, desc:"cathode", "anode"*/
        "buzzerPolarity":  "cathode",
        /*opt, enum, buzzer polarity, subType:string, desc:"cathode", "anode"*/
        "swipeInterval":  1,
        /*opt, int, time interval of repeated authentication, unit:s, desc:it is valid for authentication modes such as fingerprint, card, face, etc.*/
        "pressTimeout":  1,
        /*opt, int, timeout to reset entry on keypad, unit:s*/
        "enableFailAlarm":  true,
        /*opt, bool, whether to enable excessive failed authentication attempt alarm*/
        "maxReadCardFailNum":  1,
        /*opt, int, maximum number of failed authentication attempts*/
        "enableTamperCheck":  true,
        /*opt, bool, whether to enable tampering detection*/
        "offlineCheckTime":  1,
        /*opt, int, time to detect after the card reader is offline, unit:s*/
        "fingerPrintCheckLevel":  1,
        /*opt, enum, fingerprint recognition level, subType:int, desc:1-1/10 false acceptance rate (FAR), 2-1/100 false acceptance rate (FAR), 3-1/1000 false acceptance rate (FAR), 4-1/10000 false acceptance rate (FAR), 5-1/100000 false acceptance rate (FAR), 6-1/1000000 false acceptance rate (FAR), 7-1/10000000 false acceptance rate (FAR), 8-1/100000000 false acceptance rate (FAR), 9-3/100 false acceptance rate (FAR), 10-3/1000 false acceptance rate (FAR), 11-3/10000 false acceptance rate (FAR), 12-3/100000 false acceptance rate (FAR), 13-3/1000000 false acceptance rate (FAR), 14-3/10000000 false acceptance rate (FAR), 15-3/100000000 false acceptance rate (FAR), 16-Automatic Normal, 17-Automatic Secure, 18-Automatic More Secure (currently not support)*/
        "useLocalController":  true,
        /*opt, bool, whether it is connected to the distributed controller*/
        "localControllerID":  1,
        /*opt, int, distributed controller No., range:[0,64], dep:and,{$.CardReaderCfg.localControllerID,eq,true}, desc:0-unregistered. This field is valid only when useLocalController is "true"*/
        "localControllerReaderID":  1,
        /*opt, int, card reader ID of the distributed controller, dep:and,{$.CardReaderCfg.localControllerID,eq,true}, desc:0-unregistered. This field is valid only when useLocalController is "true"*/
        "cardReaderChannel":  1,
        /*opt, enum, communication channel No. of the card reader, subType:int, dep:and,{$.CardReaderCfg.localControllerID,eq,true}, desc:0-Wiegand or offline, 1-RS-485A, 2-RS-485B. This field is valid only when useLocalController is "true"*/
        "fingerPrintImageQuality":  1,
        /*opt, enum, fingerprint image quality, subType:int, desc:1-low quality (V1), 2-medium quality (V1), 3-high quality (V1), 4-highest quality (V1), 5-low quality (V2), 6-medium quality (V2), 7-high quality (V2), 8-highest quality (V2)*/
        "fingerPrintContrastTimeOut":  1,
        /*opt, enum, fingerprint comparison timeout, subType:int, desc:it is between 1 and 20, unit: second, 255-infinite*/
        "fingerPrintRecogizeInterval":  1,
        /*opt, enum, fingerprint scanning interval, subType:int, desc:it is between 1 and 10, unit: second, 255-no delay*/
        "fingerPrintMatchFastMode":  1,
        /*opt, enum, fingerprint matching quick mode, subType:int, desc:1-quick mode 1, 2-quick mode 2, 3-quick mode 3, 4-quick mode 4, 5-quick mode 5, 255-automatic*/
        "fingerPrintModuleSensitive":  1,
        /*opt, enum, fingerprint module sensitivity, subType:int, desc:fingerprint module sensitivity,which is between 1 and 8*/
        "fingerPrintModuleLightCondition": "outdoor",
        /*opt, enum, fingerprint module light condition, subType:string, desc:"outdoor", "indoor”*/
        "faceMatchThresholdN":  1,
        /*opt, int, threshold of face picture 1:N comparison, range:[0,100], desc:threshold of face picture 1:N comparison,which is between 0 and 100*/
        "faceQuality":  1,
        /*opt, int, face picture quality, range:[0,100]*/
        "faceRecogizeTimeOut":  1,
        /*opt, enum, face recognition timeout, subType:int, desc:it is between 1 and 20, unit: second, 255-infinite*/
        "faceRecogizeInterval":  1,
        /*opt, enum, face recognition interval, subType:int, desc:it is between 1 and 10, unit: second, 255-no delay*/
        "cardReaderFunction":  ["fingerPrint", "face", "fingerVein", "iris", "card"],
        /*opt, enumarray, card reader type, subType:string, desc:"fingerPrint"-fingerprint, "face", "fingerVein"-finger vein, “iris”. For example, ["fingerPrint","face"] indicates that the card reader supports both fingerprint and face*/
        "cardReaderDescription":  "Wiegand\u000485Offline",
        /*opt, string, card reader description, desc:if the card reader is the Wiegand card reader or if offline, this field will be set to "Wiegand" or "485Offline”*/
        "faceImageSensitometry":  1,
        /*opt, int, face picture exposure, range:[0,655535]*/
        "livingBodyDetect":  true,
        /*opt, bool, whether to enable human detection*/
        "faceMatchThreshold1":  1,
        /*opt, int, threshold of face picture 1:1 comparison, range:[0,100], desc:threshold of face picture 1:1 comparison,which is between 0 and 100*/
        "buzzerTime":  1,
        /*opt, int, buzzing duration, range:[0,59999], unit:s, desc:0-long buzzing*/
        "faceMatch1SecurityLevel":  1,
        /*opt, enum, security level of face 1:1 recognition, subType:int, desc:1-normal, 2-high, 3-higher*/
        "faceMatchNSecurityLevel":  1,
        /*opt, enum, security level of face 1:N recognition, subType:int, desc:1-normal, 2-high, 3-higher*/
        "envirMode":  "indoor",
        /*opt, enum, environment mode of face recognition, subType:string, desc:"indoor", "other”*/
        "liveDetLevelSet":  "low",
        /*opt, enum, threshold level of liveness detection, subType:string, desc:"low", "middle"-medium, "high"*/
        "liveDetThreshold":  1,
        /*opt, int, range:[0,100]*/
        "liveDetAntiAttackCntLimit":  1,
        /*opt, int, number of anti-attacks of liveness detection,, range:[1,255], desc:this value should be configured as the same one on both client and device*/
        "enableLiveDetAntiAttack":  true,
        /*opt, bool, whether to enable anti-attack for liveness detection*/
        "liveDetAntiAttackLockedTime":  1,
```

```
        /*opt, int, range:[0,300], unit:s*/
        "supportDelFPByID":  true,
        /*opt, bool, whether the card reader supports deleting fingerprint by fingerprint ID, desc:true-yes, false-no*/
        "fingerPrintCapacity":  1,
        /*opt, int, fingerprint capacity*/
        "fingerPrintNum":  1,
        /*opt, int, number of added fingerprints*/
        "defaultVerifyMode":  "cardAndPw",
        /*opt, enum, default authentication mode of the fingerprint and card reader (factory defaults):, subType:string, desc:factory defaults; "cardAndPw"
(card+password), "card" (card), "cardOrPw" (card or password), "fp" (fingerprint), "fpAndPw" (fingerprint+password), "fpOrCard" (fingerprint or card),
"fpAndCard" (fingerprint+card), "fpAndCardAndPw" (fingerprint+card+password), "faceOrFpOrCardOrPw" (face or fingerprint or card or password), "faceAndFp"
(face+fingerprint), "faceAndPw" (face+password), "faceAndCard" (face+card), "face" (face), "employeeNoAndPw" (employee No.+password), "fpOrPw" (fingerprint
or password), "employeeNoAndFp" (employee No.+fingerprint), "employeeNoAndFpAndPw" (employee No.+fingerprint+password), "faceAndFpAndCard"
(face+fingerprint+card), "faceAndPwAndFp" (face+password+fingerprint), "employeeNoAndFace" (employee No.+face), "faceOrfaceAndCard" (face or face+card),
"fpOrface" (fingerprint or face), "cardOrfaceOrPw" (card or face or password), "cardOrFace" (card or face), "cardOrFaceOrFp" (card or face or fingerprint),
"cardOrFpOrPw" (card or fingerprint or password), "iris" (iris), "faceOrFpOrCardOrPwOrIris" (face or fingerprint or card or password or iris),
"faceOrCardOrPwOrIris" (face or card or password or iris), "cardOrFace" (card or face), "cardOrFaceOrFp" (card or face or password), "faceOrPw" (face or
password), "employeeNoAndFaceAndPw" (employee No.+face+password), "cardOrFaceOrFaceAndCard" (card or face or face+card), "faceOrFpOrPw" (face or fingerprint
or password), "cardOrFpOrFaceOrIris" (card or fingerpriont or face or iris), "fpOrFaceOrIrisOrPw" (fingerprint or face or iris or password),
"cardOrFpOrIrisOrPw" (card or fingerprint or iris or password), "cardOrIrisOrPw" (card or iris or password), "cardAndIris" (card+ris), "fpAndIri"
(fingerprint+iris), "faceAndIris" (face+iris), "irisAndPw" (iris+password), "cardAndIrisAndPw" (card+iris+password), "faceAndIrisAndPw"
(face+iris+password), "cardAndFaceAndIris" (card+face+iris)*/
        "faceRecogizeEnable":  1,
        /*opt, enum, whether to enable facial recognition, subType:int, desc:1-enable, 2-disable, 3-attendence checked in/out by recognition of multiple
faces*/
        "FPAlgorithmVersion":  "test",
        /*opt, string, fingerprint algorithm library version, range:[1,32]*/
        "cardReaderVersion":  "test",
        /*opt, string, card reader version, range:[1,32]*/
        "enableReverseCardNo":  true,
        /*opt, bool, whether to enable reversing the card No.*/
        "independSwipeIntervals":  0,
        /*opt, int, time interval of person authentication, unit: second. This time interval is calculated for each person separately and is different from
swipeInterval, desc:time interval of person authentication,unit: second. This time interval is calculated for each person separately and is different from
swipeInterval*/
        "maskFaceMatchThresholdN":  1,
        /*opt, int, 1:N face picture (face with mask and normal background) comparison threshold, range:[0,100], desc:1:N face picture (face with mask and
normal background) comparison threshold,value range: [0,100]*/
        "maskFaceMatchThreshold1":  1,
        /*opt, int, 1:1 face picture (face with mask and normal background) comparison threshold, range:[0,100], desc:1:1 face picture (face with mask and
normal background) comparison threshold,value range: [0,100]*/
        "faceMotionDetLevel":  "low",
        /*opt, enum, face motion detection level, subType:string, desc:"high", "medium", "low"*/
        "showMode":  "normal",
        /*opt, enum, display mode, subType:string, desc:this node is not valid; simple mode indicates that the device displays authentication results
exclude employee No., name, etc.; the device applies normal mode by default; advertisement mode indicates that the device displays both advertisement and
authentication results; meeting mode indicates that the device displays check-in page of the conference; custom mode indicates that the device displays
layout of the interface customized by users; "concise"-simple mode, "normal"-normal mode (default), "advertising"-advertisement mode, "meeting"-meeting
mode, "selfDefine"-custom mode*/
        "enableScreenOff":  true,
        /*opt, bool, whether to enable auto locking the screen*/
        "screenOffTimeout":  1
        /*opt, int, time, step:1, unit:s*/
    }
}
```


Response Message

```
{
    "statusCode":  1,
    /*ro, opt, int, status code, desc:1 (succeeded). It is required when an error occurred*/
    "statusString":  "OK",
    /*ro, opt, string, status description, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "subStatusCode":  "ok",
    /*ro, opt, string, sub status code, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:when the value of statusCode is not 1, it corresponds to subStatusCode*/
    "errorMsg":  "ok"
    /*ro, opt, string, error description, desc:this node is required when the value of statusCode is not 1*/
}
```

10.3.7.2 Get the card reader configuration parameters

Request URL

GET /ISAPI/AccessControl/CardReaderCfg/<cardReaderID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
cardReaderID	string	--

Request Message

None

Response Message

```
{
    "CardReaderCfg": {
    /*ro, req, object, card reader information*/
        "enable":  true,
        /*ro, req, bool, whether to enable, desc:true (enable), false (disable)*/
        "okLedPolarity":  "cathode",
        /*ro, opt, enum, OK LED polarity, subType:string, desc:"cathode", "anode"*/
        "errorLedPolarity":  "cathode",
        /*ro, opt, enum, Error LED polarity, subType:string, desc:"cathode", "anode"*/
        "buzzerPolarity":  "cathode",
        /*ro, opt, enum, buzzer polarity, subType:string, desc:"cathode", "anode"*/
        "swipeInterval":  1,
        /*ro, opt, int, time interval of repeated authentication, unit:s, desc:which is valid for authentication modes such as fingerprint, card, face, 
etc.*/
        "pressTimeout":  1,
        /*ro, opt, int, timeout to reset entry on keypad, unit:s*/
        "enableFailAlarm":  true,
        /*ro, opt, bool, whether to enable excessive failed authentication attempts alarm*/
        "maxReadCardFailNum":  1,
        /*ro, opt, int, maximum number of failed authentication attempts*/
        "enableTamperCheck":  true,
        /*ro, opt, bool, whether to enable tampering detection*/
        "offlineCheckTime":  1,
        /*ro, opt, int, time to detect after the card reader is offline, unit:s*/
        "fingerPrintCheckLevel":  1,
        /*ro, opt, enum, fingerprint recognition level, subType:int, desc:1-1/10 false acceptance rate (FAR), 2-1/100 false acceptance rate (FAR), 3-1/1000 
false acceptance rate (FAR), 4-1/10000 false acceptance rate (FAR), 5-1/100000 false acceptance rate (FAR), 6-1/1000000 false acceptance rate (FAR), 7-
1/10000000 false acceptance rate (FAR), 8-1/100000000 false acceptance rate (FAR), 9-3/100 false acceptance rate (FAR),  10-3/1000 false acceptance rate 
(FAR), 11-3/10000 false acceptance rate (FAR), 12-3/100000 false acceptance rate (FAR), 13-3/1000000 false acceptance rate (FAR), 14-3/10000000 false 
acceptance rate (FAR), 15-3/100000000 false acceptance rate (FAR), 16-Automatic Normal, 17-Automatic Secure, 18-Automatic More Secure (currently not 
support)*/
        "useLocalController":  true,
        /*ro, opt, bool, whether it is connected to the distributed controller*/
        "localControllerID":  1,
        /*ro, opt, int, distributed controller No., range:[0,64], dep:and,{$.CardReaderCfg.localControllerID,eq,true}, desc:which is between 1 and 64, 0-
unregistered. This field is valid only when useLocalController is "true"*/
        "localControllerReaderID":  1,
        /*ro, opt, int, card reader ID of the distributed controller, 0-unregistered, dep:and,{$.CardReaderCfg.localControllerID,eq,true}, desc:this field 
is valid only when useLocalController is "true"*/
        "cardReaderChannel":  1,
        /*ro, opt, enum, communication channel No. of the card reader, subType:int, dep:and,{$.CardReaderCfg.localControllerID,eq,true}, desc:0-Wiegand or 
offline, 1-RS-485A, 2-RS-485B. This field is valid only when useLocalController is "true"*/
        "fingerPrintImageQuality":  1,
        /*ro, opt, enum, fingerprint image quality, subType:int, desc:1-low quality (V1), 2-medium quality (V1), 3-high quality (V1), 4-highest quality 
(V1), 5-low quality (V2), 6-medium quality (V2), 7-high quality (V2), 8-highest quality (V2)*/
        "fingerPrintContrastTimeOut":  1,
        /*ro, opt, enum, fingerprint comparison timeout, subType:int, desc:fingerprint comparison timeout,which is between 1 and 20,unit: second,255-
infinite*/
        "fingerPrintRecogizeInterval":  1,
        /*ro, opt, enum, fingerprint scanning interval, subType:int, desc:fingerprint scanning interval,which is between 1 and 10,unit: second,255-no 
delay*/
        "fingerPrintMatchFastMode":  1,
        /*ro, opt, enum, fingerprint matching quick mode, subType:int, desc:1-quick mode 1, 2-quick mode 2, 3-quick mode 3, 4-quick mode 4, 5-quick mode 5, 
255-automatic*/
        "fingerPrintModuleSensitive":  1,
        /*ro, opt, enum, fingerprint module sensitivity, subType:int, desc:fingerprint module sensitivity,which is between 1 and 8*/
        "fingerPrintModuleLightCondition":  "outdoor",
        /*ro, opt, enum, fingerprint module light condition, subType:string, desc:"outdoor", "indoor"*/
        "faceMatchThresholdN":  1,
        /*ro, opt, int, threshold of face picture 1:N comparison,which is between 0 and 100, range:[0,100]*/
        "faceQuality":  1,
        /*ro, opt, int, face picture quality, range:[0,100]*/
        "faceRecogizeTimeOut":  1,
        /*ro, opt, enum, face recognition timeout, subType:int, desc:face recognition timeout,which is between 1 and 20,unit: second,255-infinite*/
        "faceRecogizeInterval":  1,
        /*ro, opt, enum, face recognition interval, subType:int, desc:face recognition interval,which is between 1 and 10,unit: second,255-no delay*/
        "cardReaderFunction":  ["fingerPrint", "face", "card"],
        /*ro, opt, enumarray, card reader type, subType:string, desc:"fingerPrint", "face", "fingerVein". For example, ["fingerPrint", "face"] indicates 
that the card reader supports both fingerprint and face*/
        "cardReaderDescription":  "Wiegand\u0004485Offline",
        /*ro, opt, string, card reader description, desc:if the card reader is the Wiegand card reader or if offline, this field will be set to "Wiegand" or 
"485Offline"*/
        "faceImageSensitometry":  1,
        /*ro, opt, int, face picture exposure, range:[0,655535]*/
        "livingBodyDetect":  true,
        /*ro, opt, bool, whether to enable human detection*/
        "faceMatchThreshold1":  1,
        /*ro, opt, int, threshold of face picture 1:1 comparison, range:[0,100]*/
        "buzzerTime":  1,
        /*ro, opt, int, buzzing duration, range:[0,59999], unit:s, desc:buzzing duration,which is between 0 and 5999,unit: second,0-long buzzing*/
        "faceMatch1SecurityLevel":  1,
        /*ro, opt, enum, security level of face 1:1 recognition, subType:int, desc:1 (normal), 2 (high), 3 (higher)*/
        "faceMatchNSecurityLevel":  1,
        /*ro, opt, enum, security level of face 1:N recognition: 1-normal,2-high,3-higher, subType:int, desc:1 (normal), 2 (high), 3 (higher)*/
        "envirMode":  "other",
        /*ro, opt, enum, environment mode of face recognition, subType:string, desc:"indoor", "other"*/
        "liveDetLevelSet":  "low",
        /*ro, opt, enum, threshold level of liveness detection, subType:string, desc:"low", "middle", "high"*/
```

```
        /*ro, opt, enum, threshold level of liveness detection, subType:string, desc:"low", "middle", "high"*/
        "liveDetThreshold":  1,
        /*ro, opt, int, range:[0,100]*/
        "liveDetAntiAttackCntLimit":  1,
        /*ro, opt, int, number of anti-attacks of liveness detection, range:[1,255], desc:this value should be configured as the same one on both client and
device*/
        "enableLiveDetAntiAttack":  true,
        /*ro, opt, bool, whether to enable anti-attack for liveness detection*/
        "liveDetAntiAttackLockedTime":  1,
        /*ro, opt, int, range:[0,300], unit:s*/
        "supportDelFPByID":  true,
        /*ro, opt, bool, whether the card reader supports deleting fingerprint by fingerprint ID, desc:"true"-yes, "false"-no*/
        "fingerPrintCapacity":  1,
        /*ro, opt, int, fingerprint capacity*/
        "fingerPrintNum":  1,
        /*ro, opt, int, number of added fingerprints*/
        "defaultVerifyMode":  "cardAndPw",
        /*ro, opt, enum, default authentication mode of the fingerprint and card reader (factory defaults), subType:string, desc:factory defaults;
"cardAndPw" (card+password), "card" (card), "cardOrPw" (card or password), "fp" (fingerprint), "fpAndPw" (fingerprint+password), "fpOrCard" (fingerprint or
card), "fpAndCard" (fingerprint+card), "fpAndCardAndPw" (fingerprint+card+password), "faceOrFpOrCardOrPw" (face or fingerprint or card or password),
"faceAndFp" (face+fingerprint), "faceAndPw" (face+password), "faceAndCard" (face+card), "face" (face), "employeeNoAndPw" (employee No.+password), "fpOrPw"
(fingerprint or password), "employeeNoAndFp" (employee No.+fingerprint), "employeeNoAndFpAndPw" (employee No.+fingerprint+password), "faceAndFpAndCard"
(face+fingerprint+card), "faceAndPwAndFp" (face+password+fingerprint), "employeeNoAndFace" (employee No.+face), "faceOrfaceAndCard" (face or face+card),
"fpOrface" (fingerprint or face), "cardOrfaceOrPw" (card or face or password), "cardOrFace" (card or face), "cardOrFaceOrFp" (card or face or fingerprint),
"cardOrFpOrPw" (card or fingerprint or password), "iris" (iris), "faceOrFpOrCardOrPwOrIris" (face or fingerprint or card or password or iris),
"faceOrCardOrPwOrIris" (face or card or password or iris), "cardOrFace" (card or face), "cardOrFaceOrFp" (card or face or password), "faceOrPw" (face or
password), "employeeNoAndFaceAndPw" (employee No.+face+password), "cardOrFaceOrFaceAndCard" (card or face or face+card), "faceOrFpOrPw" (face or fingerprint
or password), "cardOrFpOrFaceOrIris" (card or fingerpiont or face or iris), "fpOrFaceOrIrisOrPw" (fingerprint or face or iris or password),
"cardOrFpOrIrisOrPw" (card or fingerprint or iris or password), "cardOrIrisOrPw" (card or iris or password), "cardAndIris" (card+ris), "fpAndIri"
(fingerprint+iris), "faceAndIris" (face+iris), "irisAndPw" (iris+password), "cardAndIrisAndPw" (card+iris+password), "faceAndIrisAndPw"
(face+iris+password), "cardAndFaceAndIris" (card+face+iris)*/
        "faceRecogizeEnable":  1,
        /*ro, opt, enum, whether to enable facial recognition, subType:int, desc:1 (enable), 2 (disable), 3 (attendence checked in/out by recognition of
multiple faces)*/
        "FPAlgorithmVersion":  "test",
        /*ro, opt, string, fingerprint algorithm library version, range:[1,32]*/
        "cardReaderVersion":  "test",
        /*ro, opt, string, card reader version, range:[1,32]*/
        "enableReverseCardNo":  true,
        /*ro, opt, bool, whether to enable reversing the card No.*/
        "independSwipeIntervals":  0,
        /*ro, opt, int, time interval of person authentication, desc:unit: second. This time interval is calculated for each person separately and is
different from swipeInterval*/
        "maskFaceMatchThresholdN":  1,
        /*ro, opt, int, 1:N face picture (face with mask and normal background) comparison threshold, range:[0,100]*/
        "maskFaceMatchThreshold1":  1,
        /*ro, opt, int, 1:1 face picture (face with mask and normal background) comparison threshold, range:[0,100]*/
        "faceMotionDetLevel":  "low",
        /*ro, opt, enum, face motion detection level, subType:string, desc:"high", "medium", "low"*/
        "showMode":  "normal",
        /*ro, opt, enum, display mode, subType:string, desc:this node is not valid; simple mode indicates that the device displays authentication results
exclude employee No., name, etc.; the device applies normal mode by default; advertisement mode indicates that the device displays both advertisement and
authentication results; meeting mode indicates that the device displays check-in page of the conference; custom mode indicates that the device displays
layout of the interface customized by users; "concise"-simple mode, "normal"-normal mode (default), "advertising"-advertisement mode, "meeting"-meeting
mode, "selfDefine"-custom mode*/
        "enableScreenOff":  true,
        /*ro, opt, bool, whether to enable auto locking the screen*/
        "screenOffTimeout":  1
        /*ro, opt, int, time, step:1, unit:s*/
    }
}
```


10.3.7.3 Get the configuration capability of the card reader

Request URL

GET /ISAPI/AccessControl/CardReaderCfg/capabilities?format=json&cardReaderID=<cardReaderID>

Query Parameter

Parameter Name	Parameter Type	Description
cardReaderID	string	--

Request Message

None

Response Message

```
{
    "CardReaderCfg": {
    /*ro, req, object, Set Card Reader Parameters*/
        "cardReaderNo": {
        /*ro, opt, object, card reader No., desc:card reader No.*/
            "@min":  1,
            /*ro, opt, int, the minimum value*/
            "@max":  512,
            /*ro, opt, int, the maximum value*/
        },
        "enable":  "true,false",
        /*ro, opt, string, whether to enable, desc:"true"-yes,"false"-no*/
        "swipeInterval": {
        /*ro, opt, object, time interval of repeated authentication, desc:it is valid for authentication modes such as fingerprint, card, face, etc., unit:
second*/
            "@min":  1,
            /*ro, req, int, the minimum value*/
            "@max":  255
            /*ro, req, int, the maximum value*/
        },
        "enableFailAlarm":  "true,false",
        /*ro, opt, string, whether to enable excessive failed authentication attempts alarm*/
        "maxReadCardFailNum": {
        /*ro, opt, object, maximum number of failed authentication attempts*/
            "@min":  1,
            /*ro, req, int, the minimum value*/
            "@max":  255
            /*ro, req, int, the maximum value*/
        },
        "enableTamperCheck":  "true,false",
        /*ro, opt, string, whether to enable tampering detection*/
        "offlineCheckTime": {
        /*ro, opt, object, time to detect after the card reader is offline, desc:unit: second*/
            "@min":  1,
            /*ro, req, int, the minimum value*/
            "@max":  255
            /*ro, req, int, the maximum value*/
        },
        "cardReaderDescription": {
        /*ro, opt, object, card reader description, desc:if the card reader is the Wiegand card reader or if offline, this field will be set to "Wiegand" or
"485Offline"*/
            "@min":  1,
            /*ro, req, int, the minimum value*/
            "@max":  16
            /*ro, req, int, the maximum value*/
        },
        "buzzerTime": {
        /*ro, opt, object, buzzing duration, desc:it is between 0 and 5999, unit: second, 0 (long buzzing)*/
            "@min":  0,
            /*ro, req, int, the minimum value*/
            "@max":  5999
            /*ro, req, int, the maximum value*/
        },
        "fingerPrintCapacity": {
        /*ro, opt, object, maximum number of fingerprints that can be added*/
            "@min":  1,
            /*ro, req, int, the minimum value*/
            "@max":  100
            /*ro, req, int, the maximum value*/
        },
        "fingerPrintNum": {
        /*ro, opt, object, number of added fingerprints*/
            "@min":  1,
            /*ro, req, int, the minimum value*/
            "@max":  100
            /*ro, req, int, the maximum value*/
        },
        "FPAlgorithmVersion": {
        /*ro, opt, object, fingerprint algorithm library version*/
            "@min":  1,
            /*ro, req, int, the minimum value*/
            "@max":  1
            /*ro, req, int, the maximum value*/
        },
        "cardReaderVersion": {
        /*ro, opt, object, card reader version*/
            "@min":  1,
            /*ro, req, int, the minimum value*/
            "@max":  1
            /*ro, req, int, the maximum value*/
        },
    }
}
```


10.3.7.4 Get the configuration capability of enabling NFC (Near-Field Communication) function

Request URL

GET /ISAPI/AccessControl/Configuration/NFCCfg/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message


```
{
    "NFCCfgCap": {
    /*ro, opt, object, configuration capability of enabling NFC (Near-Field Communication) function*/
        "enable":  "true,false"
        /*ro, req, string, whether to enable NFC function, desc:true-yes, false-no (default)*/
    }
}
```


10.3.7.5 Get the parameters of enabling NFC (Near-Field Communication) function

Request URL

GET /ISAPI/AccessControl/Configuration/NFCCfg?format=json

Query Parameter

None

Request Message

None

Response Message


```
{
    "NFCCfg": {
    /*ro, req, object*/
        "enable":  true
        /*ro, req, bool, whether to enable NFC function, desc:true (yes), false (no). The value of this node is "false" by default*/
    }
}
```


10.3.7.6 Set the parameters of enabling NFC (Near-Field Communication) function

Request URL

PUT /ISAPI/AccessControl/Configuration/NFCCfg?format=json

Query Parameter

None

Request Message


```
{
    "NFCCfg": {
    /*req, object, configuration capability of enabling NFC (Near-Field Communication) function*/
        "enable":  true
        /*req, bool, whether to enable NFC function, desc:true-yes, false-no (default)*/
    }
}
```


Response Message

```
{
    "requestURL":  "test",
    /*ro, opt, string, URI*/
    "statusCode":  1,
    /*ro, opt, int, status code, desc:1 (succeeded). It is required when an error occurred*/
    "statusString":  "test",
    /*ro, opt, string, status description, desc:"ok" (succeeded). It is required when an error occurred*/
    "subStatusCode":  "test",
    /*ro, opt, string, sub status code, desc:"ok" (succeeded). It is required when an error occurred*/
    "errorCode":  1,
    /*ro, req, int, error code, desc:it is required when the value of statusCode is not 1, and it corresponds to subStatusCode*/
    "errorMsg":  "ok"
    /*ro, req, string, error information, desc:this node is required when the value of statusCode is not 1*/
}
```

10.3.7.7 Get the configuration capability of enabling RF (Radio Frequency) card recognition

Request URL

GET /ISAPI/AccessControl/Configuration/RFCardCfg/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "RFCardCfgCap": {
    /*ro, req, object*/
        "cardType": {
        /*ro, opt, object, card type, desc:"EMCard"-EM card, "M1Card"-M1 card, "CPUCard"-CPU card, "IDCard"-ID card, "DesfireCard"-DESFire card,
"FelicaCard"-FeliCa card*/
            "@opt": ["EMCard", "M1Card", "CPUCard", "IDCard", "FelicaCard"]
            /*ro, req, array, options, subType:string*/
        },
        "enabled": {
        /*ro, opt, object, whether to enable RF card recognition*/
            "@opt": [true, false]
            /*ro, req, array, options, subType:bool*/
        }
    }
}
```

10.3.7.8 Get the parameters of enabling RF (Radio Frequency) card recognition

Request URL

GET /ISAPI/AccessControl/Configuration/RFCardCfg?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "RFCardCfg": [
    /*ro, req, array, subType:object*/
        {
            "cardType":  "EMCard",
            /*ro, req, enum, card type, subType:string, desc:"EMCard" (EM card), "M1Card" (M1 card), "CPUCard" (CPU card), "IDCard" (ID card), "DesfireCard"
(DESFire card), "FelicaCard" (FeliCa card)*/
            "enabled":  true
            /*ro, req, bool, whether to enable RF card recognition, desc:true-yes, false-no*/
        }
    ]
}
```

10.3.7.9 Set the parameters of enabling RF (Radio Frequency) card recognition

Request URL

PUT /ISAPI/AccessControl/Configuration/RFCardCfg?format=json

Query Parameter

None

Request Message

```
{
    "RFCardCfg": [
    /*req, array, the parameters of enabling RF (Radio Frequency) card recognition, subType:object*/
        {
            "cardType":  "EMCard",
            /*req, enum, card type, subType:string, desc:"EMCard"(EM card), "M1Card"(M1 card), "CPUCard"(CPU card), "IDCard"(ID card), "DesfireCard"(DESFire
card), "FelicaCard"(FeliCa card)*/
            "enabled":  true
            /*req, bool, whether to enable RF card recognition, desc:"true"(yes), "false"(no)*/
        }
    ]
}
```

Response Message

```
{
    "requestURL":  "test",
    /*ro, opt, string, URI*/
    "statusCode":  1,
    /*ro, opt, int, status code, desc:1 (succeeded). It is required when an error occurred*/
    "statusString":  "test",
    /*ro, opt, string, status description, desc:"ok" (succeeded). It is required when an error occurred*/
    "subStatusCode":  "test",
    /*ro, opt, string, sub status code, desc:"ok" (succeeded). It is required when an error occurred*/
    "errorCode":  1,
    /*ro, req, int, error code, desc:it is required when the value of statusCode is not 1, and it corresponds to subStatusCode*/
    "errorMsg":  "ok"
    /*ro, req, string, error information, desc:this node is required when the value of statusCode is not 1*/
}
```

10.3.7.10 Get the parameters of face recognition terminal

Request URL

GET /ISAPI/AccessControl/IdentityTerminal

Query Parameter

None

Request Message

None

Response Message

```
<?xml version="1.0" encoding="UTF-8"?>
<IdentityTerminal xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, parameters of face recognition terminal, attr:version{req, string, protocolVersion}-->
  <terminalMode>
    <!--ro, opt, enum, terminal mode, subType:string, desc:"authMode" (authentication mode), "registerMode" (registration mode)-->authMode
  </terminalMode>
  <idCardReader>
    <!--ro, opt, enum, ID card reader model, subType:string, desc:“iDR210”, “DS-K1F110-I”, “DS-K1F1110-B”, “DS-K1F1110-AB”, “none”, “DS-K1F1001-I(USB)”,
“DS-K1F1002-I(USB)”-->iDR210
  </idCardReader>
  <camera>
    <!--ro, opt, enum, camera, subType:string, desc:camera model: C270,DS-2CS5432B-S-->C270
  </camera>
  <fingerPrintModule>
    <!--ro, opt, enum, fingerprint module type, subType:string, desc:“ALIWARD”, “”-->ALIWARD
  </fingerPrintModule>
  <videoStorageTime>
    <!--ro, opt, int, time for saving video, range:[0,10], desc:unit: second-->1
  </videoStorageTime>
  <faceContrastThreshold>
    <!--ro, opt, int, face picture comparison threshold, range:[0,100]-->1
  </faceContrastThreshold>
  <twoDimensionCode>
    <!--ro, opt, enum, whether to enable QR code recognition, subType:string, desc:“enable”, “disable”-->enable
  </twoDimensionCode>
  <blackListCheck>
    <!--ro, opt, enum, whether to enable blocklist verification, subType:string, desc:“enable”, “disable”-->enable
  </blackListCheck>
```

```
  <idCardCheckCenter>
    <!--ro, opt, enum, ID card comparison mode, subType:string, desc:"local" (compare with ID card of local storage), "server" (compare with ID card of remote server storage)-->local
  </idCardCheckCenter>
  <faceAlgorithm>
    <!--ro, opt, enum, face picture algorithm, subType:string, desc:"DeepLearn" (deep learning algorithm), "Tradition" (third-party algorithm)-->DeepLearn
  </faceAlgorithm>
  <comNo>
    <!--ro, opt, int, COM No., range:[1,9]-->1
  </comNo>
  <memoryLearning>
    <!--ro, opt, enum, whether to enable learning and memory function, subType:string, desc:"enable", "disable"-->enable
  </memoryLearning>
  <saveCertifiedImage>
    <!--ro, opt, enum, whether to enable saving authenticated picture, subType:string, desc:"enable", "disable"-->enable
  </saveCertifiedImage>
  <MCUVersion>
    <!--ro, opt, string, MCU version information-->test
  </MCUVersion>
  <usbOutput>
    <!--ro, opt, enum, whether to enable USB output of ID card reader, subType:string, desc:"enable", "disable"-->enable
  </usbOutput>
  <serialOutput>
    <!--ro, opt, enum, whether to enable serial port output of ID card reader, subType:string, desc:"enable", "disable"-->enable
  </serialOutput>
  <readInfoOfCard>
    <!--ro, opt, enum, set content to be read from CPU card, subType:string, desc:"serialNo" (read serial No.), "file" (read file)-->serialNo
  </readInfoOfCard>
  <workMode>
    <!--ro, opt, enum, authentication mode, subType:string, desc:"passMode", "accessControlMode"-->passMode
  </workMode>
  <ecoMode>
    <!--ro, opt, object, ECO mode-->
    <eco>
      <!--ro, opt, enum, whether to enable ECO mode, subType:string, desc:"enable", "disable"-->enable
    </eco>
    <faceMatchThreshold1>
      <!--ro, opt, int, 1V1 face picture comparison threshold of ECO mode, range:[0,100]-->1
    </faceMatchThreshold1>
    <faceMatchThresholdN>
      <!--ro, opt, int, 1:N face picture comparison threshold of ECO mode, range:[0,100]-->1
    </faceMatchThresholdN>
    <changeThreshold>
      <!--ro, opt, int, switching threshold of ECO mode, range:[0,8], desc:switching threshold of ECO mode,which is between 0 and 8-->0
    </changeThreshold>
    <maskFaceMatchThresholdN>
      <!--ro, opt, int, 1:N face picture (face with mask and normal background picture) comparison threshold of ECO mode, range:[0,100]-->1
    </maskFaceMatchThresholdN>
    <maskFaceMatchThreshold1>
      <!--ro, opt, int, 1:1 face picture (face with mask and normal background picture) comparison threshold of ECO mode, range:[0,100]-->1
    </maskFaceMatchThreshold1>
    <alwaysInfrared>
      <!--ro, opt, bool, whether to enable infrared recognition, desc:if this node exists and changeThreshold is 8, it indicates that the device will not always enable the infrared recognition-->true
    </alwaysInfrared>
    <ageFaceMatchList>
      <!--ro, opt, array, list of different age groups in ECO mode, subType:object, range:[0,5], desc:list of different age groups in ECO mode-->
      <ageFaceMatch>
        <!--ro, opt, object, age group matching in ECO mode-->
        <ageLevel>
          <!--ro, opt, enum, age groups, subType:int, desc:0 (child who are 0~6 years old), 1 (teenager who are 7~17years old), 2 (youth and prime who are 18~40 years old), 3 (middle age who are 41~65 years old), 4 (elderly who are more than 66 years old)-->1
        </ageLevel>
        <ageFaceMatchThreshold1>
          <!--ro, opt, int, matching threshold when authenticating via age groups in ECO mode (1:1), range:[0,100]-->1
        </ageFaceMatchThreshold1>
        <ageFaceMatchThresholdN>
          <!--ro, opt, int, matching threshold when authenticating via age groups in ECO mode (1:N), range:[0,100]-->1
        </ageFaceMatchThresholdN>
      </ageFaceMatch>
    </ageFaceMatchList>
  </ecoMode>
  <readCardRule>
    <!--ro, opt, enum, card No. setting rule, subType:string, desc:"wiegand26", "wiegand34"-->wiegand26
  </readCardRule>
  <enableScreenOff>
    <!--ro, opt, bool, whether the device enters the sleep mode when there is no operation after the configured sleep time-->true
  </enableScreenOff>
  <screenOffTimeout>
    <!--ro, opt, int, sleep time, range:[0,3600], unit:s, dep:or,{$.IdentityTerminal.enableScreenOff,eq,true}, desc:unit: second-->1
  </screenOffTimeout>
  <enableScreensaver>
    <!--ro, opt, bool, whether to enable the screen saver function-->true
  </enableScreensaver>
  <faceModuleVersion>
    <!--ro, opt, string, face recognition module version, range:[1,32]-->test
  </faceModuleVersion>
  <showMode>
    <!--ro, opt, enum, display mode, subType:string, desc:"concise" (simple mode,only the authentication result will be displayed), "normal" (normal mode). The default mode is normal mode. If this node does not exist, the default mode is normal mode-->concise
  </showMode>
  <popUpPreviewWindow>
    <!--ro, opt, bool, whether to pop up live view window, dep:or,{$.IdentityTerminal.showMode,eq,advertising}-->true
```

```
    </popUpPreviewWindow>
    <needDeviceCheck>
      <!--ro, opt, bool, whether it need device check in permission free mode, dep:or,{$.IdentityTerminal.workMode,eq,passMode}-->true
    </needDeviceCheck>
    <previewShowTime>
      <!--ro, opt, int, display duration in live view, range:[1,99], unit:s, dep:or,{$.IdentityTerminal.popUpPreviewWindow,eq,true}-->1
    </previewShowTime>
    <screensaverTimeout>
      <!--ro, opt, int, range:[0,3600], unit:s, dep:or,{$.IdentityTerminal.enableScreensaver,eq,true}-->1
    </screensaverTimeout>
    <screensaverDuration>
      <!--ro, opt, int, range:[0,3600], unit:s, dep:or,{$.IdentityTerminal.enableScreensaver,eq,true}-->1
    </screensaverDuration>
    <standbyTimeout>
      <!--ro, opt, int, range:[30,1800], unit:s-->30
    </standbyTimeout>
    <advertisingDisplayType>
      <!--ro, opt, enum, subType:string, dep:or,{$.IdentityTerminal.showMode,eq,advertising}-->full
    </advertisingDisplayType>
  </IdentityTerminal>
```

10.3.7.11 Set the parameters of face recognition terminal

Request URL

PUT /ISAPI/AccessControl/IdentityTerminal

Query Parameter

None

Request Message

```
<?xml version="1.0" encoding="UTF-8"?>
<IdentityTerminal xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--req, object, the parameters of face recognition terminal, attr:version{req, string, protocolVersion}-->
  <terminalMode>
    <!--opt, enum, terminal mode, subType:string, desc:"authMode” (authentication mode), "registerMode” (registration mode)-->authMode
  </terminalMode>
  <idCardReader>
    <!--opt, enum, ID card reader, subType:string, desc:“iDR210”, “DS-K1F110-I”, “DS-K1F1110-B”, “DS-K1F1110-AB”, “none”, “DS-K1F1001-I (USB)”, “DS-K1F1002-
I (USB)”-->iDR210
  </idCardReader>
  <camera>
    <!--opt, enum, camera, subType:string, desc:"C270", "DS-2CS5432B-S"-->C270
  </camera>
  <fingerPrintModule>
    <!--opt, enum, fingerprint module, subType:string, desc:fingerprint module type: ALIWARD,-->ALIWARD
  </fingerPrintModule>
  <videoStorageTime>
    <!--opt, int, time for saving video, range:[0,10], desc:unit: second-->1
  </videoStorageTime>
  <faceContrastThreshold>
    <!--opt, int, face picture comparison threshold, range:[0,100]-->1
  </faceContrastThreshold>
  <twoDimensionCode>
    <!--opt, enum, whether to enable QR code recognition, subType:string, desc:“enable”, “disable”-->enable
  </twoDimensionCode>
  <blackListCheck>
    <!--opt, enum, whether to enable blocklist verification, subType:string, desc:“enable”, “disable”-->enable
  </blackListCheck>
  <idCardCheckCenter>
    <!--opt, enum, ID card comparison mode, subType:string, desc:“local” (compare with ID card of local storage), “server” (compare with ID card of remote
server storage)-->local
  </idCardCheckCenter>
  <faceAlgorithm>
    <!--opt, enum, face picture algorithm, subType:string, desc:"DeepLearn" (deep learning algorithm), "Tradition" (third-party algorithm)-->DeepLearn
  </faceAlgorithm>
  <comNo>
    <!--opt, int, COM No., range:[1,9]-->1
  </comNo>
  <memoryLearning>
    <!--opt, enum, whether to enable learning and memory function, subType:string, desc:“enable”, “disable”-->enable
  </memoryLearning>
  <saveCertifiedImage>
    <!--opt, enum, whether to enable saving authenticated picture, subType:string, desc:“enable”, “disable”-->enable
  </saveCertifiedImage>
  <MCUVersion>
    <!--opt, string, MCU Version-->test
  </MCUVersion>
  <usbOutput>
    <!--opt, enum, whether to enable USB output of ID card reader, subType:string, desc:“enable”, “disable”-->enable
  </usbOutput>
  <serialOutput>
    <!--opt, enum, whether to enable serial port output of ID card reade, subType:string, desc:“enable”, “disable”-->enable
  </serialOutput>
  <readInfoOfCard>
    <!--opt, enum, set content to be read from CPU card, subType:string, desc:“serialNo” (read serial No.), “file” (read file)-->serialNo
  </readInfoOfCard>
```

```xml
    <workMode>
      <!--opt, enum, authentication mode, subType:string, desc:"passMode", "accessControlMode"-->passMode
    </workMode>
    <ecoMode>
      <!--opt, object, ECO mode-->
      <eco>
        <!--opt, enum, whether to enable ECO mode, subType:string, desc:"enable", "disable"-->enable
      </eco>
      <faceMatchThreshold1>
        <!--opt, int, 1V1 face picture comparison threshold of ECO mode, range:[0,100]-->1
      </faceMatchThreshold1>
      <faceMatchThresholdN>
        <!--opt, int, 1VN face picture comparison threshold of ECO mode, range:[0,100]-->1
      </faceMatchThresholdN>
      <changeThreshold>
        <!--opt, int, ECO mode threshold, range:[0,8], desc:switching threshold of ECO mode,which is between 0 and 8-->0
      </changeThreshold>
      <maskFaceMatchThresholdN>
        <!--opt, int, 1:N face picture (face with mask and normal background picture) comparison threshold of ECO mode, range:[0,100]-->1
      </maskFaceMatchThresholdN>
      <maskFaceMatchThreshold1>
        <!--opt, int, 1:1 face picture (face with mask and normal background picture) comparison threshold of ECO mode, range:[0,100]-->1
      </maskFaceMatchThreshold1>
      <alwaysInfrared>
        <!--opt, bool, whether to enable infrared recognition, desc:if this node exists and changeThreshold is 8, it indicates that the device will not always enable the infrared recognition-->true
      </alwaysInfrared>
      <ageFaceMatchList>
        <!--opt, array, list of different age groups in ECO mode, subType:object, range:[0,5], desc:before set this node, age group matching should be enabled, see URL: PUT /ISAPI/AccessControl/FaceCompareCond, and ageFaceMatchEnabled should be true-->
        <ageFaceMatch>
          <!--opt, object, age group matching in ECO mode-->
          <ageLevel>
            <!--opt, enum, age groups, subType:int, desc:0 (child who are 0~6 years old), 1 (teenager who are 7~17years old), 2 (youth and prime who are 18~40 years old), 3 (middle age who are 41~65 years old), 4 (elderly who are more than 66 years old)-->1
          </ageLevel>
          <ageFaceMatchThreshold1>
            <!--opt, int, matching threshold when authenticating via age groups in ECO mode (1:1), range:[0,100]-->1
          </ageFaceMatchThreshold1>
          <ageFaceMatchThresholdN>
            <!--opt, int, matching threshold when authenticating via age groups in ECO mode (1:N), range:[0,100]-->1
          </ageFaceMatchThresholdN>
        </ageFaceMatch>
      </ageFaceMatchList>
    </ecoMode>
    <readCardRule>
      <!--opt, enum, card No. setting rule, subType:string, desc:"wiegand26", "wiegand34"-->wiegand26
    </readCardRule>
    <enableScreenOff>
      <!--opt, bool, whether the device enters the sleep mode when there is no operation after the configured sleep time-->true
    </enableScreenOff>
    <screenOffTimeout>
      <!--opt, int, sleep time, range:[0,3600], unit:s, dep:or,{$.IdentityTerminal.enableScreenOff,eq,true}, desc:unit: second-->1
    </screenOffTimeout>
    <enableScreensaver>
      <!--opt, bool, whether to enable the screen saver function-->true
    </enableScreensaver>
    <faceModuleVersion>
      <!--opt, string, face recognition module version, range:[1,32]-->test
    </faceModuleVersion>
    <showMode>
      <!--opt, enum, display mode, subType:string, desc:simple mode indicates that the device displays authentication results exclude employee No., name, etc.; the device applies normal mode by default; advertisement mode indicates that the device displays both advertisement and authentication results; meeting mode indicates that the device displays check-in page of the conference; custom mode indicates that the device displays layout of the interface customized by users; "concise"-simple mode, "normal"-normal mode (default), "advertising"-advertisement mode, "meeting"-meeting mode, "selfDefine"-custom mode-->concise
    </showMode>
    <popUpPreviewWindow>
      <!--opt, bool, whether to pop up live view window, dep:or,{$.IdentityTerminal.showMode,eq,advertising}-->true
    </popUpPreviewWindow>
    <needDeviceCheck>
      <!--opt, bool, whether it need device check in permission free mode, dep:or,{$.IdentityTerminal.workMode,eq,passMode}-->true
    </needDeviceCheck>
    <previewShowTime>
      <!--opt, int, display duration in live view, range:[1,99], unit:s, dep:or,{$.IdentityTerminal.popUpPreviewWindow,eq,true}-->1
    </previewShowTime>
    <screensaverTimeout>
      <!--opt, int, range:[0,3600], unit:s, dep:or,{$.IdentityTerminal.enableScreensaver,eq,true}-->1
    </screensaverTimeout>
    <screensaverDuration>
      <!--opt, int, range:[0,3600], unit:s, dep:or,{$.IdentityTerminal.enableScreensaver,eq,true}-->1
    </screensaverDuration>
    <standbyTimeout>
      <!--opt, int, range:[30,1800], unit:s-->30
    </standbyTimeout>
    <advertisingDisplayType>
      <!--opt, enum, subType:string, dep:or,{$.IdentityTerminal.showMode,eq,advertising}-->full
    </advertisingDisplayType>
  </IdentityTerminal>
```


Response Message

```xml
<?xml version="1.0" encoding="UTF-8"?>
<ResponseStatus xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, response status, attr:version{req, string, protocolVersion}-->
  <requestURL>
    <!--ro, req, string, request URL, desc:request URL-->test
  </requestURL>
  <statusCode>
    <!--ro, req, enum, status code, subType:int, desc:read-only,status code: 0,1-OK,2-Device Busy,3-Device Error,4-Invalid Operation,5-Invalid XML Format,6-Invalid XML Content,7-Reboot Required,9-Additional Error-->1
  </statusCode>
  <statusString>
    <!--ro, req, enum, read-only,status description, subType:string, desc:“OK” (succeeded), “Device Busy”, “Device Error”, “Invalid Operation”, “Invalid XML Format”, “Invalid XML Content”, “Reboot” (reboot device)-->OK
  </statusString>
  <subStatusCode>
    <!--ro, req, string, sub status code, desc:read-only,describe the error reason in detail-->test
  </subStatusCode>
</ResponseStatus>
```


10.3.7.12 Get configuration capability of face recognition terminal

Request URL

GET /ISAPI/AccessControl/IdentityTerminal/capabilities

Query Parameter

None

Request Message

None

Response Message


```xml
<?xml version="1.0" encoding="UTF-8"?>
<IdentityTerminal xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, parameters of face recognition terminal, attr:version{req, string, protocolVersion}-->
  <terminalMode opt="authMode,registerMode">
    <!--ro, opt, enum, terminal mode, subType:string, attr:opt{req, string}, desc:"authMode" (authentication mode), "registerMode" (registration mode)-->authMode
  </terminalMode>
  <idCardReader opt="iDR210,DS-K1F110-I,DS-K1F1110-B,DS-K1F1110-AB,none ">
    <!--ro, opt, enum, ID card reader model, subType:string, attr:opt{req, string}, desc:“iDR210”, “DS-K1F110-I”, “DS-K1F1110-B”, “DS-K1F1110-AB”, “none”, “DS-K1F1001-I (USB)”, “DS-K1F1002-I (USB)”-->iDR210
  </idCardReader>
  <camera opt="C270,DS-2CS5432B-S">
    <!--ro, opt, enum, camera, subType:string, attr:opt{req, string}, desc:"C270", "DS-2CS5432B-S"-->C270
  </camera>
  <fingerPrintModule opt="ALIWARD,">
    <!--ro, opt, enum, fingerprint module, subType:string, attr:opt{req, string}, desc:fingerprint module-->ALIWARD
  </fingerPrintModule>
  <videoStorageTime min="0" max="10">
    <!--ro, opt, int, time for saving video (unit: second), range:[0,10], attr:min{req, int},max{req, int}, desc:unit: second-->1
  </videoStorageTime>
  <faceContrastThreshold min="0" max="100">
    <!--ro, opt, int, face picture comparison threshold, range:[0,100], attr:min{req, int},max{req, int}-->1
  </faceContrastThreshold>
  <twoDimensionCode opt="enable,disable">
    <!--ro, opt, enum, whether to enable QR code recognition, subType:string, attr:opt{req, string}, desc:“enable”, “disable”-->enable
  </twoDimensionCode>
  <blackListCheck opt="enable,disable">
    <!--ro, opt, enum, whether to enable blocklist verification, subType:string, attr:opt{req, string}, desc:“enable”, “disable”-->enable
  </blackListCheck>
  <idCardCheckCenter opt="local,server">
    <!--ro, opt, enum, ID card comparison mode, subType:string, attr:opt{req, string}, desc:“local” (compare with ID card of local storage), “server” (compare with ID card of remote server storage)-->local
  </idCardCheckCenter>
  <faceAlgorithm opt="DeepLearn,Tradition">
    <!--ro, opt, enum, face picture algorithm, subType:string, attr:opt{req, string}, desc:"DeepLearn" (deep learning algorithm), "Tradition" (third-party algorithm)-->DeepLearn
  </faceAlgorithm>
  <comNo min="1" max="9">
    <!--ro, opt, int, COM No., range:[1,9], attr:min{req, int},max{req, int}-->1
  </comNo>
  <memoryLearning opt="enable,disable">
    <!--ro, opt, enum, whether to enable learning and memory function, subType:object, attr:opt{req, string}, desc:“enable”, “disable”-->enable
  </memoryLearning>
  <saveCertifiedImage opt="enable,disable">
    <!--ro, opt, enum, whether to enable saving authenticated picture, subType:string, attr:opt{req, string}, desc:“enable”, “disable”-->enable
  </saveCertifiedImage>
  <MCUVersion min="1" max="10">
    <!--ro, opt, string, MCU version information, attr:min{req, int},max{req, int}-->test
  </MCUVersion>
  <usbOutput opt="enable,disable">
    <!--ro, opt, enum, whether to enable USB output of ID card reader, subType:string, attr:opt{req, string}, desc:“enable”, “disable”-->enable
  </usbOutput>
```

```
    <serialOutput opt="enable,disable">
      <!--ro, opt, enum, whether to enable serial port output of ID card reader, subType:string, attr:opt{req, string}, desc:“enable”, “disable”-->enable
    </serialOutput>
    <readInfoOfCard opt="serialNo,file">
      <!--ro, opt, enum, set content to be read from CPU card, subType:string, attr:opt{req, string}, desc:"serialNo" (read the serial No.), "file" (read the file)-->serialNo
    </readInfoOfCard>
    <workMode opt="passMode,accessControlMode">
      <!--ro, opt, enum, authentication mode, subType:string, attr:opt{req, string}, desc:authentication mode-->passMode
    </workMode>
    <ecoMode>
      <!--ro, opt, object, ECO mode-->
      <eco opt="enable,disable">
        <!--ro, opt, enum, whether to enable ECO mode, subType:string, attr:opt{req, string}, desc:“enable”, “disable”-->enable
      </eco>
      <faceMatchThreshold1 min="0" max="100">
        <!--ro, opt, int, 1V1 face picture comparison threshold of ECO mod, range:[0,100], attr:min{req, int},max{req, int}-->1
      </faceMatchThreshold1>
      <faceMatchThresholdN min="0" max="100">
        <!--ro, opt, int, 1:N face picture comparison threshold of ECO mode, range:[0,100], attr:min{req, int},max{req, int}-->1
      </faceMatchThresholdN>
      <changeThreshold min="0" max="8">
        <!--ro, opt, int, switching threshold of ECO mode, range:[0,8], attr:min{req, int},max{req, int}, desc:switching threshold of ECO mode,which is between 0 and 8-->0
      </changeThreshold>
      <maskFaceMatchThresholdN min="0" max="100">
        <!--ro, opt, int, 1:N face picture (face with mask and normal background picture) comparison threshold of ECO mode, range:[0,100], attr:min{req, int},max{req, int}-->1
      </maskFaceMatchThresholdN>
      <maskFaceMatchThreshold1 min="0" max="100">
        <!--ro, opt, int, 1:1 face picture (face with mask and normal background picture) comparison threshold of ECO mode, range:[0,100], attr:min{req, int},max{req, int}-->1
      </maskFaceMatchThreshold1>
      <alwaysInfrared opt="true,false">
        <!--ro, opt, bool, whether to enable infrared recognition, attr:opt{req, string}, desc:if this node exists and changeThreshold is 8, it indicates that the device will not always enable the infrared recognition-->true
      </alwaysInfrared>
      <ageFaceMatchList size="5">
        <!--ro, opt, array, list of different age groups in ECO mode, subType:object, range:[0,5], attr:size{req, int}-->
        <ageFaceMatch>
          <!--ro, opt, object, age group matching in ECO mode-->
          <ageLevel opt="0,1,2,3,4">
            <!--ro, opt, enum, age groups, subType:int, attr:opt{req, string}, desc:0 (child who are 0~6 years old), 1 (teenager who are 7~17years old), 2 (youth and prime who are 18~40 years old), 3 (middle age who are 41~65 years old), 4 (elderly who are more than 66 years old)-->1
          </ageLevel>
          <ageFaceMatchThreshold1 min="0" max="100">
            <!--ro, opt, int, matching threshold when authenticating via age groups in ECO mode (1:1), range:[0,100], attr:min{req, int},max{req, int}-->1
          </ageFaceMatchThreshold1>
          <ageFaceMatchThresholdN min="0" max="100">
            <!--ro, opt, int, matching threshold when authenticating via age groups in ECO mode (1:N), range:[0,100], attr:min{req, int},max{req, int}-->1
          </ageFaceMatchThresholdN>
        </ageFaceMatch>
      </ageFaceMatchList>
    </ecoMode>
    <readCardRule opt="wiegand26,wiegand34">
      <!--ro, opt, enum, card No. setting rule, subType:string, attr:opt{req, string}, desc:card No. setting rule: "wiegand26","wiegand34"-->wiegand26
    </readCardRule>
    <enableScreenOff opt="true,false">
      <!--ro, opt, bool, whether the device enters the sleep mode when there is no operation after the configured sleep time, attr:opt{req, string}-->true
    </enableScreenOff>
    <screenOffTimeout min="0" max="3600">
      <!--ro, opt, int, sleep time, range:[0,3600], unit:s, attr:min{req, int},max{req, int}-->1
    </screenOffTimeout>
    <enableScreensaver opt="true,false">
      <!--ro, opt, bool, whether to enable the screen saver function, attr:opt{req, string}-->true
    </enableScreensaver>
    <faceModuleVersion min="0" max="10">
      <!--ro, opt, string, face recognition module version, attr:min{req, int},max{req, int}-->test
    </faceModuleVersion>
    <showMode opt="concise,normal,advertising,meeting,selfDefine,boxStatus,clock">
      <!--ro, opt, enum, display mode, subType:string, attr:opt{req, string}, desc:"concise" (simple mode,only the authentication result will be displayed), "normal" (normal mode). The default mode is normal mode. If this node does not exist, the default mode is normal mode-->concise
    </showMode>
    <popUpPreviewWindow opt="true,false">
      <!--ro, opt, bool, whether to pop up live view window, dep:or,{$.IdentityTerminal.showMode,eq,advertising}, attr:opt{req, string}-->true
    </popUpPreviewWindow>
    <needDeviceCheck opt="true,false">
      <!--ro, opt, bool, whether it need device check in permission free mode, dep:or,{$.IdentityTerminal.workMode,eq,passMode}, attr:opt{req, string}-->true
    </needDeviceCheck>
    <previewShowTime min="1" max="99">
      <!--ro, opt, int, display duration in live view, range:[1,99], unit:s, attr:min{req, int},max{req, int}-->1
    </previewShowTime>
    <screensaverTimeout min="0" max="3600">
      <!--ro, opt, int, range:[0,3600], unit:s, attr:min{req, int},max{req, int}-->1
    </screensaverTimeout>
    <screensaverDuration min="0" max="3600">
      <!--ro, opt, int, range:[0,3600], unit:s, attr:min{req, int},max{req, int}-->1
    </screensaverDuration>
    <standbyTimeout min="30" max="1800">
      <!--ro, opt, int, range:[30,1800], unit:s, attr:min{req, int},max{req, int}-->30
    </standbyTimeout>
    <advertisingDisplayType opt="full,split">
      <!--ro, opt, enum, subType:string, attr:opt{req, string}-->full
    </advertisingDisplayType>
```

```
</IdentityTerminal>
```

10.3.7.13 Set the parameters of M1 card encryption verification

Request URL

PUT /ISAPI/AccessControl/M1CardEncryptCfg

Query Parameter

None

Request Message

```xml
<?xml version="1.0" encoding="UTF-8"?>

<M1CardEncryptCfg xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--req, object, configuration capability of the M1 card encryption verification, attr:version{req, string, protocolVersion}-->
  <enable>
    <!--req, bool, whether to enable the function-->true
  </enable>
  <sectionID>
    <!--req, int, sector ID, desc:only one sector can be configured at a time-->1
  </sectionID>
</M1CardEncryptCfg>
```

Response Message

```xml
<?xml version="1.0" encoding="UTF-8"?>

<ResponseStatus xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, response status, attr:version{req, string, protocolVersion}-->
  <requestURL>
    <!--ro, req, string, request URL, desc:request URL-->test
  </requestURL>
  <statusCode>
    <!--ro, req, enum, status code, subType:string, desc:0 (OK), 1 (OK), 2 (Device Busy), 3 (Device Error), 4 (Invalid Operation), 5 (Invalid XML Format), 6 
(Invalid XML Content), 7 (Reboot Required)-->1
  </statusCode>
  <statusString>
    <!--ro, req, enum, status description, subType:string, desc:"OK,Device Busy,Device Error,Invalid Operation,Invalid XML Format,Invalid XML 
Content,Reboot"-->OK
  </statusString>
  <subStatusCode>
    <!--ro, req, string, sub status code, desc:sub status code description-->test
  </subStatusCode>
</ResponseStatus>
```

10.3.7.14 Get the configuration parameters of M1 card encryption verification

Request URL

GET /ISAPI/AccessControl/M1CardEncryptCfg

Query Parameter

None

Request Message

None

Response Message

```xml
<?xml version="1.0" encoding="UTF-8"?>

<M1CardEncryptCfg xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, configuration capability of the M1 card encryption verification, attr:version{req, string, protocolVersion}-->
  <enable>
    <!--ro, req, bool, whether to enable the function-->true
  </enable>
  <sectionID>
    <!--ro, req, int, sector ID, desc:sector ID,only one sector can be configured at a time-->1
  </sectionID>
</M1CardEncryptCfg>
```

10.3.7.15 Get the configuration capability of the M1 card encryption verification

Request URL

GET /ISAPI/AccessControl/M1CardEncryptCfg/capabilities

Query Parameter

None

Request Message

None

Response Message


```xml
<?xml version="1.0" encoding="UTF-8"?>

<M1CardEncryptCfg xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, configuration capability of the M1 card encryption verification, attr:version{req, string, protocolVersion}-->
  <enable opt="true,false">
    <!--ro, req, bool, whether to enable, attr:opt{req, string}-->true
  </enable>
  <sectionID min="0" max="100">
    <!--ro, req, int, sector ID, range:[0,100], attr:min{req, int},max{req, int}-->1
  </sectionID>
</M1CardEncryptCfg>
```


10.3.7.16 Get the capability of Wiegand parameters

Request URL

GET /ISAPI/AccessControl/WiegandCfg/capabilities

Query Parameter

None

Request Message

None

Response Message


```xml
<?xml version="1.0" encoding="UTF-8"?>

<WiegandCfg xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, configuration of Wiegand parameters, attr:version{req, string, protocolVersion}-->
  <wiegandNo min="1" max="4" opt="1,4">
    <!--ro, req, int, Wiegand interface No., range:[0,4], step:1, attr:min{opt, int},max{opt, int},opt{opt, string}, desc:Wiegand interface No.-->1
  </wiegandNo>
  <communicateDirection opt="receive,send">
    <!--ro, req, enum, communication direction, subType:string, attr:opt{req, string}, desc:"receive", "send"-->receive
  </communicateDirection>
  <wiegandMode
opt="wiegand26,wiegand34,wiegand27,wiegand35,Corporate1000_35,Corporate1000_48,H10302_37,H10304_37,wiegand_26CSN,H103130_32CSN,wiegand_56CSN,wiegand_58">
    <!--ro, opt, enum, Wiegand mode, subType:string, attr:opt{req, string}, desc:Wiegand mode-->wiegand26
  </wiegandMode>
  <inputWiegandMode>
    <!--ro, opt, enum, subType:string, dep:or,{$.WiegandCfg.communicateDirection,eq,receive}-->wiegand26
  </inputWiegandMode>
  <signalInterval min="1" max="20">
    <!--ro, opt, int, it is between 1 and 20,unit: ms, range:[1,20], attr:min{req, int},max{req, int}, desc:it is between 1 and 20,unit: ms-->1
  </signalInterval>
  <enable opt="true,false">
    <!--ro, opt, bool, whether to enable Wiegand parameters, attr:opt{req, string}-->true
  </enable>
  <pulseDuration min="1" max="10">
    <!--ro, opt, int, pulse duration, range:[1,10], attr:min{req, int},max{req, int}, desc:pulse duration-->1
  </pulseDuration>
  <facilityCodeEnabled opt="true,false">
    <!--ro, opt, bool, whether to enable facilityCode, attr:opt{req, string}-->true
  </facilityCodeEnabled>
  <facilityCode min="0" max="65535">
    <!--ro, opt, int, range:[0,65535], dep:and,{$.WiegandCfg.facilityCodeEnabled,eq,true}, attr:min{req, int, range:[0,65535]},max{req, int, range:
[0,65535]}-->1
  </facilityCode>
  <dataType opt="employeeNo,cardNo">
    <!--ro, opt, enum, data type, subType:string, dep:or,{$.WiegandCfg.wiegandMode,eq,send}, attr:opt{req, string}, desc:data type-->employeeNo
  </dataType>
</WiegandCfg>
```


10.3.7.17 Set Wiegand parameters

Request URL

PUT /ISAPI/AccessControl/WiegandCfg/wiegandNo/<wiegandID>

Query Parameter

Parameter Name	Parameter Type	Description
wiegandID	string	--

Request Message

```
<?xml version="1.0" encoding="UTF-8"?>

<WiegandCfg xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--wo, req, object, Wiegand parameters, attr:version{req, string, protocolVersion}-->
  <communicateDirection>
    <!--wo, req, enum, communication direction, subType:string, desc:"receive", "send"-->receive
  </communicateDirection>
  <wiegandMode>
    <!--wo, opt, enum, Wiegand mode, subType:string, dep:or,{$.WiegandCfg.communicateDirection,eq,send}, desc:"wiegand26", "wiegand34", "wiegand27",
"wiegand35", "Corporate1000_35", "Corporate1000_48", "H10302_37", "H10304_37", "wiegand_26CSN", "H103130_32CSN", "wiegand_56CSN", "wiegand_58"-->wiegand26
  </wiegandMode>
  <inputWiegandMode>
    <!--wo, opt, enum, subType:string, dep:or,{$.WiegandCfg.communicateDirection,eq,receive}-->wiegand26
  </inputWiegandMode>
  <signalInterval>
    <!--wo, opt, int, it is between 1 and 20,unit: ms, range:[1,20], desc:unit: ms-->1
  </signalInterval>
  <enable>
    <!--wo, opt, bool, whether to enable the function or not-->true
  </enable>
  <pulseDuration>
    <!--wo, opt, int, range:[1,10]-->1
  </pulseDuration>
  <facilityCodeEnabled>
    <!--opt, bool-->true
  </facilityCodeEnabled>
  <facilityCode>
    <!--opt, int, range:[0,65535], dep:and,{$.WiegandCfg.facilityCodeEnabled,eq,true}-->1
  </facilityCode>
  <dataType>
    <!--opt, enum, data type, subType:string, dep:or,{$.WiegandCfg.wiegandMode,eq,send}, desc:data type-->employeeNo
  </dataType>
</WiegandCfg>
```

Response Message

```
<?xml version="1.0" encoding="UTF-8"?>

<ResponseStatus xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, response message, attr:version{ro, req, string, protocolVersion}-->
  <requestURL>
    <!--ro, req, string, request URL-->null
  </requestURL>
  <statusCode>
    <!--ro, req, enum, status code, subType:int, desc:0 (OK), 1 (OK), 2 (Device Busy), 3 (Device Error), 4 (Invalid Operation), 5 (Invalid XML Format), 6
(Invalid XML Content), 7 (Reboot Required)-->0
  </statusCode>
  <statusString>
    <!--ro, req, enum, status information, subType:string, desc:"OK" (succeeded), "Device Busy", "Device Error", "Invalid Operation", "Invalid XML Format",
"Invalid XML Content", "Reboot" (reboot device)-->OK
  </statusString>
  <subStatusCode>
    <!--ro, req, string, sub status code, which describes the error in details, desc:sub status code, which describes the error in details-->OK
  </subStatusCode>
</ResponseStatus>
```


10.3.7.18 Get Wiegand parameters

Request URL

GET /ISAPI/AccessControl/WiegandCfg/wiegandNo/<wiegandID>

Query Parameter

Parameter Name	Parameter Type	Description
wiegandID	string	--

Request Message

None

Response Message

```
<?xml version="1.0" encoding="UTF-8"?>

<WiegandCfg xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, Wiegand parameters, attr:version{req, string, protocolVersion}-->
  <communicateDirection>
    <!--ro, req, enum, communication direction, subType:string, desc:"receive", "send"-->receive
  </communicateDirection>
  <wiegandMode>
    <!--ro, opt, enum, Wiegand mode, subType:string, dep:or,{$.WiegandCfg.communicateDirection,eq,send}, desc:"wiegand26", "wiegand34", "wiegand27",
"wiegand35", "Corporate1000_35", "Corporate1000_48", "H10302_37", "H10304_37", "wiegand_26CSN", "H103130_32CSN", "wiegand_56CSN", "wiegand_58"-->wiegand26
  </wiegandMode>
  <inputWiegandMode>
    <!--ro, opt, enum, subType:string, dep:or,{$.WiegandCfg.communicateDirection,eq,receive}-->wiegand26
  </inputWiegandMode>
  <signalInterval>
    <!--ro, opt, int, Wiegand signal sending interval, range:[1,20], desc:unit: ms-->1
  </signalInterval>
  <enable>
    <!--ro, opt, bool, whether to enable the function or not-->true
  </enable>
  <pulseDuration>
    <!--ro, opt, int, range:[1,10]-->1
  </pulseDuration>
  <facilityCodeEnabled>
    <!--ro, opt, bool-->true
  </facilityCodeEnabled>
  <facilityCode>
    <!--ro, opt, int, range:[0,65535], dep:and,{$.WiegandCfg.facilityCodeEnabled,eq,true}-->1
  </facilityCode>
  <dataType>
    <!--ro, opt, enum, data type, subType:string, dep:or,{$.WiegandCfg.wiegandMode,eq,send}, desc:data type-->employeeNo
  </dataType>
</WiegandCfg>
```

10.3.8 Multi-Factor Authentication Management

10.3.8.1 Get the capability of clearing group parameters

Request URL

GET /ISAPI/AccessControl/ClearGroupCfg/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "ClearGroupCfg": {
    /*ro, opt, object*/
        "ClearFlags": {
        /*ro, opt, object*/
            "groupCfg":  "true,false"
            /*ro, req, string, group parameters*/
        }
    }
}
```

10.3.8.2 Clear group parameters

Request URL

PUT /ISAPI/AccessControl/ClearGroupCfg?format=json

Query Parameter

None

Request Message

```
{
    "ClearGroupCfg": {
    /*opt, object, group parameters*/
        "ClearFlags": {
        /*opt, object*/
            "groupCfg":  true
            /*req, bool, whether to clear group parameters*/
        }
    }
}
```

Response Message

```
{
    "requestURL":  "test",
    /*ro, opt, string, URI*/
    "statusCode":  "test",
    /*ro, opt, string, status code*/
    "statusString":  "test",
    /*ro, opt, string, status description*/
    "subStatusCode":  "test",
    /*ro, opt, string, sub status code*/
    "errorCode":  1,
    /*ro, req, int, error code*/
    "errorMsg":  "ok"
    /*ro, req, string, error description*/
}
```

10.3.8.3 Get the group configuration parameters

Request URL

GET /ISAPI/AccessControl/GroupCfg/<groupID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
groupID	string	--

Request Message

None

Response Message

```
{
    "GroupCfg": {
    /*ro, opt, object, group parameters*/
        "enable":  true,
        /*ro, req, bool, whether to enable the function*/
        "ValidPeriodCfg": {
        /*ro, opt, object, validity period of the group*/
            "enable":  true,
            /*ro, req, bool, whether to enable validity period*/
            "beginTime":  "1970-01-01T00:00:00+08:00",
            /*ro, req, datetime, start time of the validity period (UTC time)*/
            "endTime":  "1970-01-01T00:00:00+08:00"
            /*ro, req, datetime, end time of the validity period (UTC time)*/
        },
        "groupName":  "test"
        /*ro, opt, string*/
    }
}
```


10.3.8.4 Set the group parameters

Request URL

PUT /ISAPI/AccessControl/GroupCfg/<groupID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
groupID	string	--

Request Message

```
{
    "GroupCfg": {
    /*opt, object, group parameters*/
        "enable":  true,
        /*req, bool, whether to enable the group*/
        "ValidPeriodCfg": {
        /*opt, object, validity period of the group*/
            "enable":  true,
            /*req, bool, whether to enable validity period*/
            "beginTime":  "1970-01-01T00:00:00+08:00",
            /*req, datetime, start time of the validity period (UTC time)*/
            "endTime":  "1970-01-01T00:00:00+08:00"
            /*req, datetime, end time of the validity period (UTC time)*/
        },
        "groupName":  "test"
        /*opt, string, group name*/
    }
}
```

Response Message

```
{
    "requestURL":  "test",
    /*ro, opt, string, request URL*/
    "statusCode":  "test",
    /*ro, opt, string, status code*/
    "statusString":  "test",
    /*ro, opt, string, status description*/
    "subStatusCode":  "test",
    /*ro, opt, string, sub status code*/
    "errorCode":  1,
    /*ro, req, int, error code*/
    "errorMsg":  "ok"
    /*ro, req, string, error information*/
}
```

10.3.8.5 Get the group configuration capability

Request URL

GET /ISAPI/AccessControl/GroupCfg/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "GroupCfg": {
    /*ro, opt, object, group parameters*/
        "groupNo": {
        /*ro, opt, object, group No.*/
            "@min":  1,
            /*ro, opt, int, the minimum value*/
            "@max":  1
            /*ro, opt, int, the maximum value*/
        },
        "enable":  "true,false",
        /*ro, req, string, whether to enable the group*/
        "ValidPeriodCfg": {
        /*ro, req, object, whether to enable validity period parameters of the group*/
            "enable":  "true,false",
            /*ro, req, string, whether to enable validity period*/
            "beginTime": {
            /*ro, req, object, start time of the validity period (UTC time)*/
                "@min":  1,
                /*ro, opt, int, the minimum value*/
                "@max":  32
                /*ro, opt, int, the maximum value*/
            },
            "endTime": {
            /*ro, req, object, end time of the validity period (UTC time)*/
                "@min":  1,
                /*ro, opt, int, the minimum value*/
                "@max":  32
                /*ro, opt, int, the maximum value*/
            }
        },
        "groupName": {
        /*ro, opt, object, group name*/
            "@min":  1,
            /*ro, opt, int, the minimum value*/
            "@max":  32
            /*ro, opt, int, the maximum value*/
        }
    }
}
```

10.3.8.6 Get the configuration parameters of multi-factor authentication mode

Request URL

GET /ISAPI/AccessControl/MultiCardCfg/<doorID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
doorID	string	--

Request Message

None

Response Message

```
{
    "MultiCardCfg": {
    /*ro, opt, object*/
        "enable":  true,
        /*ro, req, bool, whether to enable multi-factor authentication*/
        "swipeIntervalTimeout":  10,
        /*ro, opt, int, timeout of swiping (authentication) interval*/
        "GroupCfg": [
        /*ro, opt, array, multi-factor authentication parameters, subType:object*/
            {
                "id":  1,
                /*ro, opt, int, multi-factor authentication No.*/
                "enable":  true,
                /*ro, opt, bool, whether to enable multi-factor authentication*/
                "enableOfflineVerifyMode":  true,
                /*ro, opt, bool, whether to enable verification mode when the access control device is offline (the super password will replace opening door remotely)*/
                "templateNo":  1,
                /*ro, opt, int, schedule template No. to enable the multi-factor authentication*/
                "GroupCombination": [
                /*ro, opt, array, multi-factor authentication parameters, subType:object*/
                    {
                        "enable":  true,
                        /*ro, opt, bool, whether to enable multi-factor authentication*/
                        "memberNum":  3,
                        /*ro, opt, int, number of members swiping cards*/
                        "sequenceNo":  1,
                        /*ro, opt, int, serial No. of swiping cards of the multi-factor authentication group*/
                        "groupNo":  1
                        /*ro, opt, int, group No.*/
                    }
                ]
            }
        ]
    }
}
```

10.3.8.7 Set parameters of multi-factor authentication mode

Request URL

PUT /ISAPI/AccessControl/MultiCardCfg/<doorID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
doorID	string	--

Request Message

```
{
    "MultiCardCfg": {
    /*opt, object, parameters of multi-factor authentication mode*/
        "enable":  true,
        /*req, bool, whether to enable multi-factor authentication*/
        "swipeIntervalTimeout":  10,
        /*opt, int, timeout of swiping (authentication) interval*/
        "GroupCfg": [
        /*opt, array, multi-factor authentication parameters, subType:object*/
            {
                "id":  1,
                /*opt, int, multi-factor authentication No.*/
                "enable":  true,
                /*opt, bool, whether to enable multi-factor authentication*/
                "enableOfflineVerifyMode":  true,
                /*opt, bool, whether to enable verification mode when the access control device is offline (the super password will replace opening door remotely)*/
                "templateNo":  1,
                /*opt, int, schedule template No. to enable the multi-factor authentication*/
                "GroupCombination": [
                /*opt, array, parameters of the multi-factor authentication group, subType:object*/
                    {
                        "enable":  true,
                        /*opt, bool, whether to enable multi-factor authentication*/
                        "memberNum":  3,
                        /*opt, int, number of members swiping cards*/
                        "sequenceNo":  1,
                        /*opt, int, serial No. of swiping cards of the multi-factor authentication group*/
                        "groupNo":  1
                        /*opt, int, group No.*/
                    }
                ]
            }
        ]
    }
}
```

Response Message

```
{
    "requestURL":  "test",
    /*ro, opt, string, URI*/
    "statusCode":  "test",
    /*ro, opt, string, status code*/
    "statusString":  "test",
    /*ro, opt, string, status description*/
    "subStatusCode":  "test",
    /*ro, opt, string, sub status code*/
    "errorCode":  1,
    /*ro, req, int, error code*/
    "errorMsg":  "ok"
    /*ro, req, string, error information*/
}
```


10.3.8.8 Get the configuration capability of multi-factor authentication mode

Request URL

GET /ISAPI/AccessControl/MultiCardCfg/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "MultiCardCfg": {
    /*ro, opt, object*/
        "doorNo": {
        /*ro, opt, object, door No.*/
            "@min":  1,
            /*ro, opt, int, minimum value*/
            "@max":  256
            /*ro, opt, int, maximum value*/
        },
        "enable":  "true,false",
        /*ro, req, string, whether to enable multi-factor authentication*/
        "swipeIntervalTimeout": {
        /*ro, opt, object, timeout of swiping (authentication) interval*/
            "@min":  1,
            /*ro, opt, int, minimum value*/
            "@max":  255
            /*ro, opt, int, maximum value*/
        },
        "GroupCfg": {
        /*ro, opt, object, multi-factor authentication parameters*/
            "maxSize":  20,
            /*ro, opt, int, maximum value*/
            "id": {
            /*ro, opt, object, multi-factor authentication No.*/
                "@min":  1,
                /*ro, opt, int, minimum value*/
                "@max":  20
                /*ro, opt, int, maximum value*/
            },
            "enable":  "true,false",
            /*ro, opt, string, whether to enable multi-factor authentication*/
            "enableOfflineVerifyMode":  "true,false",
            /*ro, opt, string, whether to enable verification mode when the access control device is offline (the super password will replace opening door 
remotely)*/
            "templateNo": {
            /*ro, opt, object, schedule template No. to enable the multi-factor authentication*/
                "@min":  1,
                /*ro, opt, int, minimum value*/
                "@max":  20
                /*ro, opt, int, maximum value*/
            },
            "GroupCombination": {
            /*ro, opt, object, parameters of the multi-factor authentication group*/
                "maxSize":  8,
                /*ro, opt, int, maximum value*/
                "enable":  "true,false",
                /*ro, opt, string, whether to enable multi-factor authentication*/
                "memberNum": {
                /*ro, opt, object, number of members swiping cards*/
                    "@min":  1,
                    /*ro, opt, int, minimum value*/
                    "@max":  20
                    /*ro, opt, int, maximum value*/
                },
                "sequenceNo": {
                /*ro, opt, object, serial No. of swiping cards of the multi-factor authentication group*/
                    "@min":  1,
                    /*ro, opt, int, minimum value*/
                    "@max":  8
                    /*ro, opt, int, maximum value*/
                },
                "groupNo": {
                /*ro, opt, object, group No.*/
                    "@min":  1,
                    /*ro, opt, int, minimum value*/
                    "@max":  20
                    /*ro, opt, int, the maximum value*/
                }
            }
        }
    }
}
```


10.3.9 Permission Schedules for Persons and Access Points

10.3.9.1 Set the holiday group parameters of the access permission control schedule

Request URL

PUT /ISAPI/AccessControl/UserRightHolidayGroupCfg/<holidayGroupID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
holidayGroupID	string	--

Request Message

```
{
    "UserRightHolidayGroupCfg": {
    /*req, object, the holiday group parameters of the access permission control schedule*/
        "enable":  true,
        /*req, bool, whether to enable, desc:true (yes), false (no)*/
        "groupName":  "test",
        /*req, string, holiday group name*/
        "holidayPlanNo":  "1,3,5",
        /*req, string, holiday group schedule No., desc:holiday group schedule No.*/
        "operateType":  "byTerminal",
        /*opt, enum, operation type, subType:string, desc:"byTerminal" (by terminal), "byOrg" (by organization), "byTerminalOrg" (by terminal
organization)*/
        "terminalNoList": [1, 2, 3, 4],
        /*opt, array, terminal ID list, subType:int, desc:this node is required when operation type is "byTerminal" or "byTerminalOrg"*/
        "orgNoList": [1, 2, 3, 4]
        /*opt, array, organization ID list, subType:int, desc:this node is required when operation type is "byOrg" or "byTerminalOrg"*/
    }
}
```

Response Message

```
{
    "requestURL":  "test",
    /*ro, opt, string, URI*/
    "statusCode":  "test",
    /*ro, opt, string, status code, desc:1 (succeeded). It is required when an error occurred*/
    "statusString":  "test",
    /*ro, opt, string, status description, desc:"ok" (succeeded). It is required when an error occurred*/
    "subStatusCode":  "test",
    /*ro, opt, string, sub status code, desc:"ok" (succeeded). It is required when an error occurred*/
    "errorCode":  1,
    /*ro, req, int, error code, desc:it is required when the value of statusCode is not 1, and it corresponds to subStatusCode*/
    "errorMsg":  "ok"
    /*ro, req, string, error information, desc:this node is required when the value of statusCode is not 1*/
}
```


10.3.9.2 Get the holiday group configuration parameters of the access permission control schedule

Request URL

GET /ISAPI/AccessControl/UserRightHolidayGroupCfg/<holidayGroupID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
holidayGroupID	string	--

Request Message

None

Response Message

```
{
    "UserRightHolidayGroupCfg": {
    /*ro, req, object*/
        "enable":  true,
        /*ro, req, bool, whether to enable, desc:true (yes), false (no)*/
        "groupName":  "test",
        /*ro, req, string, holiday group name*/
        "holidayPlanNo":  "1,3,5"
        /*ro, req, string, holiday group schedule No., desc:holiday group schedule No.*/
    }
}
```

10.3.9.3 Get the holiday group configuration capability of the access permission control

Request URL

GET /ISAPI/AccessControl/UserRightHolidayGroupCfg/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "UserRightHolidayGroupCfg": {
    /*ro, req, object*/
        "groupNo": {
        /*ro, opt, object, holiday group No.*/
            "@min":  1,
            /*ro, opt, int, the minimum value*/
            "@max":  16
            /*ro, opt, int, the maximum value*/
        },
        "enable":  "true,false",
        /*ro, opt, string, whether it is enabled, desc:"true" (enabled), "false" (disabled)*/
        "groupName": {
        /*ro, opt, object, holiday group name*/
            "@min":  1,
            /*ro, opt, int, the minimum value*/
            "@max":  32
            /*ro, opt, int, the maximum value*/
        },
        "holidayPlanNo": {
        /*ro, opt, object, holiday group schedule No.*/
            "@min":  1,
            /*ro, opt, int, the minimum value*/
            "@max":  16
            /*ro, opt, int, the maximum value*/
        }
    }
}
```

10.3.9.4 Set the holiday schedule parameters of the access permission control

Request URL

PUT /ISAPI/AccessControl/UserRightHolidayPlanCfg/<holidayPlanID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
holidayPlanID	string	--

Request Message

```
{
    "UserRightHolidayPlanCfg": {
    /*req, object, the holiday schedule parameters of the access permission control*/
        "enable":  true,
        /*req, bool, whether to enable, desc:"true" (enable), "false" (disable)*/
        "beginDate":  "1970-01-01",
        /*req, date, start date of the holiday, desc:device local time*/
        "endDate":  "1970-01-01",
        /*req, date, end date of the holiday, desc:device local time*/
        "HolidayPlanCfg": [
        /*req, array, subType:object*/
            {
                "id":  1,
                /*req, int, time period No., range:[1,8], desc:it is between 1 and 8*/
                "enable":  true,
                /*req, bool, whether to enable, desc:"true" (enable), "false" (disable)*/
                "TimeSegment": {
                /*opt, object, time period*/
                    "beginTime":  "00:00:00",
                    /*req, time, start time of the time period, desc:device local time*/
                    "endTime":  "00:00:00"
                    /*req, time, end time of the time period, desc:device local time*/
                },
                "authenticationTimesEnabled":  true,
                /*opt, bool*/
                "authenticationTimes":  10
                /*opt, int, range:[1,255], step:1*/
            }
        ]
    }
}
```

Response Message

```
{
    "requestURL":  "test",
    /*ro, opt, string, URI*/
    "statusCode":  "test",
    /*ro, opt, string, status code, desc:1 (succeeded). It is required when an error occurred*/
    "statusString":  "test",
    /*ro, opt, string, status description, desc:"ok" (succeeded). It is required when an error occurred*/
    "subStatusCode":  "test",
    /*ro, opt, string, sub status code, desc:"ok" (succeeded). It is required when an error occurred*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:it is required when the value of statusCode is not 1, and it corresponds to subStatusCode*/
    "errorMsg":  "ok"
    /*ro, opt, string, error information, desc:this node is required when the value of statusCode is not 1*/
}
```


10.3.9.5 Get holiday schedule configuration parameters

Request URL

GET /ISAPI/AccessControl/UserRightHolidayPlanCfg/<holidayPlanID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
holidayPlanID	string	--

Request Message

None

Response Message

```
{
    "UserRightHolidayPlanCfg": {
    /*ro, req, object, holiday schedule configuration parameters*/
        "enable":  true,
        /*ro, req, bool, whether to enable, desc:true (enable), false (disable)*/
        "beginDate":  "1970-01-01",
        /*ro, req, date, start date of the holiday, desc:device local time*/
        "endDate":  "1970-01-01",
        /*ro, req, date, end date of the holiday, desc:device local time*/
        "HolidayPlanCfg": [
        /*ro, req, array, holiday schedule parameters, subType:object*/
            {
                "id":  1,
                /*ro, req, int, time period No., range:[1,8], desc:it is between 1 and 8*/
                "enable":  true,
                /*ro, req, bool, whether to enable, desc:true (enable), false (disable)*/
                "TimeSegment": {
                /*ro, req, object, time period*/
                    "beginTime":  "00:00:00",
                    /*ro, req, time, start time of the time period, desc:device local time*/
                    "endTime":  "00:00:00"
                    /*ro, req, time, end time of the time period, desc:device local time*/
                },
                "authenticationTimesEnabled":  true,
                /*ro, opt, bool*/
                "authenticationTimes":  10
                /*ro, opt, int, range:[1,255], step:1*/
            }
        ]
    }
}
```


10.3.9.6 Get the holiday schedule configuration capability of the access permission control

Request URL

GET /ISAPI/AccessControl/UserRightHolidayPlanCfg/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "UserRightHolidayPlanCfg": {
    /*ro, req, object*/
        "planNo": {
        /*ro, opt, object, holiday schedule No.*/
            "@min":  1,
            /*ro, opt, int, the minimum value*/
            "@max":  16
            /*ro, opt, int, the maximum value*/
        },
        "enable":  "true,false",
        /*ro, opt, string, whether it is enabled, desc:"true" (enabled), "false" (disabled)*/
        "beginDate":  "1970-01-01",
        /*ro, opt, date, start date of the holiday, desc:(device local time)*/
        "endDate":  "1970-01-01",
        /*ro, opt, date, end date of the holiday, desc:(device local time)*/
        "HolidayPlanCfg": {
        /*ro, opt, object, holiday schedule parameter*/
            "maxSize":  8,
            /*ro, opt, int, the maximum value*/
            "id": {
            /*ro, opt, object, time period No.*/
                "@min":  1,
                /*ro, opt, int, the minimum value*/
                "@max":  8
                /*ro, opt, int, the maximum value*/
            },
            "enable":  "true,false",
            /*ro, opt, string, whether it is enabled, desc:"true" (enabled), "false" (disabled)*/
            "TimeSegment": {
            /*ro, opt, object, time period*/
                "beginTime":  "00:00:00",
                /*ro, opt, time, start time, desc:(device local time)*/
                "endTime":  "00:00:00",
                /*ro, opt, time, end time, desc:(device local time)*/
                "validUnit":  "minute"
                /*ro, opt, enum, time accuracy, subType:string, desc:"hour", "minute", "second". If this node is not returned, the default time accuracy is
"minute"*/
            },
            "authenticationTimesEnabled": {
            /*ro, opt, object*/
                "@opt": [true, false]
                /*ro, req, array, subType:bool*/
            },
            "authenticationTimes": {
            /*ro, opt, object*/
                "@min":  1,
                /*ro, req, int, range:[1,255], step:1*/
                "@max":  255
                /*ro, req, int, range:[1,255], step:1*/
            }
        }
    }
}
```


10.3.9.7 Get the schedule template configuration parameters of the access permission control

Request URL

GET /ISAPI/AccessControl/UserRightPlanTemplate/<planTemplateID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
planTemplateID	string	--

Request Message

None

Response Message

```
{
    "UserRightPlanTemplate": {
    /*ro, req, object*/
        "enable":  true,
        /*ro, req, bool, whether to enable, desc:true (yes), false (no)*/
        "templateName":  "test",
        /*ro, req, string, template name*/
        "weekPlanNo":  1,
        /*ro, req, int, week schedule No.*/
        "holidayGroupNo":  "1,3,5"
        /*ro, req, string, holiday group No., desc:holiday group No.*/
    }
}
```

10.3.9.8 Set the schedule template parameters of the access permission control

Request URL

PUT /ISAPI/AccessControl/UserRightPlanTemplate/<planTemplateID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
planTemplateID	string	--

Request Message

```
{
    "UserRightPlanTemplate": {
    /*req, object, the schedule template of the access permission control*/
        "enable":  true,
        /*req, bool, whether to enable, desc:true (yes), false (no)*/
        "templateName":  "test",
        /*req, string, template name*/
        "weekPlanNo":  1,
        /*req, int, week schedule No.*/
        "holidayGroupNo":  "1,3,5",
        /*req, string, holiday group No., desc:holiday group No.*/
        "operateType":  "byTerminal",
        /*opt, enum, operation type, subType:string, desc:"byTerminal" (by terminal), "byOrg" (by organization), "byTerminalOrg" (by terminal
organization)*/
        "terminalNoList": [1, 2, 3, 4],
        /*opt, array, terminal ID list, subType:int, desc:this node is required when operation type is "byTerminal" or "byTerminalOrg"*/
        "orgNoList": [1, 2, 3, 4]
        /*opt, array, organization ID list, subType:int, desc:this node is required when operation type is "byOrg" or "byTerminalOrg"*/
    }
}
```


Response Message

```
{
    "requestURL":  "test",
    /*ro, opt, string, URI*/
    "statusCode":  "test",
    /*ro, opt, string, status code, desc:1 (succeeded). It is required when an error occurred*/
    "statusString":  "test",
    /*ro, opt, string, status description, desc:"ok" (succeeded). It is required when an error occurred*/
    "subStatusCode":  "test",
    /*ro, opt, string, sub status code, desc:"ok" (succeeded). It is required when an error occurred*/
    "errorCode":  1,
    /*ro, req, int, error code, desc:it is required when the value of statusCode is not 1, and it corresponds to subStatusCode*/
    "errorMsg":  "ok"
    /*ro, req, string, error information, desc:this node is required when the value of statusCode is not 1*/
}
```

10.3.9.9 Get the schedule template configuration capability of the access permission control

Request URL

GET /ISAPI/AccessControl/UserRightPlanTemplate/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message


```
{
    "UserRightPlanTemplate": {
    /*ro, opt, object*/
        "templateNo": {
        /*ro, opt, object, schedule template No.*/
            "@min":  1,
            /*ro, opt, int, the minimum value*/
            "@max":  16
            /*ro, opt, int, the maximum value*/
        },
        "enable":  "true,false",
        /*ro, opt, string, whether it is enabled, desc:"true" (enabled), "false" (disabled)*/
        "templateName": {
        /*ro, opt, object, template name*/
            "@min":  1,
            /*ro, opt, int, the minimum value*/
            "@max":  32
            /*ro, opt, int, the maximum value*/
        },
        "weekPlanNo": {
        /*ro, opt, object, weekly schedule No.*/
            "@min":  1,
            /*ro, opt, int, the minimum value*/
            "@max":  16
            /*ro, opt, int, the maximum value*/
        },
        "holidayGroupNo": {
        /*ro, opt, object, holiday group No.*/
            "@min":  1,
            /*ro, opt, int, the minimum value*/
            "@max":  16
            /*ro, opt, int, the maximum value*/
        }
    }
}
```


10.3.9.10 Get weekly schedule configuration parameters

Request URL

GET /ISAPI/AccessControl/UserRightWeekPlanCfg/<weekPlanID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
weekPlanID	string	--

Request Message

None

Response Message

```
{
    "UserRightWeekPlanCfg": {
    /*ro, opt, object, weekly schedule configuration parameters*/
        "enable":  true,
        /*ro, req, bool, whether to enable, desc:true (enable), false (disable)*/
        "WeekPlanCfg": [
        /*ro, req, array, weekly schedule parameters, subType:object*/
            {
                "week":  "Monday",
                /*ro, req, enum, day of the week, subType:string, desc:"Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"*/
                "id":  1,
                /*ro, req, int, time period No., range:[1,8], desc:it is between 1 and 8*/
                "enable":  true,
                /*ro, req, bool, whether to enable, desc:true (enable), false (disable)*/
                "TimeSegment": {
                /*ro, req, object, time period*/
                    "beginTime":  "10:10:00",
                    /*ro, req, string, start time of the time period, desc:device local time*/
                    "endTime":  "12:10:00"
                    /*ro, req, string, end time of the time period, desc:device local time*/
                },
                "authenticationTimesEnabled":  true,
                /*ro, opt, bool*/
                "authenticationTimes":  10
                /*ro, opt, int, range:[1,255], step:1*/
            }
        ]
    }
}
```

10.3.9.11 Set the week schedule parameters of the access permission control

Request URL

PUT /ISAPI/AccessControl/UserRightWeekPlanCfg/<weekPlanID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
weekPlanID	string	--

Request Message

```
{
    "UserRightWeekPlanCfg": {
    /*opt, object, the week schedule parameters of the access permission control*/
        "enable":  true,
        /*req, bool, whether to enable, desc:"true" (enable), "false" (disable)*/
        "WeekPlanCfg": [
        /*req, array, week schedule parameters, subType:object*/
            {
                "week":  "Monday",
                /*req, enum, days of the week, subType:string, desc:"Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"*/
                "id":  1,
                /*req, int, time period No., range:[1,8], desc:it is between 1 and 8*/
                "enable":  true,
                /*req, bool, whether to enable, desc:"true" (enable), "false" (disable)*/
                "TimeSegment": {
                /*req, object, time period*/
                    "beginTime":  "10:10:00",
                    /*req, string, start time of the time period, desc:(device local time)*/
                    "endTime":  "12:10:00"
                    /*req, string, end time of the time period, desc:(device local time)*/
                },
                "authenticationTimesEnabled":  true,
                /*opt, bool*/
                "authenticationTimes":  10
                /*opt, int, range:[1,255], step:1*/
            }
        ]
    }
}
```

Response Message

```
{
    "requestURL":  "test",
    /*ro, opt, string, URI*/
    "statusCode":  "test",
    /*ro, opt, string, status code, desc:1 (succeeded). It is required when an error occurred*/
    "statusString":  "test",
    /*ro, opt, string, status description, desc:"ok" (succeeded). It is required when an error occurred*/
    "subStatusCode":  "test",
    /*ro, opt, string, sub status code, desc:"ok" (succeeded). It is required when an error occurred*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:it is required when the value of statusCode is not 1, and it corresponds to subStatusCode*/
    "errorMsg":  "ok"
    /*ro, opt, string, error information, desc:this node is required when the value of statusCode is not 1*/
}
```

10.3.9.12 Get the weekly schedule configuration capability of the access permission control

Request URL

GET /ISAPI/AccessControl/UserRightWeekPlanCfg/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "UserRightWeekPlanCfg": {
    /*ro, opt, object*/
        "planNo": {
        /*ro, opt, object, weekly schedule No.*/
            "@min":  1,
            /*ro, opt, int, the minimum value*/
            "@max":  16
            /*ro, opt, int, the maximum value*/
        },
        "enable":  "true,false",
        /*ro, opt, string, whether it is enabled, desc:"true" (enabled), "false" (disabled)*/
        "WeekPlanCfg": {
        /*ro, opt, object, weekly schedule parameters*/
            "maxSize":  56,
            /*ro, opt, int, the maximum value*/
            "week": {
            /*ro, opt, object, week*/
                "@opt":  "Monday"
                /*ro, opt, enum, days of the week, subType:string, desc:"Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"*/
            },
            "id": {
            /*ro, opt, object*/
                "@min":  1,
                /*ro, opt, int, the minimum value*/
                "@max":  8
                /*ro, opt, int, the maximum value*/
            },
            "enable":  "true,false",
            /*ro, opt, string, whether it is enabled, desc:"true" (enabled), "false" (disabled)*/
            "TimeSegment": {
            /*ro, opt, object, time period*/
                "beginTime":  "test",
                /*ro, opt, string, start time, desc:(device local time)*/
                "endTime":  "test",
                /*ro, opt, string, end time, desc:(device local time)*/
                "validUnit":  "minute"
                /*ro, opt, enum, time accuracy, subType:string, desc:"hour", "minute", "second". If this node is not returned, the default time accuracy is
"minute"*/
            },
            "authenticationTimesEnabled": {
            /*ro, opt, object*/
                "@opt": [true, false]
                /*ro, req, array, subType:bool*/
            },
            "authenticationTimes": {
            /*ro, opt, object*/
                "@min":  1,
                /*ro, req, int, range:[1,255], step:1*/
                "@max":  255
                /*ro, req, int, range:[1,255], step:1*/
            }
        }
    }
}
```


10.3.10 Person and Credential Management

10.3.10.1 Get the capability of deleting person information (including linked cards, fingerprints, and faces) and permissions

Request URL

GET /ISAPI/AccessControl/UserInfoDetail/Delete/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "UserInfoDetail": {
    /*ro, opt, object, user Information*/
        "mode": {
        /*ro, req, object*/
            "@opt":  "all,byEmployeeNo"
            /*ro, opt, string, deleting mode, desc:all (delete all), byEmployeeNo (delete by employee No. (person ID))*/
        },
        "EmployeeNoList": {
        /*ro, opt, object, person ID list, desc:person ID list*/
            "maxSize":  50,
            /*ro, opt, int*/
            "employeeNo": {
            /*ro, opt, object, employee No. (person ID)*/
                "@min":  1,
                /*ro, opt, int, the maximum value*/
                "@max":  32
                /*ro, opt, int, the minimum value*/
            }
        }
    }
}
```

10.3.10.2 Start deleting all person information (including linked cards, fingerprints, and faces) and permissions by employee No.

Request URL

PUT /ISAPI/AccessControl/UserInfoDetail/Delete?format=json

Query Parameter

None

Request Message

```
{
    "UserInfoDetail": {
    /*opt, object, user Information*/
        "mode":  "all",
        /*req, enum, deleting mode, subType:string, desc:deleting mode*/
        "EmployeeNoList": [
        /*opt, array, person ID list, subType:object*/
            {
                "employeeNo":  "test"
                /*opt, string, employee No.*/
            }
        ],
        "operateType":  "byTerminal",
        /*opt, enum, operation mode, subType:string, desc:"byTerminal" (by terminal), "byOrg" (by organization), "byTerminalOrg" (by terminal
organization)*/
        "terminalNoList": [1, 2, 3, 4],
        /*opt, array, terminal list, subType:int, dep:and,{$.UserInfoDetail.operateType,eq,byTerminal}*/
        "orgNoList": [1, 2, 3, 4]
        /*opt, array, organization list, subType:int, dep:or,{$.UserInfoDetail.operateType,eq,byOrg},{$.UserInfoDetail.operateType,eq,byTerminalOrg}*/
    }
}
```

Response Message

```
{
    "statusCode":  1,
    /*ro, opt, int, status code, desc:1 (succeeded). It is required when an error occurred*/
    "statusString":  "ok",
    /*ro, opt, string, status description, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "subStatusCode":  "ok",
    /*ro, opt, string, sub status code, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:when the value of statusCode is not 1, it corresponds to subStatusCode*/
    "errorMsg":  "ok"
    /*ro, opt, string, error information, desc:this field is required when the value of statusCode is not 1*/
}
```

10.3.10.3 Get the status of deleting all person information (including linked cards, fingerprints, and faces) and permissions by employee No

Request URL

GET /ISAPI/AccessControl/UserInfoDetail/DeleteProcess?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "UserInfoDetailDeleteProcess": {
    /*ro, req, object*/
        "status":  "processing",
        /*ro, req, enum, status, subType:string, desc:status*/
        "percent":  100
        /*ro, opt, int, range:[0,100], dep:or,{$.UserInfoDetailDeleteProcess.status,eq,processing}*/
    }
}
```

10.3.11 Person and Credential Management

10.3.11.1 Get the capability of deleting person information (including linked cards, fingerprints, and faces) and permissions

Request URL

GET /ISAPI/AccessControl/UserInfoDetail/Delete/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "UserInfoDetail": {
    /*ro, opt, object, user Information*/
        "mode": {
        /*ro, req, object*/
            "@opt":  "all,byEmployeeNo"
            /*ro, opt, string, deleting mode, desc:all (delete all), byEmployeeNo (delete by employee No. (person ID))*/
        },
        "EmployeeNoList": {
        /*ro, opt, object, person ID list, desc:person ID list*/
            "maxSize":  50,
            /*ro, opt, int*/
            "employeeNo": {
            /*ro, opt, object, employee No. (person ID)*/
                "@min":  1,
                /*ro, opt, int, the maximum value*/
                "@max":  32
                /*ro, opt, int, the minimum value*/
            }
        }
    }
}
```

10.3.11.2 Start deleting all person information (including linked cards, fingerprints, and faces) and permissions by employee No.

Request URL

PUT /ISAPI/AccessControl/UserInfoDetail/Delete?format=json

Query Parameter

None

Request Message

```
{
    "UserInfoDetail": {
    /*opt, object, user Information*/
        "mode":  "all",
        /*req, enum, deleting mode, subType:string, desc:deleting mode*/
        "EmployeeNoList": [
        /*opt, array, person ID list, subType:object*/
            {
                "employeeNo":  "test"
                /*opt, string, employee No.*/
            }
        ],
        "operateType":  "byTerminal",
        /*opt, enum, operation mode, subType:string, desc:"byTerminal" (by terminal), "byOrg" (by organization), "byTerminalOrg" (by terminal 
organization)*/
        "terminalNoList": [1, 2, 3, 4],
        /*opt, array, terminal list, subType:int, dep:and,{$.UserInfoDetail.operateType,eq,byTerminal}*/
        "orgNoList": [1, 2, 3, 4]
        /*opt, array, organization list, subType:int, dep:or,{$.UserInfoDetail.operateType,eq,byOrg},{$.UserInfoDetail.operateType,eq,byTerminalOrg}*/
    }
}
```

Response Message

```
{
    "statusCode":  1,
    /*ro, opt, int, status code, desc:1 (succeeded). It is required when an error occurred*/
    "statusString":  "ok",
    /*ro, opt, string, status description, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "subStatusCode":  "ok",
    /*ro, opt, string, sub status code, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:when the value of statusCode is not 1, it corresponds to subStatusCode*/
    "errorMsg":  "ok"
    /*ro, opt, string, error information, desc:this field is required when the value of statusCode is not 1*/
}
```

10.3.11.3 Get the status of deleting all person information (including linked cards, fingerprints, and faces) and permissions by employee No

Request URL

GET /ISAPI/AccessControl/UserInfoDetail/DeleteProcess?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "UserInfoDetailDeleteProcess": {
    /*ro, req, object*/
        "status":  "processing",
        /*ro, req, enum, status, subType:string, desc:status*/
        "percent":  100
        /*ro, opt, int, range:[0,100], dep:or,{$.UserInfoDetailDeleteProcess.status,eq,processing}*/
    }
}
```

10.3.12 Remote Door Control

10.3.12.1 Remotely control the door or elevator

Request URL

PUT /ISAPI/AccessControl/RemoteControl/door/<doorID>

Query Parameter

Parameter Name	Parameter Type	Description
doorID	string	--

Request Message


```xml
<?xml version="1.0" encoding="UTF-8"?>

<RemoteControlDoor xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--req, object, attr:version{req, string, protocolVersion}-->
  <cmd>
    <!--req, enum, command, subType:string, desc:"open" (open the door), "close" (close the door (controlled)), "alwaysOpen" (remain unlocked (free)), "alwaysClose" (remain open (disabled)), "visitorCallLadder" (call elevator (visitor)), "householdCallLadder" (call elevator (resident))-->open
  </cmd>
  <password>
    <!--opt, string, range:[8,16]-->test
  </password>
  <employeeNo>
    <!--wo, opt, string, employee No., range:[0,32], desc:employee No.-->test
  </employeeNo>
  <channelNo>
    <!--opt, int, range:[0,256], step:1-->1
  </channelNo>
  <controlType>
    <!--opt, enum, subType:string-->monitor
  </controlType>
</RemoteControlDoor>
```


Response Message


```xml
<?xml version="1.0" encoding="UTF-8"?>

<ResponseStatus xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, response message, attr:version{ro, req, string, protocolVersion}-->
  <requestURL>
    <!--ro, req, string, request URL-->null
  </requestURL>
  <statusCode>
    <!--ro, req, enum, status code, subType:int, desc:0 (OK), 1 (OK), 2 (Device Busy), 3 (Device Error), 4 (Invalid Operation), 5 (Invalid XML Format), 6 (Invalid XML Content), 7 (Reboot Required)-->0
  </statusCode>
  <statusString>
    <!--ro, req, enum, status information, subType:string, desc:"OK" (succeeded), "Device Busy", "Device Error", "Invalid Operation", "Invalid XML Format", "Invalid XML Content", "Reboot" (reboot device)-->OK
  </statusString>
  <subStatusCode>
    <!--ro, req, string, sub status code, desc:sub status code, which describes the error in details-->OK
  </subStatusCode>
</ResponseStatus>
```


10.3.12.2 Get the capability set of remote door status control

Request URL

GET /ISAPI/AccessControl/RemoteControl/door/capabilities

Query Parameter

None

Request Message

None

Response Message

```xml
<?xml version="1.0" encoding="UTF-8"?>

<RemoteControlDoor xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, remote door status control, attr:version{req, float, protocolVersion}-->
  <doorNo min="1" max="10">
    <!--ro, opt, int, range of the door No., attr:min{req, int},max{req, int}-->1
  </doorNo>
  <cmd opt="open,close,alwaysOpen,alwaysClose,visitorCallLadder,householdCallLadder,resume">
    <!--ro, req, enum, command, subType:string, attr:opt{req, string}, desc:"open" (open the door), "close" (close the door (controlled)), "alwaysOpen" (remain unlocked (free)), "alwaysClose" (remain locked (disabled)), "visitorCallLadder" (call elevator (visitor)), "householdCallLadder" (call elevator (resident))-->open
  </cmd>
  <password min="8" max="16">
    <!--ro, opt, string, door opening password, range:[8,16], attr:min{req, int, range:[8,16]},max{req, int, range:[8,16]}-->test
  </password>
  <employeeNo min="0" max="32">
    <!--ro, opt, string, employee No., range:[0,32], attr:min{req, int, range:[0,32]},max{req, int, range:[0,32]}-->test
  </employeeNo>
  <channelNo min="0" max="10">
    <!--ro, opt, int, attr:min{req, int},max{req, int}-->0
  </channelNo>
  <controlType opt="monitor,calling">
    <!--ro, opt, enum, subType:string, attr:opt{req, string}-->monitor
  </controlType>
</RemoteControlDoor>
```

10.3.12.3 Get the capability of configuring password for remote door control

Request URL

GET /ISAPI/AccessControl/remoteControlPWCfg/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```json
{
    "RemoteControlPWCfg": {
    /*ro, opt, object*/
        "password": {
        /*ro, opt, object, password for remote door control*/
            "@min":  "6",
            /*ro, opt, string*/
            "@max":  "6"
            /*ro, opt, string*/
        }
    }
}
```


10.3.12.4 Configure the password for remote door control

Request URL

PUT /ISAPI/AccessControl/remoteControlPWCfg/door/<doorID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
doorID	string	--

Request Message

```json
{
    "RemoteControlPWCfg": {
    /*opt, object, password configuration for remote door control*/
        "password":  "test"
        /*opt, string, password for remote door control*/
    }
}
```

Response Message

```
{
    "requestURL":  "test",
    /*ro, opt, string, URI*/
    "statusCode":  "test",
    /*ro, opt, string, status code*/
    "statusString":  "test",
    /*ro, opt, string, status description*/
    "subStatusCode":  "test",
    /*ro, opt, string, sub status code*/
    "errorCode":  1,
    /*ro, req, int, error code*/
    "errorMsg":  "ok"
    /*ro, req, string, error information*/
}
```

10.3.12.5 Get the password for remotely controlling the door

Request URL

GET /ISAPI/AccessControl/remoteControlPWCfg/door/<doorID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
doorID	string	--

Request Message

None

Response Message

```
{
    "RemoteControlPWCfg": {
    /*ro, opt, object, password settings for remotely controlling the door*/
        "password":  "test"
        /*ro, opt, string, password for remotely controlling the door*/
    }
}
```


10.3.12.6 Get the capability of verifying the password for remote door control

Request URL

GET /ISAPI/AccessControl/remoteControlPWCheck/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "RemoteControlPWCfg": {
    /*ro, opt, object*/
        "password": {
        /*ro, opt, object, password for remote door control, desc:password for remote door control (or EZ verification code). The password must contain 6 
digits and it ranges from 000000 to 999999*/
            "@min":  6,
            /*ro, opt, int, the minimum value*/
            "@max":  6
            /*ro, opt, int, the maximum value*/
        }
    }
}
```

10.3.12.7 Verify the password for remote door control

Request URL

PUT /ISAPI/AccessControl/remoteControlPWCheck/door/<doorID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
doorID	string	--

Request Message

```
{
    "RemoteControlPWCheck": {
    /*opt, object, password verification for remote door control*/
        "password":  "test"
        /*opt, string, password for remote door control (or EZVIZ verification code)*/
    }
}
```

Response Message

```
{
    "statusCode":  1,
    /*ro, opt, int, status code, desc:1 (succeeded). It is required when an error occurred*/
    "statusString":  "ok",
    /*ro, opt, string, status description, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "subStatusCode":  "ok",
    /*ro, opt, string, sub status code, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:it corresponds to subStatusCode when statusCode is not 1*/
    "errorMsg":  "ok"
    /*ro, opt, string, error information, desc:this field is required when the value of statusCode is not 1*/
}
```

10.4 Credentials Collection

10.4.1 Card Online Adding

10.4.1.1 Get the capability of collecting card information

Request URL

GET /ISAPI/AccessControl/CaptureCardInfo/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "CardInfoCap": {
    /*ro, req, object*/
        "cardNo": {
        /*ro, opt, object, card No.*/
            "@min":  1,
            /*ro, req, int, the minimum length, range:[1,32]*/
            "@max":  32
            /*ro, req, int, the maximum length, range:[1,32]*/
        },
        "cardType": ["TypeA_M1", "TypeA_CPU", "TypeB", "ID_125K", "FelicaCard", "DesfireCard"],
        /*ro, opt, array, card type, subType:string, range:[1,6]*/
        "readerID": {
        /*ro, opt, object*/
            "@min":  1,
            /*ro, req, int*/
            "@max":  8
            /*ro, req, int*/
        }
    }
}
```

10.4.1.2 Collect card information by the card reading module of the device

Request URL

GET /ISAPI/AccessControl/CaptureCardInfo?format=json&readerID=<readerID>

Query Parameter

Parameter Name	Parameter Type	Description
readerID	string	--

Request Message

None

Response Message

```
{
    "CardInfo": {
    /*ro, req, object, card information*/
        "cardNo":  "abcd1234",
        /*ro, req, string, card No.*/
        "cardType":  "TypeA_M1",
        /*ro, opt, enum, card type, subType:string, desc:"TypeA_M1", "TypeA_CPU", "TypeB", "ID_125K", "FelicaCard" (FeliCa card), "DesfireCard" (DESFire card)*/
        "readerID":  1
        /*ro, opt, int, range:[1,8]*/
    }
}
```

10.4.2 Fingerprint Online Adding

10.4.2.1 Collect fingerprint information

Request URL

POST /ISAPI/AccessControl/CaptureFingerPrint

Query Parameter

None

Request Message

```
<?xml version="1.0" encoding="UTF-8"?>

<CaptureFingerPrintCond xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--opt, object, Collect fingerprint information conditions, attr:version{req, string, protocolVersion}-->
  <fingerNo>
    <!--req, int, finger No., range:[1,10]-->1
  </fingerNo>
</CaptureFingerPrintCond>
```

Response Message

```
<?xml version="1.0" encoding="UTF-8"?>

<CaptureFingerPrint xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, attr:version{req, string, protocolVersion}-->
  <fingerData>
    <!--ro, opt, string, fingerprint data, range:[1,768], desc:which, should be encoded by Base64-->test
  </fingerData>
  <fingerNo>
    <!--ro, req, int, finger No., range:[1,10]-->1
  </fingerNo>
  <fingerPrintQuality>
    <!--ro, req, int, fingerprint quality, range:[1,100]-->1
  </fingerPrintQuality>
</CaptureFingerPrint>
```

Parameter Name	Parameter Value	Parameter Type(Content-Type)	Content-ID	File Name	Description
CaptureFingerPrint	[Message content]	application/xml	--	--	--
fingerPrintPic	[Binary picture data]	image/jpeg		fingerPrintPic.jpg	--

Note: The protocol is transmitted in form format. See Chapter 4.5.1.4 for form framework description, as shown in the instance below.

```
--<frontier>
Content-Disposition: form-data; name=Parameter Name;filename=File Name
Content-Type: Parameter Type
Content-Length: ****
Content-ID: Content ID
Parameter Value
```

- Parameter Name: the name property of Content-Disposition in the header of form unit; it refers to the form unit name.
- Parameter Type (Content-Type): the Content-Type property in the header of form unit.
- File Name (filename): the filename property of Content-Disposition of form unit Headers. It exists only when the transmitted data of form unit is file, and it refers to the file name of form unit body.
- Parameter Value: the body content of form unit.

10.4.2.2 Get the fingerprint collection capability

Request URL

GET /ISAPI/AccessControl/CaptureFingerPrint/capabilities

Query Parameter

None

Request Message

None

Response Message


```xml
<?xml version="1.0" encoding="UTF-8"?>

<CaptureFingerPrint xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, collect fingerprint information, attr:version{req, string, protocolVersion}-->
  <CaptureFingerPrintCond>
    <!--ro, req, object, finger No.-->
    <fingerNo min="1" max="10">
      <!--ro, opt, int, fingerprint No., range:[1,10], attr:min{req, int},max{req, int}-->1
    </fingerNo>
  </CaptureFingerPrintCond>
  <fingerData min="1" max="768">
    <!--ro, opt, string, fingerprint data, range:[1,768], attr:min{req, int},max{req, int}-->test
  </fingerData>
  <fingerNo min="1" max="10">
    <!--ro, opt, int, fingerprint No., range:[1,10], attr:min{req, int},max{req, int}-->1
  </fingerNo>
  <fingerPrintQuality min="1" max="100">
    <!--ro, opt, int, fingerprint quality, range:[1,100], attr:min{req, int},max{req, int}-->1
  </fingerPrintQuality>
</CaptureFingerPrint>
```


10.5 Security Control Device (General)

10.5.1 Event Message Push Management

10.5.1.1 Get the parameters of the report uploading method

Request URL

GET /ISAPI/SecurityCP/ReportCenterCfg/<centerID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
centerID	string	--

Request Message

None

Response Message

```
{
    "ReportCenterCfg": {
    /*ro, req, object*/
        "enable":  true,
        /*ro, opt, bool, whether to enable the function*/
        "ChanAlarmMode": [
        /*ro, opt, array, alarm channel of the center group, subType:object*/
            {
                "id":  1,
                /*ro, opt, enum, channel ID, subType:int, desc:channel ID: 1-main channel,2-backup channel 1,3-backup channel 2,4-backup channel 3*/
                "chanAlarmMode":  "T1"
                /*ro, opt, enum, alarm channel mode, subType:string, desc:"T1" (T1 channel), "T2" (T2 channel), "N1" (N1 channel), "N2" (N2 channel), "G1"
(G1 channel), "G2" (G2 channel), "N3" ()N3 channel, ”N4" (N4 channel)*/
            }
        ]
    }
}
```

10.5.1.2 Set the parameters of the report uploading method

Request URL

PUT /ISAPI/SecurityCP/ReportCenterCfg/<centerID>?format=json

Query Parameter

Parameter Name	Parameter Type	Description
centerID	string	--

Request Message

```
{
    "ReportCenterCfg": {
    /*req, object*/
        "enable":  true,
        /*opt, bool, whether to enable uploading report*/
        "ChanAlarmMode": [
        /*opt, array, alarm channel of the center group, subType:object*/
            {
                "id":  1,
                /*opt, enum, channel ID, subType:int, desc:1 (main channel), 2 (backup channel 1), 3 (backup channel 2), 4 (backup channel 3)*/
                "chanAlarmMode":  "T1"
                /*opt, enum, alarm channel mode, subType:string, desc:"T1" (T1 channel), "T2" (T2 channel), "N1" (N1 channel), "N2" (N2 channel), "G1" (G1
channel), "G2" (G2 channel), "N3" ()N3 channel, ”N4" (N4 channel)*/
            }
        ]
    }
}
```

Response Message

```
{
    "statusCode":  1,
    /*ro, opt, int, status code, desc:1 (succeeded). It is required when an error occurred*/
    "statusString":  "ok",
    /*ro, opt, string, status description, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "subStatusCode":  "ok",
    /*ro, opt, string, sub status code, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:it is required when the value of statusCode is not 1, it corresponds to subStatusCode*/
    "errorMsg":  "ok"
    /*ro, opt, string, error description, desc:this field is required when the value of statusCode is not 1*/
}
```

10.5.1.3 Get the configuration capability of the report uploading method

Request URL

GET /ISAPI/SecurityCP/ReportCenterCfg/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "ReportCenterCfg": {
    /*ro, req, object*/
        "CenterID": {
        /*ro, opt, object, center group No.*/
            "@min":  1,
            /*ro, opt, int, the minimum value*/
            "@max":  1
            /*ro, opt, int, the maximum value*/
        },
        "enable":  "true,false",
        /*ro, opt, string, whether to enable uploading report*/
        "ChanAlarmMode": {
        /*ro, opt, object, alarm channel of the center group*/
            "maxSize":  1,
            /*ro, opt, int, the maximum number of channels*/
            "id": {
            /*ro, opt, object, channel ID, desc:1 (main channel), 2 (backup channel 1), 3 (backup channel 2), 4 (backup channel 3)*/
                "@min":  1,
                /*ro, opt, int, the minimum value*/
                "@max":  2
                /*ro, opt, int, the maximum value*/
            },
            "chanAlarmMode": {
            /*ro, opt, object, alarm channel mode, desc:"T1" (T1 channel), "T2" (T2 channel), "N1" (N1 channel), "N2" (N2 channel), "G1" (G1 channel), "G2" 
(G2 channel), "N3" (N3 channel), "N4" (N4 channel)*/
                "@opt":  "T1,T2,N1,N2,G1,G2,N3,N4"
                /*ro, opt, string*/
            }
        }
    }
}
```

10.6 Zone Alarm

10.6.1 Security Control Panel Management

10.6.1.1 Get the capability of security control panel

Request URL

GET /ISAPI/SecurityCP/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "SecurityCPCap": {
    /*ro, req, object, capability node*/
        "partitionNum":  64,
        /*ro, opt, int, number of partitions that can be set, desc:1 by default*/
        "localZoneNum":  16,
        /*ro, req, int, number of local zones*/
        "extendZoneNum":  64,
        /*ro, opt, int, number of extended zones*/
        "wirelessZoneNum":  64,
        /*ro, opt, int, number of wireless zones*/
        "localRelayNum":  1,
        /*ro, req, int, number of local triggers*/
        "extendRelayNum":  1,
        /*ro, opt, int, number of extended triggers*/
        "wirelessRelayNum":  1,
        /*ro, opt, int, number of wireless triggers*/
        "repeater":  4,
        /*ro, opt, int, number of repeaters*/
        "sirenNum":  8,
        /*ro, opt, int, number of sounders*/
        "userNum":  1,
        /*ro, req, int, number of users*/
        "adminNum":  2,
        /*ro, opt, int, number of administrators*/
        "installerNum":  21,
        /*ro, opt, int, number of installers*/
```

```
        "operatorNum":  41,
        /*ro, opt, int, number of operators*/
        "ARCNum":  4,
        /*ro, opt, int, number of alarm receiving centers*/
        "phoneNum":  3,
        /*ro, opt, int, number of phone numbers*/
        "outputModNum":  8,
        /*ro, opt, int, number of output modules*/
        "cardNum":  64,
        /*ro, opt, int, number of cards*/
        "keypadNum":  8,
        /*ro, opt, int, number of keypads*/
        "cardReaderNum":  8,
        /*ro, opt, int, number of card readers*/
        "protocolOptimizationVersion":  "v1.0",
        /*ro, opt, string, supported protocol optimizing version*/
        "remoteCtrlNum":  64,
        /*ro, opt, int, number of remote controls*/
        "alarmLampNum":  8,
        /*ro, opt, int, number of strobe lights*/
        "electricLockNum":  64,
        /*ro, opt, int, number of electric locks*/
        "detectorModel": {
        /*ro, opt, object, supported detector models*/
            "@opt": ["0x00001", "0x00002"]
            /*ro, opt, array, options, subType:string*/
        },
        "sirenModel": {
        /*ro, opt, object, supported sounder models*/
            "@opt": ["0x7A001", "0x7A011"]
            /*ro, opt, array, options, subType:string*/
        },
        "keypadModel": {
        /*ro, opt, object, supported keypad models*/
            "@opt": ["0x92000", "0x92010"]
            /*ro, opt, array, options, subType:string*/
        },
        "cardReaderModel": {
        /*ro, opt, object, supported card reader models*/
            "@opt": ["0x90000", "0x90010"]
            /*ro, opt, array, options, subType:string*/
        },
        "remoteCtrlModel": {
        /*ro, opt, object, supported remote control models*/
            "@opt": ["0x81000", "0x81010"]
            /*ro, opt, array, options, subType:string*/
        },
        "repeaterModel": {
        /*ro, opt, object, supported repeater models*/
            "@opt": ["0x80000", "0x80010"]
            /*ro, opt, array, options, subType:string*/
        },
        "outputModuleModel": {
        /*ro, opt, object, supported output module models*/
            "@opt": ["0x71001", "0x71011"]
            /*ro, opt, array, options, subType:string*/
        },
        "isSptLogSearch":  true,
        /*ro, opt, bool, whether it supports log search*/
        "isSptLocalUser":  true,
        /*ro, opt, bool, whether it supports local users*/
        "isSptConfiguration":  true,
        /*ro, opt, bool, whether it supports configuring security control panel, desc:/ISAPI/SecurityCP/Configuration*/
        "isSptControl":  true,
        /*ro, opt, bool, whether it supports security control panel control, desc:/ISAPI/SecurityCP/control*/
        "isSptStatus":  true,
        /*ro, opt, bool, whether it supports monitoring security control panel status, desc:/ISAPI/SecurityCP/status*/
        "isSptZoneInfo":  true,
        /*ro, opt, bool, whether it supports getting the information of a specific zone, desc:/ISAPI/SecurityCP/Info/zones/<ID>?format=json*/
        "isSptSirenInfo":  true,
        /*ro, opt, bool, whether it supports getting the information of a specific sounder, desc:/ISAPI/SecurityCP/Info/siren/<ID>?format=json*/
        "isSptPaceTest":  true,
        /*ro, opt, bool, whether it supports pacing, desc:/ISAPI/SecurityCP/paceTest*/
        "isSptStandardCfg":  true,
        /*ro, opt, bool, whether it supports standard configuration for security control panel, desc:/ISAPI/SecurityCP/standardCfg*/
        "isSptPircamCapture":  true,
        /*ro, opt, bool, whether it supports pircam (detector equipped with camera) capture, desc:/ISAPI/SecurityCP/pircam/channels/<ID>/picture?
format=json*/
        "isSptManualControlCapture":  true,
        /*ro, opt, bool*/
        "isSptGetPictureByURL":  true,
        /*ro, opt, bool*/
        "isSptOneKeyAlarm":  true,
        /*ro, opt, bool, whether it supports one-push alarm, desc:/ISAPI/SecurityCP/control/oneKeyAlarm. For compatibility, this node will also be returned
in the capability message JSON_HostControlCap after calling the URI /ISAPI/SecurityCP/control/capabilities?format=json by GET method*/
        "isSptOneKeyAlarmSoundCtrl":  true,
        /*ro, opt, bool*/
        "transmitterNum":  8,
        /*ro, opt, int, the number of transmitters*/
        "transmitterModel": {
        /*ro, opt, object, supported model of transmitters*/
            "@opt": ["0x71001"]
            /*ro, opt, array, options, subType:string*/
        },
```

```
        },
        "localAccessModuleType": {
        /*ro, opt, object, onboard access module type*/
            "@opt": ["localTransmitter", "localZone", "localRelay", "localSiren"]
            /*ro, opt, array, options, subType:string, desc:"localTransmitter" (onboard transmitter), "localZone" (onboard zone module), "localRelay"
(onboard relay module), "localSiren" (onboard sounder module)*/
        },
        "networkZoneModuleNum":  8
        /*ro, opt, int, number of network zone modules*/
    }
}
```

10.6.1.2 Get the configuration capability of security control panel

Request URL

GET /ISAPI/SecurityCP/Configuration/capabilities?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "HostConfigCap": {
    /*ro, req, object, capability node*/
        "ExDevice": {
        /*ro, opt, object, peripheral node*/
            "isSptOutput":  true,
            /*ro, opt, bool, whether it supports relay management, desc:/ISAPI/SecurityCP/Configuration/outputs*/
        },
        "isSptReportCenterCfg":  true,
        /*ro, opt, bool, whether it supports configuring the report uploading method, desc:/ISAPI/SecurityCP/ReportCenterCfg/<ID>?format=json*/
        "isSptAlarmOutCfg":  true,
        /*ro, opt, bool, whether it supports configuring alarm output parameters, desc:/ISAPI/SecurityCP/AlarmOutCfg/<ID>?format=json*/
        "isSptSetAlarmHostOut":  true,
        /*ro, opt, bool, whether it supports setting alarm output, desc:/ISAPI/SecurityCP/SetAlarmHostOut?format=json*/
    }
}
```

11 How-To Video Guidance

If you need access to corresponding video guidance for device integration, please register on https://tpp.hikvision.com and visit our Training Center: https://tpp.hikvision.com/tpp/Training. The Training Center is specifically designed to provide technical training and guidance resources for our partners. On this platform, you can find integration video tutorials for various devices, enabling better understanding and learning of the integration process. To offer more personalized service, our Training Center also supports filtering by integration protocols, devices, and applications.